

spintent

SPANISH PARSE INTENTS

V0.99 2026-10-05*

©2026 by Pablo González L[†]

CTAN: <https://www.ctan.org/pkg/spintent>

 <https://github.com/pablgonz/spintent>

Resumen

El paquete **spintent** proporciona una serie de utilidades para docentes de educación primaria y secundaria que necesiten crear documentos PDF *accesibles* (*tagged* PDF) en español usando Lua $\text{\LaTeX}$.

Índice

1	Descripción	1	3.3	El comando <code>\spmoney</code>	6
1.1	Carga del paquete	1	3.3.1	Llaves para <code>\spmoney</code>	7
1.2	El paquete <code>babel</code>	2	3.3.2	Monedas y alias para <code>\spmoney</code>	7
1.3	Fuentes tipográficas	2	3.4	El comando <code>\spshort</code>	8
1.4	El paquete <code>hyperref</code>	2	3.4.1	Llaves para <code>\spshort</code>	8
2	El comando <code>\setspintent</code>	3	3.4.2	Argumentos para <code>\spshort</code>	9
3	The big four	3	4	Utilidades generales	10
3.1	El comando <code>\spnum</code>	3	5	Utilidades para símbolos	11
3.1.1	Llaves para <code>\spnum</code>	3	6	Utilidades para números	13
3.2	El comando <code>\spunit</code>	4	7	Utilidades para geometría	19
3.2.1	Llaves para <code>\spunit</code>	4	8	Referencias	35
3.2.2	Argumentos para <code>\spunit</code>	4	9	Change history	36
3.2.3	El comando <code>\spnewunit</code>	6	10	Índice de la Documentación	37
3.2.4	El comando <code>\spunitalias</code>	6	11	Implementation	40
			12	Índice de la Implementación	166

Motivación y agradecimientos

Desde que desarrollé `scontents`[7] en 2019 y, más tarde, `enumext`[8] en 2024, me he ido interesando cada vez más por la accesibilidad en los documentos PDF. Al principio era un tema prácticamente desconocido para mí, pero a medida que fui aprendiendo sobre él comprendí la importancia que tiene, especialmente en el ámbito educativo.

El equipo del proyecto de $\text{\LaTeX}$ [19] ha realizado un trabajo extraordinario en este campo. Hoy es posible generar documentos etiquetados, validarlos con herramientas como `veraPDF`[20] o inspeccionar su estructura con `ngpdf`[21]. Sin embargo, siempre me quedó una duda: cuando un estudiante utiliza NVDA[11] junto con `MathCAT`[12], ¿realmente escucha lo que el o la docente quiso expresar?

Basta con generar un documento accesible siguiendo las recomendaciones de `The LaTeX Tagged PDF Project` y escucharlo con NVDA/`MathCAT` para darse cuenta de que, en muchos casos, la *verbalización* no coincide con lo que uno espera. Esto no resulta extraño: las reglas del español, y en particular el lenguaje matemático, dependen en gran medida del contexto en que se utilizan.

El objetivo de **spintent** nace precisamente de esta necesidad: proporcionar herramientas para que el o la docente pueda indicar explícitamente el contexto de determinadas expresiones matemáticas desde el propio código fuente de $\text{\LaTeX}$, de modo que el documento PDF resultante no solo sea *accesible*, sino que también produzca una *verbalización más natural* y adecuada para el estudiante al utilizar NVDA/`MathCAT`.

El nuevo estándar PDF 2.0[14, 18, 16], las especificaciones de WTPDF[15] y la guía más reciente de la PDF Association, «Best Practice Guide: Math in PDF»[17], coinciden en que el camino hacia una accesibilidad matemática de calidad pasa por el uso de `MathML`. En ese contexto, Lua $\text{\LaTeX}$ ofrece una plataforma especialmente adecuada para incorporar esta tecnología.

*Este archivo describe la documentación para la versión v0.99 revisada por última vez el 2026-10-05.

[†]E-mail: «pablgonz@educarchile.cl».

Licencia y requerimientos

El paquete `spintent` carga y requiere el paquete `unicode-math`[1] y que se ejecute bajo Lua $\TeX$ por lo que es necesario contar con una distribución moderna de $\TeX$ como $\TeX$ Live 2026. Ha sido probado con las clases estándar proporcionadas por $\LaTeX$: `book`, `report`, `article` y `letter` en tamaños `10pt`, `11pt` y `12pt`.

Se otorga permiso para copiar, distribuir y/o modificar este paquete bajo los términos de $\LaTeX$ Project Public License (LPPL), versión 1.3 o posterior (<https://www.latex-project.org/lppl.txt>). El paquete tiene la condición de mantenido al estilo «*A Work in Progress*»¹.

- El requerimiento mínimo es $\LaTeX$ release 2026-11-01.

Este paquete solo se puede utilizar solo con Lua $\TeX$ y *tagged* PDF activo. Está presente en $\TeX$ Live y MiK $\TeX$, para instalarlo, utilice el gestor de paquetes de su distribución. Si desea una instalación manual, descargue `spintent.zip` y descomprímalo, luego ejecute `luatex spintent.ins`, mueva todos los archivos a las ubicaciones correspondientes y luego ejecute `mktexlsr`. Para generar la documentación, ejecute `arara spintent.dtx`.

```
spintent.sty  ⇒ TDS:tex/lualatex/spintent/
spintent.lua  ⇒ TDS:tex/lualatex/spintent/
spintent.pdf  ⇒ TDS:doc/lualatex/spintent/
README.md    ⇒ TDS:doc/lualatex/spintent/
spintent.dtx ⇒ TDS:source/lualatex/spintent/
spintent.ins  ⇒ TDS:source/lualatex/spintent/
```

1 Descripción

El nombre `spintent` puede entenderse como «*Spanish Parse Intent*» o también como «*School Parse Intent*». Ambas interpretaciones reflejan el objetivo del paquete y el público para el que fue diseñado: docentes de educación primaria y secundaria.

Está pensado para crear «*apuntes*», «*hojas de ejercicios*», «*clases escritas*» enfocadas en el estudiante. Solo un docente sabe cual es el nivel de aprendizaje del estudiante. El paquete intenta sobrescribir muchas de las reglas dictadas por la heurística de NVDA/MathCAT teniendo esto presente.

- El paquete `spintent` usa y abusa del atributo `:intent` de MathML por medio del comando `\MathMLintent` definido en `latex-lab-mathintent`[30], un encapsulado de `\luamml_attribute:een` proveída por `luamml`[2]. ¡Gracias Marcel!

1.1 Carga del paquete

El paquete `spintent` trabaja solo con Lua $\TeX$ y *tagged* PDF activo usando `tagging=on` dentro del comando `\DocumentMetadata` al inicio del archivo. Las llaves (*keys*) aceptadas por `\DocumentMetadata` y en general para el código de *tagged* PDF están documentadas en los paquetes y `tagpdf`[29] `documentmetadata-support`[30], es deber del usuario revisar la documentación asociada a ellos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage{spintent}
```

1.2 El paquete babel

El paquete `babel-spanish`[10] está marcado como «parcialmente compatible» para *tagged* PDF, pero, es absolutamente necesario para la creación de documentos en español. En caso de encontrar problemas de compatibilidad con otros paquetes se puede desactivar usando:

```
\usepackage[spanish,shorthands=off]{babel}
```

Muchas de las «shorthands» proveídas por `babel-spanish` hoy no son necesarias, más allá de `\sptext` para las abreviaturas y ordinales el resto son solo atajos de teclas para escritura que hoy en día no se ocupan. El paquete `spintent` provee el comando `\spshort` (§3.4) que cubre estos casos, de esta manera podemos cargar `babel`[9] con todas sus funcionalidades sin problemas.

¹A *Work in Progress* es un álbum en vivo de Neil Peart (* 1952–† 2020), baterista y principal letrista de Rush. El nombre no pretende formar parte de la terminología de la LPPL; simplemente describe el estado permanente de este paquete mientras sigo aprendiendo sobre *accesibilidad* en PDF.

1.3 Fuentes tipográficas

Como `spintent` es un paquete Lua $\TeX$ debemos distinguir entre las fuentes tipográficas OpenType/TrueType versus las fuentes de 8bit estándar usadas por $\TeX$ para el *modo texto* y el *modo matemático*. Para evitar problemas con *tagged* PDF preferiremos no combinar fuentes OpenType/TrueType con fuentes 8bit y trabajaremos solo con fuentes OpenType/TrueType que tengan soporte para matemáticas.

La carga de las fuentes tipográficas para Lua $\TeX$ se maneja por medio del paquete `fontspec`[3], si no se especifica ninguna se usarán las fuentes Latin Modern para el *modo texto* y Latin Modern Math para el *modo matemático*.

El tipo de fuente tipográfica escogida es preferencia de cada usuario, en lo personal utilizo las fuente Libertinus para el *modo texto* por medio del paquete `libertinus-otf`[4] y la fuente Erewhon-Math para el modo matemático cargada por el paquete `fourier-otf`[5]. Adicionalmente cargo la fuente Source Code Pro para el texto tipo «máquina de escribir» por medio del paquete `sourcecodepro`[6].

```
% \DocumentMetadata{...}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\usepackage{spintent}
```

1.4 El paquete hyperref

El paquete `hyperref`[23] es «casi obligatorio» cuando se crean documentos *tagged* PDF, se encarga de las referencias cruzadas `\label` y `\ref`, las notas al pie `\footnote`, los hipervínculos `\href` y muchos otros elementos como marcadores, índices y metadatos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\setmathfont{Erewhon-Math}[range={cal,bfcal},StylisticSet=1]
\usepackage{spintent, hyperref}
\hypersetup
{
  colorlinks      = true,
  pdfstartview    = FitH,
  pdfdisplaydoctitle = true,
  pdftitle        = {Test para el paquete spintent},
  pdfinfo         =
  {
    Title   = {Test},
    Subject = {spintent},
  },
}
```

2 El comando \setspintent

`\setspintent` `\setspintent{comando}{key-uno = valor, key-dos = valor, key-tres = valor, ...}`

El comando `\setspintent` establece las llaves (*keys*) de forma global para los comandos provistos por `spintent`. Puede utilizarse tanto en el preámbulo como en el cuerpo del documento tantas veces como se desee. Las llaves (*keys*) definidas en el *argumento opcional* [*key = val*] de los comandos tienen prioridad sobre las establecidas por este, anulando las configuraciones globales de `\setspintent`.

Muchas de las utilidades que provee el paquete `spintent` implementan el sistema [*key = val*] y están pensadas para ser utilizadas en la generación de «*hojas de ejercicios*» o «*clases escritas*». Se recomienda usar `\setspintent` y establecerlas de manera global para cada documento, para mantener la coherencia en la verbalización de NVDA/MathCAT a lo largo del mismo.

El sistema [*key = val*] utilizado por `spintent` se implementa usando el módulo `l3keys` de `expl3`[24]. Por diseño, las llaves que reciben un *valor* después del signo '=' NO aceptan *valores vacíos* y es obligatorio pasar uno válido; las llaves documentadas como (*valor prohibido*) no reciben *ningun valor*. En caso de omitir un *valor* válido o pasar un *valor vacío* a una llave que acepte *valor* o pasar un *valor* a una llave del tipo (*valor prohibido*) se devolverá un "error" y se detendrá la compilación del documento.

3 The big four

El paquete `spintent` provee cuatro grandes comandos («*the big four*»): `\spnum` para el manejo de números, `\spunit` para el manejo de unidades, `\spmoney` para el manejo de dinero y `\spshort` para el manejo de abreviaturas, acrónimos y números ordinales.

- También se incluyen las utilidades `\spnewunit` y `\spunitalias` asociadas a al comando `\spunit`.

3.1 El comando \spnum

`\spnum` `\spnum{<número>}` `\spnum{<número,número>}` `\spnum{<número.número>}` `\spnum{<número;número>}`
`\spnum[key = val]{<número,número;número>}` `\spnum[key = val]{<número,número;número e exponente>}`
`\spnum[key = val]{<número.número;número>}` `\spnum[key = val]{<número.número;número e exponente>}`

El comando `\spnum` trabaja en *modo matemático* y recibe el *argumento obligatorio* {<número>}, donde la «*parte entera*» puede iniciar con los signos '+' o '-', la «*parte decimal finita*» **debe** iniciar con una coma ',' o con un punto '.', y la «*parte decimal infinita (periódica)*» **debe** iniciar con un punto y coma '; '.

Si no se proporciona el *argumento opcional* [*key = val*] y se ejecuta `\$ \spnum{+23} \$` se imprimirá '+23' en la salida PDF y NVDA/MathCAT verbalizará «*más veintitrés*», si se ejecuta `\$ \spnum{3,14} \$` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce*», si se ejecuta `\$ \spnum{3,1;4} \$` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma uno con cuatro periódico*».

Además, el *argumento obligatorio* {<número>} admite al final de este un sufijo de «*notación científica*». Este se indica mediante las letras 'e' o 'E', acompañadas de los signos opcionales '+' o '-' seguido por los dígitos del exponente. Por ejemplo `\$ \spnum{3,14e3} \$` imprimirá '3,14 · 10³' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

- La coma ',' o el punto '.' son *separadores decimales* y NO de millares, por lo tanto solo pueden aparecer una «única vez» dentro del {<argumento>}. El {<argumento>} puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo. Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.1.1 Llaves para \spnum

`cifras-sep = {<true | false>}` default: true

Establece el *espaciado* utilizado para agrupar los dígitos en bloques de tres, tanto en la parte entera como en la parte decimal finita. Si se establece en `true` se siguen las normas de la RAE y se utiliza un *espacio fino* '\,' entre las cifras; si se establece en `false`, se respetan los espacios matemáticos introducidos en el argumento. Por ejemplo `\$ \spnum{100000} \$` imprimirá '100 000' en la salida PDF y `\$ \spnum[cifras-sep=false]{10\,00\,00} \$` imprimirá '10 00 00' en la salida PDF.

`read-sign = {<terse | clear>}` default: terse

Establece la *forma* en la que NVDA/MathCAT verbalizará los signos '+' o '-'. Si se establece en `terse`, NVDA/MathCAT verbalizará «*más*» o «*menos*» **antes** de leer el número; si se establece en `clear`, NVDA/MathCAT verbalizará «*positivo*» o «*negativo*» **después** de leer el número. Por ejemplo `\$ \spnum{-23} \$` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*menos veintitrés*» y `\$ \spnum[read-sign=clear]{-23} \$` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés negativo*».

`read-period-sep = {<coma | punto>}` default: coma

Establece la *forma* en la que NVDA/MathCAT verbalizará e imprimirá el signo de separación ';' de la «*parte decimal infinita (periódica)*» cuando no se tiene la «*parte decimal finita*». Si se establece en `coma`, se imprimirá ',' en la salida PDF y NVDA/MathCAT verbalizará «*coma*»; si se establece en `punto`, se imprimirá '.' en la

²Los «*cuatro grandes*» del Thrash Metal: Metallica, Megadeth, Slayer y Anthrax, que me acompañan mientras desarrollo `spintent`.

salida PDF y NVDA/MathCAT verbalizará «*punto*». Por ejemplo `\spnum{3;4}` imprimirá ‘3,4’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma cuatro periódico*» y `\spnum[read-period-sep=punto]{3;4}` imprimirá ‘3.4’ en la salida PDF y NVDA/MathCAT verbalizará «*tres punto cuatro periódico*»

- Esta llave es necesaria SOLO cuando se utilizan «*decimales periódicos puros*», es decir, cuando no se ha incluido en el `{(argumento)}` una coma ‘,’ o un punto ‘.’ antes del signo de separación ‘;’ periódica.

`notation-sym = {(times | cdot)}`

default: `cdot`

Establece el *símbolo* que se utilizará para imprimir la «*notación científica*» al usar ‘e’ o ‘E’. Si se establece en `times` imprimirá ‘×’ en la salida PDF y NVDA/MathCAT verbalizará «*por*»; si se establece en `cdot` imprimirá ‘·’ en la salida PDF y NVDA/MathCAT verbalizará «*por*». Por ejemplo `\spnum{3,14e3}` imprimirá ‘3,14 · 10³’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*» y `\spnum[notation-sym=times]{3,14e3}` imprimirá ‘3,14 × 10³’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

- Decir «*notación científica*» aquí es un poco exagerado, en realidad es «*multiplicación por potencias de 10*», no se verifica si la entrada cumple o no con la condición de estar escrita en «*notación científica*».

3.2 El comando \spunit

<code>\spunit{(unidad)}</code>	<code>\spunit[(key = val)]{(unidad)}</code>	<code>\spunit[(key = val)]{(número unidad)}</code>
<code>\spunit[(key = val)]{(número unidad*unidad)}</code>	<code>\spunit[(key = val)]{(número unidad*unidad/unidad)}</code>	
<code>\spunit[(key = val)]{(número unidad*unidad/unidad*...)}</code>	<code>\spunit[(key = val)]{(número e exponente unidad ...)}</code>	

El comando `\spunit` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{(unidad)}`, el cual puede contener una «*parte numérica*» opcional la cual se procesa internamente por el comando `\spnum` al inicio del `{(argumento)}` seguido de una `{(unidad)}` o un `{(alias)}` valido descritos en §3.2.2 o una `{(unidad)}` creada por el comando `\spnewunit` (§3.2.3) o un `{(alias)}` creado por el comando `\spunitalias` (§3.2.4).

Si no se proporciona el *argumento opcional* `[(key = val)]`, `\spunit{23 m}` imprimirá ‘23 m’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros*».

La multiplicación entre las `{(unidades)}` se realiza mediante un *asterisco* ‘*’ y las potencias de estas se indican con un *circunflejo seguido del exponente entre llaves* ‘^`{(exponente)}`’. Por ejemplo si se ejecuta `\spunit{23 m*s^{2}}` se imprimirá ‘23 m s²’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros segundos al cuadrado*».

Se pueden usar *fracciones lineales* con las `{(unidades)}` utilizando una «*única barra*» ‘/’ para la separación entre las `{(unidades)}` del numerador y el denominador, teniendo presente que el denominador NO puede contener números de ningún tipo. Por ejemplo: `\spunit{23 m/s^{2}}` imprimirá ‘23 m/s²’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros por segundos al cuadrado*».

- Solo imprimiremos *fracciones lineales*, si se requiere otra forma fraccionaria para explicaciones y otros usos se debe usar el comando por separado. El `{(argumento)}` puede contener espacios matemáticos validos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo, NO puede contener más de una «*única barra*» ‘/’ y NO puede contener números en el denominador. Cualquier entrada fuera de esta sintaxis devolverá un “error” deteniendo la compilación del documento.

3.2.1 Llaves para \spunit

El comando `\spunit` posee las *(meta-keys)* `cifras-sep`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1).

`print-cdot = {(true | false)}`

default: `false`

Establece si se *imprime \cdot* ‘·’ o un *espacio fino* ‘\,’ para la multiplicación de `{(unidades)}`. Si se establece en `true`, se imprimirá ‘·’ entre las `{(unidades)}` en la salida PDF; si se establece en `false` se imprimirá un *espacio fino* ‘\,’ entre las `{(unidades)}` en la salida PDF.

`read-cdot = {(true | false)}`

default: `false`

Establece la *forma* en la que NVDA/MathCAT verbalizará la multiplicación de `{(unidades)}`. Si se establece en `true`, NVDA/MathCAT verbalizará «*por*» entre las `{(unidades)}`; si se establece en `false`, NVDA/MathCAT no verbalizará nada entre las `{(unidades)}`.

`read-slash = {(por | partido-por)}`

default: `por`

Establece la *forma* en la que NVDA/MathCAT verbalizará la «*barra*» ‘/’ de separación entre el numerador y el denominador. Si se establece en `por`, NVDA/MathCAT verbalizará «*por*»; si se establece en `partido-por`, NVDA/MathCAT verbalizará «*partido por*».

3.2.2 Argumentos para \spunit

Resumen de `{(unidades)}` y `{(alias)}` soportados por `\spunit`.

Entrada	Alias soportados	Nombre unidad	Salida	NVDA/MathCAT
m	metro, metros	Metro	m	«metro»
g	gramo, gramos	Gramo	g	«gramo»
s	segundo, segundos	Segundo (Tiempo)	s	«segundo (Tiempo)»
l	litro, l-litro	Litro (minúscula)	l	«litro (minúscula)»
h	hora, horas	Hora	h	«hora»
d	dia, dias	Día	d	«día»
A	amperio, ampere	Amperio	A	«amperio»
V	voltio, volt	Voltio	V	«voltio»
W	vatio, watt	Vatio	W	«vatio»
J	julio, joule	Julio	J	«julio»
N	newton	Newton	N	«newton»
Pa	pascal	Pascal	Pa	«pascal»
Hz	hercio, hertz	Hercio	Hz	«hercio»
Ω	ohmio, ohm, Omega	Ohmio	Ω	«Ohmio»
F	faradio	Faradio	F	«Faradio»
C	culombio	Culombio	C	«Culombio»
K	kelvin, Kelvin	Kelvin	K	«Kelvin»
°C	celsius, degC, °C	Grados Celsius	°C	«Grados Celsius»
°F	fahrenheit, degF, °F	Grados Fahrenheit	°F	«Grados Fahrenheit»
cal	caloria, calorías	Caloría	cal	«Caloría»
b	bit	Bit	b	«Bit»
B	byte, bytes	Byte	B	«Byte»
in	pulgada, pulgadas	Pulgada	in	«Pulgada»
ft	pie, pies	Pie	ft	«Pie»
yd	yarda, yardas	Yarda	yd	«Yarda»
mi	milla, millas	Milla	mi	«Milla»
lb	libra, libras	Libra	lb	«Libra»
oz	onza, onzas	Onza	oz	«Onza»
gal	galon, galones	Galón	gal	«Galón»
Å	angstrom	Angstrom	Å	«Angstrom»
μ	micro	Micro (Prefijo)	μ	«Micro (Prefijo)»
Ω	mho	Mho / Siemens	Ω	«Mho / Siemens»
'	minmin	Minuto (Arco)	'	«Minuto (Arco)»
"	segseg	Segundo (Arco)	"	«Segundo (Arco)»
u	u-masa	Unidad masa atómica	u	«Unidad masa atómica»
Unidades directas (sin alias):				
L	-	Litro (Mayúscula)	L	«Litro (Mayúscula)»
ℓ	-	Litro (Cursiva)	ℓ	«Litro (Cursiva)»
w	-	Vatio (Minúscula)	w	«Vatio (Minúscula)»
°K	-	Grado Kelvin	°K	«Grado Kelvin»
Bq	-	Becquerel	Bq	«Becquerel»
Da	-	Dalton	Da	«Dalton»
Gy	-	Gray	Gy	«Gray»
S	-	Siemens	S	«Siemens»
Sv	-	Sievert	Sv	«Sievert»
T	-	Tesla	T	«Tesla»
Wb	-	Weber	Wb	«Weber»
atm	-	Atmósfera	atm	«Atmósfera»
au	-	Unidad astronómica	au	«Unidad astronómica»
bar	-	Bar	bar	«Bar»
cd	-	Candela	cd	«Candela»
dB	-	Decibelio	dB	«Decibelio»
dyn	-	Dina	dyn	«Dina»
eV	-	Electronvoltio	eV	«Electronvoltio»
erg	-	Ergio	erg	«Ergio»
ha	-	Hectárea	ha	«Hectárea»
kat	-	Katal	kat	«Katal»
kg	-	Kilogramo	kg	«Kilogramo»
km	-	Kilómetro	km	«Kilómetro»
lm	-	Lumen	lm	«Lumen»

Continúa en la siguiente página...

Continuación de la tabla 1

Entrada	Alias soportados	Nombre unidad	Salida	MathCAT
lx	-	Lux	lx	« <i>Lux</i> »
cm	-	Centímetro	cm	« <i>Centímetro</i> »
min	-	Minuto (Tiempo)	min	« <i>Minuto (Tiempo)</i> »
mm	-	Milímetro	mm	« <i>Milímetro</i> »
mol	-	Mol	mol	« <i>Mol</i> »
pc	-	Pársec	pc	« <i>Pársec</i> »
rad	-	Radián	rad	« <i>Radián</i> »
rpm	-	Rev. por minuto	rpm	« <i>Rev. por minuto</i> »
sr	-	Estereorradián	sr	« <i>Estereorradián</i> »
t	-	Tonelada	t	« <i>Tonelada</i> »
a	-	Año / Área (are)	a	« <i>Año / Área (are)</i> »
y	-	Año (Year)	y	« <i>Año (Year)</i> »
°	-	Grado (Arco)	°	« <i>Grado (Arco)</i> »

3.2.3 El comando \spnewunit

```
\spnewunit \spnewunit{\nueva unidad}\{verbalización NVDA}
```

El comando `\spnewunit` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: $\{\langle \text{nueva unidad} \rangle\}$ que establece lo que se imprimirá en la salida PDF y $\{\langle \text{verbalización NVDA} \rangle\}$ que establece como NVDA/MathCAT verbalizará la $\{\langle \text{nueva unidad} \rangle\}$.

La `{\nueva unidad}` no debe estar registrada como `{\unidad}` o `{\alias}` para `\spunit` descritos en §3.2.2. Si la `{\nueva unidad}` ya está registrada se devolverá un “error” y se detendrá la compilación del documento.

La declaración de la $\{\langle\textit{nueva unidad}\rangle\}$ es «global» y se almacena en el módulo `spintent.lua` como una $\{\langle\textit{unidad}\rangle\}$ valida para pasar como $\{\langle\textit{argumento}\rangle\}$ a `\spunit`. Por ejemplo: `\spnewunit{px}{píxeles}` creará la $\{\langle\textit{nueva unidad}\rangle\}$ ‘px’ y cuando se utilice `\$ \spunit{100 px} \$` se imprimirá ‘100 px’ en la salida PDF y NVDA/MathCAT verbalizará «cien píxeles».

3.2.4 El comando \spunitalias

`\spunitalias \spunitalias{⟨nuevo alias⟩}{⟨expresión de unidades⟩}`

El comando `\spunitalias` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: $\{\langle \text{nuevo alias} \rangle\}$ que establece es el nuevo $\{\langle \text{alias} \rangle\}$ de $\{\langle \text{argumento} \rangle\}$ para `\spunit` y $\{\langle \text{expresión de unidades} \rangle\}$ que puede contener multiplicación o división de $\{\langle \text{unidades} \rangle\}$ ya registradas que se imprimirán en la salida PDF.

El `{\nuevo alias}` no debe estar registrado como `{\unidad}` o `{\alias}` para `\spunit` descritos en §3.2.2 o estar definido como `{\unidad}` por medio del comando `\spnewunit`. Si el `{\nuevo alias}` ya está registrado se devolverá un “error” y se detendrá la compilación del documento.

La declaración de un `{\nuevo alias}` es «global» y se almacena en el módulo `spintent`. Lua como una `{\alias}` valido para pasar como `{\argumento}` a `\spunit`. Por ejemplo: `\spunitalias{caudal}{m^3}/s` creará el `{\alias}` ‘caudal’ y cuando se utilice `\spunit{50 caudal}` se imprimirá ‘50 m³/s’ en la salida PDF y NVDA/MathCAT verbalizará «cincuenta metros al cubo por segundos».

3.3 El comando \spmoney

<u>\spmoney</u>	<code>\spmoney{<i>(cifra)</i>}</code>	<code>\spmoney[<i>(key = val)</i>]{<i>(cifra)</i>}</code>
	<code>\spmoney{<i>(cifra símbolo moneda)</i>}</code>	<code>\spmoney[<i>(key = val)</i>]{<i>(cifra símbolo moneda)</i>}</code>
	<code>\spmoney{<i>(cifra alias moneda)</i>}</code>	<code>\spmoney[<i>(key = val)</i>]{<i>(cifra alias moneda)</i>}</code>

El comando `\spmoney` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{ \langle \text{cifra} \rangle \}$ que contiene la «cifra numérica» positiva o negativa la cual se procesa internamente por el comando `\spnum` acompañada de «símbolo moneda» o «alias moneda» opcionales descritos en §3.3.2.

Si no se proporciona el *argumento opcional* [`<key = val>`], `\spmoney{500}` imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos».

El comportamiento de `\spmoney` depende de como se pase «*símbolo moneda*» o «*alias moneda*» en el argumento *obligatorio* donde tienen prioridad por sobre los valores por defecto. Por ejemplo: `$\spmoney{500 dolares}$` imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «*quinientos dólares*» aunque la moneda por defecto sean pesos.

Si se pasa un «*alias moneda*» del tipo ISO (código de divisas) como CLP o USD (mayúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país y se ajusta la salida PDF al estándar ISO. Por ejemplo: `$\spmoney{500 CLP}` imprimirá 'CLP500' en la salida PDF y NVDA/MathCAT verbalizará «*quinientos pesos chilenos*».

Si se pasa un «*alias moneda*» del tipo ISO (código de divisas) como `clp` o `usd` (minúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país, pero, NO se modifica la salida PDF imprimiendo el «*símbolo de moneda*» correspondiente. Por ejemplo: `\spmoney{500 clp}` imprimirá '\$500' en la salida PDF y NVDA/MathCAT verbalizará «*quinientos pesos chilenos*»

Pasar de manera directa en el *argumento obligatorio* un «*símbolo moneda*» como '\$', '€', '£' o un «*alias moneda*» como peso, euro o libra produce la salida PDF y verbalización estándar esperada para NVDA/MathCAT. Los «*alias moneda*» se pueden pasar con o sin tilde, por ejemplo dolares o dólares.

- Los «*alias moneda*» se deben escribir en minúsculas y los códigos ISO en minúsculas o todo en mayúsculas. Cualquier otra escritura, por ejemplo PESO o Usd, devolverá un “error” y se detendrá la compilación del documento.
- La posición del «*símbolo moneda*» a la izquierda o a la derecha de la «*cifra numérica*» y la verbalización para NVDA/MathCAT se manejan desde el módulo Lua `spintent.lua` y no se proveen llaves para modificar este comportamiento.
- El «*símbolo moneda*» es tomado de la fuente tipográfica definida para el *modo matemático* al momento de ejecutar el comando (o del *modo texto* si se establece `money-math=false`), se debe tener presente en caso de querer usar un «*símbolo moneda*» poco habitual que esté soportada por `spintent`. El `{(argumento)}` puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo, con la única excepción de `\$`. La coma ‘,’ o el punto ‘.’ como *separador decimal* solo pueden aparecer una «única vez». Cualquier entrada fuera de esta sintaxis devolverá un “error” deteniendo la compilación del documento.

3.3.1 Llaves para \spmoney

El comando `\spmoney` posee la «*meta-key*» `cifras-sep` pasada al comando `\spnum` (§3.1.1).

`currency = {(moneda | alias)}` default: peso

Establece el *símbolo moneda* por defecto que se imprimirá en la salida PDF y cómo lo verbalizará NVDA/MathCAT cuando SOLO se pasa una «*cifra numérica*» como `{(argumento)}` al comando `\spmoney`. El «*símbolo moneda*» o el «*alias moneda*» pasado a esta llave debe ser alguno de los descritos en §3.3.2.

`sub-units = {(true | false)}` default: false

Establece si se usarán *subunidades* de la moneda en curso para la verbalización en NVDA/MathCAT. La moneda en curso debe aceptar *subunidades*. Por ejemplo: `\spmoney[sub-units=true]{100,5 dolares}` imprimirá '\$100,5' en la salida PDF y NVDA/MathCAT verbalizará «*cient dólares con cincuenta centavos*».

- Si la «*cifra numérica*» no tiene parte decimal, la llave `sub-units` no tiene efecto. Si la moneda en curso no admite *subunidades* se emite una advertencia y la llave tampoco tiene efecto.

`money-math = {(true | false)}` default: true

Establece si el *símbolo moneda* se imprimirá en *modo texto* o en *modo matemático*. Si se establece en `true`, se usará la fuente tipográfica definida para el *modo matemático* en curso para imprimir el *símbolo moneda* en la salida PDF; si se establece en `false`, se usará la fuente tipográfica definida para el *modo texto* para imprimir el *símbolo moneda* en la salida PDF.

3.3.2 Monedas y alias para \spmoney

Resumen de los «*símbolos moneda*» y «*alias moneda*» soportados en el `{(argumento)}` de `\spmoney`.

Entrada	Alias	Nombre	Salida	NVDA/MathCAT
\$	peso, pesos	peso	\$	« <i>peso / pesos</i> »
dolar	dolares, dólar, dólares	dólar	\$	« <i>dólar / dólares</i> »
€	euro, euros	euro	€	« <i>euro / euros</i> »
£	libra, libras	libra	£	« <i>libra / libras</i> »
¥	yen, yenes	yen	¥	« <i>yen / yenes</i> »
yuan	yuanes	yuan	¥	« <i>yuan / yuanes</i> »
₩	won, wons	won	₩	« <i>won / wons</i> »
₪	shekel, shekels, sequel, sequeles	séquel	₪	« <i>séquel / séqueles</i> »
₹	rupia, rupias	rupia	₹	« <i>rupia / rupias</i> »
₰	rublo, rublos	rublo	₰	« <i>rublo / rublos</i> »
₺	lira, liras	lira	₺	« <i>lira / liras</i> »
₴	grivna, grivnas	grivna	₴	« <i>grivna / grivnas</i> »
¢	centavo, centavos	centavo	¢	« <i>centavo / centavos</i> »
Divisas en formato ISO (Mayúsculas)				
CLP	—	peso chileno	CLP	« <i>peso chileno / pesos chilenos</i> »
MXN	—	peso mexicano	MXN	« <i>peso mexicano / pesos mexicanos</i> »
USD	—	dólar	USD	« <i>dólar estadounidense / dólares estadounidenses</i> »

Continúa en la siguiente página...

Cuadro 2 – Continuación de la página anterior

Entrada	Alias	Nombre	Salida	NVDA/MathCAT
EUR	—	euro	EUR	«euro / euros»
GBP	—	libra	GBP	«libra / libras»
JPY	—	yen	JPY	«yen / yenes»
CNY	—	yuan	CNY	«yuan / yuanes»
KRW	—	won	KRW	«won / won's»
ILS	—	séquel	ILS	«séquel / séqueles»
INR	—	rupia	INR	«rupia / rupias»
RUB	—	rublo	RUB	«rublo / rublos»
TRY	—	lira	TRY	«lira / liras»
UAH	—	grivna	UAH	«grivna / grivnas»

Códigos ISO en minúsculas

clp	—	peso chileno	\$	«peso chileno / pesos chilenos»
mxn	—	peso mexicano	\$	«peso mexicano / pesos mexicanos»
usd	—	dólar	\$	«dólar estadounidense / dólares estadounidenses»
eur	—	euro	€	«euro / euros»
gbp	—	libra	£	«libra / libras»
jpy	—	yen	¥	«yen / yenes»
cny	—	yuan	¥	«yuan / yuanes»
krw	—	won	₩	«won / won's»
ils	—	séquel	₪	«séquel / séqueles»
inr	—	rupia	₹	«rupia / rupias»
rub	—	rublo	₽	«rublo / rublos»
try	—	lira	₺	«lira / liras»
uah	—	grivna	₴	«grivna / grivnas»

3.4 El comando \spshort

```
\spshort{<abreviaturas | acrónimos | números ordinales>}
\spshort[<key = val>]{<abreviaturas | acrónimos | números ordinales>}
\spshort[<read-arg={<verbalización NVDA>}]{<comandos LATEX>}
```

El comando `\spshort` trabaja solo en modo texto y recibe un argumento obligatorio del tipo `{<abreviaturas>}`, `{<acrónimos>}` o `{<números ordinales>}` descritos en §3.4.2.

Si no se proporciona el argumento opcional `[<key = val>]` y se ejecuta `\spshort{dr.a}` se imprimirá ‘Dr.^a’ en la salida PDF y NVDA/MathCAT verbalizará «doctora»; si se ejecuta `\spshort{onu}` se imprimirá ‘ONU’ en la salida PDF y NVDA/MathCAT verbalizará «organización de las naciones unidas» y si se ejecuta `\spshort{mineduc}` se imprimirá ‘Mineduc’ en la salida PDF y NVDA/MathCAT verbalizará «ministerio de educación».

Para la escritura de `{<números ordinales>}` se puede ejecutar `\spshort{1.a}` que imprimirá ‘1.^a’ en la salida PDF y NVDA/MathCAT verbalizará «primera»; si se ejecuta `\spshort{1.o}` imprimirá ‘1.^o’ en la salida PDF y NVDA/MathCAT verbalizará «primero».

Para crear un nueva `{<abreviatura>}`, `{<acrónimo>}` o `{<número ordinal>}` que no se encuentre registrada por spintent dentro del módulo Lua spintent.lua descritas en §3.4.2 se debe utilizar la llave `read-arg` y definir esta dentro del argumento obligatorio que puede contener `{<comandos LATEX>}` en modo texto los cuales se imprimirán en salida PDF.

- Para la escritura de `{<números ordinales>}` se debe tener cuidado y no confundir la «letra o voladita» ‘o’ con el símbolo de grado ‘°’. Si la fuente tipográfica del modo texto está utilizando `Numbers=OldStyle` los números pueden quedar muy separados de la «letra voladita», la solución para esto es agregar `\addfontfeatures{Numbers=Lining}` dentro de un grupo junto al comando: `{\addfontfeatures{Numbers=Lining}\spshort{1.o}}`.
- El comando `\spshort` está pensado como un reemplazo para `shorthands` y el comando `\sptext` provistos por el paquete `babel` en español compatible con `tagged` PDF. Es de las pocas utilidades que provee el paquete `spintent` que trabaja solo en modo texto. Internamente utiliza la llave `E` (expansion text) proveída por el paquete `tagpdf`[29].

3.4.1 Llaves para \spshort

```
read-arg = {<verbalización NVDA>} default: no usado

Establece la verbalización del argumento {<comandos LATEX>} de la nueva {<abreviatura>}, {<acrónimo>}
o {<número ordinal>} para NVDA. El valor de esta llave siempre debe pasarse entre ‘{ }’. Por ejemplo:
\spshort[read-arg={octavo}]{8.\textsuperscript{\emph{avo}}} creará una nueva {<abreviatura>}
que imprimirá ‘8.avo’ en la salida PDF y NVDA verbalizará «octavo».
```

```
small-caps = {<true | false>} default: false
```

Establece si se usarán versalitas para imprimir `{<acrónimos>}` y para la «letra volada» en `{<abreviaturas>}` y `{<números ordinales>}` respetando las reglas de la RAE para el uso de estas. Si se establece en `true`, las abreviaturas

con mayúscula como EE. UU. y los acrónimos de cinco o más letras como Unicef no se verán afectados; si se establece en `false`, no se aplicarán *versalitas*. Por ejemplo: `\spshort[small-caps=true]{dr.a}` imprimirá ‘Dr.^A’ en la salida PDF y NVDA verbalizará «*doctora*».

3.4.2 Argumentos para `\spshort`

El cuadro 3 resume las abreviaturas, expresiones latinas y títulos registrados en el módulo Lua `spintent.lua`.

Cuadro 3: Abreviaturas soportadas

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Abreviaturas con letras voladas			
dr.a	dr. ^a	Dr. ^a	« <i>doctora</i> »
dr.as	—	Dr. ^{as}	« <i>doctoras</i> »
prof.a	—	Prof. ^a	« <i>profesora</i> »
d.a	d. ^a	D. ^a	« <i>doña</i> »
m.a	—	M. ^a	« <i>maría</i> »
n.o	n. ^o	N. ^o	« <i>número</i> »
n.os	—	N. ^{os}	« <i>números</i> »
c.ia	—	C. ^{ia}	« <i>compañía</i> »
Abreviaturas comunes y expresiones latinas			
pág.	pag.	pág.	« <i>página</i> »
vol.	—	vol.	« <i>volumen</i> »
etc.	—	etc.	« <i>etcétera</i> »
et al.	et. al.	et al.	« <i>y otros</i> »
ibíd.	ibid.	ibíd.	« <i>ibídem</i> »
op. cit.	op.cit.	op. cit.	« <i>obra citada</i> »
loc. cit.	loc.cit.	loc. cit.	« <i>lugar citado</i> »
v. gr.	v.gr.	v. gr.	« <i>verbigracia</i> »
i. e.	i.e.	i. e.	« <i>esto es</i> »
e. g.	e.g.	e. g.	« <i>por ejemplo</i> »
p. ej.	p.ej.	p. ej.	« <i>por ejemplo</i> »
Títulos, profesiones y cargos			
sr.	—	Sr.	« <i>señor</i> »
sra.	—	Sra.	« <i>señora</i> »
srta.	—	Srta.	« <i>señorita</i> »
dr.	—	Dr.	« <i>doctor</i> »
dra.	—	Dra.	« <i>doctora</i> »
dres.	—	Dres.	« <i>doctores</i> »
dras.	—	Dras.	« <i>doctoras</i> »
prof.	—	Prof.	« <i>profesor</i> »
profa.	—	Profa.	« <i>profesora</i> »
profs.	—	Profs.	« <i>profesores</i> »
ing.	—	Ing.	« <i>ingeniero</i> »
ings.	—	Ings.	« <i>ingenieros</i> »
lic.	—	Lic.	« <i>licenciado</i> »
v. b.	v.b.	V. B.	« <i>visto bueno</i> »
Abreviaturas compuestas e institucionales			
s. a.	s.a.	S. A.	« <i>sociedad anónima</i> »
ee. uu.	ee.uu.	EE. UU.	« <i>estados unidos</i> »
dd. hh.	dd.hh.	DD. HH.	« <i>derechos humanos</i> »
jj. oo.	jj.oo.	JJ. OO.	« <i>juegos olímpicos</i> »
ff. aa.	ff.aa.	FF. AA.	« <i>fuerzas armadas</i> »
rr. ee.	rr.ee.	RR. EE.	« <i>relaciones exteriores</i> »
cc. aa.	cc.aa.	CC. AA.	« <i>comunidades autónomas</i> »
p. d.	p.d.	P. D.	« <i>posdata</i> »
a. c.	a.c.	a. C.	« <i>antes de Cristo</i> »
d. c.	d.c.	d. C.	« <i>después de Cristo</i> »
d. n. i.	d.n.i.	D. N. I.	« <i>documento nacional de identidad</i> »
r. u. t.	r.u.t.	R. U. T.	« <i>rol único tributario</i> »
r. u. n.	r.u.n.	R. U. N.	« <i>rol único nacional</i> »

El cuadro 4 detalla las siglas y acrónimos registrados en el módulo Lua `spintent.lua`.

Cuadro 4: Siglas y acrónimos soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Siglas (Deletreo o lectura expandida)			
dni	—	DNI	«documento nacional de identidad»
rut	—	RUT	«rol único tributario»
run	—	RUN	«rol único nacional»
onu	—	ONU	«organización de las naciones unidas»
rae	—	RAE	«real academia española»
ong	—	ONG	«organización no gubernamental»
ongs	—	ONGs	«organizaciones no gubernamentales»
urss	—	URSS	«unión de repúblicas socialistas soviéticas»
oea	—	OEA	«organización de los estados americanos»
oms	—	OMS	«organización mundial de la salud»
fmi	—	FMI	«fondo monetario internacional»
bid	—	BID	«banco interamericano de desarrollo»
otan	—	OTAN	«organización del tratado del atlántico norte»
ue	—	UE	«unión europea»
ru	—	RU	«reino unido»
Acrónimos silábicos			
unicef	—	Unicef	«fondo de las naciones unidas para la infancia»
unesco	—	Unesco	«organización de las naciones unidas para la educación, la ciencia y la cultura»
mercosur	—	Mercosur	«mercado común del sur»
cepal	—	Cepal	«comisión económica para américa latina»
mineduc	—	Mineduc	«ministerio de educación»
conicet	—	Conicet	«consejo nacional de investigaciones»

El cuadro 5 muestra ejemplos de números ordinales registrados en el módulo Lua `spintent.lua`.

Cuadro 5: Números ordinales soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Sufijos para ordinales (Ejemplos)			
1.a	1. ^a	1. ^a	«primera»
2.o	2. ^o	2. ^o	«segundo»
3.er	—	3. ^{er}	«tercer»
4.os	—	4. ^{os}	«cuartos»
5.as	—	5. ^{as}	«quintas»

Los números ordinales se verbalizan con palabras desde el 1 hasta el 99. Desde el 100 el ordinal no se convierte en palabras y se entrega a NVDA/MathCAT tal como se imprime, por ejemplo 100. o como «100^o».

4 Utilidades generales

Además de los «cuatro grandes» y sus asociados, el paquete `spintent` provee tres utilidades de uso general: `\spsiglo` para la escritura y lectura de siglos, `\sptime` para la escritura y lectura de tiempo y `\spdate` para la escritura y lectura de fechas.

El comando `\spsiglo`

`\spsiglo` `\spsiglo{<siglo>}` `\spsiglo[<key = val>]{<siglo>}`

El comando `\spsiglo` trabaja en *modo texto* y en *modo matemático*, recibe el argumento obligatorio `{<siglo>}`, el cual puede ser un *número arábigo* ‘21’ o un *número romano* ‘XXI’ mayor que ‘0’ y menor que ‘4000’.

Si no se proporciona el *argumento opcional* `[<key = val>]` y se ejecuta `\spsiglo{21}` o `\spsiglo{XXI}` se imprimirá ‘xxi’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

- El argumento obligatorio $\{\langle siglo \rangle\}$ «*siempre*» es convertido a *número romano en versalitas* y es verbalizado por NVDA/MathCAT como *número arábigo*. Esta es la única utilidad que provee el paquete `spintent` que trabaja en *modo texto* y en *modo matemático*. Internamente utiliza la llave `alt` proveída por el paquete `tagpdf` en *modo texto* y usa `\MathMLintent` para el *modo matemático*.

Llaves para `\spsiglo`

print-era = $\{\langle ac \mid dc \mid none \rangle\}$

default: none

Establece si se *imprime* o no la abreviatura después de imprimir el argumento $\{\langle siglo \rangle\}$. Si se establece en `ac`, `\spsiglo[print-era=ac]{XXI}` imprimirá ‘xxi a. C.’ en la salida PDF y NVDA/MathCAT verbalizará «*veintiuno antes de cristo*»; si se establece en `dc`, `\spsiglo[print-era=dc]{XXI}` imprimirá ‘xxi d. C.’ en la salida PDF y NVDA/MathCAT verbalizará «*veintiuno después de cristo*»; si se establece en `none`, `\spsiglo[print-era=none]{XXI}` imprimirá ‘xxi’ en la salida PDF y NVDA/MathCAT verbalizará «*veintiuno*».

El comando `\sptime`

`\sptime`

`\sptime{\langle horas : minutos \rangle}``\sptime{\langle horas : minutos : segundos \rangle}`

El comando `\sptime` trabaja en *modo matemático* y recibe el argumento obligatorio $\{\langle horas : minutos \rangle\}$ o $\{\langle horas : minutos : segundos \rangle\}$. Por ejemplo: `\sptime{23:45}` imprimirá en «*modo texto*» la salida PDF ‘23:45’ y NVDA/MathCAT verbalizará «*las veintitrés y cuarenta y cinco minutos*».

El comando `\spdate`

`\spdate`

`\spdate{\langle DD/MM/YYYY \rangle}``\spdate{\langle YYYY/MM/DD \rangle}``\spdate{\langle YYYY-MM-DD \rangle}``\spdate{\langle DD-MM-YYYY \rangle}`

El comando `\spdate` trabaja en *modo matemático* y recibe el argumento obligatorio $\{\langle DD/MM/YYYY \rangle\}$, $\{\langle DD-MM-YYYY \rangle\}$ o $\{\langle YYYY-MM-DD \rangle\}$. Por ejemplo: `\spdate{1981-01-02}` y `\spdate{02/01/1981}` imprimirán en «*modo texto*» la salida PDF ‘1981-01-02’ y ‘02/01/1981’ respectivamente y en ambos casos, NVDA/MathCAT verbalizará «*dos de enero de mil novecientos ochenta y uno*».

- Si se pasa el argumento $\{\langle YYYY/MM/DD \rangle\}$, internamente se convertirá en $\{\langle DD/MM/YYYY \rangle\}$ y se imprimirá de esa forma por compatibilidad con NVDA/MathCAT.

5 Utilidades para símbolos

El paquete `spintent` provee cuatro comandos `\spnewread`, `\spread`, `\spdsym` y `\spmsym` para trabajar con «*símbolos matemáticos*» descritos en esta sección. Están pensando desde el punto de vista docente, para situaciones donde la lectura de un «*símbolo*» (acorde al contexto) es relevante.

- A diferencia de las otras utilidades implementados por `spintent` (que analizan y dan formato a la salida), estos no procesan los «*símbolos matemáticos*», solo afectan la *forma* (semántica) en que serán verbalizados por NVDA/MathCAT.

El comando `\spnewread`

`\spnewread`

`\spnewread{\langle símbolo \rangle}{\langle verbalización NVDA \rangle}`

El comando `\spnewread` trabaja en *modo texto* y recibe dos argumentos obligatorios: $\{\langle símbolo \rangle\}$ y $\{\langle verbalización NVDA \rangle\}$ los cuales se almacenan de forma interna para ser utilizados como $\{\langle argumento \rangle\}$ en `\spread`. Solo se establece como NVDA/MathCAT verbalizará el $\{\langle símbolo \rangle\}$ al utilizarlo con `\spread` sin modificar la impresión de este.

El argumento $\{\langle símbolo \rangle\}$ no debe estar registrado. Si ya está registrado se devolverá un “error” y se detendrá la compilación del documento. La declaración es «global» y se almacena en el módulo `spintent.lua`. Por ejemplo: `\spnewread{\cong}{es-congruente-con}` registrará ‘`\cong`’ y cuando se utilice `\spread{\cong}` se imprimirá ‘≅’ en la salida PDF y NVDA/MathCAT verbalizará «*es congruente con*».

- El $\{\langle símbolo \rangle\}$ debe ser un único comando, pero puede ser uno definido por el usuario. Por ejemplo, con `\newcommand{\xcuad}{x^2}`, `\spnewread{\xcuad}{x-al-cuadrado}` registrará ‘`\xcuad`’ y cuando se utilice `\spread{\xcuad}` se imprimirá ‘ x^2 ’ en la salida PDF y NVDA/MathCAT verbalizará «*x al cuadrado*».
- Los espacios y los guiones bajos de $\{\langle verbalización NVDA \rangle\}$ se convierten en guiones ‘-’, por lo que {es congruente con} equivale a {es-congruente-con}. Los guiones repetidos se reducen a uno y se eliminan los del inicio y del final. Si el resultado queda vacío se devolverá un “error” y se detendrá la compilación del documento.

Llaves para \spdsym

prefix = { *prefijo* } default: no usado

Establece el *prefijo* que será *verbalizado* por NVDA/MathCAT «antes» de *verbalizar* el $\{\langle \textit{valor} \rangle\}$ establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

suffix = {⟨sufijo⟩} default: no usado

Establece el *sufijo* que será *verbalizado* por NVDA/MathCAT «después» de *verbalizar* el $\{\{\textit{valor}\}\}$ establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

$$\text{set-sym} = \{\langle \textit{old-style} \mid \textit{two-dots} \mid \textit{slash} \rangle\} \quad \text{default: } \textit{two-dots}$$

Establece el *símbolo de división* que se imprimirá en la salida PDF. Si se establece en `old-style`, imprimirá ‘÷’; si se establece en `two-dots`, imprimirá ‘:’ y si se establece `slash`, imprimirá ‘/’.

read-sym = { *dividido* | *dividido-por* | *dividido-entre* } default: *dividido*

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de división*. Si se establece en **dividido**, NVDA/MathCAT verbalizará «*dividido*»; si se establece en **dividido-por**, NVDA/MathCAT verbalizará «*dividido por*» y si se establece en **dividido-entre**, NVDA/MathCAT verbalizará «*dividido entre*».

El comando \spmsym

`\spmsym` `\spmsym` `\spmsym[⟨key = val⟩]`

El comando `\spmsym` trabaja en *modo matemático* y recibe el *argumento opcional* [`(key = val)`] mediante el cual se establece la «*forma*» en la que se imprimirá el *símbolo de multiplicación* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* [`key = val`], `\$spsym$` imprimirá ‘.’ en la salida PDF y NV-DA/MathCAT verbalizará «por».

Llaves para \spmsym

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será *verbalizado* por NVDA/MathCAT «antes» de *verbalizar* el $\{\langle \textit{valor} \rangle\}$ establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

suffix = {⟨sufijo⟩} default: no usado

Establece el *sufijo* que será *verbalizado* por NVDA/MathCAT «después» de *verbalizar* el $\{\{\textit{valor}\}\}$ establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves `{ }`.

set-sym = { *cross* | *dot* | *asterisk* } default: *dot*

Establece el *símbolo de multiplicación* que se imprimirá en la salida PDF. Si se establece en **cross**, imprimirá '×'; si se establece en **dot**, imprimirá '·' y si se establece en **asterisk**, imprimirá '*'.
 El símbolo de multiplicación se establece en **cross** por defecto.

read-sym = { $\langle$ *por* | *multiplicado-por* | *veces* $\rangle$ } default: *por*

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de multiplicación*. Si se establece en **por**, NVDA/MathCAT verbalizará «*por*»; si se establece en **multiplicado-por**, NVDA/MathCAT verbalizará «*multiplicado por*» y si se establece en **veces**, NVDA/MathCAT verbalizará «*veces*».

```
not-print = {⟨true | false⟩}                                     default: false
```

Deshabilita la *impresión del símbolo de multiplicación*. Cuando se establece esta llave, NVDA/MathCAT verbalizará el *símbolo de multiplicación* acorde al valor establecido por la llave `read-sym`, pero, no se imprimirá este en la salida PDF.

6 Utilidades para números

El paquete `spintent` provee utilidades para trabajar con números descritas en esta sección.

El comando \spdiv

`\spdiv` `\spdiv{<dividendo>}{<divisor>}` `\spdiv[<key = val>]{<dividendo>}{<divisor>}`

El comando `\spdiv` trabaja en *modo matemático* y recibe dos argumentos obligatorios $\{\langle \text{dividendo} \rangle\}$ y $\{\langle \text{divisor} \rangle\}$.

Si no se proporciona el *argumento opcional* [`[key = val]`], `\$spdiv{20}{26}` imprimirá ‘20 : 26’ en la salida PDF y **NVDA**/**MathCAT** verbalizará «veinte dividido veintiséis»

Llaves para `\spdiv`

El comando `\spdiv` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spdsym` (pág. 13).

`wrap-parens` = { *true* | *false* } default: *false*

Establece si se *imprimen* paréntesis redondos ‘(...)’ alrededor de los *argumentos* { *dividendo* } y { *divisor* } pasados a `\spdiv`. Si se establece en *true*, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en *false*, no se imprimirán los paréntesis.

`read-parens` = { *true* | *false* } default: *true*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(...)’ alrededor de los *argumentos* { *dividendo* } y { *divisor* } pasados a `\spdiv`. Si se establece en *true*, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en *false*, NVDA/MathCAT no los verbalizará.

El comando `\spmul`

`\spmul` `\spmul{factor}{factor}` `\spmul[key = val]{factor}{factor}`

El comando `\spmul` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* { *factor* } y { *factor* }. Si no se proporciona el *argumento opcional* [*key = val*], `\spmul{20}{26}` imprimirá ‘20 · 26’ en la salida PDF y NVDA/MathCAT verbalizará «veinte por veintiséis».

Llaves para `\spmul`

El comando `\spmul` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spmsym` (pág. 13).

`wrap-parens` = { *true* | *false* } default: *false*

Establece si se *imprimen* paréntesis redondos ‘(...)’ alrededor de los *argumentos* { *factor* } y { *factor* } pasados a `\spmul`. Si se establece en *true*, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en *false*, no se imprimirán los paréntesis.

`read-parens` = { *true* | *false* } default: *true*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(...)’ alrededor de los *argumentos* { *factor* } y { *factor* } pasados a `\spmul`. Si se establece en *true*, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en *false*, NVDA/MathCAT no los verbalizará.

El comando `\spnD`

`\spnD` `\spnD` `\spnD(número)` `\spnD[key = val](número)` `\spnD(letra)` `\spnD[key = val](letra)`

El comando `\spnD` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (*número*) que se procesa internamente por `\spnum` o (*letra*) que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* [*key = val*] ni el *argumento opcional delimitado por paréntesis*, `\spnD` imprimirá ‘D(*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de ene».

Si se usa el *argumento* (*número*), `\spnD(6)` imprimirá ‘D(6)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de seis».

Si se usa el *argumento* (*letra*), `\spnD(m)` imprimirá ‘D(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de eme».

🚫 El número 0 no es válido: `\spnD(0)` devolverá un “error” y se detendrá la compilación del documento.

Llaves para `\spnD`

`concept` = { *true* | *false* } default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «divisores de» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «divisores de» y el (*argumento*); si se establece en *false*, NVDA/MathCAT solo verbalizará el (*argumento*).

`prefix` = { *prefijo* } default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «divisores de». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result` = { *true* | *false* } default: *false*

Establece si se *imprime* el resultado de calcular los *divisores* del *argumento* (*número*). Si se establece en *true*, `\spnD[result=true](6)` imprimirá ‘D(6) = {1, 2, 3, 6}’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de seis son uno, dos, tres y seis»; si se establece en *false*, no se imprimirá ni verbalizará el resultado de calcular los *divisores*.

`result-num` = { *número natural* } default: 5

Establece la *cantidad* de números que se mostrarán en la salida PDF al establecer `result=true`. Si se establece en `2`, solo se imprimirán dos valores en la salida PDF; si no se establece, se imprimirán como máximo `5` valores y luego se añadirá ‘...’ en la salida PDF.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «divisores de» y del (⟨*argumento*⟩) pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spnM`

`\spnM` `\spnM` `\spnM(⟨número⟩)` `\spnM[⟨key = val⟩](⟨número⟩)` `\spnM(⟨letra⟩)` `\spnM[⟨key = val⟩](⟨letra⟩)`

El comando `\spnM` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (⟨*número*⟩) que se procesa internamente por `\spnum` o (⟨*letra*⟩) que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* [⟨*key = val*⟩] ni el *argumento opcional delimitado por paréntesis*, `\spnM` imprimirá ‘M(*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de ene».

Si se usa el argumento (⟨*número*⟩), `\spnM(6)` imprimirá ‘M(6)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de seis».

Si se usa el argumento (⟨*letra*⟩), `\spnM(m)` imprimirá ‘M(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de eme».

🚫 El número `0` no es válido: `\spnM(0)` devolverá un “error” y se detendrá la compilación del documento.

Llaves para `\spnM`

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «múltiplos de» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará «múltiplos de» y el (⟨*argumento*⟩); si se establece en `false`, NVDA/MathCAT solo verbalizará el (⟨*argumento*⟩).

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «múltiplos de». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = {⟨true | false⟩}` default: *false*

Establece si se *imprime* el resultado de calcular los *múltiplos* del argumento (⟨*número*⟩). Si se establece en `true`, `\spnM[result=true](6)` imprimirá ‘M(6) = {6, 12, 18, 24, 30, ...}’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de seis son seis, doce, dieciocho, veinticuatro, treinta puntos suspensivos»; si se establece en `false`, no se imprimirá ni verbalizará el resultado de calcular los *múltiplos*.

`result-num = {⟨número natural⟩}` default: *5*

Establece la *cantidad* de números que se mostrarán en la salida PDF al establecer `result=true`. Si se establece en `2`, solo se imprimirán dos valores en la salida PDF; si no se establece, se imprimirán como máximo `5` valores y luego se añadirá ‘...’ en la salida PDF.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «múltiplos de» y del (⟨*argumento*⟩) pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmcD`

`\spmcD` `\spmcD` `\spmcD(⟨número, número, ...⟩)` `\spmcD(⟨letra, letra, ...⟩)` `\spmcD[⟨key = val⟩](⟨número, número, ...⟩)`

El comando `\spmcD` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (⟨*número, número, ...*⟩) que se procesa internamente por `\spnum` o (⟨*letra, letra, ...*⟩) que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* [⟨*key = val*⟩] ni el *argumento opcional delimitado por paréntesis*, `\spmcD` imprimirá ‘mcd’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor».

Si se usa el argumento (⟨*número, número, ...*⟩), `\spmcD(2, 3, 6)` imprimirá ‘mcd(2, 3, 6)’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre dos, tres y seis».

Si se usa el argumento (⟨*letra, letra, ...*⟩), `\spmcD(m, n)` imprimirá ‘mcd(*m, n*)’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre eme y ene».

🚫 Los argumentos (⟨*número, número, ...*⟩) o (⟨*letra, letra, ...*⟩) requiere al menos dos elementos.

Llaves para `\spmc`

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto establecido por la llave `read-cmd` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «*máximo común divisor ...*» o «*m c d ...*» y el (*argumento*); si se establece en *false*, NVDA/MathCAT solo verbalizará el (*argumento*).

`read-cmd = {⟨terse | clear⟩}` default: *clear*

Establece la *forma* en la que NVDA/MathCAT verbalizará el *concepto* asociado al comando sin modificar la impresión de este. Si se establece en *terse*, NVDA/MathCAT verbalizará «*m c d*»; si se establece en *clear*, NVDA/MathCAT verbalizará «*máximo común divisor*».

`upper-case = {⟨true | false⟩}` default: *false*

Establece si se usarán *mayúsculas* en la salida PDF al ejecutar el comando sin modificar la verbalización de este. Si se establece en *true*, se imprimirá 'MCD' en la salida PDF; si se establece en *false*, se imprimirá 'mcd' en la salida PDF.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`result = {⟨true | false⟩}` default: *false*

Establece si se *imprime* el resultado de calcular el «*máximo común divisor*» del argumento (*número, número, ...*). Si se establece en *true*: `\spmc[result=true](2, 3, 6)` imprimirá 'mcd(2, 3, 6) = 1' en la salida PDF y NVDA/MathCAT verbalizará «*máximo común divisor entre dos, tres y seis es igual a uno*»; si se establece en *false* no se imprimirá ni verbalizará el resultado de calcular el «*máximo común divisor*».

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*máximo común divisor ...*» y del (*argumento*) pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmc`

`\spmc` `\spmc` `\spmc(⟨número, número, ...⟩)` `\spmc(⟨letra, letra, ...⟩)` `\spmc[⟨key = val⟩](⟨número, número, ...⟩)`

El comando `\spmc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (*número, número, ...*) que se procesa internamente por `\spnum` o (*letra, letra, ...*) que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* [*key = val*] ni el *argumento opcional delimitado por paréntesis*, `\spmc` imprimirá 'mcm' en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo*».

Si se usa el argumento (*número, número, ...*), `\spmc(2, 3, 6)` imprimirá 'mcm(2, 3, 6)' en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre dos, tres y seis*».

Si se usa el argumento (*letra, letra, ...*), `\spmc(m, n)` imprimirá 'mcm(m, n)' en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre eme y ene*».

🔹 Los argumentos (*número, número, ...*) o (*letra, letra, ...*) requiere al menos dos elementos.

Llaves para `\spmc`

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto establecido por la llave `read-cmd` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «*mínimo común múltiplo ...*» o «*m c m ...*» y el (*argumento*); si se establece en *false*, NVDA/MathCAT solo verbalizará el (*argumento*).

`read-cmd = {⟨terse | clear⟩}` default: *clear*

Establece la *forma* en la que NVDA/MathCAT verbalizará el *concepto* asociado al comando sin modificar la impresión de este. Si se establece en *terse*, NVDA/MathCAT verbalizará «*m c m*»; si se establece en *clear*, NVDA/MathCAT verbalizará «*mínimo común múltiplo*».

`upper-case = {⟨true | false⟩}` default: *false*

Establece si se usarán *mayúsculas* en la salida PDF al ejecutar el comando sin modificar la verbalización de este. Si se establece en *true*, se imprimirá 'MCM' en la salida PDF; si se establece en *false*, se imprimirá 'mcm' en la salida PDF.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`result = {⟨true | false⟩}` default: *false*

Establece si se *imprime* el resultado de calcular el «*mínimo común múltiplo*» del argumento (*⟨número, número, ...⟩*). Si se establece en *true*: `$\spmc{result=true}(2, 3, 6)$` imprimirá ‘mcm(2, 3, 6) = 6’ en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre dos, tres y seis es igual a seis*»; si se establece en *false* no se imprimirá ni verbalizará el resultado de calcular el «*mínimo común múltiplo*».

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*mínimo común múltiplo ...*» y del (*⟨argumento⟩*) pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spnZ`

`\spnZ` `\spnZ{⟨número entero⟩}` `\spnZ[⟨key = val⟩]{⟨número entero⟩}`

El comando `\spnZ` trabaja en *modo matemático* y recibe el *argumento* obligatorio *{⟨número entero⟩}* el cual se procesa internamente bajo el comando `\spnum`. Este comando está dedicado exclusivamente al manejo de «*números enteros*», y añade algunas utilidades prácticas para el demarcado e interacción con NVDA/MathCAT.

Llaves para `\spnZ`

El comando `\spnZ` posee las *⟨meta-keys⟩* *cifras-sep* y *read-sign* pasadas al comando `\spnum` (§3.1.1).

`wrap-parens = {⟨true | false⟩}` default: *false*

Establece si se *imprimen* paréntesis redondos ‘(…)’ alrededor del argumento *{⟨número entero⟩}* pasado a `\spnZ`. Si se establece en *true*, se imprimirán paréntesis redondos alrededor del *{⟨número entero⟩}*; si se establece en *false*, no se imprimirán los paréntesis.

`read-parens = {⟨true | false⟩}` default: *true*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(…)’ alrededor del *argumento {⟨número entero⟩}* pasado a `\spnZ`. Si se establece en *true*, NVDA/MathCAT verbalizará la lectura del paréntesis ‘(…)»; si se establece en *false*, NVDA/MathCAT no los verbalizará.

El comando `\spmQ`

`\spmQ` `\spmQ{⟨entero⟩}{⟨numerador⟩}{⟨denominador⟩}` `\spmQ[⟨key = val⟩]{⟨entero⟩}{⟨numerador⟩}{⟨denominador⟩}`

El comando `\spmQ` trabaja en *modo matemático* y recibe *tres argumentos obligatorios*: *{⟨número entero⟩}*, *{⟨numerador⟩}* y *{⟨denominador⟩}*, los procesa internamente y devuelve un «*número mixto*».

Si no se proporciona el *argumento opcional* *[⟨key = val⟩]*, `$\spmQ{2}{5}{7}$` imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*dos y la fracción con numerador cinco y denominador siete*».

- En el caso de las fracciones, y en especial con los «*números mixtos*», no se pueden manejar muchas cosas desde el lado de *spintent*. Por ejemplo, la entrada `$3\frac{5}{7}$` con *tagged* PDF se verbaliza por NVDA/MathCAT como «*tres más cinco séptimos*», lo cual no está mal, pero pedagógicamente hablando no es del todo correcto.

Llaves para `\spmQ`

`join-word = {⟨entero | enteros | y⟩}` default: *y*

Establece la *forma* en la que NVDA/MathCAT verbaliza la conexión entre el entero y la fracción que forman el *número mixto*. Si se establece en *entero*, `$\spmQ[join-word=entero]{1}{5}{7}$` imprimirá ‘ $1\frac{5}{7}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*un entero la fracción con numerador cinco y denominador siete*»; si se establece en *enteros*, `$\spmQ[join-word=enteros]{2}{5}{7}$` imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*dos enteros la fracción con numerador cinco y denominador siete*»; si se establece en *y*, `$\spmQ[join-word=y]{2}{5}{7}$` imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*dos y la fracción con numerador cinco y denominador siete*».

`use-spnZ = {⟨true | false⟩}` default: *true*

Establece si el *{⟨numerador⟩}* y el *{⟨denominador⟩}* que forman la fracción del *número mixto* se procesan internamente bajo el comando `\spnZ`, añadiendo así *semántica de entero* en la estructura *MathML* de la salida PDF. Si se establece en *true*, *{⟨numerador⟩}* y *{⟨denominador⟩}* se pasarán a `\spnZ` y, por ejemplo, `$\spmQ{2}{5}{7}$` se verbalizará «*dos y la fracción con numerador cinco y denominador siete*»; si se establece en *false*, los argumentos se tipografían directamente sin procesamiento semántico adicional y NVDA/MathCAT aplica sus propias reglas a la fracción: `$\spmQ[use-spnZ=false]{2}{5}{7}$` se verbalizará «*dos y cinco séptimos*».

`use-dfrac = {⟨true | false⟩}` default: *false*

Establece si la fracción del *número mixto* se imprime usando `\dfrac` en lugar de `\frac`. Si se establece en *true*, al usar `$\spmQ[use-dfrac=true]{2}{5}{7}$` imprimirá la fracción en tamaño *display* ‘ $2\frac{5}{7}$ ’ en la salida PDF; si se establece en *false*, la fracción se imprime con `\frac` siguiendo el tamaño matemático del contexto en curso.

`wrap-parens = {⟨true | false⟩}` default: *false*

Establece si se *imprimen* paréntesis redondos ‘(...)’ alrededor del *número mixto* completo devuelto por `\spmQ`. Si se establece en `true`, se imprimirán paréntesis redondos cuyo tamaño se ajusta automáticamente al estilo matemático en curso; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {\true | false}` default: `true`

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(...)’ alrededor del *número mixto* cuando la llave `wrap-parens` está activa. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\sprazon`

`\sprazon` `\sprazon{\langle antecendente \rangle}{\langle consecuente \rangle}` `\sprazon[\langle key = val \rangle]{\langle antecendente \rangle}{\langle consecuente \rangle}`

El comando `\sprazon` trabaja en *modo matemático* y recibe *dos argumentos obligatorios*: `{\langle antecendente \rangle}` y `{\langle consecuente \rangle}`, los tipografía como una fracción y devuelve una «razón».

Si no se proporciona el *argumento opcional* `[\langle key = val \rangle]`, `\sprazon{3}{4}` imprimirá ‘ $\frac{3}{4}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «tres es a cuatro».

- El `{\langle antecendente \rangle}` y el `{\langle consecuente \rangle}` se tipografían directamente, sin procesamiento semántico adicional.

Llaves para `\sprazon`

`silent = {\true | false}` default: `false`

Establece si se *silencia* en NVDA/MathCAT la verbalización de la *razón* pasada al comando sin modificar la impresión de esta. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

`use-dfrac = {\true | false}` default: `false`

Establece si la fracción de la *razón* se imprime usando `\dfrac` en lugar de `\frac`. Si se establece en `true`, al usar `\sprazon[use-dfrac=true]{3}{4}` imprimirá la fracción en tamaño *display* ‘ $\frac{3}{4}$ ’ en la salida PDF; si se establece en `false`, la fracción se imprime con `\frac` siguiendo el tamaño matemático del contexto en curso.

`inline = {\true | false}` default: `false`

Establece si la *razón* se imprime en línea con dos puntos ‘:’ en lugar de una fracción. Si se establece en `true`, `\sprazon[inline=true]{3}{4}` imprimirá ‘3 : 4’ en la salida PDF y NVDA/MathCAT verbalizará «tres es a cuatro»; si se establece en `false`, la razón se imprime como una fracción.

- La llave `use-dfrac` no tiene efecto cuando la llave `inline` se establece en `true`, porque en ese caso no se imprime una fracción.

El comando `\spProp`

`\spProp` `\spProp{\langle a \rangle}{\langle b \rangle}{\langle c \rangle}{\langle d \rangle}` `\spProp[\langle key = val \rangle]{\langle a \rangle}{\langle b \rangle}{\langle c \rangle}{\langle d \rangle}`

El comando `\spProp` trabaja en *modo matemático* y recibe *cuatro argumentos obligatorios*: `{\langle a \rangle}` y `{\langle b \rangle}` forman la primera *razón*, `{\langle c \rangle}` y `{\langle d \rangle}` forman la segunda, y devuelve una «proporción» con las dos razones unidas por el signo igual.

Si no se proporciona el *argumento opcional* `[\langle key = val \rangle]`, `\spProp{3}{4}{6}{8}` imprimirá ‘ $\frac{3}{4} = \frac{6}{8}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «tres es a cuatro como seis es a ocho».

- Los cuatro argumentos se tipografían directamente, sin procesamiento semántico adicional.

Llaves para `\spProp`

`silent = {\true | false}` default: `false`

Establece si se *silencia* en NVDA/MathCAT la verbalización de la *proporción* pasada al comando sin modificar la impresión de esta. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

`use-dfrac = {\true | false}` default: `false`

Establece si las fracciones de la *proporción* se imprimen usando `\dfrac` en lugar de `\frac`. Si se establece en `true`, al usar `\spProp[use-dfrac=true]{3}{4}{6}{8}` imprimirá las fracciones en tamaño *display* ‘ $\frac{3}{4} = \frac{6}{8}$ ’ en la salida PDF; si se establece en `false`, las fracciones se imprimen con `\frac` siguiendo el tamaño matemático del contexto en curso.

`inline = {\true | false}` default: `false`

Establece si las *razones* de la *proporción* se imprimen en línea con dos puntos ‘:’ en lugar de fracciones. Si se establece en `true`, `\spProp[inline=true]{3}{4}{6}{8}` imprimirá ‘3 : 4 = 6 : 8’ en la salida PDF y NVDA/MathCAT verbalizará «tres es a cuatro como seis es a ocho»; si se establece en `false`, las razones se imprimen como fracciones.

- La llave `use-dfrac` no tiene efecto cuando la llave `inline` se establece en `true`, porque en ese caso no se imprimen fracciones.

7 Utilidades para geometría

El paquete **spintent** provee comandos para trabajar con *geometría* descritos en esta sección. Todos los comandos trabajan en *modo matemático* y reciben como $\{\langle \text{argumento} \rangle\}$ base una *única* letra (mayúscula en el caso de puntos) con un modificador del tipo subíndice/prima, un número o comando.

Algunos comandos reciben $\{\langle \text{argumento} \rangle\}$ único (modo “*single*”) o más de uno (modo “*batch*”) donde opcionalmente se pueden separar por comas ‘,’.

- Las primas se debe pasar de forma directa ‘’ y los subíndices siempre deben escribirse usando llaves ‘A_{...}’ y nunca sin estas ‘A_n’ que funciona solo por coincidencia. .

El comando `\spcoord`

<code>\spcoord</code>	<code>\spcoord{\langle abscisa, ordenada \rangle}</code>	<code>\spcoord[\langle key = val \rangle]{\langle abscisa, ordenada \rangle}</code>
	<code>\spcoord{\langle abscisa; ordenada \rangle}</code>	<code>\spcoord[\langle key = val \rangle]{\langle abscisa; ordenada \rangle}</code>

El comando `\spcoord` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{\langle abscisa, ordenada \rangle\}$ o $\{\langle abscisa; ordenada \rangle\}$ el cual se procesa internamente bajo el comando `\spnum` cuando es numérico no fraccionario.

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$, `\spcoord{-3, 4}` imprimirá ‘(-3, 4)’ en la salida PDF y NVDA/MathCAT verbalizará «menos tres coma cuatro»; `\spcoord{x_{1}, y_{1}}` imprimirá ‘(x₁, y₁)’ y verbalizará «exis subíndice uno y subíndice uno».

- El comando `\spcoord` solo acepta un *separador* ‘,’ o ‘;’ para la abscisa y la ordenada cuando son valores numéricos no fraccionarios.

Llaves para `\spcoord`

`silent = {\langle true | false \rangle}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del $\{\langle \text{argumento} \rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPoint`

<code>\spPoint</code>	<code>\spPoint{\langle punto \rangle}</code>	<code>\spPoint[\langle key = val \rangle]{\langle punto \rangle}</code>
	<code>\spPoint{\langle punto \rangle}(\langle abscisa, ordenada \rangle)</code>	<code>\spPoint[\langle key = val \rangle]{\langle punto \rangle}(\langle abscisa, ordenada \rangle)</code>
	<code>\spPoint{\langle punto \rangle}(\langle abscisa; ordenada \rangle)</code>	<code>\spPoint[\langle key = val \rangle]{\langle punto \rangle}(\langle abscisa; ordenada \rangle)</code>

El comando `\spPoint` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{\langle punto \rangle\}$ el cual es una *única letra mayúscula* con prima ‘’ o subíndice ‘_...’ y el *argumento opcional delimitado por paréntesis* $(\langle abscisa, ordenada \rangle)$ el cual se procesa internamente bajo el comando `\spcoord`.

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$, `\spPoint{A}` imprimirá ‘A’ en la salida PDF y NVDA/MathCAT verbalizará «punto A»; `\spPoint{A_{1}}` imprimirá ‘A₁’ y verbalizará «punto A sub uno».

Si se proporciona el *argumento opcional delimitado por paréntesis*, `\spPoint{A}(-3, 4)` imprimirá ‘A(-3, 4)’ en la salida PDF y NVDA/MathCAT verbalizará «punto A menos tres coma cuatro»; `\spPoint{A_{1}}(-3, 4)` imprimirá ‘A₁(-3, 4)’ y verbalizará «punto A sub uno menos tres coma cuatro».

Llaves para `\spPoint`

`read-cmd = {\langle school | mathcat \rangle}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento $\{\langle punto \rangle\}$. Si se establece en *school*, NVDA/MathCAT verbalizará el argumento $\{\langle punto \rangle\}$ sin el prefijo «mayúscula»; `\spPoint{A}` se verbaliza «punto A»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {\langle prefijo \rangle}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «punto». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`concept = {\langle true | false \rangle}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «punto» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto y el $\{\langle \text{argumento} \rangle\}$; si se establece en *false*, NVDA/MathCAT solo verbalizará el $\{\langle \text{argumento} \rangle\}$ acorde al valor establecido por la llave `read-cmd`.

`silent = {\langle true | false \rangle}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «punto» y del $\{\langle \text{argumento} \rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spvec`

<code>\spvec</code>	<code>\spvec{⟨letra⟩}</code>	<code>\spvec{⟨punto, punto⟩}</code>
	<code>\spvec{⟨letra⟩} (⟨abscisa, ordenada⟩)</code>	<code>\spvec{⟨punto, punto⟩} (⟨abscisa, ordenada⟩)</code>
	<code>\spvec[⟨key = val⟩]{⟨letra⟩} (⟨abscisa, ordenada⟩)</code>	<code>\spvec[⟨key = val⟩]{⟨punto, punto⟩} (⟨abscisa, ordenada⟩)</code>

El comando `\spvec` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{⟨letra⟩}` (minúscula) o `{⟨punto, punto⟩}` junto al *argumento opcional delimitado por paréntesis* `(⟨abscisa, ordenada⟩)` el cual se procesa internamente bajo el comando `\spcoord`.

Si el *argumento* es de la forma `{⟨punto, punto⟩}` con prima(s) ‘`’` o subíndice(s) ‘`_{}...`’ opcionales se aplica `\overrightarrow` en la impresión; si el *argumento* es de la forma `{⟨letra⟩}` (minúscula) con subíndice ‘`_{}...`’ opcional se aplica `\vec` en la impresión.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spvec{a}` imprimirá ‘ $\vec{a}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «vector a »; `\spvec{AB}` imprimirá ‘ $\overrightarrow{AB}$ ’ y verbalizará «vector $A B$ ».

Si se proporciona el *argumento opcional delimitado por paréntesis*, `\spvec{a}(-3,4)` imprimirá ‘ $\vec{a} = (-3, 4)$ ’ en la salida PDF y NVDA/MathCAT verbalizará «vector a igual a menos tres coma cuatro»; `\spvec{AB}(-3,4)` imprimirá ‘ $\overrightarrow{AB} = (-3, 4)$ ’ y verbalizará «vector $A B$ igual a menos tres coma cuatro».

Llaves para `\spvec`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `{⟨punto, punto⟩}`. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spvec{AB}` se verbaliza «vector $A B$ »; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «vector». El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

`autofit = {⟨true | false⟩}` default: *false*

Establece si se *utiliza* `\vphantom{b}` al ejecutar el comando cuando se utiliza el argumento `{⟨letra⟩}`. Si se establece en *true*, se aplicará `\vphantom{b}` al inicio del argumento `{⟨letra⟩}`; si se establece en *false*, no se aplicará `\vphantom{b}`.

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «vector» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto y el `{⟨argumento⟩}`; si se establece en *false*, NVDA/MathCAT solo verbalizará el `{⟨argumento⟩}` acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «vector» y del `{⟨argumento⟩}` pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\sprecta`

<code>\sprecta</code>	<code>\sprecta{⟨punto, punto⟩}</code>	<code>\sprecta{⟨letra⟩}</code>
	<code>\sprecta[⟨key = val⟩]{⟨punto, punto⟩}</code>	<code>\sprecta[⟨key = val⟩]{⟨letra⟩}</code>

El comando `\sprecta` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{⟨letra⟩}` o `{⟨punto, punto⟩}` con prima(s) ‘`’` o subíndice(s) ‘`_{}...`’ opcionales. Si el *argumento* es de la forma `{⟨punto, punto⟩}` se aplica `\overleftarrow` en la impresión; si el *argumento* es de la forma `{⟨letra⟩}` (mayúscula o minúscula), se imprimirá solo la `{⟨letra⟩}` sin aplicar `\overleftarrow`.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\sprecta{AB}` imprimirá ‘ $\overleftarrow{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «recta $A B$ »; `\sprecta{m}` imprimirá ‘ m ’ en la salida PDF y NVDA/MathCAT verbalizará «recta m ».

Llaves para `\sprecta`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `{⟨punto, punto⟩}` y el argumento `{⟨letra⟩}` cuando está en mayúscula. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sprecta{AB}` se verbaliza «recta $A B$ »; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «recta». El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

`concept = {⟨true | false⟩}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*recta*» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto y el {⟨*argumento*⟩}; si se establece en *false*, NVDA/MathCAT solo verbalizará el {⟨*argumento*⟩} acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*recta*» y del {⟨*argumento*⟩} pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\sprayo`

`\sprayo` `\sprayo{⟨punto, punto⟩}`
`\sprayo[⟨key = val⟩]{⟨punto, punto⟩}`

El comando `\sprayo` trabaja en *modo matemático* y recibe el *argumento obligatorio* {⟨*punto, punto*⟩} con prima(s) ‘*’* o subíndice(s) ‘*_*{...}’ opcionales. Representa un *rayo* o *semirrecta* con origen en el primer punto y dirección hacia el segundo, aplicando `\overrightarrow` en la impresión.

Si no se proporciona el *argumento opcional* [⟨*key = val*⟩], `\sprayo{AB}` imprimirá ‘ $\overrightarrow{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «rayo A B».

Llaves para `\sprayo`

`read-cmd = {⟨school | mathcat⟩}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento {⟨*punto, punto*⟩}. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sprayo{AB}` se verbaliza «rayo A B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty = {⟨rayo | semirrecta⟩}`

default: *rayo*

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\sprayo[read-sty=rayo]{AB}`, NVDA/MathCAT verbalizará «rayo A B»; si se utiliza `\sprayo[read-sty=semirrecta]{AB}`, NVDA/MathCAT verbalizará «semirrecta A B».

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «*concepto*» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`concept = {⟨true | false⟩}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del «*concepto*» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto y el {⟨*argumento*⟩}; si se establece en *false*, NVDA/MathCAT solo verbalizará el {⟨*argumento*⟩} acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del «*concepto*» establecido por la llave `read-sty` y del {⟨*argumento*⟩} pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

- La impresión ‘ $\overrightarrow{AB}$ ’ en la salida PDF es idéntica en ambos casos. La elección depende de la terminología del libro de texto o del currículo nacional: en Chile y España se prefiere «semirrecta»; en México y otros países de América Latina se usa «rayo».

El comando `\spside`

`\spside` `\spside{⟨punto, punto⟩}`
`\spside[⟨key = val⟩]{⟨punto, punto⟩}`

El comando `\spside` trabaja en *modo matemático* y recibe el *argumento obligatorio* {⟨*punto, punto*⟩} con prima(s) ‘*’* o subíndice(s) ‘*_*{...}’ opcionales. Representa un *lado*, *segmento* o *trazo* determinado por dos puntos, aplicando `\overline` en la impresión.

Si no se proporciona el *argumento opcional* [⟨*key = val*⟩], `\spside{AB}` imprimirá ‘ $\overline{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «lado A B».

Llaves para `\spside`

`read-cmd` = { `<school | mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto, punto>` }. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spside{AB}` se verbaliza «lado A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty` = { `<lado | segmento | trazo>` } default: `lado`

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\spside[read-sty=lado]{AB}`, NVDA/MathCAT verbalizará «lado A B»; si se utiliza `\spside[read-sty=segmento]{AB}`, NVDA/MathCAT verbalizará «segmento A B» y si se utiliza `\spside[read-sty=trazo]{AB}`, NVDA/MathCAT verbalizará «trazo A B»

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept` = { `<true | false>` } default: `true`

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto y el { `<argumento>` }; si se establece en `false`, NVDA/MathCAT solo verbalizará el { `<argumento>` } acorde al valor establecido por la llave `read-cmd`.

`read-arg` = { `<verbalización NVDA>` } default: `no usado`

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spside[read-arg={hipotenusa}]{AB}` imprimirá $\overline{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «hipotenusa A B».

`silent` = { `<true | false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` y del { `<argumento>` } pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmside`

`\spmside` `\spmside{<punto, punto>}`
 `\spmside[<key = val>]{<punto, punto>}`

El comando `\spmside` trabaja en *modo matemático* y recibe el *argumento obligatorio* { `<punto, punto>` } con prima(s) ' ' o subíndice(s) ' _{...}' opcionales. Representa la *medida* del *lado*, *segmento* o *trazo* determinado por dos puntos. La distinción entre `\spside` y `\spmside` es semántica: `\spside` representa el *objeto geométrico* $\overline{AB}$; `\spmside` representa su *medida* (un número real positivo).

Si no se proporciona el *argumento opcional* [`<key = val>`], `\spmside{AB}` imprimirá $\overline{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «medida del lado A B».

Llaves para `\spmside`

`read-cmd` = { `<school | mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto, punto>` }. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmside{AB}` se verbaliza «medida del lado A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty` = { `<lado | segmento | trazo>` } default: `lado`

Establece el *concepto* que verbalizará NVDA/MathCAT después de verbalizar el término «medida del» al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\spmside[read-sty=lado]{AB}`, NVDA/MathCAT verbalizará «medida del lado A B»; si se utiliza `\spmside[read-sty=segmento]{AB}`, NVDA/MathCAT verbalizará «medida del segmento A B» y si se utiliza `\spmside[read-sty=trazo]{AB}`, NVDA/MathCAT verbalizará «medida del trazo A B»

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del» seguido por el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept` = { `<true | false>` } default: `true`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del» y del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el término «medida del», el concepto y el { `<argumento>` }; si se establece en `false`, NVDA/MathCAT solo verbalizará el { `<argumento>` } acorde al valor establecido por la llave `read-cmd`.

`read-arg = {⟨verbalización NVDA⟩}`

default: *no usado*

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`.

Por ejemplo: `$\spmside[concept=false,read-arg={medida de la cuerda}]{AB}$` imprimirá 'AB' en la salida PDF y NVDA/MathCAT verbalizará «medida de la cuerda A B».

`print-m = {⟨true | false⟩}`

default: *false*

Establece si se *imprime* la notación visual 'm' antes aplicar `\overline` al {⟨argumento⟩}. Si se establece en *true*, `$\spmside[print-m=true]{AB}$` imprimirá 'm $\overline{AB}$ ' en la salida PDF, si se establece en *false* imprimirá 'AB' en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «medida del lado A B».

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del», del «concepto» establecido por la llave `read-sty` y del {⟨argumento⟩} pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spangle`

<code>\spangle</code>	<code>{⟨punto letra griega número⟩}</code>	<code>\spangle{⟨punto, punto, punto⟩}</code>
<code>\spangle</code>	<code>[⟨key = val⟩]{⟨punto letra griega número⟩}</code>	<code>\spangle[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\spangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma {⟨punto | letra griega | número⟩} o {⟨punto, punto, punto⟩} aplicando `\angle` o `\rightanglesqr` en la impresión. Solo los argumentos de la forma {⟨punto⟩} o {⟨punto, punto, punto⟩} admiten prima(s) ' ' o subíndice(s) '_{...}' opcionales.

Si no se proporciona el *argumento opcional* [⟨key = val⟩], `$\spangle{A}$` imprimirá '∠A' en la salida PDF y NVDA/MathCAT verbalizará «ángulo en A»; `$\spangle{\alpha}$` imprimirá '∠α' en la salida PDF y NVDA/MathCAT verbalizará «ángulo alfa»; `$\spangle{1}$` imprimirá '∠1' en la salida PDF y NVDA/MathCAT verbalizará «ángulo uno» y `$\spangle{AOB}$` imprimirá '∠AOB' en la salida PDF y NVDA/MathCAT verbalizará «ángulo A O B».

- Si no se tiene instalada la fuente matemática NewCMath-Regular.otf parte del paquete `newcomputermodern` en el sistema o no se provee una fuente matemática con soporte para `\rightanglesqr` se usará `\rightangle` en su reemplazo.

Llaves para `\spangle`

`read-cmd = {⟨school | mathcat⟩}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento {⟨punto⟩} o {⟨punto, punto, punto⟩}. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `$\spangle{AOB}$` se verbaliza «ángulo A O B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «ángulo». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`recto = {⟨true | false⟩}`

default: *false*

Establece si se *usará* `\rightanglesqr` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en *true*, `$\spangle[recto=true]{A}$` imprimirá '⊥A' en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto en A», `$\spangle[recto=true]{AOB}$` imprimirá '⊥AOB' en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto A O B»; si se establece en *false*, imprimirá usando `\angle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

`concept = {⟨true | false⟩}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «ángulo» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto «ángulo» y el {⟨argumento⟩}; si se establece en *false*, NVDA/MathCAT solo verbalizará el {⟨argumento⟩} acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «ángulo» y del {⟨argumento⟩} pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmangle`

```
\spmangle <{punto | letra griega | número}> <{punto, punto, punto}>
\spmangle[key = val]{<punto | letra griega | número>} <{punto, punto, punto}>
```

El comando `\spmangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{<punto | letra griega | número>}` o `{<punto, punto, punto>}` aplicando `\measuredangle` o `\measuredrightangle` en la impresión. Solo los argumentos de la forma `{<punto>}` o `{<punto, punto, punto>}` admiten prima(s) ‘`’` o subíndice(s) ‘`_`’ opcionales.

Si no se proporciona el *argumento opcional* `[key = val]`, `\spmangle{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo en A»; `\spmangle{\alpha}` imprimirá ‘ $\angle \alpha$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo alfa»; `\spmangle{1}` imprimirá ‘ $\angle 1$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo uno» y `\spmangle{AOB}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo A O B».

- La distinción entre `\spangle` y `\spmangle` es semántica: `\spangle` representa el *objeto geométrico* ‘ $\angle AOB$ ’; `\spmangle` representa su *medida* (un valor en grados).

Llaves para `\spmangle`

```
read-cmd = {<school | mathcat>} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{<punto>}` o `{<punto, punto, punto>}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmangle{AOB}` se verbaliza «medida del ángulo A O B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del» seguido por el concepto «ángulo». El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

```
recto = {<true | false>} default: false
```

Establece si se *usará* `\measuredrightangle` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en `true`, `\spmangle[recto=true]{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto en A»; `\spmangle[recto=true]{AOB}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto A O B»; si se establece en `false`, imprimirá usando `\measuredangle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

```
print-m = {<true | false>} default: false
```

Establece si se *imprime* la notación visual ‘m’ antes del argumento pasado al comando sin modificar la verbalización de este. Si se establece en `true`, `\spmangle[print-m=true]{AOB}` imprimirá ‘m $\angle AOB$ ’ en la salida PDF, si se establece en `false` imprimirá ‘ $\angle AOB$ ’ en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «medida del ángulo A O B».

```
concept = {<true | false>} default: true
```

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del» seguido por el concepto «ángulo» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará «medida del ángulo» y el `{<argumento>}`; si se establece en `false`, NVDA/MathCAT solo verbalizará el `{<argumento>}` acorde al valor establecido por la llave `read-cmd`.

```
silent = {<true | false>} default: false
```

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del», del concepto «ángulo» y del `{<argumento>}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spang`

```
\spang {<grados>}
\spang {<grados : minutos>}
\spang {<grados : minutos : segundos>}
```

El comando `\spang` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<grados>}`, `{<grados : minutos>}` o `{<grados : minutos : segundos>}`. Los procesa internamente bajo los comandos `\spnum` y `\spunit` para imprimir y verbalizar. Por ejemplo, `\spang{45:30:15}` imprimirá ‘45°30’15” en la salida PDF y NVDA/MathCAT verbalizará «cuarenta y cinco grados treinta minutos quince segundos».

El comando `\sparc`

<code>\sparc</code>	<code>\sparc{⟨punto, punto⟩}</code>	<code>\sparc{⟨punto, punto, punto⟩}</code>
	<code>\sparc[⟨key = val⟩]{⟨punto, punto⟩}</code>	<code>\sparc[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\sparc` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{⟨punto, punto⟩}` o `{⟨punto, punto, punto⟩}` con prima(s) `'` o subíndice(s) `'_{...}'` opcionales aplicando `\widearc` en la impresión.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `$_\sparc{AB}$` imprimirá $\widehat{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «arco A B».

- El acento `\widearc` es proporcionado por algunos paquetes como `fourier-otf` o `kpfonts-otf`, en caso de no estar disponible se utilizará una versión interna para este.

Llaves para `\sparc`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{⟨punto, punto⟩}` o `{⟨punto, punto, punto⟩}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `$_\sparc{AB}$` se verbaliza «arco A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «arco». El valor de esta llave *siempre* debe pasarse entre llaves `{ }`.

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «arco» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará «arco» y el `{⟨argumento⟩}`; si se establece en `false`, NVDA/MathCAT solo verbalizará el `{⟨argumento⟩}` acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «arco» y del `{⟨argumento⟩}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmarc`

<code>\spmarc</code>	<code>\spmarc{⟨punto, punto⟩}</code>	<code>\spmarc{⟨punto, punto, punto⟩}</code>
	<code>\spmarc[⟨key = val⟩]{⟨punto, punto⟩}</code>	<code>\spmarc[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\spmarc` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{⟨punto, punto⟩}` o `{⟨punto, punto, punto⟩}` con prima(s) `'` o subíndice(s) `'_{...}'` opcionales colocando `'m'` antes de aplicar `\widearc` en la impresión.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `$_\spmarc{AB}$` imprimirá $m\widehat{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «medida del arco A B».

- La distinción entre `\sparc` y `\spmarc` es semántica: `\sparc` representa el *objeto geométrico* $\widehat{AB}$; `\spmarc` representa su *medida* (un valor en grados).

Llaves para `\spmarc`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{⟨punto, punto⟩}` o `{⟨punto, punto, punto⟩}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `$_\spmarc{AB}$` se verbaliza «medida del arco A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del» seguido por el concepto «arco». El valor de esta llave *siempre* debe pasarse entre llaves `{ }`.

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del» y el concepto «arco» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará «medida del arco» y el `{⟨argumento⟩}`; si se establece en `false`, NVDA/MathCAT solo verbalizará el `{⟨argumento⟩}` acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del», el concepto «arco» y el `{⟨argumento⟩}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPoly`

```
\spPoly \spPoly{⟨punto, punto, punto, ...⟩}
\spPoly[⟨key = val⟩]{⟨punto, punto, punto, ...⟩}
```

El comando `\spPoly` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{\langle \text{punto, punto, punto, ...} \rangle\}$, una lista de al menos tres $\{\langle \text{puntos} \rangle\}$ con prima(s) ‘ $\prime$ ’ o subíndice(s) ‘ $_{\dots}$ ’ opcionales. El *símbolo* usado en la impresión se maneja por medio de la llave `print-sym` y la verbalización del concepto asociado se determina acorde al número de puntos en el argumento y la llave `concept`.

Si son tres puntos se aplica `\triangle` y añade el concepto «*triángulo*» a la verbalización; si son cuatro puntos se aplica `\mdlgwhtsquare` y añade el concepto «*cuadrado*» a la verbalización; si son cinco o más puntos no se aplica ningún símbolo y se añade a la verbalización el nombre del polígono: «*pentágono*», «*hexágono*», «*heptágono*» u «*octágono*» de cinco a ocho puntos, y «*polígono*» desde nueve puntos.

Si no se proporciona el *argumento opcional* $[\langle \text{key} = \text{val} \rangle]$, `\spPoly{ABC}` imprimirá ‘ $\triangle ABC$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*triángulo A B C*»; `\spPoly{ABCD}` imprimirá ‘ $\square ABCD$ ’ y verbalizará «*cuadrado A B C D*»; `\spPoly{ABCDE}` imprimirá ‘ $ABCDE$ ’ y verbalizará «*pentágono A B C D E*».

Llaves para `\spPoly`

```
read-cmd = {⟨school | mathcat⟩} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento $\{\langle \text{punto, punto, punto, ...} \rangle\}$. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spPoly{ABC}` se verbaliza «*triángulo A B C*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {⟨prefijo⟩} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el $\{\langle \text{argumento} \rangle\}$ o el concepto «*triángulo*», «*cuadrado*», «*pentágono*», «*hexágono*», «*heptágono*», «*octágono*» o «*polígono*» establecido por la llave `concept`. El valor de esta llave *siempre* debe pasarse entre llaves ‘ $\{ \}$ ’.

```
concept = {⟨true | false⟩} default: true
```

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto («*triángulo*», «*cuadrado*», «*pentágono*», «*hexágono*», «*heptágono*», «*octágono*» o «*polígono*») acorde al número de puntos en el $\{\langle \text{argumento} \rangle\}$ al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto y el $\{\langle \text{argumento} \rangle\}$; si se establece en `false`, NVDA/MathCAT solo verbalizará el $\{\langle \text{argumento} \rangle\}$ acorde al valor establecido por la llave `read-cmd`.

```
print-sym = {⟨true | false⟩} default: true
```

Establece si se *usará* `\triangle` o `\mdlgwhtsquare` en la impresión sin modificar la verbalización de este. Si se establece en `true`, `\spPoly[print-sym=true]{ABC}` imprimirá ‘ $\triangle ABC$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*triángulo A B C*»; `\spPoly[print-sym=true]{ABCD}` imprimirá ‘ $\square ABCD$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*cuadrado A B C D*»; si se establece en `false`, solo se imprimirá el $\{\langle \text{argumento} \rangle\}$ en la salida PDF y la verbalización será la misma.

🔊 Esta llave no tiene efecto con cinco o más puntos, porque no hay un símbolo definido para ellos.

```
silent = {⟨true | false⟩} default: false
```

Establece si se *silencia* la verbalización en NVDA/MathCAT de los conceptos establecido por la llave `concept` y el $\{\langle \text{argumento} \rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spAfig`

```
\spAfig \spAfig(⟨punto, punto, punto, ...⟩)
\spAfig[⟨key = val⟩] \spAfig[⟨key = val⟩](⟨punto, punto, punto, ...⟩)
\spAfig(⟨1 | 2 | 3 | ... | n⟩) \spAfig((F | P | B | L | T))
\spAfig[⟨key = val⟩](⟨1 | 2 | 3 | ... | n⟩) \spAfig[⟨key = val⟩]((F | P | B | L | T))
```

El comando `\spAfig` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma $(\langle 1 | 2 | 3 | \dots | n \rangle)$, $(\langle \text{punto, punto, punto, ...} \rangle)$ o $((F | P | B | L | T))$ aplicando `\mathcal{A}` a la impresión. El *símbolo* usado en la impresión se maneja por medio de la llave `print-sym` y la verbalización del concepto asociado se determina acorde al $(\langle \text{argumento} \rangle)$ y la llave `concept`.

Si se proporciona el *argumento* $(\langle 1 | 2 | 3 | \dots | n \rangle)$, `\spAfig(1)` imprimirá ‘ $\mathcal{A}_1$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*área uno*»; `\spAfig(n)` imprimirá ‘ $\mathcal{A}_n$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*área ene*».

Si se proporciona el *argumento* $(\langle \text{punto, punto, punto, ...} \rangle)$, `\spAfig(ABC)` imprimirá ‘ $\mathcal{A}_{\triangle ABC}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*área del triángulo A B C*»; `\spAfig(ABCD)` imprimirá ‘ $\mathcal{A}_{\square ABCD}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*área del cuadrado A B C D*»; `\spAfig(ABCDE)` imprimirá ‘ $\mathcal{A}_{ABCDE}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*área del pentágono A B C D E*». Con cinco o más puntos no

se imprime ningún símbolo y se añade a la verbalización el nombre del polígono: «pentágono», «hexágono», «heptágono» u «octágono» de cinco a ocho puntos, y «polígono» desde nueve puntos.

Si se proporciona el *argumento* ((F | P | B | L | T)), $\$ \backslash \text{spAfig}(F) \$$ imprimirá ' $\mathcal{A}_F$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la figura»; $\$ \backslash \text{spAfig}(F_{\{1\}}) \$$ imprimirá ' $\mathcal{A}_{F_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la figura uno».

Si no se proporciona el *argumento opcional* [(key = val)] ni el *argumento opcional delimitado por paréntesis*, $\$ \backslash \text{spAfig} \$$ imprimirá ' $\mathcal{A}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área».

- El *argumento* ((punto, punto, punto, ...)) acepta prima(s) ' ' o subíndice(s) '_{...}'. El *argumento* ((F | P | B | L | T)) solo acepta subíndices '_{...}' y tiene una impresión y verbalización fija para cada letra: F «figura», P «polígono», B «basal», L «lateral» y T «total».

Llaves para \spAfig

`read-cmd = { (school | mathcat) }` default: school

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: $\$ \backslash \text{spAfig}(ABC) \$$ se verbaliza «área del triángulo A B C»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = { (prefijo) }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT antes de verbalizar el término «área del» y el concepto «nombre de la figura» acorde al ((argumento)) pasado al comando. El valor de esta llave siempre debe pasarse entre llaves '{ }'.

`concept = { (true | false) }` default: true

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área del» y el concepto «nombre de la figura» acorde al ((argumento)) pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el término «área del», el concepto «nombre de la figura» y el ((argumento)); si se establece en `false`, NVDA/MathCAT solo verbalizará el ((argumento)) acorde al valor establecido por la llave `read-cmd`.

- Utilizar `concept=false` sin el ((argumento)) es equivalente a usar `silent=true`.

`read-arg = { (nombre) }` default: no usado

Sobrescribe el término «área del» y el concepto «nombre de la figura» cuando se utiliza el argumento ((punto, punto, punto, ...)) que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave siempre debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`. Por ejemplo: $\$ \backslash \text{spAfig}[\text{read-arg}=\{\text{trapezio}\}](ABCD) \$$ imprimirá ' $\mathcal{A}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del trapecio A B C D».

- Esta llave no se puede utilizar con los argumentos ((F | P | B | L | T)) y ((1 | 2 | 3 | ... | n)).

`concept* (valor prohibido)` default: no usado

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «nombre de la figura» acorde al ((argumento)) pasado al comando sin modificar la impresión de este. Si se establece, NVDA/MathCAT verbalizará el término «área» y el ((argumento)) acorde al valor establecido por la llave `read-cmd`.

`read-sub = { (true | false) }` default: false

Establece si se *antepone* la palabra «sub» al verbalizar los argumentos ((1 | 2 | 3 | ... | n)) y ((F | P | B | L | T)) para NVDA/MathCAT. Si se utiliza ((1 | 2 | 3 | ... | n)) y se establece en `true`, $\$ \backslash \text{spAfig}[\text{read-sub}=true](1) \$$ imprimirá ' $\mathcal{A}_1$ ' en la salida PDF y NVDA/MathCAT verbalizará «área sub uno»; si se establece en `false`, la impresión será la misma y se verbaliza «área uno».

Si se utiliza ((F | P | B | L | T)) y se establece en `true`, $\$ \backslash \text{spAfig}[\text{read-sub}=true](F_{\{1\}}) \$$ imprimirá ' $\mathcal{A}_{F_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la figura sub uno»; si se establece en `false`, la impresión será la misma y se verbaliza «área de la figura uno».

`print-sym = { (auto | triangle | square | none) }` default: auto

Establece si se *usará* `\triangle` o `\mdlgwhtsquare` en la impresión para los argumentos ((punto, punto, punto, ...)) y ((1 | 2 | 3 | ... | n)) pasado al comando sin modificar la verbalización de este. Si se utiliza ((punto, punto, punto, ...)) se infiere el *símbolo* acorde al número de puntos; si establece en `auto` se imprime y si se establece en `none` se omite: $\$ \backslash \text{spAfig}[\text{print-sym}=auto](ABC) \$$ y $\$ \backslash \text{spAfig}[\text{print-sym}=none](ABC) \$$ imprimirán ' $\mathcal{A}_{\triangle ABC}$ ' y ' $\mathcal{A}_{ABC}$ ' en la salida PDF y en ambos casos NVDA/MathCAT verbalizará «área del triángulo A B C».

Si se utiliza ((1 | 2 | 3 | ... | n)) los valores `auto` y `none` quedan sin efecto; si se establece en `triangle` o `square` anida el argumento bajo el *símbolo*: $\$ \backslash \text{spAfig}[\text{print-sym}=triangle](1) \$$ imprimirá ' $\mathcal{A}_{\triangle 1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del triángulo uno»; $\$ \backslash \text{spAfig}[\text{print-sym}=square](1) \$$ imprimirá ' $\mathcal{A}_{\square 1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del cuadrado uno».

- Esta llave no se puede utilizar con el argumento ((F | P | B | L | T)).

`silent = { (true | false) }` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área del», el concepto «nombre de la figura» y el $\langle \text{argumento} \rangle$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPfig`

<code>\spPfig</code>	<code>\spPfig</code>	<code>\spPfig(\punto, punto, punto, ...)</code>
<code>\spPfig[key = val]</code>	<code>\spPfig[key = val](\punto, punto, punto, ...)</code>	<code>\spPfig(F P B L T)</code>
<code>\spPfig(1 2 3 ... n)</code>	<code>\spPfig(F P B L T)</code>	<code>\spPfig[key = val](F P B L T)</code>

El comando `\spPfig` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma $\langle 1|2|3|...|n \rangle$, $\langle \text{punto, punto, punto, ...} \rangle$ o $\langle F|P|B|L|T \rangle$ aplicando `\mathcal{P}` a la impresión. El *símbolo* usado en la impresión se maneja por medio de la llave `print-sym` y la verbalización del concepto asociado se determina acorde al $\langle \text{argumento} \rangle$ y la llave `concept`.

Si se proporciona el *argumento* $\langle 1|2|3|...|n \rangle$, `\spPfig(1)` imprimirá ' $\mathcal{P}_1$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro uno»; `\spPfig(n)` imprimirá ' $\mathcal{P}_n$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro ene».

Si se proporciona el *argumento* $\langle \text{punto, punto, punto, ...} \rangle$, `\spPfig(ABC)` imprimirá ' $\mathcal{P}_{\triangle ABC}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro del triángulo A B C»; `\spPfig(ABCD)` imprimirá ' $\mathcal{P}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro del cuadrado A B C D»; `\spPfig(ABCDE)` imprimirá ' $\mathcal{P}_{ABCDE}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro del pentágono A B C D E». Con cinco o más puntos no se imprime ningún símbolo y se añade a la verbalización el nombre del polígono: «pentágono», «hexágono», «heptágono» u «octágono» de cinco a ocho puntos, y «polígono» desde nueve puntos.

Si se proporciona el *argumento* $\langle F|P|B|L|T \rangle$, `\spPfig(F)` imprimirá ' $\mathcal{P}_F$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la figura»; `\spPfig(F_{1})` imprimirá ' $\mathcal{P}_{F_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la figura uno».

Si no se proporciona el *argumento opcional* $\langle \text{key} = \text{val} \rangle$ ni el *argumento opcional delimitado por paréntesis*, `\spPfig` imprimirá ' $\mathcal{P}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro».

- El *argumento* $\langle \text{punto, punto, punto, ...} \rangle$ acepta prima(s) ' ' o subíndice(s) ' _{...}' . El *argumento* $\langle F|P|B|L|T \rangle$ solo acepta subíndices ' _{...}' y tiene una impresión y verbalización fija para cada letra: F «figura», P «polígono», B «basal», L «lateral» y T «total».

Llaves para `\spPfig`

`read-cmd` = $\{ \langle \text{school} | \text{mathcat} \rangle \}$ default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spPfig(ABC)` se verbaliza «perímetro del triángulo A B C»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = $\{ \langle \text{prefijo} \rangle \}$ default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «perímetro del» y el concepto «nombre de la figura» acorde al $\langle \text{argumento} \rangle$ pasado al comando. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept` = $\{ \langle \text{true} | \text{false} \rangle \}$ default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «perímetro del» y el concepto «nombre de la figura» acorde al $\langle \text{argumento} \rangle$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el término «perímetro del», el concepto «nombre de la figura» y el $\langle \text{argumento} \rangle$; si se establece en *false*, NVDA/MathCAT solo verbalizará el $\langle \text{argumento} \rangle$ acorde al valor establecido por la llave `read-cmd`.

- Utilizar `concept=false` sin el $\langle \text{argumento} \rangle$ es equivalente a usar `silent=true`.

`read-arg` = $\{ \langle \text{nombre} \rangle \}$ default: *no usado*

Sobrescribe el término «perímetro del» y el concepto «nombre de la figura» cuando se utiliza el argumento $\langle \text{punto, punto, punto, ...} \rangle$ que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`. Por ejemplo: `\spPfig[read-arg={trapecio}](ABCD)` imprimirá ' $\mathcal{P}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro del trapecio A B C D».

- Esta llave no se puede utilizar con los argumentos $\langle F|P|B|L|T \rangle$ y $\langle 1|2|3|...|n \rangle$.

`concept*` $\langle \text{valor prohibido} \rangle$ default: *no usado*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «nombre de la figura» acorde al $\langle \text{argumento} \rangle$ pasado al comando sin modificar la impresión de este. Si se establece, NVDA/MathCAT verbalizará el término «perímetro» y el $\langle \text{argumento} \rangle$ acorde al valor establecido por la llave `read-cmd`.

`read-sub` = $\{ \langle \text{true} | \text{false} \rangle \}$ default: *false*

Establece si se *antepone* la palabra «sub» al verbalizar los argumentos $(\langle 1 \mid 2 \mid 3 \mid \dots \mid n \rangle)$ y $(\langle F \mid P \mid B \mid L \mid T \rangle)$ para NVDA/MathCAT. Si se utiliza $(\langle 1 \mid 2 \mid 3 \mid \dots \mid n \rangle)$ y se establece en `true`, $\$ \backslash \text{spPfig}[\text{read-sub}=\text{true}](1) \$$ imprimirá ‘ $\mathcal{P}_1$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*perímetro sub uno*»; si se establece en `false`, la impresión será la misma y se verbaliza «*perímetro uno*».

Si se utiliza $(\langle F \mid P \mid B \mid L \mid T \rangle)$ y se establece en `true`, $\$ \backslash \text{spPfig}[\text{read-sub}=\text{true}](F_{\{1\}}) \$$ imprimirá ‘ $\mathcal{P}_{F_1}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*perímetro de la figura sub uno*»; si se establece en `false`, la impresión será la misma y se verbaliza «*perímetro de la figura uno*».

`print-sym = { \langle auto \mid triangle \mid square \mid none \rangle }`

default: *auto*

Establece si se usará `\triangle` o `\mdlgwhtsquare` en la impresión para los argumentos $(\langle \text{punto}, \text{punto}, \text{punto}, \dots \rangle)$ y $(\langle 1 \mid 2 \mid 3 \mid \dots \mid n \rangle)$ pasado al comando sin modificar la verbalización de este. Si se utiliza $(\langle \text{punto}, \text{punto}, \text{punto}, \dots \rangle)$ se infiere el *símbolo* acorde al número de puntos; si establece en `auto` se imprime y si se establece en `none` se omite: $\$ \backslash \text{spPfig}[\text{print-sym}=\text{auto}](ABC) \$$ y $\$ \backslash \text{spPfig}[\text{print-sym}=\text{none}](ABC) \$$ imprimirán ‘ $\mathcal{P}_{\triangle ABC}$ ’ y ‘ $\mathcal{P}_{ABC}$ ’ en la salida PDF y en ambos casos NVDA/MathCAT verbalizará «*perímetro del triángulo A B C*».

Si se utiliza $(\langle 1 \mid 2 \mid 3 \mid \dots \mid n \rangle)$ los valores `auto` y `none` quedan sin efecto; si se establece en `triangle` o `square` anida el argumento bajo el *símbolo*: $\$ \backslash \text{spPfig}[\text{print-sym}=\text{triangle}](1) \$$ imprimirá ‘ $\mathcal{P}_{\triangle_1}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*perímetro del triángulo uno*»; $\$ \backslash \text{spPfig}[\text{print-sym}=\text{square}](1) \$$ imprimirá ‘ $\mathcal{P}_{\square_1}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*perímetro del cuadrado uno*».

🔗 Esta llave no se puede utilizar con el argumento $(\langle F \mid P \mid B \mid L \mid T \rangle)$.

`silent = { \langle true \mid false \rangle }`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «*perímetro del*», el concepto «*nombre de la figura*» y el $\{ \langle \text{argumento} \rangle \}$ pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

Los comandos `\sprad` y `\spdiam`

`\sprad`
`\spdiam`

`\sprad[\langle key = val \rangle]`
`\spdiam[\langle key = val \rangle]`

Los comandos `\sprad` y `\spdiam` trabajan en *modo matemático* y reciben el *argumento opcional* $[\langle key = val \rangle]$ mediante el cual se establece la «*forma*» en la que se imprimirán en la salida PDF las letras ‘*r*’ y ‘*d*’ y como se verbalizarán en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$, $\$ \backslash \text{sprad} \$$ imprimirá ‘*r*’ en la salida PDF y NVDA/MathCAT verbalizará «*radio*»; $\$ \backslash \text{spdiam} \$$ imprimirá ‘*d*’ en la salida PDF y NVDA/MathCAT verbalizará «*diámetro*».

Llaves para `\sprad` y `\spdiam`

`concept = { \langle true \mid false \rangle }`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT de los conceptos «*radio*» o «*diámetro*» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, $\$ \backslash \text{sprad} \$$ y $\$ \backslash \text{spdiam} \$$ se verbalizan como «*radio*» y «*diámetro*» en NVDA/MathCAT; si se establece en `false`, se verbalizarán las letras ‘*r*’, ‘*R*’, ‘*d*’ o ‘*D*’ acorde al valor establecido por la llave `read-cmd`.

`read-cmd = { \langle school \mid mathcat \rangle }`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará las letras ‘*R*’ o ‘*D*’ cuando `concept=false` y `upper-case=true`. Si se establece en `school`, NVDA/MathCAT verbalizará las letras ‘*R*’ o ‘*D*’ sin el prefijo «mayúscula»: si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`upper-case = { \langle true \mid false \rangle }`

default: *false*

Establece si se *imprime* usando mayúsculas. Si se establece en `true`, $\$ \backslash \text{sprad}[\text{upper-case}=\text{true}] \$$ y $\$ \backslash \text{spdiam}[\text{upper-case}=\text{true}] \$$ imprimirán ‘*R*’ y ‘*D*’ en la salida PDF; si se establece en `false` se imprimirá usando minúsculas.

🔗 La verbalización al establecer esta llave es controlada por la llave `read-cmd`.

`print-sym = { \langle true \mid false \rangle }`

default: *false*

Establece si se usará `\diameter` antes de la letra ‘*d*’ o ‘*D*’ en la impresión del comando `\spdiam`. Si se establece en `true`, $\$ \backslash \text{spdiam}[\text{print-sym}=\text{true}] \$$ imprimirá ‘ $\emptyset d$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*diámetro*».

🔗 La llave `print-sym` no tiene ningún efecto en `\sprad`.

`sub-index = { \langle subíndice \rangle }`

default: *no usado*

Establece un *subíndice* sobre las letras ‘*r*’, ‘*R*’, ‘*d*’ o ‘*D*’. El valor de esta llave *siempre* debe pasarse entre llaves ‘ $\{ \}$ ’. Por ejemplo: $\$ \backslash \text{sprad}[\text{sub-index}=\{1\}] \$$ imprimirá ‘ r_1 ’ en la salida PDF y NVDA/MathCAT verbalizará «*radio uno*».

`read-sub = {⟨true | false⟩}`

default: *false*

Establece si se *antepone* la palabra «*sub*» al verbalizar el subíndice establecido por la llave `sub-index` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, `\sprad[sub-index={1}, read-sub=true]` se verbalizará «*radio sub uno*» en NVDA/MathCAT; si se establece en *false*, NVDA/MathCAT verbalizará «*radio uno*».

`read-arg = {⟨verbalización NVDA⟩}`

default: *no usado*

Sobrescribe el *concepto* establecido por la llave `concept` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`. Por ejemplo: `\sprad[read-arg={medida exacta}]` imprimirá 'r' en la salida PDF y NVDA/MathCAT verbalizará «*radio medida exacta*»; `\sprad[concept=false, read-arg={medida exacta}]` imprimirá 'r' y NVDA/MathCAT verbalizará «*medida exacta*».

- Las llaves `sub-index` y `read-sub` no tienen efecto al usar `read-arg`.

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del *concepto* establecido por la llave `concept` y de las letras 'r', 'R', 'd' o 'D' al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spcirc`

<code>\spcirc</code>	<code>\spcirc</code>	<code>\spcirc[⟨key = val⟩]</code>
<code>\spcirc(⟨centro⟩)</code>	<code>\spcirc(⟨centro⟩)</code>	<code>\spcirc[⟨key = val⟩](⟨centro⟩)</code>
<code>\spcirc(⟨centro, radio⟩)</code>	<code>\spcirc(⟨centro, radio⟩)</code>	<code>\spcirc[⟨key = val⟩](⟨centro, radio⟩)</code>

El comando `\spcirc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma `(⟨centro⟩)` o `(⟨centro, radio⟩)`, donde `⟨radio⟩` es opcional y debe pasarse junto al argumento `(⟨centro⟩)` separado por una coma ',' obligatoria.

La forma en la que se procesa el `⟨radio⟩` junto a su impresión y verbalización se establece de manera interna: Si es un `⟨número⟩` entero o decimal se usa `\spnum`; si es una `⟨medida⟩` se usa `\spunit`; si es un segmento `⟨punto punto⟩` se usa `\spside` o `\spmside` acorde al valor establecido por la llave `radio-sty`; y si es una letra se imprime tal cual. Cualquier otra forma devuelve un "error" y se detiene la compilación del documento.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]` ni el *argumento opcional delimitado por paréntesis*, `\spcirc` imprimirá '⊙' en la salida PDF y NVDA/MathCAT verbalizará «*circunferencia*».

Si se proporciona el *argumento* `(⟨centro⟩)`, `\spcirc(0)` imprimirá '⊙(O)' en la salida PDF y NVDA/MathCAT verbalizará «*circunferencia de centro en O*».

Si se proporciona el *argumento* `(⟨centro, radio⟩)`, `\spcirc(0,3)` imprimirá '⊙(O,3)' en la salida PDF y NVDA/MathCAT verbalizará «*circunferencia de centro en O y radio tres*».

- El argumento `(⟨centro⟩)` y el argumento `⟨radio⟩` cuando tiene la forma `⟨punto punto⟩` aceptan prima ' ' o subíndice `_{...}`.

LLaves para `\spcirc`

El comando `\spcirc` posee las *meta-keys* `spside` pasada al comando `\spside` (v. pág. 21) y `spmside` pasada al comando `\spmside` (v. pág. 22).

`read-cmd = {⟨school | mathcat⟩}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `(⟨centro⟩)`. Si se establece en *school*, NVDA/MathCAT verbaliza el argumento `(⟨centro⟩)` sin el prefijo «mayúscula»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

- Esta llave tiene efecto sobre el argumento `⟨radio⟩` cuando tiene la forma `⟨punto punto⟩`.

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «*concepto*» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`print-sym = {⟨usual | odot⟩}`

default: *odot*

Establece si se *usará* `\bigodot` o la letra 'C' en la impresión del comando sin modificar la verbalización de este. Si se establece en *usual*, `\spcirc[print-sym=usual](0)` imprimirá 'C(O)' en la salida PDF; si se establece en *odot*, `\spcirc[print-sym=odot](0)` imprimirá '⊙(O)' en la salida PDF.

`read-sty = {⟨circun | circle⟩}`

default: *circun*

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se establece en *circun*, `\spcirc[read-sty=circun](0)` se verbalizará «*circunferencia de centro en O*» en NVDA/MathCAT; si se establece en *circle*, `\spcirc[read-sty=circle](0)` se verbalizará «*círculo de centro en O*» en NVDA/MathCAT.

`diameter = {⟨true | false⟩}`

default: *false*

Establece *como* se verbaliza el argumento *(radio)* al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, `\spc[radio=true](O, 3)` se verbalizará «circunferencia de centro en *O* y diámetro tres» en NVDA/MathCAT; si se establece en `false`, `\spc[radio=false](O, 3)` se verbalizará «circunferencia de centro en *O* y radio tres» en NVDA/MathCAT.

`radio-sty = { (side | mside) }`

default: *mside*

Establece el *comando* que procesa internamente el argumento *(radio)* cuando tiene la forma *(punto punto)*. Si se establece en *side* se usará `\spside{(punto punto)}`, `\spc[radio-sty=side](O, AB)` imprimirá $\odot(O, \overline{AB})$ en la salida PDF y NVDA/MathCAT verbalizará «circunferencia de centro en *O* y radio lado *A B*»; si se establece en *mside* se usará `\spmside{(punto punto)}`, `\spc[radio-sty=mside](O, AB)` imprimirá $\odot(O, AB)$ en la salida PDF y NVDA/MathCAT verbalizará «circunferencia de centro en *O* y radio medida del lado *A B*».

- Las *(meta-keys)* `spside` y `spmside` permiten modificar la impresión y verbalización del argumento *(radio)* cuando tiene la forma *(punto punto)*.

`concept = { (true | false) }`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el «concepto» y el argumento *(centro)* o *(centro, radio)*; si se establece en `false`, NVDA/MathCAT solo verbalizará el argumento *(centro)* o *(centro, radio)* acorde al valor establecido por la llave `read-cmd`.

- Utilizar `concept=false` sin el argumento *(centro)* es equivalente a usar `silent=true`.

`sub-index = { (subíndice) }`

default: *no usado*

Establece un *subíndice* sobre `\bigodot` o la letra 'C' establecida por la llave `print-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spc[sub-index={1},radio-sty=mside](O, AB)` imprimirá $\odot_1(O, AB)$ en la salida PDF y NVDA/MathCAT verbalizará «circunferencia uno de centro en *O* y radio medida del lado *A B*».

`read-sub = { (true | false) }`

default: *false*

Establece si se *antepone* la palabra «sub» al verbalizar el subíndice establecido por la llave `sub-index` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, `\spc[sub-index={1},read-sub=true](O)` se verbalizará «circunferencia sub uno de centro en *O*»; si se establece en `false`, NVDA/MathCAT verbalizará «circunferencia uno de centro en *O*».

`read-arg = { (verbalización NVDA) }`

default: *no usado*

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spc[read-arg={disco}](O)` imprimirá $\odot(O)$ en la salida PDF y NVDA/MathCAT verbalizará «disco de centro en *O*».

`silent = { (true | false) }`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty`, del argumento *(centro)* o *(centro, radio)* pasados al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spAcirc`

<code>\spAcirc</code>	<code>\spAcirc</code>	<code>\spAcirc[(key = val)]</code>
	<code>\spAcirc((centro))</code>	<code>\spAcirc[(key = val)]((centro))</code>
	<code>\spAcirc(1 2 3 ... n)</code>	<code>\spAcirc[(key = val)](1 2 3 ... n)</code>

El comando `\spAcirc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma *(centro)* o *(1 | 2 | 3 | ... | n)*, aplicando `\mathcal{A}` a la impresión.

Si no se proporciona el *argumento opcional* `[(key = val)]` ni el *argumento opcional delimitado por paréntesis*, `\spAcirc` imprimirá ' $\mathcal{A}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia».

Si se proporciona el *argumento* *(centro)*, `\spAcirc(O)` imprimirá ' $\mathcal{A}_{\odot(O)}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia con centro en *O*».

Si se proporciona el *argumento* *(1 | 2 | 3 | ... | n)*, `\spAcirc(1)` imprimirá ' $\mathcal{A}_{\odot_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia uno».

- El *argumento* *(centro)* acepta prima '' o subíndice '_{...}' y no acepta un *(radio)*. Una coma ',' en el argumento siempre produce un "error".

Llaves para `\spAcirc`

`read-cmd` = { `<school | mathcat>` }

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento (`<centro>`). Si se establece en *school*, NVDA/MathCAT verbaliza el argumento (`<centro>`) sin el prefijo «mayúscula»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` }

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «área de ...» seguido por el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`read-sty` = { `<circun | circle>` }

default: *circun*

Establece el *concepto* que verbalizará NVDA/MathCAT después de verbalizar el término «área de ...» al ejecutar el comando sin modificar la impresión de este. Si se establece en *circun*, `\spAcirc[read-sty=circun](0)` se verbalizará «área de la circunferencia con centro en O» en NVDA/MathCAT; si se establece en *circle*, `\spAcirc[read-sty=circle](0)` se verbalizará «área del círculo con centro en O» en NVDA/MathCAT.

`concept` = { `<true | false>` }

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área de ...» y del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el término, el concepto y el argumento (`<centro>`) o (`<1 | 2 | 3 | ... | n>`); si se establece en *false*, NVDA/MathCAT solo verbalizará el argumento (`<centro>`) acorde al valor establecido por la llave `read-cmd`.

- Utilizar `concept=false` sin el argumento (`<centro>`) o con el argumento (`<1 | 2 | 3 | ... | n>`) es equivalente a usar `silent=true`.

`read-arg` = { `<verbalización NVDA>` }

default: *no usado*

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`. Por ejemplo: `\spAcirc[read-arg={esfera}](0)` se verbalizará «área de la esfera con centro en O» en NVDA/MathCAT; `\spAcirc[concept=false,read-arg={esfera}](0)` se verbalizará «esfera con centro en O» en NVDA/MathCAT.

- Utilizar esta llave con `concept=false` y el argumento (`<1 | 2 | 3 | ... | n>`) es equivalente a usar `silent=true`.

`print-sym` = { `<usual | odot>` }

default: *odot*

Establece si se *usará* `\bigodot` o la letra 'C' en la impresión del comando sin modificar la verbalización de este. Si se establece en *usual*, `\spAcirc[print-sym=usual](0)` imprimirá ' $\mathcal{A}_{C(O)}$ ' en la salida PDF; si se establece en *odot*, `\spAcirc[print-sym=odot](0)` imprimirá ' $\mathcal{A}_{\odot(O)}$ ' en la salida PDF.

`sub-index` = { `<subíndice>` }

default: *no usado*

Establece un *subíndice* sobre `\bigodot` o la letra 'C' establecida por la llave `print-sym` cuando se utiliza el argumento (`<centro>`). El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spAcirc[sub-index={1}](0)` imprimirá ' $\mathcal{A}_{\odot_1(O)}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia uno con centro en O».

`read-sub` = { `<true | false>` }

default: *false*

Establece si se *antepone* la palabra «sub» al verbalizar el subíndice establecida por la llave `sub-index` o al usar el argumento (`<1 | 2 | 3 | ... | n>`) al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, `\spAcirc[read-sub=true](1)` se verbalizará «área de la circunferencia sub uno» en NVDA/MathCAT; si se establece en *false*, NVDA/MathCAT verbalizará «área de la circunferencia uno».

`silent` = { `<true | false>` }

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área de ...», del concepto establecido por la llave `read-sty` y del argumento (`<centro>`) o (`<1 | 2 | 3 | ... | n>`) pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPcirc`

`\spPcirc` `\spPcirc`

`\spPcirc(<centro>)`

`\spPcirc(<1 | 2 | 3 | ... | n>)`

`\spPcirc[<key = val>]`

`\spPcirc[<key = val>](<centro>)`

`\spPcirc[<key = val>](<1 | 2 | 3 | ... | n>)`

El comando `\spPcirc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (`<centro>`) o (`<1 | 2 | 3 | ... | n>`), aplicando `\mathcal{P}` a la impresión.

Si no se proporciona el *argumento opcional* [`<key = val>`] ni el *argumento opcional delimitado por paréntesis*, `\spPcirc` imprimirá ' $\mathcal{P}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la circunferencia».

Si se proporciona el argumento ($\langle centro \rangle$), $\$ \backslash spPcirc(0) \$$ imprimirá ' $\mathcal{P}_{O(0)}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la circunferencia con centro en O».

Si se proporciona el argumento ($\langle 1 | 2 | 3 | \dots | n \rangle$), $\$ \backslash spPcirc(1) \$$ imprimirá ' $\mathcal{P}_{O_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la circunferencia uno».

- El argumento ($\langle centro \rangle$) acepta prima ' ' o subíndice '_{...}' y no acepta un $\langle radio \rangle$. Una coma ',' en el argumento siempre produce un "error".

Llaves para $\backslash spPcirc$

`read-cmd = {  $\langle school | mathcat \rangle$  }` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento ($\langle centro \rangle$). Si se establece en *school*, NVDA/MathCAT verbaliza el argumento ($\langle centro \rangle$) sin el prefijo «mayúscula»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {  $\langle prefijo \rangle$  }` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «perímetro de ...» seguido por el «concepto» establecido por la llave *read-sty*. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`read-sty = {  $\langle circun | circle \rangle$  }` default: *circun*

Establece el *concepto* que verbalizará NVDA/MathCAT después de verbalizar el término «perímetro de ...» al ejecutar el comando sin modificar la impresión de este. Si se establece en *circun*, $\$ \backslash spPcirc[read-sty=circun](0) \$$ se verbalizará «perímetro de la circunferencia con centro en O» en NVDA/MathCAT; si se establece en *circle*, $\$ \backslash spPcirc[read-sty=circle](0) \$$ se verbalizará «perímetro del círculo con centro en O» en NVDA/MathCAT.

`concept = {  $\langle true | false \rangle$  }` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «perímetro de ...» y del «concepto» establecido por la llave *read-sty* al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el término, el concepto y el argumento ($\langle centro \rangle$) o ($\langle 1 | 2 | 3 | \dots | n \rangle$); si se establece en *false*, NVDA/MathCAT solo verbalizará el argumento ($\langle centro \rangle$) acorde al valor establecido por la llave *read-cmd*.

- Utilizar *concept=false* sin el argumento ($\langle centro \rangle$) o con el argumento ($\langle 1 | 2 | 3 | \dots | n \rangle$) es equivalente a usar *silent=true*.

`read-arg = {  $\langle verbalización NVDA \rangle$  }` default: *no usado*

Sobrescribe el *concepto* establecido por la llave *read-sty* que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave *concept*. Por ejemplo: $\$ \backslash spPcirc[read-sty=circle, read-arg=\{disco\}](0) \$$ se verbalizará «perímetro del disco con centro en O» en NVDA/MathCAT; $\$ \backslash spPcirc[concept=false, read-arg=\{disco\}](0) \$$ se verbalizará «disco con centro en O» en NVDA/MathCAT.

- Utilizar esta llave con *concept=false* y el argumento ($\langle 1 | 2 | 3 | \dots | n \rangle$) es equivalente a usar *silent=true*.

`print-sym = {  $\langle usual | odot \rangle$  }` default: *odot*

Establece si se *usará* $\backslash bigodot$ o la letra 'C' en la impresión del comando sin modificar la verbalización de este. Si se establece en *usual*, $\$ \backslash spPcirc[print-sym=usual](0) \$$ imprimirá ' $\mathcal{P}_{C(0)}$ ' en la salida PDF; si se establece en *odot*, $\$ \backslash spPcirc[print-sym=odot](0) \$$ imprimirá ' $\mathcal{P}_{O(0)}$ ' en la salida PDF.

`sub-index = {  $\langle subíndice \rangle$  }` default: *no usado*

Establece un *subíndice* sobre $\backslash bigodot$ o la letra 'C' establecida por la llave *print-sym* cuando se utiliza el argumento ($\langle centro \rangle$). El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: $\$ \backslash spPcirc[sub-index=\{1\}](0) \$$ imprimirá ' $\mathcal{P}_{O_1(0)}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la circunferencia uno con centro en O».

`read-sub = {  $\langle true | false \rangle$  }` default: *false*

Establece si se *antepone* la palabra «sub» al verbalizar el subíndice establecida por la llave *sub-index* o al usar el argumento ($\langle 1 | 2 | 3 | \dots | n \rangle$) al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, $\$ \backslash spPcirc[read-sub=true](1) \$$ se verbalizará «perímetro de la circunferencia sub uno» en NVDA/MathCAT; si se establece en *false*, NVDA/MathCAT verbalizará «perímetro de la circunferencia uno».

`silent = {  $\langle true | false \rangle$  }` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «perímetro de ...», del concepto establecido por la llave *read-sty* y del argumento ($\langle centro \rangle$) o ($\langle 1 | 2 | 3 | \dots | n \rangle$) pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spVol`

`\spVol` `\spVol` `\spVol[⟨key = val⟩]`

El comando `\spVol` trabaja en *modo matemático* y recibe el *argumento opcional* `[⟨key = val⟩]` mediante el cual se establece la «*forma*» en la que se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spVol` imprimirá ‘ $\mathcal{V}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*volumen*».

Llaves para `\spVol`

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*volumen*» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, `\spVol` se verbaliza como «*volumen*» en NVDA/MathCAT; si se establece en *false*, se verbalizará la letra ‘ $\mathcal{V}$ ’ acorde al valor establecido por la llave `read-cmd`.

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará la letra ‘ $\mathcal{V}$ ’ cuando `concept=false`. Si se establece en *school*, NVDA/MathCAT verbalizará únicamente la letra ‘*V*’; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas sobre ‘ $\mathcal{V}$ ’.

`sub-index = {⟨subíndice⟩}` default: *no usado*

Establece un *subíndice* sobre ‘ $\mathcal{V}$ ’. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’. Por ejemplo: `\spVol[sub-index={1}]` imprimirá ‘ $\mathcal{V}_1$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*volumen uno*».

`read-sub = {⟨true | false⟩}` default: *false*

Establece si se *antepone* la palabra «*sub*» al verbalizar el subíndice establecido por la llave `sub-index` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, `\spVol[sub-index={1}, read-sub=true]` se verbalizará «*volumen sub uno*» en NVDA/MathCAT; si se establece en *false*, NVDA/MathCAT verbalizará «*volumen uno*».

`read-arg = {⟨verbalización NVDA⟩}` default: *no usado*

Sobrescribe el *concepto* establecido por la llave `concept` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’ y se debe usar en conjunto con la llave `concept`. Por ejemplo: `\spVol[read-arg={medida exacta}]` imprimirá ‘ $\mathcal{V}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*volumen medida exacta*»; `\spVol[concept=false, read-arg={medida exacta}]` imprimirá ‘ $\mathcal{V}$ ’ y NVDA/MathCAT verbalizará «*medida exacta*».

🔊 Las llaves `sub-index` y `read-sub` no tienen efecto al usar `read-arg`.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del *concepto* establecido por la llave `concept` y de ‘ $\mathcal{V}$ ’ al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

8 Referencias

- [1] ROBERTSON, WILL . “unicode-math - Unicode mathematics support for X_YTeX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/unicode-math>, 2023.
- [2] KRUEGER, MARCEL. “LuaMML - Automatically generate MathML from LuaTeX math mode material”. Available from CTAN, <https://ctan.org/pkg/luamml>, 2026.
- [3] The L^AT_EX Project. “fontspec - Advanced font selection for X_YL^AT_EX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/fontspec>, 2025.
- [4] VOß, HERBERT. “libertinus-otf - Support for Libertinus OpenType”. Available from CTAN, <https://www.ctan.org/pkg/libertinus-otf>, 2025.
- [5] FLIPO, DANIEL. “erewhon-math - Utopia based OpenType Math font”. Available from CTAN, <https://ctan.org/pkg/erewhon-math>, 2026.
- [6] HOFSTRA, SILKE. “sourcecodepro - Use SourceCodePro with T_EX(-alike) systems”. Available from CTAN, <https://ctan.org/pkg/sourcecodepro>, 2025.
- [7] GONZÁLEZ, PABLO. “scontents - Stores L^AT_EX contents in memory or files”. Available from CTAN, <https://www.ctan.org/pkg/scontents>, 2026.
- [8] GONZÁLEZ, PABLO. “enumext - Enumerate exercise sheets”. Available from CTAN, <https://www.ctan.org/pkg/enumext>, 2026.
- [9] BRAAMS, JOHANNES AND BEZOS, JAVIER. “babel - Multilingual support for L^AT_EX, LuaL^AT_EX, X_YL^AT_EX, and Plain T_EX”. Available from CTAN, <https://www.ctan.org/pkg/babel>, 2026.
- [10] BEZOS, JAVIER. “babel-spanish - Babel support for Spanish”. Available from CTAN, <https://www.ctan.org/pkg/babel-spanish>, 2026.
- [11] NV ACCESS. “NVDA - NonVisual Desktop Access screen reader”. Available from NV ACCESS, <https://www.nvaccess.org/>, 2026.
- [12] SOIFFER, NEIL. “MathCAT - Math to speech and braille translation”. Available from GITHUB, <https://nsoiffer.github.io/MathCAT/>, 2026.
- [13] ADOBE INC. “Adobe Acrobat Reader - Reliable PDF viewer”. Available from ADOBE, <https://get.adobe.com/reader/>, 2026.
- [14] ISO. “ISO 32000-2:2020 - Portable Document Format (PDF 2.0)”. Available from PDF ASSOCIATION, <https://pdfa.org/resource/iso-32000-2/>, 2020.
- [15] PDF ASSOCIATION. “WTPDF - Well-Tagged PDF specification”. Available from PDFA, <https://pdfa.org/wtpdf/>, 2024.
- [16] PDF ASSOCIATION. “PDF 2.0 Errata Collection 3”. Available from PDFA, <https://pdfa.org/PDF-2-0-errata-collection-3-now-available/>, 2026.
- [17] L^AT_EX PROJECT LWG. “Best Practice Guide: Math in PDF”. Available from PDFA, <https://pdfa.org/resource/best-practice-guide-math-in-PDF/>, 2026.
- [18] JOHNSON, DUFF. “PDF 2.0: New Features, Real-World Impact”. Available from PDFA, <https://pdfa.org/PDF-2-0-new-features-real-world-impact/>, 2026.
- [19] The L^AT_EX Project. “LaTeX Tagged PDF Project - Documentation and resources”. Available from L^AT_EX PROJECT, <https://latex3.github.io/tagging-project/>, 2026.
- [20] VERAPDF CONSORTIUM. “veraPDF - Industry supported PDF/A and PDF/UA validator”. Available from VERAPDF, <https://verapdf.org/>, 2026.
- [21] DUAL LAB. “ngPDF - Next Generation PDF demonstrator and validator”. Available from NGPDF, <https://ngpdf.com/>, 2026.
- [22] BERRY, KARL. “L^AT_EX 2_ε: An Unofficial Reference Manual”. Available from CTAN, <https://ctan.org/pkg/latex2e-help-texinfo>, 2026.
- [23] The L^AT_EX Project. “Extensive support for hypertext in L^AT_EX”. Available from CTAN, <https://www.ctan.org/pkg/hyperref>, 2026.
- [24] The L^AT_EX Project. “The expl3 package”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.

- [25] The $\text{\TeX}$ Project. “The $\text{\TeX}$ ₃ Interfaces”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.
- [26] The $\text{\TeX}$ Project. “The $\text{\TeX}$ _{2 ϵ} sources”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2026.
- [27] The $\text{\TeX}$ Project. “ $\text{\TeX}$ News, Issue 41, June 2025”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2025.
- [28] The $\text{\TeX}$ Project. “ $\text{\TeX}$ for authors current version”. Available from CTAN, <https://ctan.org/pkg/latex-base>, 2026.
- [29] FISCHER, ULRIKE. “tagpdf – $\text{\TeX}$ kernel code for PDF tagging”. Available from CTAN, <https://www.ctan.org/pkg/tagpdf>, 2026.
- [30] The $\text{\TeX}$ Project. “latex-lab – $\text{\TeX}$ laboratory”. Available from CTAN, <https://www.ctan.org/pkg/latex-lab>, 2026.

9 Change history

v0.99 (ctan), 2026-10-05 – First public release.

10 Índice de la Documentación

Los números en cursiva indican las páginas donde se describe la entrada correspondiente.

C	
Document class:	
article	1
book	1
letter	1
report	1
Commands provide by spintent :	
\setspintent	3
\spAcirc	31
\spAfig	26
\spPcirc	32
\spPfig	28
\spPoint	19
\spPoly	26
\spProp	18
\spVol	34
\spangle	23
\spang	24
\sparc	25
\spcirc	30
\spcoord	19
\spdate	11
\spdiam	29
\spdiv	13
\spdsym	12
\spmQ	17
\spmangle	24
\spmarc	25
\spmcd	15
\spmcm	16
\spmoney	6
\spmside	22
\spmsym	13
\spmul	14
\spnD	14
\spnM	15
\spnZ	17
\spnewread	11
\spnewunit	6
\spnum	3
\sprad	29
\sprayo	21
\sprazon	18
\spread	12
\sprecta	20
\spshort	8
\spside	21
\spsiglo	10
\sptime	11
\spunitalias	6
\spunit	4
\spusep	12
\spvec	20
Keys for \spAcirc provide by spintent :	
concept	32
prefix	32
print-sym	32
read-arg	32
read-cmd	32
read-sty	32
read-sub	32
silent	32
sub-index	32
Keys for \spAfig provide by spintent :	
concept*	27
concept	27
prefix	27
print-sym	27
read-arg	27
read-cmd	27
read-sub	27
silent	27
Keys for \spangle provide by spintent :	
concept	23
prefix	23
read-cmd	23
recto	23
silent	23
Keys for \sparc provide by spintent :	
concept	25
prefix	25
read-cmd	25
silent	25
Keys for \spcirc provide by spintent :	
concept	31
diameter	30
prefix	30
print-sym	30
radio-sty	31
read-arg	31
read-cmd	30
read-sty	30
read-sub	31
silent	31
spmside	30, 31
spside	30, 31
sub-index	31
Keys for \spcoord provide by spintent :	
silent	19
Keys for \spdiam provide by spintent :	
concept	29
print-sym	29
read-arg	30
read-cmd	29
read-sub	30
silent	30
sub-index	29
upper-case	29
Keys for \spdiv provide by spintent :	
cifras-sep	14
notation-sym	14
read-parens	14
read-period-sep	14
read-sign	14
read-sym	14
set-sym	14
wrap-parens	14
Keys for \spdsym provide by spintent :	
prefix	13
read-sym	13

set-sym	13	result-num	14
suffix	13	result	14
Keys for \spmangle provide by spintent :		silent	15
concept	24	Keys for \spnM provide by spintent :	
prefix	24	concept	15
print-m	24	prefix	15
read-cmd	24	result-num	15
recto	24	result	15
silent	24	silent	15
Keys for \spmarc provide by spintent :		Keys for \spnum provide by spintent :	
concept	25	cifras-sep	3
prefix	25	notation-sym	4
read-cmd	25	read-period-sep	3
silent	25	read-sign	3
Keys for \spmcd provide by spintent :		Keys for \spnZ provide by spintent :	
concept	16	cifras-sep	17
prefix	16	read-parens	17
read-cmd	16	read-sign	17
result	16	wrap-parens	17
silent	16	Keys for \spPcirc provide by spintent :	
upper-case	16	concept	33
Keys for \spmcm provide by spintent :		prefix	33
concept	16	print-sym	33
prefix	16	read-arg	33
read-cmd	16	read-cmd	33
result	16	read-sty	33
silent	17	read-sub	33
upper-case	16	silent	33
Keys for \spmoney provide by spintent :		sub-index	33
cifras-sep	7	Keys for \spPfig provide by spintent :	
currency	7	concept*	28
money-math	7	concept	28
sub-units	7	prefix	28
Keys for \spmQ provide by spintent :		print-sym	29
join-word	17	read-arg	28
read-parens	18	read-cmd	28
use-dfrac	17	read-sub	28
use-spN	17	silent	29
wrap-parens	17, 18	Keys for \spPoint provide by spintent :	
Keys for \spmside provide by spintent :		concept	19
concept	22	prefix	19
prefix	22	read-cmd	19
print-m	23	silent	19
read-arg	23	Keys for \spPoly provide by spintent :	
read-cmd	22	concept	26
read-sty	22	prefix	26
silent	23	print-sym	26
Keys for \spmsym provide by spintent :		read-cmd	26
not-print	13	silent	26
prefix	13	Keys for \spProp provide by spintent :	
read-sym	13	inline	18
set-sym	13	silent	18
suffix	13	use-dfrac	18
Keys for \spmul provide by spintent :		Keys for \sprad provide by spintent :	
cifras-sep	14	concept	29
notation-sym	14	read-arg	30
read-parens	14	read-cmd	29
read-period-sep	14	read-sub	30
read-sign	14	silent	30
read-sym	14	sub-index	29
set-sym	14	upper-case	29
wrap-parens	14	Keys for \spray provide by spintent :	
Keys for \spnD provide by spintent :		concept	21
concept	14	prefix	21
prefix	14	read-cmd	21

read-sty	21	Keys for \spvec provide by spintent :	
silent	21	autofit	20
Keys for \sprazon provide by spintent :		concept	20
inline	18	prefix	20
silent	18	read-cmd	20
use-dfrac	18	silent	20
Keys for \spread provide by spintent :		Keys for \spVol provide by spintent :	
read-arg	12	concept	34
silent	12	read-arg	34
Keys for \sprecta provide by spintent :		read-cmd	34
concept	21	read-sub	34
prefix	20	silent	34
read-cmd	20	sub-index	34
silent	21		
Keys for \spshort provide by spintent :			
read-arg	8		
small-caps	8		
Keys for \spside provide by spintent :			
concept	22		
prefix	22		
read-arg	22		
read-cmd	22		
read-sty	22		
silent	22		
Keys for \spsiglo provide by spintent :			
print-era	11		
Keys for \spunit provide by spintent :			
cifras-sep	4		
notation-sym	4		
print-cdot	4		
read-cdot	4		
read-period-sep	4		
read-slash	4		
Keys for \spusep provide by spintent :			
silent	12		
size	12		

M

Lua module provide by spintent :	
spintent.lua	6-11

P

Packages :	
babel-spanish	2
babel	2, 8
documentmetadata-support	1
enumext	1
expl3	3
fontspec	2
fourier-otf	2
hyperref	2
l3keys	3
latex-lab-mathintent	1
libertinus-otf	2
luamml	1
scontents	1
sourcecodepro	2
tagpdf	1, 8, 11
unicode-math	1

11 Implementation

The most recent publicly released version of `spintent` is available at CTAN: <https://www.ctan.org/pkg/spintent>. While general feedback via email is welcomed, specific bugs or feature requests should be reported through the issue tracker: <https://github.com/pablgonz/spintent/issues>.

- The documentation presented here is far from professional, it contains a lot of obvious information that to the eye of a TeXpert are superfluous, but, after so many years developing this project is the only way to remember what does what.

11.1 General conventions

Functions of the form `\__spintent_luafun_some:n` or `\__spintent_luafun_some:nn` are defined (created) in the Lua module `spintent.lua` and only their variants are declared in `expl3`.

Similarly, variables of the form `\l__spintent_luaset_some_tl` or `\l__spintent_luaset_some_str` are defined in the Lua module `spintent.lua` and only declared in `expl3`.

- All variables and functions defined in this package are private and are NOT intended to work or be used by another package or module.

11.2 Initial set up

Start the DocStrip guards.

```
1 (*package)
```

Identify the internal prefix (L^AT_EX3 DocStrip convention) for `l3doc` class.

```
2 (@@=spintent)
```

11.3 Declaration of the package

First we will make sure we have a minimum (super updated) release version of L^AT_EX to work correctly, otherwise, we will return an “error”.

```
3 \NeedsTeXFormat{LaTeX2e}[2026-11-01]
4 \IfFormatAtLeastF { 2026-11-01 }
5 {
6   \PackageError { spintent }
7     { LaTeX~kernel~too~old~need~2026-11-01~or~later }
8     {
9       spintent~requires~a~LaTeX2e~release~2026-11-01~or~
10      later.~Update~your~TeX~distribution.
11     }
12 }
```

Now declare the `spintent` package and add only two keys: `mathcal` and `AFghost`, which can be passed as package options.

```
13 \ProvidesExplPackage {spintent} {2026-10-05} {0.99} {Spanish parse intents}
14 \keys_define:nn { spintent }
15 {
16   mathcal .usage:n    = load,
17   mathcal .bool_set:N = \l__spintent_mathcal_bool,
18   mathcal .initial:n  = true,
19   mathcal .value_required:n = true,
20   AFghost .usage:n    = load,
21   AFghost .bool_set:N = \l__spintent_debug_ghost_bool,
22   AFghost .initial:n  = false,
23   AFghost .value_required:n = true,
24 }
25 \ProcessKeyOptions [ spintent ]
```

We check if the requirements for running the `spintent` package are met; that is: `\DocumentMetadata` must be active, Lua_LTeX must be running, and the `unicode-math` or `lua-unicode-math` package must be loaded. If neither is present, we will load `unicode-math`. If the conditions cannot be met, we will return a “fatal error”.

```
26 \IfDocumentMetadataTF
27 {
28   \sys_if_engine_luatex:TF
29   {
30     \msg_new:nnn { spintent } { package-not-load }
31     {
32       The~'#1'~package~will~be~loaded~as~a~dependency.
33     }
34   }
35   \IfPackageLoadedF { unicode-math }
36   {
37     \IfPackageLoadedF { lua-unicode-math }
38     {
39       \msg_info:nnn { spintent } { package-not-load } { unicode-math }
40       \RequirePackage{unicode-math}
41     }
42   }
43 }
```

```

40     }
41   }
42   \msg_new:nnn { spintent } { env-success }
43   { Everything~looks~correct,~spintent~will~make~the~intent! }
44   \msg_info:nn { spintent } { env-success }
45 }
46 {
47   \msg_new:nnnn { spintent } { fatal-engine-requires-luatex }
48   { The~spintent~package~requires~LuaLaTeX. }
49   { This~package~relies~on~Lua~module.~Compile~with~lualatex. }
50   \msg_fatal:nn { spintent } { fatal-engine-requires-luatex }
51 }
52 }
53 {
54   \msg_new:nnnn { spintent } { fatal-metadata-missing }
55   { \token_to_str:N \DocumentMetadata \c_space_tl is-missing. }
56   {
57     The~spintent~package~requires~tagged~PDF~active.\\
58     You~must~declare~\token_to_str:N \DocumentMetadata { tagging=on } ~
59   }
60   \msg_fatal:nn { spintent } { fatal-metadata-missing }
61 }

```

11.4 Some kernel variants

Non-standard kernel variants.

```

\str_if_eq:nVTF
\str_if_eq:nVF
\str_uppercase:V
\str_lowercase:V
\seq_set_from_clist:Ne

```

(End of definition for `\str_if_eq:nVTF` and others.)

11.5 Variables for the key `\mathcal`, `\rightanglesqr` and `\attribute`

First, we declare the string variable `\l__spintent_luaset_cal_fam_ok_str` used by the function `\__spintent_luafun_classic_mathcal:nn` (controlled by the key `mathcal`), the string variable `\l__spintent_luafun_right_angle_sqr:nn` used by the function `\__spintent_luafun_right_angle_sqr:nn` to copy `\rightanglesqr` alongside the string variable `\l__spintent_luaset_scope_attr_str` used by `\attribute` before loading the Lua module `spintent.lua`. This is necessary due to the implementation of the functions within the Lua module; otherwise, `expl3` would issue a warning regarding duplicate variable declarations.

```

67 \str_new:N \l__spintent_luaset_cal_fam_ok_str
68 \str_new:N \l__spintent_luaset_sym_fam_ok_str
69 \str_new:N \l__spintent_luaset_scope_attr_str

```

(End of definition for `\l__spintent_luaset_cal_fam_ok_str`, `\l__spintent_luaset_sym_fam_ok_str`, and `\l__spintent_luaset_scope_attr_str`.)

11.6 Loading the Lua module and functions

`spintent.lua` Loads the Lua module `spintent.lua`, where every `_luafun_` function used throughout the package is registered.

```

70 \lua_load_module:n { spintent.lua }

```

(End of definition for `spintent.lua`.)

`\__spintent_luafun_set_debug_ghost:n` Passes `AFghost` to Lua as a plain string. When `true`, both ghost characters are marked `sptmp="outer"` or `sptmp="inner"` in the file `-luamml-mathml.html`, so an external script can identify and strip them before generating the Associated~Files structure for `tagged PDF` — `math-AF` mode does not want either ghost present.

```

71 \bool_if:NTF \l__spintent_debug_ghost_bool
72 { \__spintent_luafun_set_debug_ghost:n { true } }
73 { \__spintent_luafun_set_debug_ghost:n { false } }

```

(End of definition for `\__spintent_luafun_set_debug_ghost:n`.)

`\l__spintent_scope_attr_int` A LuaTeX attribute number, allocated collision-free from Lua via `luatexbase.new_attribute` and exposed here as an integer. Every command that uses the shared function `\__spintent_luafun_mrow_intent_arg:nn` activates it, via `\attribute \l__spintent_scope_attr_int = 1`, inside its own group — marking every node typeset within as eligible for that queue to claim. Without this mark, the queue could claim content from an unrelated command sitting in the same formula.

```

74 \int_new:N \l__spintent_scope_attr_int
75 \int_set:Nn \l__spintent_scope_attr_int { \l__spintent_luaset_scope_attr_str }

```

(End of definition for `\__spintent_scope_attr_int`.)

`\__spintent_luafun_spunit_lookup_alias:n` Variants generated for `\__spintent_luafun_spunit_lookup_alias:n` (used by `\spunit`), `\__spintent_luafun_spmoney_lookup_data:n` (used by `\spmoney`), `\__spintent_luafun_spmQ_scale_int:nn` (used by `\spmQ`) and `\__spintent_luafun_clean_split_and_set:n` (used by `\spnum`).

```

76 \cs_generate_variant:Nn \__spintent_luafun_spunit_lookup_alias:n { V }
77 \cs_generate_variant:Nn \__spintent_luafun_spmoney_lookup_data:n { V }
78 \cs_generate_variant:Nn \__spintent_luafun_spmQ_scale_int:nn { Vn }
79 \cs_generate_variant:Nn \__spintent_luafun_clean_split_and_set:n { V }

```

(End of definition for `\__spintent_luafun_spunit_lookup_alias:n` and others.)

11.7 Internal implementation for `\mathcal` and `\rightanglesqr`

To simplify user configuration and obtain the expected output style pdf_{TeX} for `\mathcal`, we will provide a fallback to the font NewCMMath-Regular.otf, which contains the glyphs for these characters. Using this same font, we will obtain the symbol `\rightanglesqr` for the key `recto`.

`mathcal-fallback`
`right-angle-sqr-fallback`

First, we'll define two error messages in case our fallback fails.

```

80 \hook_gput_code:nnn { begindocument } { spintent }
81 {
82   \__spintent_luafun_init_math_fallbacks:
83 }
84 \msg_new:nnn { spintent } { mathcal-fallback }
85 {
86   Using~classic~\token_to_str:N \mathcal \c_space_tl as~fallback~
87   (font~not~found~or~no~free~math~family~available).~\msg_line_context:
88 }
89 \msg_new:nnn { spintent } { right-angle-sqr-fallback }
90 {
91   Using~\token_to_str:N \rightangle \c_space_tl as~fallback~
92   for~the~right-angle~ glyph.~\msg_line_context:
93 }

```

(End of definition for `mathcal-fallback` and `right-angle-sqr-fallback`.)

`\__spintent_mathcal:n`

Now we define the internal function `\__spintent_mathcal:n` for `\mathcal` which checks the state of the variable `\__spintent_mathcal_bool` set by the package key `mathcal`; if its state is true, we check the string variable `\__spintent_luaset_cal_fam_ok_str` and if the latter has the value true, we call the function `\__spintent_luafun_classic_mathcal:n` and otherwise (we cannot find the source in the system) we return a warning message and directly use `\mathcal`. Otherwise, that is, `mathcal=false` we simply use `\mathcal`.

```

94 \cs_new_protected:Nn \__spintent_mathcal:n
95 {
96   \bool_if:NTF \__spintent_mathcal_bool
97   {
98     \str_if_eq:VnTF \__spintent_luaset_cal_fam_ok_str { true }
99     { \__spintent_luafun_classic_mathcal:n {#1} }
100     {
101       \msg_warning:nn { spintent } { mathcal-fallback }
102       \mathcal {#1}
103     }
104   }
105   { \mathcal {#1} }
106 }

```

(End of definition for `\__spintent_mathcal:n`.)

`\__spintent_right_angle_sqr:`

The internal function `\__spintent_right_angle_sqr:` establishes the internal copy of the command `\rightanglesqr`. First, we check the string variable `\__spintent_luaset_sym_fam_ok_str`. If it is true, we call the function `\__spintent__luafun_right_angle_sqr:`; otherwise (we do not find the font in the system), we return a warning message and use `\rightangle` instead.

```

107 \cs_new_protected:Nn \__spintent_right_angle_sqr:
108 {
109   \str_if_eq:VnTF \__spintent_luaset_sym_fam_ok_str { true }
110   { \__spintent_luafun_right_angle_sqr: }
111   {
112     \msg_warning:nn { spintent } { right-angle-sqr-fallback }
113     \rightangle
114   }
115 }

```

(End of definition for `\__spintent_right_angle_sqr:`.)

11.8 Shared annotation mechanisms: the queue and the ghosts

Almost all commands in the `spintent` package share three annotation functions, the mechanism of which is defined in the Lua module `spintent.lua`.

`\__spintent_luafun_mrow_intent_arg:nn`

The internal function `\__spintent_luafun_mrow_intent_arg:nn` — called directly, with no `expl3` wrapper — pushes one entry onto a queue: a concept `intent` (`#1`, possibly empty) and a Lua table literal `arg=/intent=` (`#2`). A `pre_mlist_to_hlist_filter` callback (installed once, before `luamml[2]`’s own) walks the formula’s top-level noads in order and, for each one whose nucleus or subscript was typeset inside a `spintent` command’s own group (marked via a LuaTeX `\attribute` activated by every command that uses this queue), claims it for the next pending entry: `arg`/a literal `intent` land on that leaf, and, when the concept `intent` is non-empty, the leaf is wrapped in a native `<mrow>` carrying it — no token, no ghost involved. Entries are consumed strictly in order; any left unclaimed when the formula ends are discarded rather than carried into the next one. A native `\MathMLarg/\MathMLintent` claim is invisible to this queue: mixing the two for the same content risks the queue claiming something already tagged natively, or never being claimed at all.

(End of definition for `\__spintent_luafun_mrow_intent_arg:nn`.)

The other two are “ghost” characters (an invisible U+2063, self-tagged `:silent` through their own `mathml_filter` — so the queue above never mistakes either for one of its own free targets) dropped in as an extra sibling so the resulting `<mrow>` is not the formula’s sole top-level content, which `luamml` would otherwise promote straight to `<math intent=...>` instead of leaving it nested (issue #26).

`\__spintent_invisible_sep:`

Placed once, before opening the formula’s own outer `\MathMLintent` — protects the *whole formula* from collapsing when that outer group is its only top-level content. Fires at most once per document: past the first call, in any command, it does nothing, since one sibling anywhere in the formula is already enough.

```
116 \cs_new_protected:Nn \__spintent_invisible_sep:
117 {
118   \__spintent_luafun_invisible_sep:
119 }
```

Executing this function writes the following line directly to the output file `-luamml-mathml.html`:

```
<mi intent=":silent" mathvariant="normal"></mi>
```

and if the package key `AFghost=true` is used, the output is:

```
<mi intent=":silent" mathvariant="normal" sptmp="outer"></mi>
```

If you wish to use math-AF mode, you can remove the line generated by `AFghost=true` using the Lua script `spiafcls`.

(End of definition for `\__spintent_invisible_sep:`.)

`\__spintent_inner_sep:`

Placed just inside a native `\MathMLintent` group — protects *that specific group* when it too would otherwise hold only one child. Never deduplicates: called once per such group, however many appear, nested or sibling, in the same formula.

```
120 \cs_new_protected:Nn \__spintent_inner_sep:
121 {
122   \__spintent_luafun_inner_sep:
123 }
```

Executing this function writes the following line directly to the output file `-luamml-mathml.html`:

```
<mi intent=":silent" mathvariant="normal"></mi>
```

and if the package key `AFghost=true` is used, the output is:

```
<mi intent=":silent" mathvariant="normal" sptmp="inner"></mi>
```

If you wish to use math-AF mode, you can remove the line generated by `AFghost=true` using the Lua script `spiafcls`.

(End of definition for `\__spintent_inner_sep:`.)

`\c__spintent_invisible_sep_int`

A raw, untagged copy of the same U+2063 character, via the LuaTeX primitive `\Umathchardef` — for use as ordinary content *inside* an explicit `\MathMLintent{<intent>}` call (typically `:silent`), not as a sibling on its own.

```
124 \hook_gput_code:nnn { begindocument } { spintent }
125 {
126   \Umathchardef \c__spintent_invisible_sep_int = "0 "0 "2063
127 }
```

Documented in `source3` on page 887 for `\int_const:Nn`: “As stated, most constants can be defined as `\chardef` or `\mathchardef` but that’s engine dependent.” —so this appears to be the correct option here.

(End of definition for `\c__spintent_invisible_sep_int`.)

11.9 Normalize argument and affixes

We normalize the $\{\langle argument \rangle\}$ to support affixes (prefixes/suffixes) by trimming leading and trailing spaces, converting internal spaces to hyphens, collapsing consecutive hyphens, and removing leading or trailing hyphens.

`\__spintent_collapse_hyphens:N` A recursive helper to collapse multiple hyphens into a single one.

```

128 \cs_new_protected:Nn \__spintent_collapse_hyphens:N
129 {
130   \tl_if_in:NnT #1 { -- }
131   {
132     \tl_replace_all:Nnn #1 { -- } { - }
133     \__spintent_collapse_hyphens:N #1
134   }
135 }
```

(End of definition for `\__spintent_collapse_hyphens:N`.)

`\__spintent_sanitize_affix:Nn`
`\__spintent_sanitize_affix:cn`

The function `\__spintent_sanitize_affix:Nn` checks if the $\{\langle argument \rangle\}$ passed in ‘#2’ is *blank*. If so, it clears the target token list variable passed in ‘#1’. Otherwise, it converts the $\{\langle argument \rangle\}$ into a string (so every character has category code 12 and it no longer matters how an underscore ‘_’ was typed), trims the bounds, converts spaces ‘ ’ and underscores ‘_’ to hyphens ‘-’, collapses multiple hyphens, and safely strips leading/trailing hyphens. Because underscores are converted *before* collapsing, any run mixing hyphens and underscores is reduced to a single hyphen, and the stored value never starts or ends with ‘-’ or ‘_’: the command that uses the value is the one that places the leading ‘_’ of the intent name. For example, if you pass a target variable and raw text:

```
\__spintent_sanitize_affix:Nn \__spintent_spsomesym_prefix_tl{ foo bar baz }
```

The function will “store” the normalized string `foo-bar-baz` inside `\__spintent_spsomesym_prefix_tl`, ready to be conditionally appended. In the same way, `{ __foo__bar__ }` is stored as `foo-bar`. The only exception is the exact value ‘_-’ (once the bounds are trimmed), which is stored as a single hyphen ‘-’. After the command prepends its own leading ‘_’, this yields the *silent* literal ‘_-’ documented for `\spread`. Any other value made only of hyphens, underscores and spaces is stored as *empty*.

```

136 \cs_new_protected:Nn \__spintent_sanitize_affix:Nn
137 {
138   \tl_if_blank:nTF {#2}
139   { \tl_clear:N #1 }
140   {
141     \tl_set:Ne #1 { \tl_to_str:e {#2} }
142     \tl_trim_spaces:N #1
143     \str_if_eq:VnTF #1 { _- }
144     { \tl_set:Nn #1 { - } }
145     {
146       \tl_replace_all:Nnn #1 { ~ } { - }
147       \tl_replace_all:Nee #1 { \tl_to_str:n { _ } } { - }
148       \__spintent_collapse_hyphens:N #1
149       \tl_put_left:Nn #1 { \q_mark }
150       \tl_put_right:Nn #1 { \q_stop }
151       \tl_replace_all:Nnn #1 { \q_mark - } { \q_mark }
152       \tl_replace_all:Nnn #1 { - \q_stop } { \q_stop }
153       \tl_remove_all:Nn #1 { \q_mark }
154       \tl_remove_all:Nn #1 { \q_stop }
155     }
156   }
157 }
158 \cs_generate_variant:Nn \__spintent_sanitize_affix:Nn { cn }
```

(End of definition for `\__spintent_sanitize_affix:Nn`.)

11.10 The command `\setspintent`

The `\setspintent` command is used to configure $\langle keys \rangle$ “globally” for the utilities provided by `spintent`.

11.10.1 Internal variables and error messages for `\setspintent`

`\__spintent_setspintent_input_arg_cmd_tl` The variable `\__spintent_setspintent_input_arg_cmd_tl` will *store* the command or command name passed to the “first” $\{\langle argument \rangle\}$ ‘#1’ of the command `\setspintent`.

```

159 \tl_new:N \__spintent_setspintent_input_arg_cmd_tl
```

Error message if the $\langle argument \rangle$ ‘#1’ passed to `\setspintent` is a “non-existent” command.

```

160 \msg_new:nnnn { spintent } { unknown-command-setup }
161 {
162   Cannot~configure~unknown~command~'\exp_not:c{#1}'.
163 }
164 {
165   You~attempted~to~use~\token_to_str:N \setspintent{#1}{...},~but~the~
166   command~\exp_not:c{#1}~is~not~defined.~Please~check~for~typos.
167 }

```

(End of definition for `\__spintent_setspintent_input_arg_cmd_tl` and `unknown-command-setup`.)

```

\__spintent_setspintent_set_keys:nn
\__spintent_setspintent_set_keys:en

```

The function `\__spintent_setspintent_set_keys:nn` will check if the command passed as $\langle argument \rangle$ exists in ‘#1’ and set the $\langle keys \rangle$ passed as $\langle argument \rangle$ in ‘#2’ to the correct path; otherwise, it will return an error.

```

168 \cs_new_protected:Nn \__spintent_setspintent_set_keys:nn
169 {
170   \cs_if_exist:cTF { #1 }
171   {
172     \keys_set:nn { spintent / #1 } { #2 }
173   }
174   {
175     \msg_error:nnn { spintent } { unknown-command-setup } { #1 }
176   }
177 }
178 \cs_generate_variant:Nn \__spintent_setspintent_set_keys:nn { en }

```

(End of definition for `\__spintent_setspintent_set_keys:nn`.)

```

\__spintent_setspintent_parse_args:nn
\__spintent_setspintent_parse_args:Vn

```

The function `\__spintent_setspintent_parse_args:nn` will check if the argument passed in #1 begins with ‘\’ or is just a *command name*. If it begins with ‘\’, it will remove it using `\cs_to_str:N` and then execute `\__spintent_setspintent_set_keys:en`. Otherwise, i.e., if ‘\’ was not passed, it will directly execute `\__spintent_setspintent_set_keys:nn`.

```

179 \cs_new_protected:Nn \__spintent_setspintent_parse_args:nn
180 {
181   \tl_if_single:nTF { #1 }
182   {
183     \token_if_cs:NTF #1
184     {
185       \__spintent_setspintent_set_keys:en { \cs_to_str:N #1 } { #2 }
186     }
187     {
188       \__spintent_setspintent_set_keys:nn { #1 } { #2 }
189     }
190   }
191   {
192     \__spintent_setspintent_set_keys:nn { #1 } { #2 }
193   }
194 }
195 \cs_generate_variant:Nn \__spintent_setspintent_parse_args:nn { Vn }

```

(End of definition for `\__spintent_setspintent_parse_args:nn`.)

`\setspintent` Now we define the user command `\setspintent`.

```

196 \NewDocumentCommand \setspintent { m m }
197 {
198   \tl_if_blank:nTF { #1 }
199   {
200     \msg_error:nnn { spintent } { unknown-command-setup } { <empty> }
201   }
202   {
203     \tl_set:Nn \__spintent_setspintent_input_arg_cmd_tl { #1 }
204     \tl_trim_spaces:N \__spintent_setspintent_input_arg_cmd_tl
205     \__spintent_setspintent_parse_args:Vn \__spintent_setspintent_input_arg_cmd_tl { #2 }
206   }
207 }

```

(End of definition for `\setspintent`. This function is documented on page 3.)

11.11 Handling unknown keys

```
\l__spintent_command_name_str
    unknown-choice
    cmd-key-unknown
    cmd-key-value-unknown
```

We define the variable `\l__spintent_command_name_str` which will *store* the “*name*” of each command within the implementation along with the messages `unknown-choice` used by *(key)* that use `.choice:n` in their implementation and the messages `cmd-key-unknown`, `cmd-key-value-unknown` used by the *(key)* `unknown` in commands that use `[key = val]`.

```
208 \str_new:N \l__spintent_command_name_str
209 \msg_new:nnn { spintent } { unknown-choice }
210 {
211     The~value~'#3'~for~'#1'~key~is~invalid~use~('#2').
212 }
213 \msg_new:nnnn { spintent } { cmd-key-unknown }
214 {
215     The~key~'#1'~is~unknown~by~command~
216     \c_backslash_str \l__spintent_command_name_str \c_space_tl
217     and~is~being~ignored~\msg_line_context:.
218 }
219 {
220     The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
221     does~not~have~a~key~called~'#1'.\\
222     Check~that~you~have~spelled~the~key~name~correctly.
223 }
224 \msg_new:nnnn { spintent } { cmd-key-value-unknown }
225 {
226     The~key~'#1=#2'~is~unknown~by~command~
227     \c_backslash_str \l__spintent_command_name_str \c_space_tl
228     and~is~being~ignored~\msg_line_context:.
229 }
230 {
231     The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
232     does~not~have~a~key~called~'#1'.\\
233     Check~that~you~have~spelled~the~key~name~correctly.
234 }
```

(End of definition for `\l__spintent_command_name_str` and others.)

```
\__spintent_unknown_keys:nn
\__spintent_unknown_keys:Vn
\__spintent_unknown_keys:n
```

The internal functions `\__spintent_unknown_keys:nn` and `\__spintent_unknown_keys:n` used by the *(key)* `unknown`.

```
235 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:nn
236 {
237     \tl_if_blank:nTF {#2}
238     { \msg_error:nnn { spintent } { cmd-key-unknown } {#1} }
239     { \msg_error:nnnn { spintent } { cmd-key-value-unknown } {#1} {#2} }
240 }
241 \cs_generate_variant:Nn \__spintent_unknown_keys_cmd:nn { Vn }
242 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:n
243 {
244     \__spintent_unknown_keys_cmd:Vn \l_keys_key_str {#1}
245 }
```

(End of definition for `\__spintent_unknown_keys:nn` and `\__spintent_unknown_keys:n`.)

11.12 Definition of core base variables

```
\__spintent_luafun_clean_split_and_set:n
\__spintent_luafun_clean_split_and_set:V
```

The function `\__spintent_luafun_clean_split_and_set:n` defined in the Lua module `spintent.lua`, is common to several commands in this implementation. It analyzes, processes, cleans and splits the *{(argument)}* passed to commands using internal functions defined in the Lua module `spintent.lua`. It isolates signs, integer components, decimal separators, decimal parte, decimal periodic part, exponents and final physical “*units*”, assigning them directly to `expl3` variables.

This function assigns the states of the variables ‘`_str`’ or ‘`_tl`’ described below to `true`, `false`, `error` or *stores* information according to the role it fulfills within the implementation.

(End of definition for `\__spintent_luafun_clean_split_and_set:n`.)

```
\l__spintent_luaset_sign_tl
\l__spintent_luaset_part_int_tl
\l__spintent_luaset_dec_sep_tl
\l__spintent_luaset_part_dec_tl
\l__spintent_luaset_part_period_tl
\l__spintent_luaset_has_integer_str
\l__spintent_luaset_has_natural_str
```

The variable `\l__spintent_luaset_sign_tl` stores the explicit ‘`+`’ or ‘`-`’ sign if present. The variable `\l__spintent_luaset_part_int_tl` stores the raw digit sequence of the “*integer part*”. The variable `\l__spintent_luaset_dec_sep_tl` stores the matched ‘`,`’ or ‘`.`’ used as the decimal separator. The variable `\l__spintent_luaset_part_dec_tl` stores the raw digits of the “*finite decimal part*” that follow the separator, and the variable `\l__spintent_luaset_part_period_tl` stores the digits of the “*infinite periodic decimal part*” that follow a semicolon ‘`;`’.

The variable `\l__spintent_luaset_has_integer_str` store the state `true` when it is *only* an “integer”. The variable `\l__spintent_luaset_has_natural_str` store the state `true` when it is *only* an “natural number”.

```

246 \tl_new:N \l__spintent_luaset_sign_tl
247 \tl_new:N \l__spintent_luaset_part_int_tl
248 \tl_new:N \l__spintent_luaset_dec_sep_tl
249 \tl_new:N \l__spintent_luaset_part_dec_tl
250 \tl_new:N \l__spintent_luaset_part_period_tl
251 \str_new:N \l__spintent_luaset_has_integer_str
252 \str_new:N \l__spintent_luaset_has_natural_str

```

(End of definition for `\l__spintent_luaset_sign_tl` and others.)

The literal entries of the numbers that we will pass to the *intent function* :number of `\MathMLintent` are stored in `\l__spintent_luaset_only_part_int_str` for the “integer part”, in `\l__spintent_luaset_only_part_dec_str` for the “finite decimal part”, and in `\l__spintent_luaset_only_part_period_str` for the “infinite periodic decimal part”.

Formatted versions (`cifras-sep=true`) are stored in `\l__spintent_luaset_format_part_int_str` for the “integer part” and `\l__spintent_luaset_format_part_dec_str` for the “finite decimal part”.

```

253 \str_new:N \l__spintent_luaset_only_part_int_str
254 \str_new:N \l__spintent_luaset_only_part_dec_str
255 \str_new:N \l__spintent_luaset_only_part_period_str
256 \str_new:N \l__spintent_luaset_format_part_int_str
257 \str_new:N \l__spintent_luaset_format_part_dec_str

```

(End of definition for `\l__spintent_luaset_only_part_int_str` and others.)

The flag variable `\l__spintent_luaset_millions_str` indicates whether the integer part translates exactly to a multiple of one million when passed to `\MathMLintent`. To properly structure the reading and writing of numbers in MathML, we use the flag variables `\l__spintent_luaset_decimal_period_str`, `\l__spintent_luaset_not_decimal_period_str`, and `\l__spintent_luaset_not_decimal_not_period_str`. The flag variable `\l__spintent_luaset_dec_is_all_zeros_str` monitors whether the “finite decimal part” is completely zero (useful for “pure periodic decimals”).

The flag variable `\l__spintent_luaset_has_exponent_str` indicates whether active “scientific notation” was detected at the end of the entered number, passed with ‘e’ or ‘E’. If true, it will store the digits of the exponent in `\l__spintent_luaset_exponent_tl`.

```

258 \str_new:N \l__spintent_luaset_millions_str
259 \str_new:N \l__spintent_luaset_decimal_period_str
260 \str_new:N \l__spintent_luaset_not_decimal_period_str
261 \str_new:N \l__spintent_luaset_not_decimal_not_period_str
262 \str_new:N \l__spintent_luaset_dec_is_all_zeros_str
263 \str_new:N \l__spintent_luaset_has_exponent_str
264 \tl_new:N \l__spintent_luaset_exponent_tl

```

(End of definition for `\l__spintent_luaset_millions_str` and others.)

Since the function `\__spintent_luafun_clean_split_and_set:n` is common to `\spnum` (§11.14) and `\spunit` (§11.15) (as well as others), it determines the following unit-related variables after processing the `{⟨arguments⟩}` via the Lua module `spintent.lua`.

First, it checks if there is any residual text *after* the processed number that is a candidate for physical “units”. It sets the flag variable `\l__spintent_luaset_has_units_str` accordingly. Following this, it verifies if this text contains more than one “/”. It then sets the variable `\l__spintent_luaset_status_str` to `valid` or `multislash`. If the structure is `valid`, it splits the unit into `\l__spintent_luaset_arg_above_tl` and `\l__spintent_luaset_arg_below_tl`. Finally, it analyzes `\l__spintent_luaset_arg_below_tl`; if the denominator starts with a number, it sets `\l__spintent_luaset_denom_has_number_str` to `true`, otherwise to `false`.

```

265 \str_new:N \l__spintent_luaset_has_units_str
266 \str_new:N \l__spintent_luaset_status_str
267 \tl_new:N \l__spintent_luaset_arg_above_tl
268 \tl_new:N \l__spintent_luaset_arg_below_tl
269 \str_new:N \l__spintent_luaset_denom_has_number_str

```

(End of definition for `\l__spintent_luaset_has_units_str` and others.)

11.13 Common messages for text mode and math mode

Messages shared by the different commands for text mode or math mode.

```

270 \msg_new:nnnn { spintent } { need-math-mode }
271 {
272   Math~mode~required~for~'#1'~\msg_line_context:.
273 }
274 {
275   Use~inside~$.~$.~$~or~in~math~environment.
276 }
277 \msg_new:nnnn { spintent } { need-text-mode }
278 {
279   Text~mode~required~for~'#1'~\msg_line_context:.
280 }
281 {
282   Use~outside~of~$.~$.~$~or~math~environment.
283 }

```

11.14 The command \spnum

The user command `\spnum` is first of “the big four” provided by the `spintent` package and is fundamental to the implementation of other commands. It handles the processing of numerical input, scientific notation, and units of measurement, using the Lua module `spintent.lua`, which is responsible for separating and assigning variables defined in `expl3`.

From the `expl3` side, we handle the printing logic and the `intent` functions passed to `\MathMLintent` to achieve a valid MathML structure that is accepted by MathCAT.

11.14.1 Definition of keys and variables for \spnum

The variable `\__spintent_spnum_read_terse_bool` handled by the key `read-sign`, controls the `intent` passed to `\MathMLintent` to spoken the ‘+’ or ‘-’. If set to `clear`, it will say “positivo” or “negativo” after reading the number; if set to `terse`, it will say “más” or “menos” before reading the number.

The variable `\__spintent_spnum_read_period_sep_str` handled by the key `read-period-sep`, controls how the decimal separator ‘;’ (periodic part) is printed and spoken when the `{(argument)}` is a *pure decimal periodic* and is passed the `intent` to `\MathMLintent`. If set to `coma`, it will say “coma” and print ‘,’; if set to `punto` it will say “punto” and print ‘.’.

The variable `\__spintent_spnum__notation_sym_tl`, handled by the key `notation-sym`, determines whether ‘x’ will be printed when set to `times` or ‘.’ when set to `cdot`.

The variable `\__spintent_spnum_intent_read_sign_str` will store the spoken translation used in the `intent` function passed to `\MathMLintent` for the ‘+’ or ‘-’ according to the value passed to the key `read-sign`.

The variable `\__spintent_spnum_intent_read_dec_sep_str` stores the spoken translation that will be used in the `intent` function passed to `\MathMLintent` for reading the decimal separation as a “coma” or a “punto” according to the value found in the variable `\__spintent_luaset_dec_sep_tl`.

```

284 \bool_new:N \__spintent_spnum_read_terse_bool
285 \str_new:N \__spintent_spnum_read_period_sep_str
286 \tl_new:N \__spintent_spnum_notation_sym_tl
287 \str_new:N \__spintent_spnum_intent_read_sign_str
288 \str_new:N \__spintent_spnum_intent_read_dec_sep_str

```

(End of definition for `\__spintent_spnum_read_terse_bool` and others.)

```

cifras-sep
read-sign
read-period-sep
notation-sym

Now we add the keys cifras-sep, read-sign, read-period-sep and notation-sym.

289 \keys_define:nn { spintent/spnum }
290 {
291   cifras-sep .bool_set:N = \__spintent_spnum_cifras_sep_bool,
292   cifras-sep .initial:n = true,
293   cifras-sep .value_required:n = true,
294   read-sign .choices:nn =
295     { terse , clear }
296   {
297     \int_case:nn { \_keys_choice_int }
298     {
299       {1} { \bool_set_true:N \__spintent_spnum_read_terse_bool }
300       {2} { \bool_set_false:N \__spintent_spnum_read_terse_bool }
301     }
302   },
303   read-sign .initial:n = terse,
304   read-sign .value_required:n = true,
305   read-sign / unknown .code:n =

```

```

306     \msg_error:nneee { spintent } { unknown-choice }
307     { read-sign } { terse , ~clear } { \exp_not:n {#1} },
308 read-period-sep                .choices:nn =
309     { coma , punto }
310     {
311         \int_case:nn { \l_keys_choice_int }
312         {
313             {1} { \str_set:Nn \l__spintent_spnun_read_period_sep_str { coma } }
314             {2} { \str_set:Nn \l__spintent_spnun_read_period_sep_str { punto } }
315         }
316     },
317 read-period-sep                .initial:n = coma,
318 read-period-sep                .value_required:n = true,
319 read-period-sep / unknown .code:n =
320     \msg_error:nneee { spintent } { unknown-choice }
321     { read-period-sep } { coma , ~punto } { \exp_not:n {#1} },
322 notation-sym                    .choices:nn =
323     { times , cdot }
324     {
325         \int_case:nn { \l_keys_choice_int }
326         {
327             {1} { \tl_set:Nn \l__spintent_spnun_notation_sym_tl { \times } }
328             {2} { \tl_set:Nn \l__spintent_spnun_notation_sym_tl { \cdot } }
329         }
330     },
331 notation-sym                    .initial:n = cdot,
332 notation-sym                    .value_required:n = true,
333 notation-sym / unknown .code:n =
334     \msg_error:nneee { spintent } { unknown-choice }
335     { notation-sym } { times , ~ cdot } { \exp_not:n {#1} },
336 unknown                        .code:n =
337     {
338         \str_set:Nn \l__spintent_command_name_str { spnum }
339         \__spintent_unknown_keys_cmd:n {#1}
340     },
341 }

```

(End of definition for `cifras-sep` and others.)

11.14.2 Internal functions for `\spnum`

```

\__spintent_spnun_render_part:n
\__spintent_spnun_render_int:
\__spintent_spnun_render_dec:

```

The function `\__spintent_spnun_render_part:n` processes the $\langle argument \rangle$ passed in ‘#1’ used by the functions `\__spintent_spnun_render_int:` and `\__spintent_spnun_render_dec:` to define the *intent function* :number for the integer and decimal part passed to `\MathMLintent`. It evaluates whether the key `cifras-sep` is active or not.

```

342 \cs_new_protected:Nn \__spintent_spnun_render_part:n
343 {
344     \MathMLintent { \str_use:c { \l__spintent_luaset_only_part_#1_str } :number }
345     {
346         \bool_if:NTF \l__spintent_spnun_cifras_sep_bool
347         { \str_use:c { \l__spintent_luaset_format_part_#1_str } }
348         { \tl_use:c { \l__spintent_luaset_part_#1_tl } }
349     }
350 }
351 \cs_new_protected:Nn \__spintent_spnun_render_int:
352 {
353     \__spintent_spnun_render_part:n { int }
354 }
355 \cs_new_protected:Nn \__spintent_spnun_render_dec:
356 {
357     \__spintent_spnun_render_part:n { dec }
358 }

```

(End of definition for `\__spintent_spnun_render_part:n`, `\__spintent_spnun_render_int:`, and `\__spintent_spnun_render_dec:`.)

```

\__spintent_spnun_render_only_period:
\__spintent_spnun_print_period_bar:
\__spintent_spnun_render_period_sep:

```

The function `\__spintent_spnun_render_only_period:` isolates the “infinite periodic decimal part” from the “finite decimal part”, assigning the *intent function* :number passed to `\MathMLintent` which will be used by `\MathMLarg` in the function `\__spintent_spnun_print_period_bar:`.

```

359 \cs_new_protected:Nn \__spintent_spnun_render_only_period:
360 {
361     \MathMLintent { \l__spintent_luaset_only_part_period_str :number }

```

```

362     {
363         \l__spintent_luaset_part_period_tl
364     }
365 }

```

The function `\__spintent_snum_print_period_bar:` inserts the function `\__spintent_snum_render_only_` as an argument within the *intent function* `_periódico:postfix($dp)` passed to `\MathMLintent`, generating the correct spoken for the “*infinite periodic decimal part*”.

```

366 \cs_new_protected:Nn \__spintent_snum_print_period_bar:
367 {
368     \MathMLintent { _periódico:postfix($dp) }
369     {
370         {
371             \overline
372             {
373                 \MathMLarg { dp } { \__spintent_snum_render_only_period: }
374             }
375         }
376     }
377 }

```

The function `\__spintent_snum_render_period_sep:` sets how the decimal separator handled by the key `read-period-sep` will be printed and spoken when a “*pure periodic decimal*” is passed.

```

378 \cs_new_protected:Nn \__spintent_snum_render_period_sep:
379 {
380     \str_if_eq:VnTF \l__spintent_snum_read_period_sep_str { coma }
381     {
382         \MathMLintent { _coma:prefix($pp) }
383         {
384             \mathord{,}
385             \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
386         }
387     }
388     {
389         \MathMLintent { _punto:prefix($pp) }
390         {
391             \mathord{.}
392             \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
393         }
394     }
395 }

```

(End of definition for `\__spintent_snum_render_only_period:`, `\__spintent_snum_print_period_bar:`, and `\__spintent_snum_render_period_sep:`.)

`\__spintent_snum_render_decimal_sep:`

The function `\__spintent_snum_render_decimal_sep:` parses the value of the variable `\l__spintent_luaset_` and sets the variable `\l__spintent_snum_intent_read_dec_sep_str` with the function `_coma:prefix($dp)` for reading a “*coma*” or `_punto:prefix($dp)` for reading a “*punto*” to `\MathMLintent`. We print the decimal separator stored in `\l__spintent_luaset_dec_sep_tl` using `\mathord` to remove spaces around it and then use the command `\MathMLarg` where we insert `\__spintent_invisible_sep:` for compatibility with MathML and finally call the function `\__spintent_snum_render_dec:` to print the finite decimal part.

```

396 \cs_new_protected:Nn \__spintent_snum_render_decimal_sep:
397 {
398     \str_if_eq:VnTF \l__spintent_luaset_dec_sep_tl { , }
399     { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _coma } }
400     { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _punto } }
401     \MathMLintent { \l__spintent_snum_intent_read_dec_sep_str :prefix($dp) }
402     {
403         \mathord { \l__spintent_luaset_dec_sep_tl }
404         \MathMLarg { dp }
405         {
406             \__spintent_inner_sep:
407             \__spintent_snum_render_dec:
408         }
409     }
410     % period part
411     \str_if_eq:VnT \l__spintent_luaset_decimal_period_str { true }
412     {
413         \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
414         {
415             \__spintent_inner_sep:
416             \__spintent_snum_print_period_bar:

```

```

417     }
418     {
419         \MathMLintent { _con:prefix($pnum) }
420         {
421             \__spintent_inner_sep:
422             \MathMLarg { pnum }
423             {
424                 \__spintent_snum_print_period_bar:
425             }
426         }
427     }
428 }
429 }

```

(End of definition for `\__spintent_snum_render_decimal_sep:`.)

`\__spintent_snum_print_part_dec:`
`\__spintent_snum_print_part_period:`

The function `\__spintent_snum_print_part_dec:` checks if a decimal part exists. If it does, it determines whether the decimal separator is a comma or a period to set the correct speech intent (“*coma*” or “*punto*”). It then prints the separator, renders the decimal digits, and if a periodic sequence follows, it decides how to attach the periodic bar.

The function `\__spintent_snum_print_part_period:` handles cases where a periodic part exists. It evaluates if it should render the periodic separator directly, particularly when the number has a periodic part but lacks a standard decimal part.

```

430 \cs_new_protected:Nn \__spintent_snum_print_part_dec:
431 {
432     \tl_if_empty:NF \l__spintent_luaset_part_dec_tl
433     {
434         \__spintent_snum_render_decimal_sep:
435     }
436 }
437 \cs_new_protected:Nn \__spintent_snum_print_part_period:
438 {
439     \tl_if_empty:NF \l__spintent_luaset_part_period_tl
440     {
441         \str_if_eq:VnTF \l__spintent_luaset_not_decimal_period_str { true }
442         { \__spintent_snum_render_period_sep: }
443         {
444             \tl_if_empty:NT \l__spintent_luaset_part_dec_tl
445             {
446                 \__spintent_snum_render_period_sep:
447             }
448         }
449     }
450 }

```

(End of definition for `\__spintent_snum_print_part_dec:` and `\__spintent_snum_print_part_period:`.)

`\__spintent_snum_print_integer:`
`\__spintent_snum_print_num_start:`
`\__spintent_snum_print_number_core:`

The function `\__spintent_snum_print_integer:` checks if the number consists only of an integer. If so, it simply renders it. If a decimal or periodic part exists, it sets up a `MathML` intent to link the integer part (respecting digit separators if enabled) to the subsequent decimal or periodic functions.

The function `\__spintent_snum_print_num_start:` checks if an explicit sign is present. If it finds one, it evaluates the terse reading flag to decide whether the sign should be read as a prefix (“*más*” or “*menos*”) or as a postfix (“*positivo*” or “*negativo*”), and wraps the intent around the rest of the number.

The function `\__spintent_snum_print_number_core:` acts as the main wrapper. It triggers the sign and base number evaluation, and then checks if a scientific exponent is present. If it is, it appends the correct base-10 notation symbol and exponent to complete the sequence.

📌 Note for `\__spintent_invisible_sep:`.

```

451 \cs_new_protected:Nn \__spintent_snum_print_integer:
452 {
453     \str_if_eq:VnTF \l__spintent_luaset_not_decimal_not_period_str { true }
454     { \__spintent_snum_render_int: }
455     {
456         \MathMLintent { \l__spintent_luaset_only_part_int_str :number:prefix($next) }
457         {
458             \bool_if:NTF \l__spintent_snum_cifras_sep_bool
459             { \l__spintent_luaset_format_part_int_str }
460             { \l__spintent_luaset_part_int_tl }
461             \MathMLarg { next }
462         }

```

```

463         \tl_if_empty:NTF \l__spintent_luaset_part_dec_tl
464         { \__spintent_spnum_print_part_period: }
465         { \__spintent_spnum_print_part_dec: }
466     }
467 }
468 }
469 }
470 \cs_new_protected:Nn \__spintent_spnum_print_num_start:
471 {
472     \tl_if_empty:NTF \l__spintent_luaset_sign_tl
473     {
474         \__spintent_invisible_sep: % need
475         \__spintent_spnum_print_integer:
476     }
477     {
478         \bool_if:NTF \l__spintent_spnum_read_terse_bool
479         {
480             \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
481             { \str_set:Nn \l__spintent_spnum_intent_read_sign_str { _más:prefix($n)} }
482             { \str_set:Nn \l__spintent_spnum_intent_read_sign_str { _menos:prefix($n)} }
483         }
484         {
485             \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
486             { \str_set:Nn \l__spintent_spnum_intent_read_sign_str { _positivo:postfix($n) } }
487             { \str_set:Nn \l__spintent_spnum_intent_read_sign_str { _negativo:postfix($n) } }
488         }
489         \__spintent_invisible_sep: % need
490         \MathMLintent { \l__spintent_spnum_intent_read_sign_str }
491         {
492             \l__spintent_luaset_sign_tl
493             \MathMLarg { n } { \__spintent_spnum_print_integer: }
494         }
495     }
496 }
497 \cs_new_protected:Nn \__spintent_spnum_print_number_core:
498 {
499     \__spintent_spnum_print_num_start:
500     \str_if_eq:VnT \l__spintent_luaset_has_exponent_str { true }
501     {
502         \MathMLintent { _por } { \l__spintent_spnum_notation_sym_tl }
503         10^{\l__spintent_luaset_exponent_tl }
504     }
505 }

```

(End of definition for `\__spintent_spnum_print_integer:`, `\__spintent_spnum_print_num_start:`, and `\__spintent_spnum_print_number_core:`.)

11.14.3 Definition of error messages

spnum-has-units
spnum-no-digits

We define two error messages for the `\spnum` command. The first “error” triggers if the user includes “units” in the $\langle argument \rangle$ passed in ‘#1’, advising them to use the `\spunit` command instead. The second “error” is raised if the provided $\langle argument \rangle$ contains no numerical digits.

```

506 \msg_new:nnnn { spintent } { spnum-has-units }
507 {
508     The~\token_to_str:N \spnum \c_space_tl command~does~not~accept~units~(' #1').
509 }
510 {
511     Use~\token_to_str:N \spunit \c_space_tl if~you~need~to~print~numbers~with~units.
512 }
513 \msg_new:nnnn { spintent } { spnum-no-digits }
514 {
515     The~\token_to_str:N \spnum \c_space_tl command~requires~a~valid~numerical~value.
516 }
517 {
518     You~provided~' #1',~which~contains~no~digits.
519 }

```

(End of definition for `spnum-has-units` and `spnum-no-digits`.)

11.14.4 Implementation of the `\spum` command

`\spnum` The public user command `\spnum` accepts an optional $[key = val]$ argument in ‘#1’ and a mandatory $\langle rational number \rangle$ in ‘#2’. It first checks if it is executed inside math mode, raising an error if it is

not. If correct, it opens a local group to apply the options and passes the input to Lua for parsing via `\__spintent_luafun_clean_split_and_set:n`.

Before printing, it validates the extracted data. It throws an error if it finds trailing units in the input or if the integer part is completely empty (no valid digits). If all checks pass, it calls `\__spintent_spnum_print_number_core:` to render the final number, and finally closes the group.

```

520 \NewDocumentCommand \spnum { O{} m }
521 {
522   \mode_if_math:TF
523   {
524     \group_begin:
525       \keys_set:nn { spintent/spnum } { #1 }
526       \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
527       \str_if_eq:VnTF \__spintent_luaset_has_units_str { true }
528       { \msg_error:nnn { spintent } { spnum-has-units } { #2 } }
529       {
530         \tl_if_empty:NTF \__spintent_luaset_part_int_tl
531         { \msg_error:nnn { spintent } { spnum-no-digits } { #2 } }
532         { \__spintent_spnum_print_number_core: }
533       }
534     \group_end:
535   }
536   { \msg_error:nnn { spintent } { need-math-mode } { \spnum } }
537 }

```

(End of definition for `\spnum`. This function is documented on page 3.)

11.15 The command `\spunit`

The user command `\spunit` is the *second* of “the big four” provided by the `spintent` package and is used to parse and print physical units and angular systems. It sends the `{\argument}` to Lua module `spintent.lua` via `\__spintent_luafun_clean_split_and_set:n` to separate and clean the `{\argument}`.

This ensures that all “units” (such as SI fractions, products, or sexagesimal angles) are processed consistently by the `spintent.lua`.

11.15.1 Definition of variables

These string variables store the data returned by the Lua module `spintent.lua`. They hold the canonical “unit symbol”, the dictionary lookup status, the correct speech text for NVDA/MathCAT, and specific flags that indicate if the “unit” is compact or represents a sexagesimal angle.

```

538 \str_new:N \__spintent_spunit_luaset_canonical_str
539 \str_new:N \__spintent_spunit_luaset_lookup_status_str
540 \str_new:N \__spintent_spunit_luaset_read_str
541 \str_new:N \__spintent_spunit_luaset_is_compact_str
542 \str_new:N \__spintent_spunit_luaset_compact_exp_str
543 \str_new:N \__spintent_spunit_luaset_is_sexagesimal_str
544 \str_new:N \__spintent_spunit_luaset_status_str

```

(End of definition for `\__spintent_spunit_luaset_canonical_str` and others.)

These variables are used to process and format the “unit”. The sequences `\__spintent_spunit_mult_seq` and `\__spintent_spunit_exp_seq` split compound “units” at multiplication ‘`*`’ or exponent ‘`^`’ boundaries.

The boolean variables control specific rendering options, such as whether the multiplication dot ‘`\cdot`’ is explicitly read aloud or silently skipped.

```

545 \tl_new:N \__spintent_spunit_exponent_tl
546 \tl_new:N \__spintent_spunit_unit_name_tl
547 \str_new:N \__spintent_spunit_read_slash_str
548 \seq_new:N \__spintent_spunit_exp_seq
549 \seq_new:N \__spintent_spunit_mult_seq
550 \bool_new:N \__spintent_spunit_is_mult_bool
551 \bool_new:N \__spintent_spunit_read_cdod_bool

```

(End of definition for `\__spintent_spunit_exponent_tl` and others.)

11.15.2 Definition of error messages

We define three error messages for the `\spunit` command. The first “error” triggers if the “unit expression” provided in ‘`#1`’ is unknown or malformed. The second “error” occurs if no “units” are found in the `{\argument}`, advising the user to use `\spnum` for pure numbers. The third “error” is raised if a numeric coefficient is illegally placed in the denominator of an inline fraction (after a slash).

```

552 \msg_new:nnnn { spintent } { spunit-invalid-unit }
553 {
554   The~expression~'~#1'~contains~an~unknown~or~malformed~unit~structure.

```

```

555 }
556 {
557     Check~spelling~or~verify~the~unit~is~defined~in~the~dictionary.
558 }
559 \msg_new:nnnn { spintent } { spunit-no-units }
560 {
561     The~\token_to_str:N \spunit \c_space_tl command~requires~at~least~one~unit~('#1').
562 }
563 {
564     Use~\token_to_str:N \spnum \c_space_tl if~you~only~need~to~format~a~pure~number.
565 }
566 \msg_new:nnnn { spintent } { inline-numeric-denominator }
567 {
568     Illegal~numeric~coefficient~in~inline~denominator~'#1'.
569 }
570 {
571     The~inline~mode~of~\token_to_str:N \spunit \c_space_tl
572     does~not~allow~numbers~after~the~slash~(/).
573 }

```

(End of definition for spunit-invalid-unit, spunit-no-units, and inline-numeric-denominator.)

11.15.3 Definition of keys for \spunit

Now we add the keys `print-cdot`, `read-cdot`, `read-slash`, `cifras-sep`, `read-period-sep` and `notation-sym`.

```

574 \keys_define:nn { spintent/spunit }
575 {
576     print-cdot      .bool_set:N = \l_spintent_spunit_print_cdot_bool,
577     print-cdot      .initial:n  = false,
578     print-cdot      .value_required:n = true,
579     read-cdot       .bool_set:N = \l_spintent_spunit_read_cdot_bool,
580     read-cdot       .initial:n  = false,
581     read-cdot       .value_required:n = true,
582     read-slash      .choices:nn =
583     { por , partido-por }
584     {
585         \int_case:nn { \l_keys_choice_int }
586         {
587             {1} { \str_set:Nn \l__spintent_spunit_read_slash_str { _por } }
588             {2} { \str_set:Nn \l__spintent_spunit_read_slash_str { _partido-por } }
589         }
590     },
591     read-slash      .initial:n  = por,
592     read-slash      .value_required:n = true,
593     read-slash / unknown .code:n =
594     \msg_error:nneee { spintent } { unknown-choice }
595     { read-slash } { por , ~partido-por } { \exp_not:n {#1} },
596     cifras-sep      .meta:nn = { spintent/spnum } { cifras-sep = {#1} },
597     cifras-sep      .value_required:n = true,
598     read-period-sep .meta:nn = { spintent/spnum } { read-period-sep = {#1} },
599     read-period-sep .value_required:n = true,
600     notation-sym    .meta:nn = { spintent/spnum } { notation-sym = {#1} },
601     notation-sym    .value_required:n = true,
602     unknown         .code:n =
603     {
604         \str_set:Nn \l__spintent_command_name_str { spunit }
605         \__spintent_unknown_keys_cmd:n {#1}
606     },
607 }

```

(End of definition for read-cdot and others.)

11.15.4 Internal functions for \spunit

`\__spintent_spunit_format_canonical:` The function `\__spintent_spunit_format_canonical:` handles the visual formatting of the “unit symbol”. It specifically checks for the *liter symbol* ‘ℓ’ to print it as `\ell`, and formats all other valid “units” in standard upright text using `\mathrm`.

```

608 \cs_new_protected:Nn \__spintent_spunit_format_canonical:
609 {
610     \str_case:VnF \l__spintent_spunit_luaset_canonical_str
611     {
612         { ℓ } { \ell }

```

```

613     }
614     { \mathrm { \l__spintent_spunit_luaset_canonical_str } }
615 }

```

(End of definition for `\l__spintent_spunit_format_canonical:`)

`\l__spintent_spunit_process_base_exp:n`

The function `\l__spintent_spunit_process_base_exp:n` evaluates a “single unit” along with its exponent, ‘#1’. It first looks up the unit in the dictionary; if not found, flags it as invalid.

Otherwise, unless this is the first unit in a product (`\l__spintent_spunit_is_mult_bool`), it handles the spacing and multiplication dot: `\cdot` when `print-cdot` is `true` (read as `por` when `read-cdot` is also `true`, silent otherwise), or a thin space `\,` when `print-cdot` is `false` (still read as `por` under `read-cdot`).

```

616 \cs_new_protected:Nn \l__spintent_spunit_process_base_exp:n
617 {
618   \tl_clear:N \l__spintent_spunit_exponent_tl
619   \__spintent_luafun_spunit_lookup_alias:n { #1 }
620   \str_if_eq:VnTF \l__spintent_spunit_luaset_lookup_status_str { notfound }
621   {
622     \str_set:Nn \l__spintent_luaset_status_str { invalid }
623   }
624   {
625     \bool_if:NF \l__spintent_spunit_is_mult_bool
626     {
627       \bool_if:NTF \l__spintent_spunit_print_cdot_bool
628       {
629         \bool_if:NTF \l__spintent_spunit_read_cdot_bool
630         { \MathMLint { _por } { \cdot } }
631         { \MathMLint { :silent } { \cdot } }
632       }
633       {
634         \bool_if:NTF \l__spintent_spunit_read_cdot_bool
635         { \MathMLint { _por } { \, } }
636         { \, }
637       }
638     }

```

Then renders the base unit and its exponent: either a pre-defined compact exponent from Lua (`\l__spintent_spunit_luafun_spunit_lookup_status_str`) or by manually splitting ‘#1’ at the ‘^’ symbol otherwise.

```

639   \str_if_eq:VnTF \l__spintent_spunit_luaset_is_compact_str { true }
640   {
641     {
642       \MathMLint { \l__spintent_spunit_luaset_read_str }
643       {
644         \__spintent_spunit_format_canonical:
645       }
646     }
647     ^ { \l__spintent_spunit_luaset_compact_exp_str }
648   }
649   {
650     \seq_set_split:Nnn \l__spintent_spunit_exp_seq { ^ } { #1 }
651     \seq_pop_left:NN \l__spintent_spunit_exp_seq \l__spintent_spunit_unit_name_tl
652     \seq_if_empty:NF \l__spintent_spunit_exp_seq
653     {
654       \seq_pop_left:NN \l__spintent_spunit_exp_seq \l__spintent_spunit_exponent_tl
655     }
656     \MathMLint { \l__spintent_spunit_luaset_read_str }
657     {
658       \__spintent_spunit_format_canonical:
659     }
660     \tl_if_empty:NF \l__spintent_spunit_exponent_tl
661     {
662       ^ { \l__spintent_spunit_exponent_tl }
663     }
664   }
665   \bool_set_false:N \l__spintent_spunit_is_mult_bool
666 }
667 }

```

(End of definition for `\l__spintent_spunit_process_base_exp:n`)

`\l__spintent_spunit_process_mult:n`

`\l__spintent_spunit_process_mult:V`

The function `\l__spintent_spunit_process_mult:n` handles compound “units” joined by an explicit multiplication star ‘*’. It splits the $\{ \langle argument \rangle \}$ at each “star symbol” and applies `\l__spintent_spunit_process_base_exp:n` to evaluate and render each resulting unit individually.

```

668 \cs_new_protected:Nn \l__spintent_spunit_process_mult:n

```

```

669 {
670   \bool_set_true:N \__spintent_spunit_is_mult_bool
671   \seq_set_split:Nnn \__spintent_spunit_mult_seq { * } { #1 }
672   \seq_map_function:NN \__spintent_spunit_mult_seq \__spintent_spunit_process_base_exp:n
673 }
674 \cs_generate_variant:Nn \__spintent_spunit_process_mult:n { V }

```

(End of definition for `\__spintent_spunit_process_mult:n`.)

`\__spintent_print_spunit_hierarchy:`

The function `\__spintent_print_spunit_hierarchy:` handles the layout of inline “unit” fractions. It first processes the “units” before the slash (the numerator). If a denominator exists, it prints the slash / ‘/’ with its corresponding speech intent, and then processes the remaining “units”.

```

675 \cs_new_protected:Nn \__spintent_print_spunit_hierarchy:
676 {
677   \tl_if_empty:NF \__spintent_luaset_arg_above_tl
678   {
679     \__spintent_spunit_process_mult:V \__spintent_luaset_arg_above_tl
680     \tl_if_empty:NF \__spintent_luaset_arg_below_tl
681     {
682       \MathMLintent { \__spintent_spunit_read_slash_str } { / }
683       \__spintent_spunit_process_mult:V \__spintent_luaset_arg_below_tl
684     }
685   }
686 }

```

(End of definition for `\__spintent_print_spunit_hierarchy:`.)

`\__spintent_spunit_safe_exec:n`

The function `\__spintent_spunit_safe_exec:n` performs the main validation and layout for “units”. It first checks if the `{⟨argument⟩}` has units, is a valid expression, and does not contain illegal numbers in the denominator, throwing the appropriate “errors” if any check fails.

If the unit is valid, it checks if it is a sexagesimal angle (degrees, minutes, or seconds). For sexagesimal “units”, it prints the number and the symbol “without any space in between”, applying a slight negative space ‘\!’ for arcseconds. When the number is exactly 1, with no decimal part, the reading becomes the singular literal `_grado:unit`, `_minuto:unit` or `_segundo:unit`, since neither the fixed plural names nor `MathCAT` can see the number. For standard units, it prints the number followed by a thin space ‘\,’ and then processes the unit sequence.

```

687 \cs_new_protected:Nn \__spintent_spunit_safe_exec:n
688 {
689   \str_if_eq:VnTF \__spintent_luaset_has_units_str { false }
690   { \msg_error:nnn { spintent } { spunit-no-units } { #1 } }
691   {
692     \str_if_eq:VnTF \__spintent_luaset_status_str { valid }
693     {
694       \str_if_eq:VnTF
695       \__spintent_luaset_denom_has_number_str { true }
696       {
697         \msg_error:nnn { spintent }
698         { inline-numeric-denominator } { #1 }
699       }
700       {
701         \__spintent_luafun_spunit_lookup_alias:V \__spintent_luaset_arg_above_tl
702         \str_if_eq:VnTF \__spintent_spunit_luaset_is_sexagesimal_str { true }
703         {
704           \tl_if_empty:NF \__spintent_luaset_part_int_tl
705           {
706             \__spintent_spnum_print_number_core:
707           }
708           \bool_lazy_and:nnT
709           { \str_if_eq_p:ee { \__spintent_luaset_part_int_tl } { 1 } }
710           { \tl_if_empty_p:N \__spintent_luaset_part_dec_tl }
711           {
712             \str_set:Ne \__spintent_spunit_luaset_read_str
713             {
714               \str_case:Vn \__spintent_spunit_luaset_canonical_str
715               {
716                 { ° } { _grado:unit }
717                 { ' } { _minuto:unit }
718                 { " } { _segundo:unit }
719               }
720             }
721           }
722         }
723       }
724     }
725   }

```

```

722         \MathMLintent { \l__spintenc_spunit_lua_set_read_str }
723         {
724             \mathord
725             {
726                 \str_case:Vn \l__spintenc_spunit_lua_set_canonical_str
727                 {
728                     { " } { \prime \! \prime }
729                     { ' } { \prime }
730                     { ° } { ° }
731                 }
732             }
733         }
734     }
735     {
736         \tl_if_empty:NF \l__spintenc_lua_set_part_int_tl
737         {
738             \__spintenc_spnum_print_number_core: \,
739         }
740         \__spintenc_print_spunit_hierarchy:
741     }
742 }
743 }
744 { \msg_error:nnn { spintenc } { spunit-invalid-unit } { #1 } }
745 }
746 }

```

(End of definition for `\__spintenc_spunit_safe_exec:n`.)

11.15.5 Implementation of the `\spunit` command

`\spunit` The public user command `\spunit` accepts an optional [`(key = val)`] argument in ‘#1’ and a mandatory `{(unit expression)}` in ‘#2’. It first checks if it is executed inside *math mode*, raising an “error” if it is not. If correct, it opens a local group to apply the options and passes the `{(argument)}` to Lua module for parsing via `\__spintenc_luafun_clean_split_and_set:n` function.

Finally, it delegates the execution to the function `\__spintenc_spunit_safe_exec:n` to perform the safety checks and render the parsed data, before closing the group.

```

747 \NewDocumentCommand \spunit { O{} m }
748 {
749     \mode_if_math:TF
750     {
751         \group_begin:
752         \keys_set:nn { spintenc/spunit } { #1 }
753         \__spintenc_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
754         \__spintenc_spunit_safe_exec:n { #2 }
755         \group_end:
756     }
757     { \msg_error:nnn { spintenc } { need-math-mode } { \spunit } }
758 }

```

(End of definition for `\spunit`. This function is documented on page 4.)

11.16 The commands `\spnewunit` and `\spunitalias`

The user commands `\spnewunit` and `\spunitalias` allow users to add new custom “units” or “aliases” to the package’s dictionary. Similar to `\spunit`, these commands delegate the processing and validation to the `spintenc.lua` via `\__spintenc_luafun_define_custom_unit:nn` and `\__spintenc_luafun_define_spunit`. During the definition, the Lua module checks if the new “unit” or “alias” is valid and ensures it does not conflict with existing “units”. It returns the result of this operation to TeX using the `\l__spintenc_spunit_lua_set_status_s` variable. If a collision is detected or the `{(argument)}` is invalid, the package throws the corresponding “error” message.

11.16.1 Definition of error messages

spnewunit-duplicate
spunitalias-duplicate
spunitalias-invalid-expr

We define three error messages for the creation of “units” and “aliases”. The first two “errors” trigger if the user attempts to define a “new unit” or alias passed in ‘#1’ that already exists in the dictionary. The third “error” is raised by `\spunitalias` if the target expression provided in ‘#2’ is invalid (for example, if it contains unknown “units” or incorrect syntax).

```

759 \msg_new:nnnn { spintenc } { spnewunit-duplicate }
760 {
761     The~unit~symbol~or~alias~'#1'~is~already~defined!
762 }
763 {
764     You~cannot~redefine~with~\token_to_str:N \spnewunit.

```

```

765 }
766 \msg_new:nnnn { spintenc } { spunitalias-duplicate }
767 {
768   The~alias~'#1'~is~already~defined~or~reserved!
769 }
770 {
771   You~cannot~redefine~an~existing~unit~or~another~alias.
772 }
773 \msg_new:nnnn { spintenc } { spunitalias-invalid-expr }
774 {
775   The~expression~'#2'~for~alias~'#1'~is~invalid.
776 }
777 {
778   It~must~contain~only~defined~units,~exponents~and~one~'/' .
779 }

```

(End of definition for `spnewunit-duplicate`, `spunitalias-duplicate`, and `spunitalias-invalid-expr`.)

`\spnewunit` The public command `\spnewunit` defines a new custom unit. It passes the new unit symbol ‘#1’ and its speech text ‘#2’ to Lua via `\__spintenc_luafun_define_custom_unit:nn`. If the module reports that the unit already exists, it throws the corresponding duplicate error.

```

780 \NewDocumentCommand \spnewunit { m m }
781 {
782   \mode_if_math:TF
783   {
784     \msg_error:nnn { spintenc } { need-text-mode } { \spnewunit }
785   }
786   {
787     \__spintenc_luafun_define_custom_unit:nn { #1 } { #2 }
788     \str_if_eq:VnT \__spintenc_spunit_luaset_status_str { duplicate }
789     {
790       \msg_error:nnn { spintenc } { spnewunit-duplicate } { #1 }
791     }
792   }
793 }

```

(End of definition for `\spnewunit`. This function is documented on page 6.)

`\spunitalias` The public command `\spunitalias` creates a shortcut for a complex unit expression. It passes the new alias name ‘#1’ and the target expression ‘#2’ to Lua via `\__spintenc_luafun_define_spunit_alias:nn`. It throws an error if the alias already exists, or if the provided target expression contains invalid syntax or unknown units.

```

794 \NewDocumentCommand \spunitalias { m m }
795 {
796   \mode_if_math:TF
797   {
798     \msg_error:nnn { spintenc } { need-text-mode } { \spunitalias }
799   }
800   {
801     \__spintenc_luafun_define_spunit_alias:nn { #1 } { #2 }
802     \str_if_eq:VnT \__spintenc_spunit_luaset_status_str { duplicate }
803     {
804       \msg_error:nnn { spintenc } { spunitalias-duplicate } { #1 }
805     }
806     \str_if_eq:VnT \__spintenc_spunit_luaset_status_str { invalid-expr }
807     {
808       \msg_error:nnnn { spintenc } { spunitalias-invalid-expr } { #1 } { #2 }
809     }
810   }
811 }

```

(End of definition for `\spunitalias`. This function is documented on page 6.)

11.17 The command `\spmoney`

The user command `\spmoney` is the *third* of “the big four” provided by the `spintenc` package, designed to format monetary values and currencies. It delegates `{(argument)}` parsing to the Lua module `spintenc.lua` via `\__spintenc_luafun_clean_split_and_set:n`. The module then retrieves the correct “currency symbol”, pluralization rules, and layout structures from the database.

The command accepts *optional arguments* to override the default “currency symbol” or enable “subunit” rendering. Finally, it constructs the MathML intent for MathCAT.

```

\l__spintent_spmoney_luaset_symbol_str
\l__spintent_spmoney_luaset_position_str
\l__spintent_spmoney_luaset_sing_str
\l__spintent_spmoney_luaset_plur_str
\l__spintent_spmoney_luaset_conde_str
\l__spintent_spmoney_luaset_subsing_str
\l__spintent_spmoney_luaset_subplur_str
\l__spintent_spmoney_luaset_print_iso_str
\l__spintent_spmoney_luaset_currency_arg_str
\l__spintent_spmoney_luaset_status_str

```

These string variables store the currency data returned by the Lua module `spintent.lua`. Upon a successful database lookup, the module populates these variables with the “*currency symbol*”, its layout position (pre or post), the appropriate singular and plural names for both the main unit and subunits, and the execution status flags. T_EX then uses this information to render the visual layout and construct the correct MathML speech intents.

```

812 \str_new:N \l__spintent_spmoney_luaset_symbol_str
813 \str_new:N \l__spintent_spmoney_luaset_position_str
814 \str_new:N \l__spintent_spmoney_luaset_sing_str
815 \str_new:N \l__spintent_spmoney_luaset_plur_str
816 \str_new:N \l__spintent_spmoney_luaset_conde_str
817 \str_new:N \l__spintent_spmoney_luaset_subsing_str
818 \str_new:N \l__spintent_spmoney_luaset_subplur_str
819 \str_new:N \l__spintent_spmoney_luaset_print_iso_str
820 \str_new:N \l__spintent_spmoney_luaset_currency_arg_str
821 \str_new:N \l__spintent_spmoney_luaset_status_str

```

(End of definition for `\l__spintent_spmoney_luaset_symbol_str` and others.)

```

\l__spintent_spmoney_sub_units_bool
\l__spintent_spmoney_intent_money_str
\l__spintent_spmoney_intent_sign_str
\l__spintent_spmoney_intent_sub_unit_str
\l__spintent_spmoney_res_main_voice_str
\l__spintent_spmoney_subnum_int
\l__spintent_spmoney_extracted_curr_str
\l__spintent_spmoney_extracted_num_tl

```

The boolean variable tracks whether subunit rendering is enabled. The string variables assemble the specific speech text (intents) for NVDA/MathCAT, including the sign, the main currency, and the subunits. Finally, the remaining variables temporarily store the numeric value and currency code extracted from the user’s input.

```

822 \bool_new:N \l__spintent_spmoney_sub_units_bool
823 \str_new:N \l__spintent_spmoney_intent_money_str
824 \str_new:N \l__spintent_spmoney_intent_sign_str
825 \str_new:N \l__spintent_spmoney_intent_sub_unit_str
826 \str_new:N \l__spintent_spmoney_res_main_voice_str
827 \int_new:N \l__spintent_spmoney_subnum_int
828 \str_new:N \l__spintent_spmoney_extracted_curr_str
829 \tl_new:N \l__spintent_spmoney_extracted_num_tl

```

(End of definition for `\l__spintent_spmoney_sub_units_bool` and others.)

11.17.1 Definition of error messages

```

currency-no-subunits
invalid-currency

```

We define two messages to handle invalid currency configurations. The first “error” warns the user if subunit rendering is requested for a currency that does not support fractional divisions (such as JPY or CLP), safely ignoring the flag. The second “error” is triggered if the provided currency identifier is not registered in the Lua database or is not written validly: an alias or symbol with any uppercase letter, or an ISO code that is neither all lowercase nor all uppercase.

```

830 \msg_new:nnnn { spintent } { currency-no-subunits }
831 {
832   The~currency~'#1'~does~not~support~subunits.~
833   The~option~'sub-units=true'~is~ignored.
834 }
835 {
836   Monies~like~the~Japanese~Yen~(JPY)~or~Chilean~Peso~(CLP)~
837   do~not~have~fractional~subunits~registered~in~Lua~module.~
838 }
839 \msg_new:nnnn { spintent } { invalid-currency }
840 {
841   Invalid~currency~identifier~'#1'~in~\token_to_str:N \spmoney .
842 }
843 {
844   The~provided~symbol,~alias,~or~ISO~code~does~not~exist~in~
845   the~Lua~module.
846 }

```

(End of definition for `currency-no-subunits` and `invalid-currency`.)

11.17.2 Auxiliary functions for `\spmoney`

```

\__spintent_spmoney_set_currency:n
\__spintent_spmoney_normalize_key:n
\__spintent_spmoney_normalize_key:V

```

The function `\__spintent_spmoney_set_currency:n` validates the currency identifier provided in ‘#1’ against the database. It delegates the lookup to the helper function `\__spintent_spmoney_normalize_key:n`, which interfaces with the Lua module `spintent.lua`. If the module returns a not found status, an “error” is triggered. Otherwise, it stores the valid currency identifier in `\l__spintent_spmoney_luaset_currency_arg_str` for subsequent processing.

```

847 \cs_new_protected:Nn \__spintent_spmoney_set_currency:n
848 {
849   \__spintent_spmoney_normalize_key:n {#1}
850   \str_if_eq:VnTF \l__spintent_spmoney_luaset_status_str { notfound }
851   {
852     \msg_error:nnn { spintent } { invalid-currency } {#1}

```

```

853     }
854     { \str_set:Nn \l__spintent_spmoney_luaset_currency_arg_str {#1} }
855   }
856   \cs_new_protected:Nn \__spintent_spmoney_normalize_key:n
857   {
858     \__spintent_luafun_spmoney_normalize_key:n { \exp_not:n { #1 } }
859   }
860   \cs_generate_variant:Nn \__spintent_spmoney_normalize_key:n { V }

```

(End of definition for `\__spintent_spmoney_set_currency:n` and `\__spintent_spmoney_normalize_key:n`.)

11.17.3 Definition of keys for `\spmoney`

Now we add the keys `currency`, `sub-units`, `money-math` and `cifras-sep`.

```

currency
sub-units
money-math
cifras-sep
861 \keys_define:nn { spintent / spmoney }
862 {
863   currency      .code:n      = { \__spintent_spmoney_set_currency:n {#1} },
864   currency      .initial:n    = { pesos },
865   currency      .value_required:n = true,
866   sub-units     .bool_set:N   = \l__spintent_spmoney_sub_units_bool,
867   sub-units     .value_required:n = true,
868   money-math    .bool_set:N   = \l__spintent_spmoney_money_math_bool,
869   money-math    .initial:n    = { true },
870   money-math    .value_required:n = true,
871   cifras-sep    .meta:nn      = { spintent/spnum } { cifras-sep = {#1} },
872   cifras-sep    .value_required:n = true,
873   unknown       .code:n      =
874   {
875     \str_set:Nn \l__spintent_command_name_str { spmoney }
876     \__spintent_unknown_keys_cmd:n {#1}
877   },
878 }

```

(End of definition for `currency` and others.)

11.17.4 Internal functions for `\spmoney`

`\__spintent_spmoney_validate_money:` The function `\__spintent_spmoney_validate_money:` verifies if the parsed input contains an embedded currency identifier by checking `\l__spintent_luaset_has_units_str`. If a currency is detected, it extracts and normalizes the key from `\l__spintent_luaset_arg_above_tl`. It throws an “error” if the currency is not found in the database. If valid, it updates `\l__spintent_spmoney_luaset_currency_arg_str` and retrieves the full currency metadata from the `spintent.lua` via `\__spintent_luafun_spmoney_lookup_data:V`.

```

879 \cs_new_protected:Nn \__spintent_spmoney_validate_money:
880 {
881   \str_if_eq:VnT \l__spintent_luaset_has_units_str { true }
882   {
883     \__spintent_spmoney_normalize_key:V \l__spintent_luaset_arg_above_tl
884     \str_if_eq:VnTF \l__spintent_spmoney_luaset_status_str { notfound }
885     { \msg_error:nne { spintent } { invalid-currency } { \l__spintent_luaset_arg_above_tl } }
886     {
887       \str_set:NV
888         \l__spintent_spmoney_luaset_currency_arg_str
889         \l__spintent_luaset_arg_above_tl
890     }
891   }
892   \str_set:Ne \l__spintent_spmoney_luaset_status_str { found }
893   \__spintent_luafun_spmoney_lookup_data:V \l__spintent_spmoney_luaset_currency_arg_str
894 }

```

(End of definition for `\__spintent_spmoney_validate_money:`.)

`\__spintent_spmoney_print_symbol:` The function `\__spintent_spmoney_print_symbol:` renders the “*currency symbol*”. If `\l__spintent_spmoney_luaset_currency_arg_str` is `true`, it outputs the ISO code using `\mathrm`. Otherwise, it prints the “*standard symbol*”, directly in math mode when `\l__spintent_spmoney_money_math_bool` is `true`, or wrapped inside `\text` when it is `false`. For dollar signs ‘\$’, the symbol is typeset through the control sequence `\$` in both cases.

```

895 \cs_new_protected:Nn \__spintent_spmoney_print_symbol:
896 {
897   \str_if_eq:VnTF \l__spintent_spmoney_luaset_print_iso_str { true }
898   { \mathrm { \l__spintent_spmoney_luaset_currency_arg_str } }
899   {
900     \str_if_eq:VnTF \l__spintent_spmoney_luaset_symbol_str { $ }
901     {
902       \bool_if:NTF \l__spintent_spmoney_money_math_bool

```

```

903         { \ $ } { \text { \ $ } }
904     }
905     {
906         \bool_if:NTF \__spintent_spmoney_money_math_bool
907         { \__spintent_spmoney_luaset_symbol_str }
908         { \text { \__spintent_spmoney_luaset_symbol_str } }
909     }
910 }
911 }

```

(End of definition for `\__spintent_spmoney_print_symbol:`)

`\__spintent_spmoney_render_layout:`

The function `\__spintent_spmoney_render_layout:` determines the layout order of the “*currency symbol*” relative to the numeric value. It evaluates `\__spintent_spmoney_luaset_position_str`; if set to `pre`, it renders the “*currency symbol*” *before* the number node. Otherwise, it places the number first, followed by the “*currency symbol*”.

```

912 \cs_new_protected:Nn \__spintent_spmoney_render_layout:
913 {
914     \str_if_eq:VnTF \__spintent_spmoney_luaset_position_str { pre }
915     {
916         \__spintent_spmoney_print_symbol:
917         \MathMLarg { n } { \__spintent_spmoney_render_number_node: }
918     }
919     {
920         \MathMLarg { n } { \__spintent_spmoney_render_number_node: }
921         \__spintent_spmoney_print_symbol:
922     }
923 }

```

(End of definition for `\__spintent_spmoney_render_layout:`)

`\__spintent_spmoney_render_number_node:`
`\__spintent_spmoney_render_only_integer_node:`

The function `\__spintent_spmoney_render_number_node:` renders the full numeric value, encompassing both the integer and decimal components. Conversely, the function `\__spintent_spmoney_render_only_integer_node:` outputs exclusively the integer portion, omitting any decimal data.

```

924 \cs_new_protected:Nn \__spintent_spmoney_render_number_node:
925 {
926     \__spintent_spmoney_render_int:
927     \__spintent_spmoney_print_part_dec:
928 }
929 \cs_new_protected:Nn \__spintent_spmoney_render_only_integer_node:
930 {
931     \__spintent_spmoney_render_int:
932 }

```

(End of definition for `\__spintent_spmoney_render_number_node:` and `\__spintent_spmoney_render_only_integer_node:`)

`\__spintent_spmoney_print_int:`

The function `\__spintent_spmoney_print_int:` evaluates the presence of an explicit sign in `\__spintent_luaset_sign_tl`. If a sign is detected, it assigns a prefix speech intent ‘*más*’ or ‘*menos*’ and embeds the sign and subsequent layout inside a `\MathMLintent`. If no sign is present, it routes execution based on whether subunits are enabled.

```

933 \cs_new_protected:Nn \__spintent_spmoney_print_int:
934 {
935     \tl_if_empty:NTF \__spintent_luaset_sign_tl
936     {
937         \bool_if:NTF \__spintent_spmoney_sub_units_bool
938         { \__spintent_spmoney_set_and_print_sub_units: }
939         { \__spintent_spmoney_intent_int: }
940     }
941     {
942         \str_if_eq:VnTF \__spintent_luaset_sign_tl { + }
943         {
944             \str_set:Nn \__spintent_spmoney_intent_sign_str { _más:prefix ($n) }
945         }
946         {
947             \str_set:Nn \__spintent_spmoney_intent_sign_str { _menos:prefix ($n) }
948         }
949         \__spintent_invisible_sep:
950         \MathMLintent { \__spintent_spmoney_intent_sign_str }
951         {
952             \__spintent_luaset_sign_tl
953             \MathMLarg { n }

```

```

954         {
955             \bool_if:NTF \l__spintent_spmoney_sub_units_bool
956             { \__spintent_spmoney_set_and_print_sub_units: }
957             { \__spintent_spmoney_intent_int: }
958         }
959     }
960 }
961 }

```

(End of definition for `\__spintent_spmoney_print_int:`)

`\__spintent_spmoney_intent_int:`

The function `\__spintent_spmoney_intent_int:` determines the correct grammatical inflection for the currency string (singular, plural, or the prepositional ‘de’ variant for exact millions). It evaluates the integer component, decimal boundaries, and zero-fills to select the appropriate string register, applying it as a postfix MathML intent over the layout rendered by `\__spintent_spmoney_render_layout:`.

```

962 \cs_new_protected:Nn \__spintent_spmoney_intent_int:
963 {
964     \tl_if_empty:NTF \l__spintent_luaset_dec_sep_tl
965     {
966         \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
967         {
968             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_sing_str
969         }
970         {
971             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
972         }
973         \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
974         {
975             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_conde_s
976         }
977     }
978     {
979         \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
980         \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
981         {
982             \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
983             {
984                 \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_conde
985             }
986             {
987                 \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plu
988             }
989         }
990     }
991     \__spintent_invisible_sep:
992     \MathMLintent { _ \l__spintent_spmoney_intent_money_str :postfix($n) }
993     {
994         \__spintent_spmoney_render_layout:
995     }
996 }

```

(End of definition for `\__spintent_spmoney_intent_int:`)

`\__spintent_spmoney_check_sub_units:`

The function `\__spintent_spmoney_check_sub_units:` verifies whether the selected currency supports “subunits”. If the database returns empty subunit data, it triggers a warning and automatically disables the “subunit” rendering flag. The flag is also disabled, without a warning, when the amount has no decimal part, since there are no “subunits” to voice.

The function `\__spintent_spmoney_set_and_print_sub_units:` processes and formats the speech intents when “subunits” are active. It normalizes the decimal component (for instance, padding a single-digit decimal like ‘.5’ to ‘.50’), evaluates the correct singular or plural inflections for both the main “currency” and the “subunits”, and delegates the final MathML tree assembly to `\__spintent_spmoney_build_tree:`.

```

997 \cs_new_protected:Nn \__spintent_spmoney_check_sub_units:
998 {
999     \str_if_empty:NT \l__spintent_spmoney_luaset_subsing_str
1000     {
1001         \msg_warning:nne { spintent } { currency-no-subunits }
1002         { \l__spintent_spmoney_luaset_currency_arg_str }
1003         \bool_set_false:N \l__spintent_spmoney_sub_units_bool
1004     }
1005     \tl_if_empty:NT \l__spintent_luaset_dec_sep_tl
1006     { \bool_set_false:N \l__spintent_spmoney_sub_units_bool }

```

```

1007   }
1008   \cs_new_protected:Nn \__spintent_spmoney_set_and_print_sub_units:
1009   {
1010     \int_compare:nTF { \str_count:N \__spintent_luaset_only_part_dec_str == 1 }
1011     {
1012       \int_set:Nn \__spintent_spmoney_subnum_int
1013       { \__spintent_luaset_only_part_dec_str * 10 }
1014       \str_set:Ne \__spintent_luaset_only_part_dec_str
1015       { \__spintent_luaset_only_part_dec_str 0 }
1016     }
1017     {
1018       \int_set:Nn \__spintent_spmoney_subnum_int
1019       { \__spintent_luaset_only_part_dec_str }
1020     }
1021     \int_compare:nTF { \__spintent_spmoney_subnum_int == 1 }
1022     {
1023       \str_set:Ne \__spintent_spmoney_intent_sub_unit_str
1024       { _ \__spintent_spmoney_luaset_subsing_str :postfix($sub) }
1025     }
1026     {
1027       \str_set:Ne \__spintent_spmoney_intent_sub_unit_str
1028       { _ \__spintent_spmoney_luaset_subplur_str :postfix($sub) }
1029     }
1030     \str_if_eq:VnTF \__spintent_luaset_only_part_int_str { 1 }
1031     {
1032       \str_set:NV \__spintent_spmoney_res_main_voice_str
1033       \__spintent_spmoney_luaset_sing_str
1034     }
1035     {
1036       \str_set:NV \__spintent_spmoney_res_main_voice_str
1037       \__spintent_spmoney_luaset_plur_str
1038     }
1039     \str_if_eq:VnT \__spintent_luaset_millions_str { true }
1040     {
1041       \str_set:NV \__spintent_spmoney_res_main_voice_str
1042       \__spintent_spmoney_luaset_conde_str
1043     }
1044     \__spintent_spmoney_build_tree:
1045   }

```

(End of definition for __spintent_spmoney_check_sub_units: and __spintent_spmoney_set_and_print_sub_units:.)

The function `\__spintent_spmoney_build_tree:` structures the final MathML tree assembly for fractional “currencies”. It arranges the integer component, the decimal separator, and the “subunit” component based on the layout position (pre or post) of the “currency symbol”.

Furthermore, `\__spintent_spmoney_print_sep_con:` applies the specific speech intent ‘_con’ to the decimal separator, while `\__spintent_spmoney_print_dec_sub_unit:` binds the computed “subunit” intent to the decimal digits, ensuring accurate accessibility output.

```

1046 \cs_new_protected:Nn \__spintent_spmoney_build_tree:
1047 {
1048   \str_if_eq:VnTF \__spintent_spmoney_luaset_position_str { pre }
1049   {
1050     \MathMLintent { _ \__spintent_spmoney_res_main_voice_str :postfix($n) }
1051     {
1052       \__spintent_spmoney_print_symbol:
1053       \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1054     }
1055     \__spintent_spmoney_print_sep_con:
1056     \__spintent_spmoney_print_dec_sub_unit:
1057   }
1058   {
1059     \MathMLintent { _ \__spintent_spmoney_res_main_voice_str :postfix($n) }
1060     {
1061       \__spintent_inner_sep:
1062       \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1063     }
1064     \__spintent_spmoney_print_sep_con:
1065     \__spintent_spmoney_print_dec_sub_unit:
1066     \MathMLintent { :silent } { \__spintent_spmoney_print_symbol: }
1067   }
1068 }

```

```

1069 \cs_new_protected:Nn \__spintent_spmoney_print_sep_con:
1070 {
1071   \MathMLintent { _con }
1072   {
1073     \mathord { \l__spintent_luaset_dec_sep_tl }
1074   }
1075 }
1076 \cs_new_protected:Nn \__spintent_spmoney_print_dec_sub_unit:
1077 {
1078   \MathMLintent { \l__spintent_spmoney_intent_sub_unit_str }
1079   {
1080     \__spintent_inner_sep:
1081     \MathMLarg { sub }
1082     {
1083       \__spintent_spmoney_render_dec:
1084     }
1085   }
1086 }

```

(End of definition for `\__spintent_spmoney_build_tree:`, `\__spintent_spmoney_print_sep_con:`, and `\__spintent_spmoney_print_dec_sub_unit:`.)

`\__spintent_spmoney_parse_and_route:n`

The internal function `\__spintent_spmoney_parse_and_route:n` processes the $\{(\textit{argument})\}$ passed in ‘#1’ using Lua module to extract the numeric value and any “currency” identifier. If a “currency” is found (`\l__spintent_spmoney_extracted_curr_str`), it sets the corresponding variables. It then calls `\__spintent_spmoney_validate_money:` to verify the “currency”. If “subunit” rendering is enabled (`\l__spintent_spmoney_sub_units_bool`), it calls `\__spintent_spmoney_check_sub_units:`. Finally, it calls `\__spintent_spmoney_print_int:` to format the output.

```

1087 \cs_new_protected:Nn \__spintent_spmoney_parse_and_route:n
1088 {
1089   \__spintent_luafun_spmoney_prepare_input:n { \exp_not:n { #1 } }
1090   \__spintent_luafun_clean_split_and_set:n { \exp_not:V \l__spintent_spmoney_extracted_num_tl }
1091   \str_if_empty:NF \l__spintent_spmoney_extracted_curr_str
1092   {
1093     \str_set:Nx \l__spintent_luaset_has_units_str { true }
1094     \str_set:NV \l__spintent_luaset_arg_above_tl \l__spintent_spmoney_extracted_curr_str
1095   }
1096   \__spintent_spmoney_validate_money:
1097   \bool_if:NT \l__spintent_spmoney_sub_units_bool
1098   { \__spintent_spmoney_check_sub_units: }
1099   \__spintent_spmoney_print_int:
1100 }

```

(End of definition for `\__spintent_spmoney_parse_and_route:n`.)

`\spmoney`

The public document command `\spmoney` formats monetary values with accessible speech intents. It accepts an optional argument ‘#1’ for local key-value configurations and a mandatory argument ‘#2’ containing the numeric value and optional embedded currency identifier. It first evaluates the typesetting context, triggering a fatal error if executed outside of math mode. Upon validation, it establishes a local $\TeX$ group to safely isolate the key assignments, delegates the raw input string to `\__spintent_spmoney_parse_and_route:n` for parsing and rendering, and subsequently closes the group to prevent scope leakage.

```

1101 \NewDocumentCommand \spmoney { 0{} m }
1102 {
1103   \mode_if_math:TF
1104   {
1105     \group_begin:
1106     \keys_set:nn { spintent / spmoney } { #1 }
1107     \__spintent_spmoney_parse_and_route:n { #2 }
1108     \group_end:
1109   }
1110   { \msg_error:nnn { spintent } { need-math-mode } { \spmoney } }
1111 }

```

(End of definition for `\spmoney`. This function is documented on page 6.)

11.18 The command `\spshort`

The user command `\spshort` is the *fourth* and *final* of “the big four”. It manages the semantic injection and typographical formatting of *text mode* abbreviations, ordinals, and acronyms. Unlike *math mode* commands, which rely on `MathML` attributes, this command uses the PDF *tagging tools* provided by the `tagpdf`[29] package. Specifically, it uses the `E` (expanded) key to ensure that `NVDA` interpret the full form of the abbreviation, ordinal, or acronym in *text mode*.

```

\__spintent_spshort_luaset_status_str
\__spintent_spshort_luaset_layout_str
\__spintent_spshort_luaset_expand_str
\__spintent_spshort_luaset_output_str
\__spintent_spshort_luaset_base_str
\__spintent_spshort_luaset_suffix_str

```

These internal string variables serve as the primary data interface with the Lua module `spintent.lua`. During processing, the Lua module “stores” within these registers the execution status, the targeted typographical layout parameters, the phonetic replacement text for the `E` key intended for NVDA, the final sanitized output string, and the isolated structural components for ordinals (base and suffix).

```

1112 \str_new:N \__spintent_spshort_luaset_status_str
1113 \str_new:N \__spintent_spshort_luaset_layout_str
1114 \str_new:N \__spintent_spshort_luaset_expand_str
1115 \str_new:N \__spintent_spshort_luaset_output_str
1116 \str_new:N \__spintent_spshort_luaset_base_str
1117 \str_new:N \__spintent_spshort_luaset_suffix_str

```

(End of definition for `\__spintent_spshort_luaset_status_str` and others.)

11.18.1 Definition of error messages for `\spshort`

unknown-abbreviation

The internal “error” message `unknown-abbreviation` is triggered when the command `\spshort` encounters an invalid lookup state from the Lua module. It acts as a strict fallback exception when the provided `{⟨argument⟩}` cannot be mapped to any registered database entry in `spintent.lua` nor successfully resolved through the grammatical ordinal expansion rules.

```

1118 \msg_new:nnnn { spintent } { unknown-abbreviation }
1119 { The~abbreviation~or~ordinal~'#1'~is~invalid~or~not~recognized. }
1120 { Verify~the~spelling~or~check~the~grammar~rules~for~ordinals. }

```

(End of definition for `unknown-abbreviation`.)

11.18.2 Accessibility interface

`\__spintent_spshort_tag_span:nn`

`\__spintent_spshort_tag_span:en`

The internal function `\__spintent_spshort_tag_span:nn` processes the `{⟨argument⟩}` passed in ‘#2’ using the tools provided by `tagpdf`. By utilizing the `tag` and `E` keys, it creates a native PDF /E (`#1`) structure and expands the `{⟨argument⟩}` passed in ‘#1’ for proper reading with NVDA.

```

1121 \cs_new_protected:Nn \__spintent_spshort_tag_span:nn
1122 {
1123   \tag_mc_end_push:
1124   \tag_struct_begin:n { tag=Span, E={ #1 } }
1125   \tag_mc_begin:n { tag=Span }
1126   \tag_suspend:n { \spshort }
1127   #2
1128   \tag_resume:n { \spshort }
1129   \tag_mc_end:
1130   \tag_struct_end:
1131   \tag_mc_begin_pop:n {}
1132 }
1133 \cs_generate_variant:Nn \__spintent_spshort_tag_span:nn { en }

```

(End of definition for `\__spintent_spshort_tag_span:nn`.)

11.18.3 Definition of keys for `\spshort`

small-caps

read-arg

Now we add the keys `small-caps` and `read-arg`.

```

1134 \keys_define:nn { spintent / spshort }
1135 {
1136   small-caps .bool_set:N = \__spintent_spshort_smallcaps_bool,
1137   small-caps .initial:n = false,
1138   small-caps .value_required:n = true,
1139   read-arg .str_set:N = \__spintent_spshort_read_arg_str,
1140   read-arg .initial:n = {},
1141   read-arg .value_required:n = true,
1142   unknown .code:n =
1143   {
1144     \str_set:Nn \__spintent_command_name_str { spshort }
1145     \__spintent_unknown_keys_cmd:n {#1}
1146   },
1147 }

```

(End of definition for `small-caps` and `read-arg`.)

11.18.4 Internal functions for `\spshort`

`\__spintent_spshort_print:nn`

The internal function `\__spintent_spshort_print:nn` acts as the primary formatter for text abbreviations. It clears the string variable `\__spintent_spshort_read_arg_str` and evaluates the optional `⟨keys⟩` passed in ‘#1’. It then checks if a manual reading override was provided via those keys. If that string variable remains empty, it calls `\__spintent_spshort_process_and_render:n` to process the `{⟨argument⟩}` passed in ‘#2’. Otherwise, if an override is detected, it directly delegates the execution to `\__spintent_spshort_render_bypass:n`.

```

1148 \cs_new_protected:Nn \__spintent_spshort_print:nn

```

```

1149 {
1150   \str_clear:N \l__spintent_spshort_read_arg_str
1151   \keys_set:nn { spintent / spshort } {#1}
1152   \str_if_empty:NTF \l__spintent_spshort_read_arg_str
1153   {
1154     \__spintent_spshort_process_and_render:n {#2}
1155   }
1156   {
1157     \__spintent_spshort_render_bypass:n {#2}
1158   }
1159 }

```

(End of definition for `\__spintent_spshort_print:nn`.)

`\__spintent_spshort_process_and_render:n`

The internal function `\__spintent_spshort_process_and_render:n` interfaces with the Lua module `spintent.lua` function `\__spintent_luafun_spshort_process:n` to resolve the abbreviation or ordinal `{⟨argument⟩}` passed in ‘#1’.

It subsequently evaluates the execution state stored within the string variable `\l__spintent_spshort_luaset_status` and dynamically routes execution to the corresponding rendering subroutine success, fallback or `error`. Any unrecognized status automatically defaults to the “error” handler.

```

1160 \cs_new_protected:Nn \__spintent_spshort_process_and_render:n
1161 {
1162   \__spintent_luafun_spshort_process:n {#1}
1163   \str_case:VnF \l__spintent_spshort_luaset_status_str
1164   {
1165     { success } { \__spintent_spshort_render_success: }
1166     { fallback } { \__spintent_spshort_render_fallback: }
1167     { error } { \__spintent_spshort_render_error:n {#1} }
1168   }
1169   { \__spintent_spshort_render_error:n {#1} }
1170 }

```

(End of definition for `\__spintent_spshort_process_and_render:n`.)

`\__spintent_spshort_render_success:`

`\__spintent_spshort_render_fallback:`

`\__spintent_spshort_render_bypass:n`

`\__spintent_spshort_render_error:n`

The function `\__spintent_spshort_render_success:` injects the key `E` and executes the targeted typographical layout. The `\__spintent_spshort_render_fallback:` function handles default cases, applying optional formatting such as SMALL CAPS to unrecognized or partially resolved terms.

Conversely, the function `\__spintent_spshort_render_bypass:n` directly maps the user-defined “speech” to the `{⟨argument⟩}`, bypassing the automated lookup Lua module. Finally, the function `\__spintent_spshort_render_error:n` triggers a structural exception while yielding the unformatted `{⟨argument⟩}` to prevent fatal compilation.

```

1171 \cs_new_protected:Nn \__spintent_spshort_render_success:
1172 {
1173   \__spintent_spshort_tag_span:en
1174   { \l__spintent_spshort_luaset_expand_str }
1175   { \__spintent_spshort_execute_layout: }
1176 }
1177 \cs_new_protected:Nn \__spintent_spshort_render_fallback:
1178 {
1179   \__spintent_spshort_tag_span:en
1180   { \l__spintent_spshort_luaset_expand_str }
1181   {
1182     \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1183     { \textsc{ \l__spintent_spshort_luaset_output_str } }
1184     { \l__spintent_spshort_luaset_output_str }
1185   }
1186 }
1187 \cs_new_protected:Nn \__spintent_spshort_render_bypass:n
1188 {
1189   \__spintent_spshort_tag_span:en { \l__spintent_spshort_read_arg_str } { #1 }
1190 }
1191 \cs_new_protected:Nn \__spintent_spshort_render_error:n
1192 {
1193   \msg_error:nnn { spintent } { unknown-abbreviation } {#1}
1194 }

```

(End of definition for `\__spintent_spshort_render_success:` and others.)

`\__spintent_spshort_execute_layout:`

The internal function `\__spintent_spshort_execute_layout:` executes the typographical layout. It evaluates the descriptor “stored” within the string variable `\l__spintent_spshort_luaset_layout_str` (such as `linear_regular`, `superscript`, or `small_caps_pure`) and maps it to the corresponding $\LaTeX$

formatting. Furthermore, it manages complex nested configurations, dynamically applying combinations such as superscripts integrated with conditional SMALL CAPS based on the active boolean variables.

```

1195 \cs_new_protected:Nn \__spintent_spshort_execute_layout:
1196 {
1197   \str_case:VnF \l__spintent_spshort_luaset_layout_str
1198   {
1199     { linear_regular }
1200     {
1201       \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1202       { \textsc{ \l__spintent_spshort_luaset_output_str } }
1203       { \l__spintent_spshort_luaset_output_str }
1204     }
1205     { linear_caps }
1206     { \l__spintent_spshort_luaset_output_str }
1207     { linear_mixed }
1208     { \l__spintent_spshort_luaset_output_str }
1209     { superscript }
1210     {
1211       \l__spintent_spshort_luaset_base_str
1212       \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1213       {
1214         \textsuperscript{
1215           \textsc{
1216             \str_lowercase:V \l__spintent_spshort_luaset_suffix_str
1217           }
1218         }
1219       }
1220       {
1221         \textsuperscript{ \l__spintent_spshort_luaset_suffix_str }
1222       }
1223     }
1224     { small_caps_pure }
1225     {
1226       \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1227       {
1228         \textsc{
1229           \str_lowercase:V \l__spintent_spshort_luaset_output_str
1230         }
1231       }
1232       { \l__spintent_spshort_luaset_output_str }
1233     }
1234     { acronym_long }
1235     { \l__spintent_spshort_luaset_output_str }
1236   }
1237   { \l__spintent_spshort_luaset_output_str }
1238 }

```

(End of definition for `\__spintent_spshort_execute_layout:`.)

`\spshort` The public document command `\spshort` manages the accessible formatting of *text mode* abbreviations, acronyms, and ordinals. It first triggers an error if invoked within a *math mode*, upon validation, puts `\mode_leave_vertical:` and establishes a local group to call the function `\__spintent_spshort_print:nn` for processing and printing.

```

1239 \NewDocumentCommand \spshort { 0{} m }
1240 {
1241   \mode_if_math:TF
1242   {
1243     \msg_error:nnn { spintent } { need-text-mode } { \spshort }
1244   }
1245   {
1246     \mode_leave_vertical:
1247     \group_begin:
1248       \__spintent_spshort_print:nn {#1} {#2}
1249     \group_end:
1250   }
1251 }

```

(End of definition for `\spshort`. This function is documented on page 8.)

11.19 The command `\spsiglo`

The public document command `\spsiglo` provides a semantic interface for typesetting century designations (e.g., siglo XXI). It accepts both Arabic and Roman numerals as $\langle \textit{argument} \rangle$, interfacing with the Lua module `spintent.lua` to validate, parse, and normalize chronological values.

This command work in dual-mode context awareness, operating uniformly across both *text mode* and *math mode*. When executed within *math mode*, it constructs an accessible `MathML` intent expression tree. Conversely, when invoked in standard *text mode*, it encapsulates the rendered output within a native PDF tagging structure (such as `/Alt (#1)`) containing an `alt` key.

```
\l__spintent_spsiglo_luaset_error_str
\l__spintent_spsiglo_luaset_arabic_str
\l__spintent_spsiglo_luaset_roman_min_str
\l__spintent_spsiglo_print_era_str
```

These internal string variables handle data transfer and synchronization with the Lua module `spintent.lua`. They isolate the exception evaluation states, the Arabic numeral required for speech synthesis, the lowercase Roman numeral designated for small caps typesetting via `\textsc`, and the localized chronological era modifier.

```
1252 \str_new:N \l__spintent_spsiglo_luaset_error_str
1253 \str_new:N \l__spintent_spsiglo_luaset_arabic_str
1254 \str_new:N \l__spintent_spsiglo_luaset_roman_min_str
1255 \str_new:N \l__spintent_spsiglo_print_era_str
```

(End of definition for `\l__spintent_spsiglo_luaset_error_str` and others.)

11.19.1 Definition of error messages for `\spsiglo`

invalid-century-format

This internal error message is issued when the provided century identifier fails both Arabic and Roman numeral validation. It is triggered when the Lua module `spintent.lua` cannot map the $\langle \textit{argument} \rangle$ to an integer within the valid range of `1` to `4000`, preventing incorrect speech synthesis.

```
1256 \msg_new:nnnn { spintent } { invalid-century-format }
1257 {
1258   Invalid~century~format~in~'#1'.
1259 }
1260 {
1261   The~argument~'#2'~is~not~a~valid~Arabic~or~Roman~numeral.~
1262   Ensure~the~value~is~between~1~and~4000.
1263 }
```

(End of definition for `invalid-century-format`.)

11.19.2 Definition of keys for `\spsiglo`

print-era

Now we add the key `print-era`. User configuration determining if an era designation is appended, available values are `none`, `ac` (antes de cristo) and `dc` (después de cristo).

```
1264 \keys_define:nn { spintent / spsiglo }
1265 {
1266   print-era .choices:nn =
1267     { none , ac , dc }
1268   {
1269     \int_case:nn { \l_keys_choice_int }
1270     {
1271       {1} { \str_set:Nn \l__spintent_spsiglo_print_era_str { none } }
1272       {2} { \str_set:Nn \l__spintent_spsiglo_print_era_str { ac } }
1273       {3} { \str_set:Nn \l__spintent_spsiglo_print_era_str { dc } }
1274     }
1275   },
1276   print-era .initial:n = none,
1277   print-era .value_required:n = true,
1278   print-era / unknown .code:n =
1279     \msg_error:nnee { spintent } { unknown-choice }
1280     { print-era } { none , ~ac , ~dc } { \exp_not:n {#1} },
1281   unknown .code:n =
1282     {
1283       \str_set:Nn \l__spintent_command_name_str { spsiglo }
1284       \__spintent_unknown_keys_cmd:n {#1}
1285     },
1286 }
```

(End of definition for `print-era`.)

11.19.3 Internal functions for \spsiglo

The functions `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn` generate MathML structures when century designations are evaluated within *math mode*. They construct MathML intent structures, mapping the Arabic numerals to their phonetic equivalents. Furthermore, `\__spintent_spsiglo_render_with_era:nnn` integrates the era modifiers *ac* or *dc*. It adds invisible spacing and postfix intents to ensure correct NVDA/MathCAT pronunciation.

```

1287 \cs_new_protected:Nn \__spintent_spsiglo_render_simple:nn
1288 {
1289   \MathMLintent { #1:number } { \text { \textsc {#2} } }
1290 }
1291 \cs_generate_variant:Nn \__spintent_spsiglo_render_simple:nn { VV }
1292 \cs_new_protected:Nn \__spintent_spsiglo_render_with_era:nnn
1293 {
1294   \str_if_eq:nnTF { #1 } { ac }
1295   {
1296     \MathMLintent { _antes-de-cristo:postfix($z) }
1297     {
1298       \__spintent_inner_sep:
1299       \MathMLarg { z }
1300       {
1301         \MathMLintent { #2:number }
1302         {
1303           \text { \textsc {#3}~a.~C. }
1304         }
1305       }
1306     }
1307   }
1308   {
1309     \MathMLintent { _después-de-cristo:postfix($z) }
1310     {
1311       \__spintent_inner_sep:
1312       \MathMLarg { z }
1313       {
1314         \MathMLintent { #2:number }
1315         {
1316           \text { \textsc {#3}~d.~C. }
1317         }
1318       }
1319     }
1320   }
1321 }
1322 \cs_generate_variant:Nn \__spintent_spsiglo_render_with_era:nnn { VVV }

```

(End of definition for `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn`.)

The function `\__spintent_spsiglo_tag_span:nn` handles century formatting within *text mode*. It utilizes the `tagpdf` package to enclose the content inside a PDF /Alt (#1) structure. By passing the `{⟨argument⟩}` to the `alt` key, it embeds the chronological value directly into the PDF, ensuring correct NVDA/MathCAT pronunciation without relying on MathML intents.

```

1323 \cs_new_protected:Nn \__spintent_spsiglo_tag_span:nn
1324 {
1325   \tag_mc_end_push:
1326   \tag_struct_begin:n { tag=Span, alt={ #1 } }
1327   \tag_mc_begin:n { tag=Span }
1328   \tag_suspend:n { \spsiglo }
1329   #2
1330   \tag_resume:n { \spsiglo }
1331   \tag_mc_end:
1332   \tag_struct_end:
1333   \tag_mc_begin_pop:n {}
1334 }
1335 \cs_generate_variant:Nn \__spintent_spsiglo_tag_span:nn { en }

```

(End of definition for `\__spintent_spsiglo_tag_span:nn`.)

The functions `\__spintent_spsiglo_print_math:n` and `\__spintent_spsiglo_print_text:n` format century designations based on the active mode. Both first check the “error” status in the variable `\l__spintent_spsiglo_luaset_error_str` returned by the Lua module `spintent.lua` and issue an “error” if validation fails.

If the $\langle argument \rangle$ is valid, the function `\__spintent_spsiglo_print_math:n` operates in *math mode*, applying MathML structures depending on whether an era modifier is present. Conversely, the function `\__spintent_spsiglo_print_text:n` operates in *text mode*, utilizing the `tag` and `alt` keys to expand the value of the `print-era` option into full text (e.g., “antes de cristo”) for NVDA/MathCAT.

```

1336 \cs_new_protected:Nn \__spintent_spsiglo_print_math:n
1337 {
1338   \str_if_eq:VnTF \__spintent_spsiglo_luaset_error_str { true }
1339   {
1340     \msg_error:nnnn { spintent } { invalid-century-format }
1341     { \spsiglo } { #1 }
1342   }
1343   {
1344     \str_if_eq:VnTF \__spintent_spsiglo_print_era_str { none }
1345     {
1346       \__spintent_spsiglo_render_simple:VV
1347       \__spintent_spsiglo_luaset_arabic_str
1348       \__spintent_spsiglo_luaset_roman_min_str
1349     }
1350     {
1351       \__spintent_spsiglo_render_with_era:VVV
1352       \__spintent_spsiglo_print_era_str
1353       \__spintent_spsiglo_luaset_arabic_str
1354       \__spintent_spsiglo_luaset_roman_min_str
1355     }
1356   }
1357 }
1358 \cs_new_protected:Nn \__spintent_spsiglo_print_text:n
1359 {
1360   \str_if_eq:VnTF \__spintent_spsiglo_luaset_error_str { true }
1361   {
1362     \msg_error:nnnn { spintent } { invalid-century-format }
1363     { \spsiglo } { #1 }
1364   }
1365   {
1366     \str_case:Vn \__spintent_spsiglo_print_era_str
1367     {
1368       { none }
1369       {
1370         \__spintent_spsiglo_tag_span:en
1371         { \__spintent_spsiglo_luaset_arabic_str }
1372         { \textsc { \__spintent_spsiglo_luaset_roman_min_str } }
1373       }
1374       { ac }
1375       {
1376         \__spintent_spsiglo_tag_span:en
1377         { \__spintent_spsiglo_luaset_arabic_str ~ antes ~ de ~ cristo }
1378         { \textsc { \__spintent_spsiglo_luaset_roman_min_str } ~ a.~C. }
1379       }
1380       { dc }
1381       {
1382         \__spintent_spsiglo_tag_span:en
1383         { \__spintent_spsiglo_luaset_arabic_str ~ después ~ de ~ cristo }
1384         { \textsc { \__spintent_spsiglo_luaset_roman_min_str } ~ d.~C. }
1385       }
1386     }
1387   }
1388 }

```

(End of definition for `\__spintent_spsiglo_print_math:n` and `\__spintent_spsiglo_print_text:n`)

`\spsiglo` The public command `\spsiglo` is the main interface for century typesetting. It opens a local group and evaluates any optional $\langle keys \rangle$ passed in ‘#1’, then executes the function `\__spintent_luafun_spsiglo_parse:n` defined in the Lua module `spintent.lua` passed in ‘#2’ and formats the output for *math mode* or *text mode*.

```

1389 \NewDocumentCommand \spsiglo { O{} m }
1390 {
1391   \group_begin:
1392   \keys_set:nn { spintent / spsiglo } { #1 }
1393   \__spintent_luafun_spsiglo_parse:n { #2 }
1394   \mode_if_math:TF
1395   { \__spintent_spsiglo_print_math:n { #2 } }
1396   {

```

```

1397         \mode_leave_vertical:
1398         \__spintent_spsiglo_print_text:n { #2 }
1399     }
1400     \group_end:
1401 }

```

(End of definition for \spsiglo. This function is documented on page 10.)

11.20 The command \sptime

The command `\sptime` is the main public interface for typesetting and tagging time expressions. It parses time string formats (such as HH:MM or HH:MM:SS) using the Lua function `\__spintent_luafun_sptime_parse:n`. This separates the hours, minutes, seconds, and fractions to construct a MathML *intent* tree for NVDA/MathCAT.

```

\__spintent_sptime_luaset_seconds_str
\__spintent_sptime_luaset_has_seconds_str
\__spintent_sptime_luaset_error_str
\__spintent_sptime_luaset_is_fraction_str
\__spintent_sptime_luaset_base_str

```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the extracted time units, error flags, and boolean indicators (such as the presence of seconds or decimal fractions) used during formatting.

```

1402 \str_new:N \__spintent_sptime_luaset_seconds_str
1403 \str_new:N \__spintent_sptime_luaset_has_seconds_str
1404 \str_new:N \__spintent_sptime_luaset_error_str
1405 \str_new:N \__spintent_sptime_luaset_is_fraction_str
1406 \str_new:N \__spintent_sptime_luaset_base_str

```

(End of definition for __spintent_sptime_luaset_seconds_str and others.)

11.20.1 Definition of error messages

```
invalid-time-format
```

The error message `invalid-time-format` defines the text displayed when an invalid time input is detected. It is triggered when an argument fails the format checks or contains values outside normal ranges, such as hours exceeding 23 or minutes and seconds exceeding 59.

```

1407 \msg_new:nnnn { spintent } { invalid-time-format }
1408 {
1409     Invalid~time~format~in~#1. \\\
1410     The~argument~'#2'~does~not~match~any~supported~time~format.
1411 }
1412 {
1413     Allowed~formats:~HH:MM,~HH:MM:SS,~or~HH:MM:SS.ss. \\\
1414     Please~verify~that~hours~are~0-23~and~minutes/seconds~are~0-59.
1415 }

```

(End of definition for `invalid-time-format`.)

11.20.2 Internal functions for \sptime

```
\__spintent_sptime_build_seconds:nn
```

The internal function `\__spintent_sptime_build_seconds:nn` builds the MathML structure for the seconds component of a time expression. It wraps the numeric value ‘#2’ in speech intent tags, applying a connector prefix (passed in ‘#1’) and the “segundos” postfix so it is read correctly by NVDA/MathCAT.

```

1416 \cs_new_protected:Nn \__spintent_sptime_build_seconds:nn
1417 {
1418     \MathMLintent { #1:prefix($z) }
1419     {
1420         \text { : }
1421         \MathMLarg { z }
1422         {
1423             \MathMLintent { _segundos:postfix($m) }
1424             {
1425                 \__spintent_inner_sep:
1426                 \MathMLarg { m }
1427                 {
1428                     \text { #2 }
1429                 }
1430             }
1431         }
1432     }
1433 }

```

(End of definition for `\__spintent_sptime_build_seconds:nn`.)

```
\__spintent_sptime_build_tree:
```

The internal function `\__spintent_sptime_build_tree:` builds the complete MathML structure for the time expression. It checks the string variables `\__spintent_sptime_luaset_has_seconds_str` and `\__spintent_sptime_luaset_is_fraction_str` to determine if the time includes seconds or decimal fractions. Based on these values, it inserts the base time string and either calls `\__spintent_sptime_build_seconds:`

with the appropriate prefix (`'con'` or `'minutos-con'`), or simply appends the `'minutos'` intent tag if no seconds are present.

```

1434 \cs_new_protected:Nn \__spintent_sptime_build_tree:
1435 {
1436   \str_if_eq:VnTF \l__spintent_sptime_luaset_has_seconds_str { true }
1437   {
1438     \MathMLintent { :time }
1439     {
1440       \text { \l__spintent_sptime_luaset_base_str }
1441     }
1442     \str_if_eq:VnTF \l__spintent_sptime_luaset_is_fraction_str { true }
1443     {
1444       \__spintent_sptime_build_seconds:nn { con }
1445       { \l__spintent_sptime_luaset_seconds_str }
1446     }
1447     {
1448       \__spintent_sptime_build_seconds:nn { _minutos-con }
1449       { \l__spintent_sptime_luaset_seconds_str }
1450     }
1451   }
1452   {
1453     \MathMLintent { :time }
1454     {
1455       \text { \l__spintent_sptime_luaset_base_str }
1456     }
1457     \str_if_eq:VnTF \l__spintent_sptime_luaset_is_fraction_str { false }
1458     {
1459       \MathMLintent { _minutos }
1460       {
1461         \c__spintent_invisible_sep_int
1462       }
1463     }
1464   }
1465 }

```

(End of definition for `\__spintent_sptime_build_tree:`.)

\sptime The command `\sptime` is the public interface for formatting time expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it parses the input using `\__spintent_luafun_sptime_parse:n` and checks the error status in `\l__spintent_sptime_luaset_error_str`. If an invalid format is detected, it issues the `invalid-time-format` error; otherwise, it calls `\__spintent_sptime_build_tree:` to assemble the MathML output.

```

1466 \NewDocumentCommand \sptime { m }
1467 {
1468   \mode_if_math:TF
1469   {
1470     \group_begin:
1471     \__spintent_luafun_sptime_parse:n { #1 }
1472     \str_if_eq:VnTF \l__spintent_sptime_luaset_error_str { true }
1473     {
1474       \msg_error:nnnn { spintent } { invalid-time-format }
1475       { \sptime } { #1 }
1476     }
1477     {
1478       \__spintent_sptime_build_tree:
1479     }
1480     \group_end:
1481   }
1482   { \msg_error:nnn { spintent } { need-math-mode } { \sptime } }
1483 }

```

(End of definition for `\sptime`. This function is documented on page 11.)

11.21 The command `\spdate`

The command `\spdate` is the main public command for typesetting and tagging date expressions. It parses date string formats (such as `DD/MM/YYYY` or `YYYY-MM-DD`) using the Lua function `\__spintent_luafun_spdate_parse:n`. This verifies the input and constructs a MathML `intent` tree for NVDA/MathCAT.

`\l__spintent_spdate_luaset_output_str`
`\l__spintent_spdate_luaset_error_str`

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the formatted date string and the error status used during formatting.

```

1484 \str_new:N \l__spintent_spdate_luaset_output_str

```

```
1485 \str_new:N \__spintent_spdate_luaset_error_str
```

(End of definition for `\__spintent_spdate_luaset_output_str` and `\__spintent_spdate_luaset_error_str`.)

11.21.1 Definition of error messages

invalid-date-format

The error message `invalid-date-format` defines the text displayed when an invalid date input is detected. It is triggered when an argument fails the format checks or represents an invalid calendar date, such as `31/02/2024` or using an unsupported pattern like `YYYY/MM/DD`.

```
1486 \msg_new:nnnn { spintent } { invalid-date-format }
1487 {
1488   Invalid~date~or~unsupported~format~in~#1. \\\
1489   The~argument~'#2'~does~not~match~a~supported~format.
1490 }
1491 {
1492   Allowed~formats:~DD\slash MM\slash YYYY,~or~YYYY-MM-DD. \\\
1493   Ensure~the~day~and~month~are~valid~(e.g.,~avoid~31\slash 02\slash 2024).
1494 }
```

(End of definition for `invalid-date-format`.)

11.21.2 Internal functions for `\spdate`

`\__spintent_spdate_process:n`

The internal function `\__spintent_spdate_process:n` parses the input using `\__spintent_luafun_spdate_parse:n` and checks the error status in `\__spintent_spdate_luaset_error_str`. If an invalid format is detected, it issues the `invalid-date-format` error. Otherwise, it builds the `MathML intent` structure using the formatted date stored in `\__spintent_spdate_luaset_output_str`.

```
1495 \cs_new_protected:Nn \__spintent_spdate_process:n
1496 {
1497   \__spintent_luafun_spdate_parse:n { #1 }
1498   \str_if_eq:VnTF \__spintent_spdate_luaset_error_str { true }
1499   {
1500     \msg_error:nnnn { spintent } { invalid-date-format }
1501     { \spdate } { #1 }
1502   }
1503   {
1504     \MathMLintent { :date }
1505     {
1506       \text { \__spintent_spdate_luaset_output_str }
1507     }
1508   }
1509 }
```

(End of definition for `\__spintent_spdate_process:n`.)

`\spdate`

The command `\spdate` is the public interface for formatting date expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_spdate_process:n` to process the input.

```
1510 \NewDocumentCommand \spdate { m }
1511 {
1512   \mode_if_math:TF
1513   {
1514     \group_begin:
1515     \__spintent_spdate_process:n { #1 }
1516     \group_end:
1517   }
1518   { \msg_error:nnn { spintent } { need-math-mode } { \spdate } }
1519 }
```

(End of definition for `\spdate`. This function is documented on page 11.)

11.22 Semantic Operators

This section provides purely semantic operators. Unlike their formatting counterparts (which parse and format numbers), these commands do not process mandatory arguments. Instead, they solely inject a mathematical glyph wrapped in its corresponding regional `MathCAT intent`. This grants teachers absolute control when building complex algebraic expressions, mixed numbers, or fraction divisions.

11.23 The command `\spusep`

For the use of parentheses, we will provide the command `\spusep` with only two *(keys)* `size` and `silent`. Most of the time we can use `\left` and `\right` or `\Uleft` and `\Uright`, but both forms create a group, and this hinders the code for *tagged* PDF when we want to interrupt something between them.

spusep-empty-arg
spusep-invalid-arg

The first “error” message spusep-empty-arg is raised when `\spusep` receives an empty or blank *{(argument)}*, the second this is when an input is invalid.

```
1520 \msg_new:nnnn { spintent } { spusep-empty-arg }
1521 {
1522   Empty~argument~'#1'~for~\token_to_str:N \spusep.
1523 }
1524 {
1525   \token_to_str:N \spusep~requires~a~non-empty~argument.
1526 }
1527 \msg_new:nnn { spintent } { spusep-invalid-arg }
1528 {
1529   Invalid~argument~'#1'~for~\token_to_str:N \spusep.
1530 }
```

(End of definition for spusep-empty-arg and spusep-invalid-arg.)

The variable `\l__spintent_spusep_arg_tl` stores the *{(argument)}*, the variable `\l__spintent_spusep_left_tl` and `\l__spintent_spusep_right_tl` stores the parentheses.

```
1531 \tl_new:N \l__spintent_spusep_arg_tl
1532 \tl_new:N \l__spintent_spusep_left_tl
1533 \tl_new:N \l__spintent_spusep_right_tl
```

(End of definition for `\l__spintent_spusep_arg_tl`, `\l__spintent_spusep_left_tl`, and `\l__spintent_spusep_right_tl`.)

size
silent
unknown

We define the *(keys)* `size` and `silent`.

```
1534 \keys_define:nn { spintent / spusep }
1535 {
1536   size .choices:nn =
1537     { none , big , Big , bigg, Bigg }
1538     {
1539       \int_case:nn { \l_keys_choice_int }
1540       {
1541         { 1 }
1542         {
1543           \tl_clear:N \l__spintent_spusep_left_tl
1544           \tl_clear:N \l__spintent_spusep_right_tl
1545         }
1546         { 2 }
1547         {
1548           \tl_set:Nn \l__spintent_spusep_left_tl { \bigl }
1549           \tl_set:Nn \l__spintent_spusep_right_tl { \bigr }
1550         }
1551         { 3 }
1552         {
1553           \tl_set:Nn \l__spintent_spusep_left_tl { \Bigl }
1554           \tl_set:Nn \l__spintent_spusep_right_tl { \Bigr }
1555         }
1556         { 4 }
1557         {
1558           \tl_set:Nn \l__spintent_spusep_left_tl { \biggl }
1559           \tl_set:Nn \l__spintent_spusep_right_tl { \biggr }
1560         }
1561         { 5 }
1562         {
1563           \tl_set:Nn \l__spintent_spusep_left_tl { \Biggl }
1564           \tl_set:Nn \l__spintent_spusep_right_tl { \Biggr }
1565         }
1566       }
1567     },
1568   size .initial:n = none,
1569   size .value_required:n = true,
1570   size / unknown .code:n =
1571     \msg_error:nneee { spintent } { unknown-choice }
1572     { size } { none,~big,~Big,~bigg,~Bigg } { \exp_not:n {#1} },
1573   silent .bool_set:N = \l__spintent_spusep_silent_bool,
1574   silent .initial:n = false,
```

```

1575     silent           .value_required:n = true,
1576     unknown          .code:n =
1577     {
1578         \str_set:Nn \__spintent_command_name_str { spusep }
1579         \__spintent_unknown_keys_cmd:n {#1}
1580     },
1581 }

```

(End of definition for `size`, `silent`, and `unknown`.)

`\__spintent_spusep_set_parens:nn`

The function `\__spintent_spusep_set_parens:nn` will place the argument passed in ‘#2’ to the right of the variables `\__spintent_spusep_left_tl` and `\__spintent_spusep_right_tl` and then check the state of the variable `\__spintent_spusep_silent_bool` handled by the key `silent` to print.

```

1582 \cs_new_protected:Nn \__spintent_spusep_set_parens:nn
1583 {
1584     \tl_put_right:cn { \__spintent_spusep_ #1 _tl } { #2 }
1585     \bool_if:NTF \__spintent_spusep_silent_bool
1586     {
1587         \MathMLintenc { :silent } { \tl_use:c { \__spintent_spusep_ #1 _tl } }
1588     }
1589     {
1590         \tl_use:c { \__spintent_spusep_ #1 _tl }
1591     }
1592 }

```

(End of definition for `\__spintent_spusep_set_parens:nn`.)

`\__spintent_spusep_convert:n`

The function `\__spintent_spusep_convert:n` captures and stores the $\langle argument \rangle$ passed in ‘#1’ in the variable `\__spintent_spusep_arg_tl`, validates the cases by converting them to the macros provided by `unicode-math`, and passes them to the function `\__spintent_spusep_set_parens:nn` for printing. If the input is not supported or is empty, we return an “error”.

```

1593 \cs_new_protected:Nn \__spintent_spusep_convert:n
1594 {
1595     \tl_set:Nc \__spintent_spusep_arg_tl { \tl_trim_spaces:n {#1} }
1596     \tl_if_empty:NT \__spintent_spusep_arg_tl
1597     {
1598         \msg_error:nnn { spintenc } { spusep-empty-arg } {#1}
1599     }
1600     \str_case:VnF \__spintent_spusep_arg_tl
1601     {
1602         { ( }
1603         {
1604             \__spintenc_spusep_set_parens:nn { left } { \lparen }
1605         }
1606         { ) }
1607         {
1608             \__spintenc_spusep_set_parens:nn { right } { \rparen }
1609         }
1610         { [ ]
1611         {
1612             \__spintenc_spusep_set_parens:nn { left } { \lbrack }
1613         }
1614         { ] }
1615         {
1616             \__spintenc_spusep_set_parens:nn { right } { \rbrack }
1617         }
1618         { \{ }
1619         {
1620             \__spintenc_spusep_set_parens:nn { left } { \lbrace }
1621         }
1622         { \} }
1623         {
1624             \__spintenc_spusep_set_parens:nn { right } { \rbrace }
1625         }
1626     }
1627     {
1628         \msg_error:nnn { spintenc } { spusep-invalid-arg } {#1}
1629     }
1630 }

```

(End of definition for `\__spintenc_spusep_convert:n`.)

`\spusep` The command `\spusep` checks that it is running in math mode, opens a local group, sets the keys passed in ‘#1’ and processes the $\langle argument \rangle$ passed in ‘#2’ by means of the function `\__spintent_spusep_convert:n`.

```

1631 \NewDocumentCommand \spusep { 0{} m }
1632 {
1633   \mode_if_math:TF
1634   {
1635     \group_begin:
1636     \keys_set:nn { spintent / spusep } {#1}
1637     \__spintent_spusep_convert:n {#2}
1638     \group_end:
1639   }
1640   { \msg_error:nnn { spintent } { need-math-mode } { \spusep } }
1641 }

```

(End of definition for `\spusep`. This function is documented on page 12.)

11.23.1 The command `\spread`

The command `\spread` provides a semantic read for MathCAT of mathematical symbols for situations in which it is necessary to specify gender or plurality in Spanish.

The “error” `spnewread-empty-args` is raised when `\spnewread` receives a blank symbol or a reading that is empty once normalized. The “error” `spnewread-duplicate` is raised when it registers a symbol already in the dictionary. The “error” `spread-unknown-sym` is raised when `\spread` is given a symbol that is neither registered nor accompanied by `read-arg`.

```

1642 \msg_new:nnnn { spintent } { spnewread-empty-args }
1643 {
1644   Empty~argument~in~\token_to_str:N \spnewread.
1645 }
1646 {
1647   \token_to_str:N \spnewread~requires~two~non-empty~arguments.
1648 }
1649 \msg_new:nnnn { spintent } { spnewread-duplicate }
1650 {
1651   The~symbol~'~#1'~is~already~defined!
1652 }
1653 {
1654   You~cannot~redefine~with~\token_to_str:N \spnewread.
1655 }
1656 \msg_new:nnnn { spintent } { spread-empty-sym }
1657 {
1658   Empty~symbol~argument~in~\token_to_str:N \spread.
1659 }
1660 {
1661   \token_to_str:N \spread~requires~a~non-empty~symbol.
1662 }
1663 \msg_new:nnnn { spintent } { spread-unknown-sym }
1664 {
1665   The~symbol~'~#1'~is~not~registered~and~no~'read-arg'~was~given.
1666 }
1667 {
1668   Register~it~first~with~\token_to_str:N \spnewread,~or~pass~'read-arg'.
1669 }

```

(End of definition for `spnewread-empty-args`, `spnewread-duplicate`, and `spread-unknown-sym`.)

The variables `\__spintent_spread_luaset_status_str` and `\__spintent_spread_luaset_reading_str` are set by `\__spintent_luafun_spread_new:nn/\__spintent_luafun_spread_lookup:n`. The variable `\__spintent_spread_read_arg_tl` stores `read-arg`.

```

1670 \str_new:N \__spintent_spread_luaset_status_str
1671 \str_new:N \__spintent_spread_luaset_reading_str
1672 \tl_new:N \__spintent_spread_read_arg_tl

```

(End of definition for `\__spintent_spread_luaset_status_str`, `\__spintent_spread_luaset_reading_str`, and `\__spintent_spread_read_arg_tl`.)

We define the $\langle keys \rangle$ `read-arg` and `silent`.

```

1673 \keys_define:nn { spintent / spread }
1674 {
1675   read-arg .code:n =
1676   {
1677     \__spintent_sanitizé_affix:Nn \__spintent_spread_read_arg_tl {#1}

```

```

1678     },
1679     read-arg      .initial:n = ,
1680     read-arg      .value_required:n = true,
1681     silent        .bool_set:N = \l__spintenc_spread_silent_bool,
1682     silent        .initial:n = false,
1683     silent        .value_required:n = true,
1684     unknown       .code:n =
1685     {
1686       \str_set:Nn \l__spintenc_command_name_str { spread }
1687       \__spintenc_unknown_keys_cmd:n {#1}
1688     },
1689   }

```

(End of definition for `read-arg`, `silent`, and `unknown`.)

`\__spintenc_spread_exec:nn`

The function `\__spintenc_spread_exec:nn` receives the optional $\langle keys \rangle$ in ‘#1’ and the $\{\langle symbol \rangle\}$ in ‘#2’. With `silent true`, the symbol is typeset inside a literal `_- \MathMLintent`, nothing else evaluated; a composite symbol (such as x^2) is not reliably silenced by `:silent` alone. In every case the symbol is preceded by `\__spintenc_inner_sep:` inside the group, so that a composite symbol (a sole child, such as x^2) keeps its `intent` instead of collapsing. Otherwise, with `read-arg` given, it is used directly, overriding any registered reading for this call alone. With `read-arg` empty, calls `\__spintenc_luafun_spread_lookup:n`, using the registered reading when found, reporting `spread-unknown-sym` otherwise.

```

1690 \cs_new_protected:Nn \__spintenc_spread_exec:nn
1691 {
1692   \keys_set:nn { spintenc / spread } { #1 }
1693   \tl_if_blank:nTF { #2 }
1694   { \msg_error:nn { spintenc } { spread-empty-sym } }
1695   {
1696     \__spintenc_invisible_sep:
1697     \bool_if:NTF \l__spintenc_spread_silent_bool
1698     {
1699       \MathMLintent { _- } { \__spintenc_inner_sep: #2 }
1700     }
1701     {
1702       \tl_if_empty:NTF \l__spintenc_spread_read_arg_tl
1703       {
1704         \__spintenc_luafun_spread_lookup:n { \exp_not:n {#2} }
1705         \str_if_eq:VnTF \l__spintenc_spread_luaaset_status_str { found }
1706         {
1707           \MathMLintent { _\l__spintenc_spread_luaaset_reading_str } { \__spintenc_inner_sep: #2 }
1708         }
1709         {
1710           \msg_error:nne { spintenc } { spread-unknown-sym } { \exp_not:n {#2} }
1711         }
1712       }
1713       {
1714         \MathMLintent { _\l__spintenc_spread_read_arg_tl } { \__spintenc_inner_sep: #2 }
1715       }
1716     }
1717   }
1718 }

```

(End of definition for `\__spintenc_spread_exec:nn`.)

`\spread` The public command `\spread` enforces *math mode*, opens a local group, and delegates execution to `\__spintenc_spread_exec:nn`.

```

1719 \NewDocumentCommand \spread { 0{} m }
1720 {
1721   \mode_if_math:TF
1722   {
1723     \group_begin:
1724     \__spintenc_spread_exec:nn { #1 } { #2 }
1725     \group_end:
1726   }
1727   { \msg_error:nnn { spintenc } { need-math-mode } { \spread } }
1728 }

```

(End of definition for `\spread`. This function is documented on page 12.)

`\spnewread` The public command `\spnewread` registers a new symbol/reading pair. It first normalizes the reading ‘#2’ with `\__spintenc_sanitize_affix:Nn`, storing the result in the scratch variable `\l_tmpa_tl`, and reports

spnewread-empty-args when the symbol ‘#1’ is blank or the normalized reading is empty. Otherwise it passes the symbol and the normalized reading to Lua via `__spintent_luafun_spread_new:nn`, reporting spnewread-duplicate when the symbol is already registered.

```

1729 \NewDocumentCommand \spnewread { m m }
1730 {
1731   \mode_if_math:TF
1732   {
1733     \msg_error:nnn { spintent } { need-text-mode } { \spnewread }
1734   }
1735   {
1736     __spintent_sanitizate_affix:Nn \l_tmpa_tl { #2 }
1737     \bool_lazy_or:nnTF { \tl_if_blank_p:n { #1 } } { \tl_if_empty_p:N \l_tmpa_tl }
1738     { \msg_error:nn { spintent } { spnewread-empty-args } }
1739     {
1740       __spintent_luafun_spread_new:nn { \exp_not:n {#1} } { \exp_not:V \l_tmpa_tl }
1741       \str_if_eq:VnT \l__spintent_spread_luaseta_status_str { duplicate }
1742       {
1743         \msg_error:nnn { spintent } { spnewread-duplicate } { #1 }
1744       }
1745     }
1746   }
1747 }

```

(End of definition for `\spnewread`. This function is documented on page 11.)

11.23.2 The command `\spdsym`

Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spdsym`.

```

\l__spintent_spdsym_read_sym_tl
\l__spintent_spdsym_symbol_tl
\l__spintent_spdsym_prefix_tl
\l__spintent_spdsym_suffix_tl
1748 \tl_new:N \l__spintent_spdsym_read_sym_tl
1749 \tl_new:N \l__spintent_spdsym_symbol_tl
1750 \tl_new:N \l__spintent_spdsym_prefix_tl
1751 \tl_new:N \l__spintent_spdsym_suffix_tl

```

(End of definition for `\l__spintent_spdsym_read_sym_tl` and others.)

```

prefix    We define the (keys) prefix, suffix, set-sym and read-sym for \spdsym.
suffix
set-sym
read-sym
unknown
1752 \keys_define:nn { spintent / spdsym }
1753 {
1754   prefix      .code:n =
1755   { __spintent_sanitizate_affix:Nn \l__spintent_spdsym_prefix_tl {#1} },
1756   prefix      .initial:n = ,
1757   prefix      .value_required:n = true,
1758   suffix      .code:n =
1759   { __spintent_sanitizate_affix:Nn \l__spintent_spdsym_suffix_tl {#1} },
1760   suffix      .initial:n = ,
1761   suffix      .value_required:n = true,
1762   set-sym     .choices:nn =
1763   { old-style , two-dots , slash }
1764   {
1765     \int_case:nn { \l_keys_choice_int }
1766     {
1767       {1} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { \div } }
1768       {2} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { : } }
1769       {3} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { / } }
1770     }
1771   },
1772   set-sym     .initial:n = two-dots,
1773   set-sym     .value_required:n = true,
1774   set-sym / unknown .code:n =
1775   \msg_error:nneee { spintent } { unknown-choice }
1776   { set-sym } { old-style, ~two-dots, ~slash } { \exp_not:n {#1} },
1777   read-sym    .choices:nn =
1778   { dividido , dividido-por , dividido-entre }
1779   {
1780     \int_case:nn { \l_keys_choice_int }
1781     {
1782       {1} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido } }
1783       {2} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido-por } }
1784       {3} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido-entre } }
1785     }
1786   },

```

```

1787     read-sym          .initial:n = dividido,
1788     read-sym          .value_required:n = true,
1789     read-sym / unknown .code:n =
1790         \msg_error:nneee { spintent } { unknown-choice }
1791         { read-sym } { dividido, ~dividido-por, ~dividido-entre }
1792         { \exp_not:n {#1} },
1793     unknown          .code:n =
1794     {
1795         \str_set:Nn \l__spintent_command_name_str { spdsym }
1796         \__spintent_unknown_keys_cmd:n {#1}
1797     },
1798 }

```

(End of definition for *prefix* and others.)

\spdsym Defines the `\spdsym` operator. It takes a single optional argument for local setup and outputs the configured intent structure, safely attaching prefixes and suffixes if present.

```

1799 \NewDocumentCommand \spdsym { 0{} }
1800 {
1801     \mode_if_math:TF
1802     {
1803         \group_begin:
1804         \keys_set:nn { spintent / spdsym } { #1 }
1805         \MathMLintent
1806         {
1807             -
1808             \tl_if_empty:NF \l__spintent_spdsym_prefix_tl
1809             { \l__spintent_spdsym_prefix_tl - }
1810             \l__spintent_spdsym_read_sym_tl
1811             \tl_if_empty:NF \l__spintent_spdsym_suffix_tl
1812             { - \l__spintent_spdsym_suffix_tl }
1813         }
1814         { \l__spintent_spdsym_symbol_tl }
1815     \group_end:
1816     }
1817     { \msg_error:nnn { spintent } { need-math-mode } { \spdsym } }
1818 }

```

(End of definition for `\spdsym`. This function is documented on page 12.)

11.23.3 The command `\spmsym`

Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spmsym`.

```

\l__spintent_spmsym_read_sym_tl
\l__spintent_spmsym_symbol_tl
\l__spintent_spmsym_prefix_tl
\l__spintent_spmsym_suffix_tl
1819 \tl_new:N \l__spintent_spmsym_read_sym_tl
1820 \tl_new:N \l__spintent_spmsym_symbol_tl
1821 \tl_new:N \l__spintent_spmsym_prefix_tl
1822 \tl_new:N \l__spintent_spmsym_suffix_tl

```

(End of definition for `\l__spintent_spmsym_read_sym_tl` and others.)

We define the *(keys)* prefix, suffix, set-sym, read-sym and not-print for `\spmsym`.

```

prefix
suffix
set-sym
read-sym
not-print
unknown
1823 \keys_define:nn { spintent / spmsym }
1824 {
1825     prefix          .code:n =
1826     { \__spintent_sanitize_affix:Nn \l__spintent_spmsym_prefix_tl {#1} },
1827     prefix          .initial:n = ,
1828     prefix          .value_required:n = true,
1829     suffix          .code:n =
1830     { \__spintent_sanitize_affix:Nn \l__spintent_spmsym_suffix_tl {#1} },
1831     suffix          .initial:n = ,
1832     suffix          .value_required:n = true,
1833     set-sym         .choices:nn =
1834     { cross , dot , asterisk }
1835     {
1836         \int_case:nn { \l_keys_choice_int }
1837         {
1838             {1} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \times } }
1839             {2} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \cdot } }
1840             {3} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \ast } }
1841         }
1842     },
1843     set-sym         .initial:n = dot,

```

```

1844     set-sym                .value_required:n = true,
1845     set-sym / unknown     .code:n =
1846         \msg_error:nneee { spintent } { unknown-choice }
1847         { set-sym } { cross, ~dot, ~asterisk } { \exp_not:n {#1} },
1848     read-sym              .choices:nn =
1849         { por , multiplicado-por , veces }
1850         {
1851             \int_case:nn { \l_keys_choice_int }
1852             {
1853                 {1} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { por } }
1854                 {2} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { multiplicado-por } }
1855                 {3} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { veces } }
1856             }
1857         },
1858     read-sym              .initial:n = por,
1859     read-sym              .value_required:n = true,
1860     read-sym / unknown     .code:n =
1861         \msg_error:nneee { spintent } { unknown-choice }
1862         { read-sym } { por, ~multiplicado-por, ~veces } { \exp_not:n {#1} },
1863     not-print             .bool_set:N = \l__spintent_spmsym_not_print_bool,
1864     not-print             .initial:n = false,
1865     not-print             .value_required:n = true,
1866     unknown               .code:n =
1867         {
1868             \str_set:Nn \l__spintent_command_name_str { spmsym }
1869             \__spintent_unknown_keys_cmd:n {#1}
1870         },
1871 }

```

(End of definition for *prefix* and others.)

\spmsym Defines the **\spmsym** operator. Similar to **\spdsym**, it outputs the correct mathematical glyph based on the local *⟨keys⟩*, safely attaching prefixes and suffixes if present.

```

1872 \NewDocumentCommand \spmsym { 0{} }
1873 {
1874     \mode_if_math:TF
1875     {
1876         \group_begin:
1877         \keys_set:nn { spintent / spmsym } { #1 }
1878         \MathMLintent
1879         {
1880             -
1881             \tl_if_empty:NF \l__spintent_spmsym_prefix_tl
1882             { \l__spintent_spmsym_prefix_tl - }
1883             \l__spintent_spmsym_read_sym_tl
1884             \tl_if_empty:NF \l__spintent_spmsym_suffix_tl
1885             { - \l__spintent_spmsym_suffix_tl }
1886         }
1887         {
1888             \bool_if:NTF \l__spintent_spmsym_not_print_bool
1889             {
1890                 \invisibletimes
1891             }
1892             { \l__spintent_spmsym_symbol_tl }
1893         }
1894         \group_end:
1895     }
1896     { \msg_error:nnn { spintent } { need-math-mode } { \spmsym } }
1897 }

```

(End of definition for **\spmsym**. This function is documented on page 13.)

11.24 The command **\spdiv**

The user-level command **\spdiv** formats division representations. It takes two mandatory arguments: the *⟨dividend⟩* and the *⟨divisor⟩*. Both *⟨arguments⟩* are internally processed by the **\spnum** command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the *⟨dividend⟩* nor the *⟨divisor⟩* is *blank*.

11.24.1 Definition of keys for **\spdiv**

We define the *⟨keys⟩* **wrap-parens** and **read-parens** for **\spdiv** command. Number-related *⟨keys⟩* **cifras-sep**, **read-sign**, **read-period-sep** and **notation-sym** are forwarded to **\spnum** command using

`.meta:nn`, while semantic operator $\langle keys \rangle$ `set-sym` and `read-sym` are forwarded to the `\spdsym` command using `.meta:nn`.

```

1898 \keys_define:nn { spintent / spdiv }
1899 {
1900   cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1901   read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1902   read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1903   notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1904   set-sym         .meta:nn = { spintent / spdsym } { set-sym         = {#1} },
1905   read-sym        .meta:nn = { spintent / spdsym } { read-sym        = {#1} },
1906   wrap-parens     .bool_set:N = \l__spintent_spdiv_wrap_parens_bool,
1907   wrap-parens     .initial:n = false,
1908   wrap-parens     .value_required:n = true,
1909   read-parens     .bool_set:N = \l__spintent_spdiv_read_parens_bool,
1910   read-parens     .initial:n = true,
1911   read-parens     .value_required:n = true,
1912   unknown         .code:n =
1913     {
1914       \str_set:Nn \l__spintent_command_name_str { spdiv }
1915       \__spintent_unknown_keys_cmd:n {#1}
1916     },
1917 }

```

(End of definition for `cifras-sep` and others.)

11.24.2 Definition of error messages

`spdiv-empty-argument` Raised when either the $\langle dividend \rangle$ or the $\langle divisor \rangle$ is missing or *blank*.

```

1918 \msg_new:nnnn { spintent } { spdiv-empty-argument }
1919 { The~arguments~for~\token_to_str:N \spdiv~cannot~be~empty. }
1920 {
1921   You~must~provide~both~a~dividend~and~a~divisor.\\
1922 }

```

(End of definition for `spdiv-empty-argument`.)

11.24.3 Internal functions for `\spdiv`

`\__spintent_spdiv_render_op:nn` The function `\__spintent_spdiv_render_op:nn` parses both $\langle arguments \rangle$ through the `\spnum` command and embeds the semantic operator from `\spdsym`.

```

1923 \cs_new_protected:Nn \__spintent_spdiv_render_op:nn
1924 {
1925   \spnum {#1}
1926   \MathMLintent { _ \l__spintent_spdsym_read_sym_tl }
1927   {
1928     \l__spintent_spdsym_symbol_tl
1929   }
1930   \spnum {#2}
1931 }

```

(End of definition for `\__spintent_spdiv_render_op:nn`.)

`\__spintent_spdiv_render_parens:nn` The function `\__spintent_spdiv_render_parens:nn` handles the structural parentheses around the division.

```

1932 \cs_new_protected:Nn \__spintent_spdiv_render_parens:nn
1933 {
1934   \bool_if:NTF \l__spintent_spdiv_read_parens_bool
1935   {
1936     ( \__spintent_spdiv_render_op:nn {#1} {#2} )
1937   }
1938   {
1939     \MathMLintent { :silent } { ( }
1940     \__spintent_spdiv_render_op:nn {#1} {#2}
1941     \MathMLintent { :silent } { ) }
1942   }
1943 }

```

(End of definition for `\__spintent_spdiv_render_parens:nn`.)

`\__spintent_spdiv_process:nnn` The function `\__spintent_spdiv_process:nnn` sets $\langle keys \rangle$, validates $\langle arguments \rangle$, and decides on parenthesis wrapping.

```

1944 \cs_new_protected:Nn \__spintent_spdiv_process:nnn
1945 {

```

```

1946 \keys_set:nn { spintent / spdiv } { #1 }
1947 \tl_if_blank:nTF { #2 }
1948 { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1949 {
1950 \tl_if_blank:nTF { #3 }
1951 { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1952 {
1953 \bool_if:NTF \l__spintent_spdiv_wrap_parens_bool
1954 { \__spintent_spdiv_render_parens:nn {#2} {#3} }
1955 { \__spintent_spdiv_render_op:nn {#2} {#3} }
1956 }
1957 }
1958 }

```

(End of definition for `\__spintent_spdiv_process:nnn`.)

`\spdiv` Defines the user command `\spdiv`. It strictly relies on *math mode*.

```

1959 \NewDocumentCommand \spdiv { 0{ } m m }
1960 {
1961 \mode_if_math:TF
1962 {
1963 \group_begin:
1964 \__spintent_spdiv_process:nnn { #1 } { #2 } { #3 }
1965 \group_end:
1966 }
1967 { \msg_error:nnn { spintent } { need-math-mode } { \spdiv } }
1968 }

```

(End of definition for `\spdiv`. This function is documented on page 13.)

11.25 The command `\spmul`

The user-level command `\spmul` formats multiplication representations. It takes two mandatory arguments: the `{⟨first factor⟩}` and the `{⟨second factor⟩}`. Both `{⟨arguments⟩}` are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the `{⟨first factor⟩}` nor the `{⟨second factor⟩}` is *blank*.

11.25.1 Definition of keys for `\spmul`

We define the `⟨keys⟩` `wrap-parens` and `read-parens` for `\spmul` command. Number-related `⟨keys⟩` `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator `⟨keys⟩` `set-sym` and `read-sym` are forwarded to the `\spmsym` command using `.meta:nn`.

```

1969 \keys_define:nn { spintent / spmul }
1970 {
1971 cifras-sep .meta:nn = { spintent / spnum } { cifras-sep = {#1} },
1972 read-sign .meta:nn = { spintent / spnum } { read-sign = {#1} },
1973 read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1974 notation-sym .meta:nn = { spintent / spnum } { notation-sym = {#1} },
1975 set-sym .meta:nn = { spintent / spmsym } { set-sym = {#1} },
1976 read-sym .meta:nn = { spintent / spmsym } { read-sym = {#1} },
1977 wrap-parens .bool_set:N = \l__spintent_spmul_wrap_parens_bool,
1978 wrap-parens .initial:n = false,
1979 wrap-parens .value_required:n = true,
1980 read-parens .bool_set:N = \l__spintent_spmul_read_parens_bool,
1981 read-parens .initial:n = true,
1982 read-parens .value_required:n = true,
1983 unknown .code:n =
1984 {
1985 \str_set:Nn \l__spintent_command_name_str { spmul }
1986 \__spintent_unknown_keys_cmd:n {#1}
1987 },
1988 }

```

(End of definition for `cifras-sep` and others.)

11.25.2 Definition of error messages

`spmul-empty-argument` Raised when either the `{⟨first factor⟩}` or the `{⟨second factor⟩}` is missing or *blank*.

```

1989 \msg_new:nnnn { spintent } { spmul-empty-argument }
1990 { The~arguments~for~\token_to_str:N \spmul~cannot~be~empty. }
1991 {
1992 You~must~provide~both~first~factor~and~second~factor.\\
1993 }

```

(End of definition for `spmul-empty-argument`.)

11.25.3 Internal functions for `\spmul`

`\__spintent_spmul_render_op:nn`

The function `\__spintent_spmul_render_op:nn` parses both $\{\langle arguments \rangle\}$ through the `\spnum` command and embeds the semantic operator from `\spmsym`.

```

1994 \cs_new_protected:Nn \__spintent_spmul_render_op:nn
1995 {
1996   \spnum {#1}
1997   \MathMLintent { _ \__spintent_spmsym_read_sym_tl }
1998   {
1999     \__spintent_spmsym_symbol_tl
2000   }
2001   \spnum {#2}
2002 }
```

(End of definition for `\__spintent_spmul_render_op:nn`.)

`\__spintent_spmul_render_parens:nn`

The function `\__spintent_spmul_render_parens:nn` handles the structural parentheses around the multiplication.

```

2003 \cs_new_protected:Nn \__spintent_spmul_render_parens:nn
2004 {
2005   \bool_if:NTF \__spintent_spmul_read_parens_bool
2006   {
2007     ( \__spintent_spmul_render_op:nn {#1} {#2} )
2008   }
2009   {
2010     \MathMLintent { :silent } { ( }
2011     \__spintent_spmul_render_op:nn {#1} {#2}
2012     \MathMLintent { :silent } { ) }
2013   }
2014 }
```

(End of definition for `\__spintent_spmul_render_parens:nn`.)

`\__spintent_spmul_process:nnn`

The function `\__spintent_spmul_process:nnn` sets $\langle keys \rangle$, validates $\{\langle arguments \rangle\}$, and decides on parenthesis wrapping.

```

2015 \cs_new_protected:Nn \__spintent_spmul_process:nnn
2016 {
2017   \keys_set:nn { spintent / spmul } { #1 }
2018   \tl_if_blank:NTF { #2 }
2019   { \msg_error:nnn { spintent } { spmul-empty-argument } }
2020   {
2021     \tl_if_blank:NTF { #3 }
2022     { \msg_error:nnn { spintent } { spmul-empty-argument } }
2023     {
2024       \bool_if:NTF \__spintent_spmul_wrap_parens_bool
2025       { \__spintent_spmul_render_parens:nn {#2} {#3} }
2026       { \__spintent_spmul_render_op:nn {#2} {#3} }
2027     }
2028   }
2029 }
```

(End of definition for `\__spintent_spmul_process:nnn`.)

`\spmul`

Defines the user command `\spmul`. It strictly relies on *math mode*.

```

2030 \NewDocumentCommand \spmul { O{} m m }
2031 {
2032   \mode_if_math:TF
2033   {
2034     \group_begin:
2035     \__spintent_spmul_process:nnn { #1 } { #2 } { #3 }
2036     \group_end:
2037   }
2038   { \msg_error:nnn { spintent } { need-math-mode } { \spmul } }
2039 }
```

(End of definition for `\spmul`. This function is documented on page 14.)

11.26 The commands `\spmcd` and `\spmcm`

The user-level commands `\spmcd` and `\spmcm` provide a semantic interface for the Greatest Common Divisor (MCD in Spanish) and the Least Common Multiple (MCM in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid `MathCAT` intent.

11.26.1 Variables for \spmc and \spmcm

Internal variables generated dynamically via a comma-separated list to avoid code duplication for both symmetric commands.

```
\__spintenc_spmcm_spmcd_luaset_error_str
\__spintenc_mc_luaset_types_str
\__spintenc_mc_luaset_all_numeric_str
\__spintenc_mc_args_seq
\__spintenc_mc_raw_args_seq
\__spintenc_spmcd_print_result_bool
\__spintenc_spmcm_print_result_bool
\__spintenc_spmcd_read_terse_bool
\__spintenc_spmcm_read_terse_bool
\__spintenc_spmcd_print_upper_bool
\__spintenc_spmcm_print_upper_bool
\__spintenc_spmcd_prefix_tl
\__spintenc_spmcm_prefix_tl
\__spintenc_spmcd_suffix_tl
\__spintenc_spmcm_suffix_tl

2040 \str_new:N \__spintenc_spmcm_spmcd_luaset_error_str
2041 \clist_new:N \__spintenc_mc_luaset_types_clist
2042 \str_new:N \__spintenc_mc_luaset_all_numeric_str
2043 \seq_new:N \__spintenc_mc_args_seq
2044 \seq_new:N \__spintenc_mc_raw_args_seq
2045 \seq_new:N \__spintenc_mc_types_seq
2046 \int_new:N \__spintenc_mc_idx_int
2047 \clist_map_inline:nn { mcd , mcm }
2048 {
2049   \tl_new:c { \__spintenc_sp #1 _luaset_ #1 _value_tl }
2050   \bool_new:c { \__spintenc_sp #1 _read_terse_bool }
2051   \tl_new:c { \__spintenc_sp #1 _prefix_tl }
2052 }
```

(End of definition for __spintenc_spmcm_spmcd_luaset_error_str and others.)

11.26.2 Definition of error messages

mc-invalid-arguments

Exception triggered when the comma-separated sequence contains negative integers, floats, or alphabetical characters.

```
2053 \msg_new:nnnn { spintenc } { mc-invalid-arguments }
2054 {
2055   Invalid~arguments~provided~to~#1.
2056 }
2057 {
2058   The~arguments~must~be~a~comma-separated~list. \l
2059 }
```

(End of definition for mc-invalid-arguments.)

11.26.3 Keys for \spmc and \spmcm

result
prefix
read-cmd
upper-case
concept
silent
unknown

We define the shared *keys* result, prefix, read-cmd, upper-case, concept and silent.

```
2060 \clist_map_inline:nn { mcd , mcm }
2061 {
2062   \keys_define:nn { spintenc / sp #1 }
2063   {
2064     result .bool_set:c = { \__spintenc_sp #1 _print_result_bool },
2065     result .initial:n = false,
2066     result .value_required:n = true,
2067     prefix .code:n =
2068       { \__spintenc_sanitizet_affix:cn { \__spintenc_sp #1 _prefix_tl } {##1} },
2069     prefix .initial:n = ,
2070     prefix .value_required:n = true,
2071     read-cmd .choices:nn =
2072       { terse , clear }
2073       {
2074         \int_case:nn { \l_keys_choice_int }
2075         {
2076           {1} { \bool_set_true:c { \__spintenc_sp #1 _read_terse_bool } }
2077           {2} { \bool_set_false:c { \__spintenc_sp #1 _read_terse_bool } }
2078         }
2079       },
2080     read-cmd .initial:n = clear,
2081     read-cmd .value_required:n = true,
2082     read-cmd / unknown .code:n =
2083       \msg_error:nneee { spintenc } { unknown-choice }
2084       { read-cmd } { terse , ~clear } { \exp_not:n {##1} },
2085     upper-case .bool_set:c = { \__spintenc_sp #1 _print_upper_bool },
2086     upper-case .initial:n = false,
2087     upper-case .value_required:n = true,
2088     concept .bool_set:c = { \__spintenc_sp #1 _concept_bool },
2089     concept .initial:n = true,
2090     concept .value_required:n = true,
2091     silent .bool_set:c = { \__spintenc_sp #1 _silent_bool },
2092     silent .initial:n = false,
2093     silent .value_required:n = true,
2094     unknown .code:n =
2095       {
2096         \str_set:Nn \__spintenc_command_name_str { sp #1 }
```

```

2097         \__spintenc_unknown_keys_cmd:n {##1}
2098     },
2099 }
2100 }

```

(End of definition for *result* and others.)

11.26.4 Internal functions for \spmc and \spmc

__spintenc_mc_process_args:nn

The function `\__spintenc_mc_process_args:nn` parses the operation ‘#1’ and its raw argument list ‘#2’. Calls `\__spintenc_luafun_mc_classify_args:n`, splits ‘#2’ into `\l__spintenc_mc_raw_args_seq` and `\l__spintenc_mc_luaset_types_clist` into `\l__spintenc_mc_types_seq`, then walks the former by position, fetching the matching type from the latter through `\seq_item:Nn`, to build `\l__spintenc_mc_args_seq`: each argument wrapped through `\spnum` when its type is `number`, verbatim otherwise. Injects MathML intents around the already-formatted `\l__spintenc_mc_args_seq`.

```

2101 \cs_new_protected:Nn \__spintenc_mc_process_args:nn
2102 {
2103     \__spintenc_luafun_mc_classify_args:n {#2}
2104     \seq_set_from_clist:Nn \l__spintenc_mc_raw_args_seq {#2}
2105     \seq_set_from_clist:NN \l__spintenc_mc_types_seq \l__spintenc_mc_luaset_types_clist
2106     \seq_clear:N \l__spintenc_mc_args_seq
2107     \int_zero:N \l__spintenc_mc_idx_int
2108     \seq_map_inline:Nn \l__spintenc_mc_raw_args_seq
2109     {
2110         \int_incr:N \l__spintenc_mc_idx_int
2111         \str_if_eq:eeTF
2112         { \seq_item:Nn \l__spintenc_mc_types_seq { \l__spintenc_mc_idx_int } } { number }
2113         {
2114             \seq_put_right:Nn \l__spintenc_mc_args_seq { \spnum {##1} }
2115         }
2116         { \seq_put_right:Nn \l__spintenc_mc_args_seq {##1} }
2117     }

```

Injects MathML intents around the already-formatted `\l__spintenc_mc_args_seq`.

```

2118     \bool_if:cTF { l__spintenc_sp #1 _silent_bool }
2119     {
2120         \seq_set_map:Nnn \l__spintenc_mc_args_seq \l__spintenc_mc_args_seq
2121         {
2122             \MathMLintent { _- } { \__spintenc_inner_sep: ##1 }
2123         }
2124         \MathMLintent { :silent } { \c__spintenc_invisible_sep_int } % prevent "por"
2125         \MathMLintent { :silent } { ( }
2126         \MathMLintent { :silent } { \c__spintenc_invisible_sep_int } % prevent "intervalo"
2127         \seq_use:Nnnn \l__spintenc_mc_args_seq
2128         { \MathMLintent { :silent } { , } }
2129         { \MathMLintent { :silent } { , } }
2130         { \MathMLintent { :silent } { , } }
2131     }
2132     {
2133         \MathMLintent { :silent } { \c__spintenc_invisible_sep_int } % prevent doble "de"
2134         \bool_if:cTF { l__spintenc_sp #1 _concept_bool }
2135         {
2136             \MathMLintent { _entre } { ( }
2137         }
2138         {
2139             \MathMLintent { :silent } { ( }
2140         }
2141         \MathMLintent { :silent } { \c__spintenc_invisible_sep_int } % prevent "intervalo"
2142         \seq_use:Nnnn \l__spintenc_mc_args_seq
2143         {
2144             \MathMLintent { :silent } { , }
2145             \MathMLintent { _y- } { \c__spintenc_invisible_sep_int } % "y" medio
2146         }
2147         {
2148             \MathMLintent { :silent } { , }
2149         }
2150         {
2151             \MathMLintent { :silent } { , }
2152             \MathMLintent { _y- } { \c__spintenc_invisible_sep_int } % "y" final
2153         }
2154     }
2155     \MathMLintent { :silent } { ) }
2156 }

```

(End of definition for `\__spintent_mc_process_args:nn`.)

`\__spintent_spmc_process:nnnn`

The internal function `\__spintent_spmc_process:nnnn` is the shared handler for both `\spmc` and `\spmcm`. Takes the short identifier in ‘#1’, its full reading string in ‘#2’, the local `(keys)` in ‘#3’, and the optional arguments in ‘#4’.

The operator name is silenced with `silent=true` — `\__spintent_mc_process_args:nn` silences the arguments too, in that case, each wrapped in a literal `_`- intent, and the printed result is silenced as well — or with `concept=false` alone, leaving the arguments untouched. With `concept=true`, it carries the literal intent instead — #1 or #2 per `read-cmd`, prefixed with `prefix` when given.

With ‘#4’ given, classifies it through `\__spintent_luafun_mc_classify_args:n`. Only when every argument is a number does it call the Lua calculation (`calculate_#1`), reporting `mc-invalid-arguments` on failure, and, when `result` is `true`, printing the computed value. `\__spintent_mc_process_args:n` prints the argument list itself either way.

• We should use `\mathrm` instead of `\operatorname`, which generates unwanted spacing when `\c__spintent_invisible_sep_int` is applied.

```

2157 \cs_new_protected:Nn \__spintent_spmc_process:nnnn
2158 {
2159   \keys_set:nn { spintent / sp #1 } { #3 }
2160   \bool_if:cTF { l__spintent_sp #1 _silent_bool }
2161   {
2162     \MathMLintent { :silent }
2163     {
2164       \mathrm
2165       {
2166         \bool_if:cTF { l__spintent_sp #1 _print_upper_bool }
2167         { \text_uppercase:n {#1} } { #1 }
2168       }
2169     }
2170   }
2171   {
2172     \bool_if:cTF { l__spintent_sp #1 _concept_bool }
2173     {
2174       \MathMLintent
2175       {
2176         -
2177         \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
2178         {
2179           \tl_use:c { l__spintent_sp #1 _prefix_tl } -
2180         }
2181         \bool_if:cTF { l__spintent_sp #1 _read_terse_bool }
2182         { #1 } { #2 }
2183       }
2184       {
2185         \mathrm
2186         {
2187           \bool_if:cTF { l__spintent_sp #1 _print_upper_bool }
2188           { \text_uppercase:n {#1} } { #1 }
2189         }
2190       }
2191     }
2192   }
2193   \MathMLintent { :silent }
2194   {
2195     \mathrm
2196     {
2197       \bool_if:cTF { l__spintent_sp #1 _print_upper_bool }
2198       { \text_uppercase:n {#1} } { #1 }
2199     }
2200   }
2201 }
2202

```

With ‘#4’ given, classifies it through `\__spintent_luafun_mc_classify_args:n`. Only when every argument is a number does it call the Lua calculation (`calculate_#1`), reporting `mc-invalid-arguments` on failure — which should not happen, since every argument already passed classification — and, when `result` is `true`, printing the computed value. With any non-numeric argument, no calculation is attempted at all. Either way, `\__spintent_mc_process_args:n` prints the argument list itself.

```

2203   \tl_if_novalue:nF { #4 }
2204   {

```

```

2205 \seq_set_from_clist:Nn \l__spintent_mc_raw_args_seq {#4}
2206 \int_compare:nNnTF { \seq_count:N \l__spintent_mc_raw_args_seq } < { 2 }
2207 {
2208   \msg_error:nne { spintent } { mc-too-few-arguments } { \exp_not:c { sp #1 } }
2209 }
2210 {
2211   \__spintent_luafun_mc_classify_args:n {#4}
2212   \str_if_eq:VnT \l__spintent_mc_luaset_all_numeric_str { true }
2213   {
2214     \use:c { __spintent_luafun_calculate_ #1 :n } { #4 }
2215     \str_if_eq:VnT \l__spintent_spmcm_spmcd_luaset_error_str { true }
2216     {
2217       \msg_error:nne { spintent } { mc-invalid-arguments } { \exp_not:c { sp #1 } }
2218     }
2219   }
2220   \__spintent_mc_process_args:nn {#1} {#4}
2221   \str_if_eq:VnT \l__spintent_mc_luaset_all_numeric_str { true }
2222   {
2223     \bool_if:cT { l__spintent_sp #1 _print_result_bool }
2224     {
2225       \bool_if:cTF { l__spintent_sp #1 _silent_bool }
2226       {
2227         \MathMLintent { :silent } { = }
2228         \MathMLintent { _- }
2229         {
2230           \__spintent_inner_sep:
2231           \tl_use:c { l__spintent_sp #1 _luaset_ #1 _value_tl }
2232         }
2233       }
2234       {
2235         \MathMLintent { _es-igual-a } { = }
2236         \tl_use:c { l__spintent_sp #1 _luaset_ #1 _value_tl }
2237       }
2238     }
2239   }
2240 }
2241 }
2242 }

```

(End of definition for `\__spintent_spmc_process:nnnn`.)

`\spmcd` The commands `\spmcd` and `\spmcm` act as the public interface wrappers for typesetting the greatest common divisor and least common multiple. They enforce math mode, open a local group, and defer execution to the main processing macro `\__spintent_spmc_process:nnnn`, passing along the appropriate reading strings and arguments.

```

2243 \NewDocumentCommand \spmcd { 0{} d() }
2244 {
2245   \mode_if_math:TF
2246   {
2247     \group_begin:
2248     \__spintent_spmc_process:nnnn { mcd } { máximo-común-divisor } {#1} {#2}
2249     \group_end:
2250   }
2251   { \msg_error:nnn { spintent } { need-math-mode } { \spmcd } }
2252 }
2253 \NewDocumentCommand \spmcm { 0{} d() }
2254 {
2255   \mode_if_math:TF
2256   {
2257     \group_begin:
2258     \__spintent_spmc_process:nnnn { mcm } { mínimo-común-múltiplo } {#1} {#2}
2259     \group_end:
2260   }
2261   { \msg_error:nnn { spintent } { need-math-mode } { \spmcm } }
2262 }

```

(End of definition for `\spmcd` and `\spmcm`. These functions are documented on page 15.)

11.27 The commands `\spnM` and `\spnD`

The user-level commands `\spnM` and `\spnD` provide a semantic interface for the Multiples of a number (“*múltiplos*” in Spanish) and the Divisors of a number (“*divisores*” in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

11.27.1 Definition of variables for \spnM and \spnD

Internal variables generated dynamically for multiples and divisors.

```

\l__spintent_md_out_seq
\l__spintent_spnmd_luaset_is_number_str
\l__spintent_spnM_spnD_luaset_error_str
\l__spintent_spnM_luaset_result_tl
\l__spintent_spnD_luaset_result_tl
\l__spintent_spnM_luaset_is_truncated_str
\l__spintent_spnD_luaset_is_truncated_str
\l__spintent_spnM_prefix_tl
\l__spintent_spnD_prefix_tl
\l__spintent_spnM_print_result_num_tl
\l__spintent_spnD_print_result_num_tl
2263 \seq_new:N \l__spintent_md_out_seq
2264 \str_new:N \l__spintent_spnM_spnD_luaset_error_str
2265 \str_new:N \l__spintent_spnmd_luaset_is_number_str
2266 \clist_map_inline:nn { nM , nD }
2267 {
2268   \tl_new:c { \l__spintent_sp #1 _luaset_result_tl }
2269   \str_new:c { \l__spintent_sp #1 _luaset_is_truncated_str }
2270   \tl_new:c { \l__spintent_sp #1 _prefix_tl }
2271 }
(End of definition for \l__spintent_md_out_seq and others.)

```

11.27.2 Keys for \spnM and \spnD

We define the shared *(keys)* result, silent, result-num, prefix and concept.

```

result
silent
result-num
prefix
concept
unknown
2272 \clist_map_inline:nn { nM , nD }
2273 {
2274   \keys_define:nn { spintent / sp #1 }
2275   {
2276     result      .bool_set:c = { \l__spintent_sp #1 _print_result_bool },
2277     result      .initial:n = false,
2278     result      .value_required:n = true,
2279     silent      .bool_set:c = { \l__spintent_sp #1 _silent_bool },
2280     silent      .initial:n = false,
2281     silent      .value_required:n = true,
2282     result-num .tl_set:c = { \l__spintent_sp #1 _print_result_num_tl },
2283     result-num .initial:n = ,
2284     prefix      .code:n =
2285       { \__spintent_sanitize_affix:cn { \l__spintent_sp #1 _prefix_tl } {##1} },
2286     prefix      .initial:n = ,
2287     prefix      .value_required:n = true,
2288     concept     .bool_set:c = { \l__spintent_sp #1 _concept_bool },
2289     concept     .initial:n = true,
2290     concept     .value_required:n = true,
2291     unknown     .code:n =
2292       {
2293         \str_set:Nn \l__spintent_command_name_str { sp #1 }
2294         \__spintent_unknown_keys_cmd:n {##1}
2295       },
2296   }
2297 }

```

(End of definition for result and others.)

11.27.3 Definition of error messages for \spnM and \spnD

Exception triggered when the argument for multiples or divisors is invalid.

```

md-invalid-argument
mc-too-few-arguments
2298 \msg_new:nnnn { spintent } { md-invalid-argument }
2299 {
2300   Invalid~argument~provided~to~#1~\msg_line_context:.
2301 }
2302 {
2303   The~argument~must~be~a~single~positive~integer.  \
2304 }

```

Exception triggered when fewer than two arguments are provided.

```

2305 \msg_new:nnnn { spintent } { mc-too-few-arguments }
2306 {
2307   Too~few~arguments~provided~to~#1~\msg_line_context:.
2308 }
2309 {
2310   At~least~two~arguments~are~required.  \
2311 }

```

(End of definition for md-invalid-argument and mc-too-few-arguments.)

11.27.4 Internal functions for \spnM and \spnD

__spintent_spnM_spnD_process:nnnn

The internal function `\__spintent_spnM_spnD_process:nnnn` acts as the primary entry point for the multiples and divisors commands. It takes the command identifier ‘#1’, its printed mathematical symbol ‘#2’, its base reading string ‘#3’, the local key-value options ‘#4’, and the optional argument ‘#5’. Its sole purpose is to evaluate and set the provided keys within the current scope before delegating the core branching and typesetting logic to `\__spintent_spn_M_and_D_exec:nnnn`.

```
2312 \cs_new_protected:Nn \__spintent_spnM_spnD_process:nnnnn
2313 {
2314   \keys_set:nn { spintent / sp #1 } { #4 }
2315   \__spintent_spnM_spnD_exec:nnnn { #1 } { #2 } { #3 } { #5 }
2316 }
```

(End of definition for __spintent_spnM_spnD_process:nnnnn.)

__spintent_spnM_spnD_intent:nn

The internal function `\__spintent_spnM_spnD_intent:nn` resolves the main reading intent string. It takes the command identifier ‘#1’ (such as “nM” or “nD”) and the base reading string ‘#2’. It automatically prepends the localized prefix if defined, normalizes accentuation for “*múltiplos*”, and appends ‘-de’.

```
2317 \cs_new:Nn \__spintent_spnM_spnD_intent:nn
2318 {
2319   -
2320   \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
2321     { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
2322   \str_case:nnF { #2 } { { múltiplos } { múltiplos } }
2323     { #2 }
2324   -de
2325 }
```

(End of definition for __spintent_spnM_spnD_intent:nn.)

__spintent_spnM_spnD_silent:nn

The internal function `\__spintent_spn_M_and_D_silent:nn` prints the atomized notation structure in full silent mode. It takes the mathematical symbol ‘#1’ (such as “D” or “M”) and the variable or explicit argument ‘#2’. Every component is wrapped in a silent intent, and an invisible separator is injected between the parentheses to prevent assistive technologies from applying default contextual reading rules. The argument is wrapped in a literal `_-` intent, not in `:silent`, so that intents of its own (such as those of `\spnum`) are silenced too.

```
2326 \cs_new_protected:Nn \__spintent_spnM_spnD_silent:nn
2327 {
2328   \MathMLintent { :silent } { \operatorname { #1 } }
2329   \MathMLintent { :silent } { ( }
2330   \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent doble "de"
2331   \MathMLintent { _- } { \__spintent_inner_sep: #2 }
2332   \MathMLintent { :silent } { ) }
2333 }
```

(End of definition for __spintent_spnM_spnD_silent:nn.)

__spintent_spnM_spnD_result:nn

The internal function `\__spintent_spn_M_and_D_result:nn` handles the calculation and accessible formatting of the computed mathematical set. It takes the command identifier ‘#1’ and the classified numeric argument ‘#2’. It invokes the corresponding Lua backend calculation, reporting `md-invalid-argument` if that fails — a safety net beyond `\__spintent_luafun_spnmd_classify:n`’s own check, since the Lua calculation never trusted that check blindly. On success, it maps the resulting clist into a sequence, and formats the final printed output utilizing custom intents for both separators and conjunctions.

```
2334 \cs_new_protected:Nn \__spintent_spnM_spnD_result:nn
2335 {
2336   \use:c { __spintent_luafun_calcular_ #1 :nn } { #2 }
2337   {
2338     \tl_use:c { l__spintent_sp #1 _print_result_num_tl }
2339   }
2340   \str_if_eq:VnTF \__spintent_spnM_spnD_luasets_error_str { true }
2341   {
2342     \msg_error:nne { spintent } { md-invalid-argument } { \exp_not:c { sp #1 } }
2343   }
2344   {
2345     \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent "por"
2346     \MathMLintent { _son } { = \{ }
2347     \seq_set_from_clist:Ne
2348       \l__spintent_md_out_seq { \tl_use:c { l__spintent_sp #1 _luasets_result_tl } }
2349     \seq_use:Nnnn \l__spintent_md_out_seq
2350       {
2351         \MathMLintent { :silent } { , }

```

```

2352         \MathMLintent { _y- } { \c__spintenc_invisible_sep_int } % "y" (solo dos)
2353     }
2354     { \MathMLintent { :silent } { , } }
2355     {
2356         \MathMLintent { :silent } { , }
2357         \str_if_eq:vnT { l__spintenc_sp #1 _luaset_is_truncated_str } { false }
2358         {
2359             \MathMLintent { _y- } { \c__spintenc_invisible_sep_int } % "y" final
2360         }
2361     }
2362     \str_if_eq:vnT { l__spintenc_sp #1 _luaset_is_truncated_str } { true }
2363     {
2364         \MathMLintent { :silent } { , } ~ \dots
2365     }
2366     \MathMLintent { :silent } { \ } }
2367 }
2368 }

```

(End of definition for `l__spintenc_spnM_spnD_result:nn`.)

`l__spintenc_spnM_spnD_exec:nnnn`

The function `l__spintenc_spnM_spnD_exec:nnnn` takes the command name in ‘#1’, the symbol in ‘#2’, the base reading string in ‘#3’, and the optional argument in ‘#4’. The parentheses are always silenced.

With ‘#4’ absent, the placeholder `n` stands in for the argument. With ‘#4’ given, `l__spintenc_luafun_spnmd_classify:n` decides whether it is wrapped through `\spnum`—and, when it is, whether `l__spintenc_spnM_spnD_result:nn` also prints the computed set, per `result`—or left verbatim, with no result attempted. A zero is flagged as an error by `l__spintenc_luafun_spnmd_classify:n` and rejected right away with `md-invalid-argument`. With `silent=true` the printed result is wrapped in a literal `_-` intent, so it stays on the page but is not verbalized.

Either way, `silent` silences the whole thing through `l__spintenc_spnM_spnD_silent:nn`; otherwise `concept=true` carries the literal intent from `l__spintenc_spnM_spnD_intent:nn`, `false` silences only the operator symbol, leaving the argument to read normally.

```

2369 \cs_new_protected:Nn l__spintenc_spnM_spnD_exec:nnnn
2370 {
2371     \tl_if_novalue:nTF { #4 }
2372     {
2373         \bool_if:cTF { l__spintenc_sp #1 _silent_bool }
2374         { \l__spintenc_spnM_spnD_silent:nn { #2 } { n } }
2375         {
2376             \bool_if:cTF { l__spintenc_sp #1 _concept_bool }
2377             {
2378                 \MathMLintent
2379                 { \l__spintenc_spnM_spnD_intent:nn { #1 } { #3 } }
2380                 { \operatorname { #2 } }
2381             }
2382             { \MathMLintent { :silent } { \operatorname { #2 } } }
2383             \MathMLintent { :silent } { ( }
2384             \MathMLintent { :silent } { \c__spintenc_invisible_sep_int } % prevent doble "de"
2385             n
2386             \MathMLintent { :silent } { ) }
2387         }
2388     }
2389     {
2390         \l__spintenc_luafun_spnmd_classify:n { \exp_not:n { #4 } }
2391         \str_if_eq:VnT \l__spintenc_spnM_spnD_luaset_error_str { true }
2392         { \msg_error:nne { spintenc } { md-invalid-argument } { \exp_not:c { sp #1 } } }
2393         \bool_if:cTF { l__spintenc_sp #1 _silent_bool }
2394         {
2395             \str_if_eq:VnTF \l__spintenc_spnmd_luaset_is_number_str { true }
2396             { \l__spintenc_spnM_spnD_silent:nn { #2 } { \spnum { #4 } } }
2397             { \l__spintenc_spnM_spnD_silent:nn { #2 } { #4 } }
2398         }
2399         {
2400             \bool_if:cTF { l__spintenc_sp #1 _concept_bool }
2401             {
2402                 \MathMLintent
2403                 { \l__spintenc_spnM_spnD_intent:nn { #1 } { #3 } }
2404                 { \operatorname { #2 } }
2405             }
2406             { \MathMLintent { :silent } { \operatorname { #2 } } }
2407             \MathMLintent { :silent } { ( }

```

```

2408     \MathMLint { :silent } { \c__spintent_invisible_sep_int } % prevent doble "de"
2409     \str_if_eq:VnTF \l__spintent_spmnd_luaset_is_number_str { true }
2410     { \spnum { #4 } }
2411     { #4 }
2412     \MathMLint { :silent } { ) }
2413   }
2414   \str_if_eq:VnTF \l__spintent_spmnd_luaset_is_number_str { true }
2415   {
2416     \bool_if:cT { l__spintent_sp #1 _print_result_bool }
2417     {
2418       \bool_if:cTF { l__spintent_sp #1 _silent_bool }
2419       {
2420         \MathMLint { :silent } { \c__spintent_invisible_sep_int } % prevent "por"
2421         \MathMLint { _- }
2422         {
2423           \__spintent_inner_sep:
2424           \__spintent_spm_spmD_result:nn { #1 } { #4 }
2425         }
2426       }
2427     }
2428     \__spintent_spm_spmD_result:nn { #1 } { #4 }
2429   }
2430 }
2431 }
2432 }
2433 }

```

(End of definition for `\__spintent_spm_spmD_exec:nnnn`.)

`\spnM` and `\spnD` The commands `\spnM` and `\spnD` act as the public interface wrappers for typesetting multiples and divisors, respectively. They enforce math mode, open a local group, and defer execution to the function `\__spintent_spm_spmD_process:nnnn`, passing along the optional *(keys)* ‘#1’ and the numeric *(argument)* ‘#2’.

```

2434 \NewDocumentCommand \spnM { O{} d() }
2435 {
2436   \mode_if_math:TF
2437   {
2438     \group_begin:
2439     \__spintent_spm_spmD_process:nnnn { nM } { M } { multiplos } {#1} {#2}
2440     \group_end:
2441   }
2442   { \msg_error:nnn { spintent } { need-math-mode } { \spnM } }
2443 }
2444 \NewDocumentCommand \spnD { O{} d() }
2445 {
2446   \mode_if_math:TF
2447   {
2448     \group_begin:
2449     \__spintent_spm_spmD_process:nnnn { nD } { D } { divisores } {#1} {#2}
2450     \group_end:
2451   }
2452   { \msg_error:nnn { spintent } { need-math-mode } { \spnD } }
2453 }

```

(End of definition for `\spnM` and `\spnD`. These functions are documented on page 15.)

11.28 The command `\sprazon`

The user command `\sprazon` typesets a ratio $a:b$ as a fraction (`\dfrac` or `\frac`, per *use-dfrac*), read natively — no Lua parsing involved.

11.28.1 Definition of keys for `\sprazon`

We define the *(keys)* `silent`, `use-dfrac` and `inline`.

```

2454 \keys_define:nn { spintent / sprazon }
2455 {
2456   silent      .bool_set:N = \l__spintent_sprazon_silent_bool,
2457   silent      .initial:n = false,
2458   silent      .value_required:n = true,
2459   use-dfrac   .bool_set:N = \l__spintent_sprazon_use_dfrac_bool,
2460   use-dfrac   .initial:n = false,
2461   use-dfrac   .value_required:n = true,
2462   inline      .bool_set:N = \l__spintent_sprazon_inline_bool,

```

```

2463     inline      .initial:n = false,
2464     inline      .value_required:n = true,
2465     unknown     .code:n =
2466     {
2467         \str_set:Nn \l__spintent_command_name_str { sprazon }
2468         \__spintent_unknown_keys_cmd:n {#1}
2469     },
2470 }

```

(End of definition for *silent* and others.)

`\__spintent_sprazon_exec:nnn` The function `\__spintent_sprazon_exec:nnn` receives the optional $\langle keys \rangle$ in ‘#1’, the numerator in ‘#2’ and the denominator in ‘#3’. Under *silent*, the whole thing is silenced as one unit — `\frac/\dfrac` (per *use-dfrac*) normally, or `#2 : #3` under *inline*.

Otherwise, with *inline*=false (the default), numerator and denominator are each tagged through a native `\MathMLarg` (num/denom) inside `\frac/\dfrac`, and the outer `\MathMLintent` reads `_es-a:infix($num,$denom)`. With *inline*=true, the same two references are tagged side by side instead — no `\frac/\dfrac` involved — with the `:` between them carrying the `_es-a:infix($num,$denom)` intent on its own.

```

2471 \cs_new_protected:Nn \__spintent_sprazon_exec:nnn
2472 {
2473     \keys_set:nn { spintent / sprazon } { #1 }
2474     \__spintent_invisible_sep:
2475     \bool_if:NTF \l__spintent_sprazon_silent_bool
2476     {
2477         \MathMLintent { _- }
2478         {
2479             \__spintent_inner_sep:
2480             \bool_if:NTF \l__spintent_sprazon_inline_bool
2481             { #2 : #3 }
2482             {
2483                 \bool_if:NTF \l__spintent_sprazon_use_dfrac_bool
2484                 { \dfrac { #2 } { #3 } }
2485                 { \frac { #2 } { #3 } }
2486             }
2487         }
2488     }
2489     {
2490         \bool_if:NTF \l__spintent_sprazon_inline_bool
2491         {
2492             \MathMLintent { _es-a:infix($num,$denom) }
2493             {
2494                 \MathMLarg { num } { #2 }
2495                 :
2496                 \MathMLarg { denom } { #3 }
2497             }
2498         }
2499         {
2500             \MathMLintent { _es-a:infix($num,$denom) }
2501             {
2502                 \__spintent_inner_sep:
2503                 \bool_if:NTF \l__spintent_sprazon_use_dfrac_bool
2504                 {
2505                     \dfrac { \MathMLarg { num } { #2 } } { \MathMLarg { denom } { #3 } }
2506                 }
2507                 { \frac { \MathMLarg { num } { #2 } } { \MathMLarg { denom } { #3 } } }
2508             }
2509         }
2510     }
2511 }

```

(End of definition for `\__spintent_sprazon_exec:nnn`.)

`\sprazon` The public command `\sprazon` enforces *math mode* and delegates execution to `\__spintent_sprazon_exec:nnn`.

```

2512 \NewDocumentCommand \sprazon { O{} m m }
2513 {
2514     \mode_if_math:TF
2515     {
2516         \group_begin:
2517         \__spintent_sprazon_exec:nnn { #1 } { #2 } { #3 }
2518         \group_end:
2519     }

```

```

2520     { \msg_error:nnn { spintent } { need-math-mode } { \sprazon } }
2521   }

```

(End of definition for \sprazon. This function is documented on page 18.)

11.29 The command \spProp

The user command \spProp typesets a proportion $a:b = c:d$ as two fractions joined by $=$, reusing __spintent_sprazon_exec:nnn directly (not the public \sprazon command) for each ratio.

\spProp The public command \spProp enforces *math mode*, and forwards the same $\langle keys \rangle$ in ‘#1’ to both ratios: a/b (‘#2’/‘#3’) and c/d (‘#4’/‘#5’), joined by a native \MathMLintent reading _como over the $=$ sign.

```

2522 \NewDocumentCommand \spProp { 0{ } m m m m }
2523 {
2524   \mode_if_math:TF
2525   {
2526     \group_begin:
2527     \__spintent_sprazon_exec:nnn { #1 } { #2 } { #3 }
2528     \bool_if:NTF \__spintent_sprazon_silent_bool
2529     {
2530       \MathMLintent { :silent } { = }
2531     }
2532     { \MathMLintent { _como } { = } }
2533     \__spintent_sprazon_exec:nnn { #1 } { #4 } { #5 }
2534     \group_end:
2535   }
2536   { \msg_error:nnn { spintent } { need-math-mode } { \spProp } }
2537 }

```

(End of definition for \spProp. This function is documented on page 18.)

11.30 The command \spang

The command \spang is the main public command for typesetting and tagging sexagesimal angle expressions (degrees, minutes, and seconds). It parses the input using the Lua function __spintent_luafun_spang_parse:n. This separates the degrees, minutes, and seconds to construct a MathML intent tree with the correct postfixes for NVDA/MathCAT.

```

\__spintent_spang_luaset_grado_str
\__spintent_spang_luaset_minuto_str
\__spintent_spang_luaset_segundo_str
\__spintent_spang_luaset_error_str

```

These internal string variables store the values returned from the Lua module spintent.lua. They hold the extracted numerical values for degrees, minutes, and seconds, as well as the error status used during formatting.

```

2538 \str_new:N \__spintent_spang_luaset_grado_str
2539 \str_new:N \__spintent_spang_luaset_minuto_str
2540 \str_new:N \__spintent_spang_luaset_segundo_str
2541 \str_new:N \__spintent_spang_luaset_error_str

```

(End of definition for __spintent_spang_luaset_grado_str and others.)

11.30.1 Definition of error messages for \spang

invalid-angle-format

The error message invalid-angle-format defines the text displayed when an invalid angle input is detected. It is triggered when an argument fails the format checks, does not match the expected sexagesimal pattern, or contains invalid measurement symbols.

```

2542 \msg_new:nnnn { spintent } { invalid-angle-format }
2543 { Invalid~angle~format~in~#1. }
2544 {
2545   The~argument~'~#2'~could~not~be~parsed. \\\
2546 }

```

(End of definition for invalid-angle-format.)

11.30.2 Internal functions for \spang

__spintent_spang_process:n

The internal function __spintent_spang_process:n parses the angle input using __spintent_luafun_spang_parse:n after resetting the error flag in __spintent_spang_luaset_error_str. If an invalid format is detected, it issues the invalid-angle-format error. Otherwise, it checks the extracted string variables __spintent_spang_luaset_grado_str, __spintent_spang_luaset_minuto_str, and __spintent_spang_luaset_segundo_str. For each non-empty variable, it constructs the corresponding MathML intent structure with the correct measurement symbols and postfixes for NVDA/MathCAT. Each value is expanded with \exp_args:N before it reaches \spunit, which protects its $\langle argument \rangle$ from expansion.

```

2547 \cs_new_protected:Nn \__spintent_spang_process:n
2548 {
2549   \str_set:Nn \__spintent_spang_luaset_error_str { false }
2550   \__spintent_luafun_spang_parse:n { #1 }

```

```

2551 \str_if_eq:VnTF \l__spintent_spang_luaset_error_str { true }
2552 {
2553   \msg_error:nnnn { spintent } { invalid-angle-format } { \spang } { #1 }
2554 }
2555 {
2556   \str_if_empty:NF \l__spintent_spang_luaset_grado_str
2557   {
2558     \exp_args:Ne \spunit { \l__spintent_spang_luaset_grado_str ° }
2559   }
2560   \str_if_empty:NF \l__spintent_spang_luaset_minuto_str
2561   {
2562     \MathMLintent { _minutos:postfix($m) }
2563     {
2564       \MathMLarg { m }
2565       {
2566         \exp_args:Ne \spunit { \l__spintent_spang_luaset_minuto_str ' }
2567       }
2568     }
2569   }
2570   \str_if_empty:NF \l__spintent_spang_luaset_segundo_str
2571   {
2572     \MathMLintent { _segundos:postfix($s) }
2573     {
2574       \MathMLarg { s }
2575       {
2576         \exp_args:Ne \spunit { \l__spintent_spang_luaset_segundo_str " }
2577       }
2578     }
2579   }
2580 }
2581 }

```

(End of definition for `\__spintent_spang_process:n`.)

`\spang` The command `\spang` is the public interface for formatting sexagesimal angle expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_spang_process:n` to process the $\langle argument \rangle$.

```

2582 \NewDocumentCommand \spang { m }
2583 {
2584   \mode_if_math:TF
2585   {
2586     \group_begin:
2587     \__spintent_spang_process:n { #1 }
2588     \group_end:
2589   }
2590   { \msg_error:nnn { spintent } { need-math-mode } { \spang } }
2591 }

```

(End of definition for `\spang`. This function is documented on page 24.)

11.31 The command `\spnZ`

The user-level command `\spnZ` formats integers (the set $\mathbb{Z}$). It includes specific options to automatically enclose numbers (particularly negative ones) in visual parentheses and allows you to control whether these parentheses are read by MathCAT.

This command verifies that the $\langle argument \rangle$ is indeed an *integer*, checking for the complete absence of decimal separators or attached units of measurement.

11.31.1 Definition of keys for `\spnZ`

Now we define the $\langle keys \rangle$ `cifras-sep` and `read-sign`. These are `.meta:nn` $\langle keys \rangle$ passed directly to the `\spnum` command. We also introduce `wrap-parens`, which determines whether the number is enclosed in parentheses, and `read-parens`, which controls if these parentheses are read by MathCAT.

```

2592 \keys_define:nn { spintent / spnZ }
2593 {
2594   cifras-sep .meta:nn = { spintent / spnum } { cifras-sep = {#1} },
2595   read-sign .meta:nn = { spintent / spnum } { read-sign = {#1} },
2596   wrap-parens .bool_set:N = \l__spintent_spnz_wrap_parens_bool,
2597   wrap-parens .initial:n = false,
2598   wrap-parens .value_required:n = true,
2599   read-parens .bool_set:N = \l__spintent_spnz_read_parens_bool,
2600   read-parens .initial:n = true,

```

```

2601     read-parens .value_required:n = true,
2602     unknown      .code:n =
2603     {
2604         \str_set:Nn \__spintent_command_name_str { spnZ }
2605         \__spintent_unknown_keys_cmd:n {#1}
2606     },
2607 }

```

(End of definition for *cifras-sep* and others.)

11.31.2 Definition of error messages

spnz-not-integer Raised when the $\langle argument \rangle$ contains decimal separators or attached units, which conflict with the constraints of a pure integer representation.

```

2608 \msg_new:nnnn { spintent } { spnz-not-integer }
2609 { The~value~'#1'~is~not~a~valid~integer. }
2610 {
2611     Command~\token_to_str:N \spnZ~only~accepts~integers.\\
2612 }

```

(End of definition for *spnz-not-integer*.)

11.31.3 Internal functions for \spnZ

__spintent_spnz_render_parens: The function `\__spintent_spnz_render_parens:` first checks the state of the boolean variable `\__spintent_spnz` (controlled by the `read-parens` key). If it is `true`, it prints the number surrounded by parentheses. Otherwise, it applies the `:silent` function intent to the parentheses while printing the number.

```

2613 \cs_new_protected:Nn \__spintent_spnz_render_parens:
2614 {
2615     \bool_if:NTF \__spintent_spnz_read_parens_bool
2616     {
2617         ( \__spintent_spnz_print_num_start: )
2618     }
2619     {
2620         \MathMLintent { :silent } { ( }
2621         \__spintent_spnz_print_num_start:
2622         \MathMLintent { :silent } { ) }
2623     }
2624 }

```

(End of definition for `\__spintent_spnz_render_parens:.`)

__spintent_spnz_process:nn The function `\__spintent_spnz_process:nn` first processes the $\langle keys \rangle$ passed in ‘#1’ and then calls `\__spintent_luafun_clean_split_and_set:n` on ‘#2’. It verifies that the input is a valid integer by checking if the variable `\__spintent_luaaset_has_integer_str` is `true`. If not `true`, an “error” is raised.

If it is `true`, it checks whether `\__spintent_luaaset_part_int_tl` is *empty* and raises an “error” if so. Otherwise, it checks `\__spintent_spnz_wrap_parens_bool` set by `wrap-parens` and calls either the function `\__spintent_spnz_render_parens:` or `\__spintent_spnz_print_num_start:.`

```

2625 \cs_new_protected:Nn \__spintent_spnz_process:nn
2626 {
2627     \keys_set:nn { spintent / spnZ } {#1}
2628     \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#2} }
2629     \str_if_eq:VnTF \__spintent_luaaset_has_integer_str { true }
2630     {
2631         \tl_if_empty:NTF \__spintent_luaaset_part_int_tl
2632         { \msg_error:nnn { spintent } { spnum-no-digits } {#2} }
2633         {
2634             \bool_if:NTF \__spintent_spnz_wrap_parens_bool
2635             { \__spintent_spnz_render_parens: }
2636             { \__spintent_spnz_print_num_start: }
2637         }
2638     }
2639     { \msg_error:nnn { spintent } { spnz-not-integer } {#2} }
2640 }

```

(End of definition for `\__spintent_spnz_process:nn`.)

\spnZ Defines the user command `\spnZ`, which is executed within a *group* and strictly enforces *math mode*.

```

2641 \NewDocumentCommand \spnZ { O{} m }
2642 {
2643     \mode_if_math:TF
2644     {

```

```

2645 \group_begin:
2646   \__spintent_spmz_process:nn { #1 } { #2 }
2647 \group_end:
2648 }
2649 { \msg_error:nnn { spintent } { need-math-mode } { \spnZ } }
2650 }

```

(End of definition for \spnZ. This function is documented on page 17.)

11.32 The command \spmQ

The command `\spmQ` typesets “mixed numbers” within Spanish mathematical contexts (e.g., $3\frac{1}{2}$). It delegates the scaling of the integer part to the Lua module `spintent.lua` via `\__spintent_luafun_spmQ_scale_int:Vn`, which measures both the integer glyph and the fraction box at the node level and computes a scaling factor to align their heights visually. Finally, it constructs the correct MathML intent for MathCAT.

11.32.1 Definition of variables for \spmQ

The variable `\__spintent_spmQ_join_word_str` stores the value set by the key `join-word` which will be verbalized by MathCAT as a grammatical connector between the integer and fractional parts.

The variable `\__spintent_spmQ_intent_read_sign_str` stores the `:intent` passed to `\MathMLintent` when the signs ‘+’ or ‘−’ are present in the integer part, used by the function `\__spintent_spmQ_print_scale_int:n`.

The variables `\__spintent_spmQ_wrap_parens_l_tl` and `\__spintent_spmQ_wrap_parens_r_tl` store the sized parentheses set by the function `\__spintent_spmQ_set_wrap_parens_size:` and used by the key `wrap-parens`.

The integer variable `\__spintent_saved_luamml_flag_int` temporarily stores the current value of `\__luamml_flag_int` before disabling it during the injection of the scaled integer node, and restores it afterwards. This prevents `luamml` from generating a spurious `<math>` element around the `sub_box` processed in isolation, before `\frac` has expanded.

```

2651 \str_new:N \__spintent_spmQ_join_word_str
2652 \str_new:N \__spintent_spmQ_intent_read_sign_str
2653 \tl_new:N \__spintent_spmQ_wrap_parens_l_tl
2654 \tl_new:N \__spintent_spmQ_wrap_parens_r_tl
2655 \int_new:N \__spintent_saved_luamml_flag_int

```

(End of definition for `\__spintent_spmQ_join_word_str` and others.)

11.32.2 Definition of error messages for \spmQ

The error message `spmQ-zero-denominator` is raised when a zero denominator is passed to `\spmQ`. The second error `spmQ-not-integer` is returned when the integer part passed in `{\argument}` is not actually an integer.

```

2656 \msg_new:nnnn { spintent } { spmQ-zero-denominator }
2657 {
2658   Zero~denominator~in~\token_to_str:N \spmQ~\msg_line_context:.
2659 }
2660 {
2661   The~denominator~of~a~fraction~cannot~be~zero.\
2662 }
2663 \msg_new:nnnn { spintent } { spmQ-not-integer }
2664 {
2665   The~value~'#1'~is~not~a~valid~integer~\msg_line_context:.
2666 }
2667 {
2668   Command~\token_to_str:N \spmQ \c_space_tl only~accepts~integers.\
2669 }

```

(End of definition for `spmQ-zero-denominator` and `spmQ-not-integer`.)

11.32.3 Definition of keys for \spmQ

Now we add the keys `join-word`, `use-spnZ`, `wrap-parens`, `read-parens` and `use-dfrac`.

```

join-word
use-spnZ
wrap-parens
read-parens
use-dfrac
unknown
2670 \keys_define:nn { spintent / spmQ }
2671 {
2672   join-word .choices:nn =
2673     { entero , enteros , y }
2674   {
2675     \int_case:nn { \l_keys_choice_int }
2676     {
2677       {1} { \str_set:Nn \__spintent_spmQ_join_word_str { entero } }
2678       {2} { \str_set:Nn \__spintent_spmQ_join_word_str { enteros } }
2679       {3} { \str_set:Nn \__spintent_spmQ_join_word_str { y } }
2680     }

```

```

2681     },
2682     join-word      .initial:n = y,
2683     join-word      .value_required:n = true,
2684     join-word/unknown .code:n =
2685       \msg_error:nneee { spintent } { unknown-choice }
2686       { join-word } { entero, ~enteros, ~y } { \exp_not:n {#1} },
2687     use-spnZ       .bool_set:N = \l__spintent_spmQ_use_spnZ_bool,
2688     use-spnZ       .initial:n = true,
2689     use-spnZ       .value_required:n = true,
2690     use-dfrac      .bool_set:N = \l__spintent_spmQ_use_dfrac_bool,
2691     use-dfrac      .initial:n = false,
2692     use-dfrac      .value_required:n = true,
2693     wrap-parens    .bool_set:N = \l__spintent_spmQ_wrap_parens_bool,
2694     wrap-parens    .initial:n = false,
2695     wrap-parens    .value_required:n = true,
2696     read-parens    .bool_set:N = \l__spintent_spmQ_read_parens_bool,
2697     read-parens    .initial:n = true,
2698     read-parens    .value_required:n = true,
2699     unknown        .code:n =
2700       {
2701         \str_set:Nn \l__spintent_command_name_str { spmQ }
2702         \__spintent_unknown_keys_cmd:n {#1}
2703       },
2704   }

```

(End of definition for `join-word` and others.)

11.32.4 Internal functions for `\spmQ`

`\__spintent_spmQ_set_wrap_parens_size:`

The function `\__spintent_spmQ_set_wrap_parens_size:` captures the current mathematical style using the primitive `\mathstyle` from LuaTeX and sets the variables `\l__spintent_spmQ_wrap_parens_l_tl` and `\l__spintent_spmQ_wrap_parens_r_tl` with the appropriate size for the parentheses. Display styles (0 and 1) receive `\Bigl\lparen` and `\Bigr\rparen`; text styles (2 and 3) receive `\bigl\lparen` and `\bigr\rparen`; any other style falls back to `\lparen` and `\rparen`.

If the key `use-dfrac` is active, the size is always forced to `\Bigl\lparen`/`\Bigr\rparen` regardless of the current mathematical style, since `\dfrac` always produces a display-height fraction.

```

2705 \cs_new_protected:Nn \__spintent_spmQ_set_wrap_parens_size:
2706 {
2707   \int_case:nnF { \mathstyle }
2708   {
2709     { 0 }
2710     {
2711       \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2712       \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2713     }
2714     { 1 }
2715     {
2716       \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2717       \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2718     }
2719     { 2 }
2720     {
2721       \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \bigl \lparen }
2722       \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2723     }
2724     { 3 }
2725     {
2726       \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \bigl \lparen }
2727       \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2728     }
2729   }
2730   {
2731     \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \lparen }
2732     \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \rparen }
2733   }

```

If the key `use-dfrac` is active, we must reset the variables `\l__spintent_spmQ_wrap_parens_l_tl` and `\l__spintent_spmQ_wrap_parens_r_tl` to `\Bigl\lparen`/`\Bigr\rparen` unconditionally.

```

2734   \bool_if:NT \l__spintent_spmQ_use_dfrac_bool
2735   {
2736     \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2737     \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }

```

```

2738     }
2739 }

```

(End of definition for `\__spintent_spmQ_set_wrap_parens_size:`)

```

\__spintent_spmQ_start_wrap_parens:
\__spintent_spmQ_stop_wrap_parens:

```

The functions `\__spintent_spmQ_start_wrap_parens:` and `\__spintent_spmQ_stop_wrap_parens:` open and close the parentheses around the “*mixed number*” when the key `wrap-parens` is active. They first call `\__spintent_spmQ_set_wrap_parens_size:` to determine the correct size for the current mathematical style, then print the appropriate parenthesis stored in `\l__spintent_spmQ_wrap_parens_l_tl` or `\l__spintent_spmQ_wrap_parens_r_tl`.

If the variable `\l__spintent_spmQ_read_parens_bool`, controlled by the key `read-parens`, is *false*, each parenthesis is wrapped inside `\MathMLintent{:silent}` so that NVDA/MathCAT does not verbalize them.

```

2740 \cs_new_protected:Nn \__spintent_spmQ_start_wrap_parens:
2741 {
2742   \bool_if:NT \l__spintent_spmQ_wrap_parens_bool
2743   {
2744     \__spintent_spmQ_set_wrap_parens_size:
2745     \bool_if:NTF \l__spintent_spmQ_read_parens_bool
2746       { \l__spintent_spmQ_wrap_parens_l_tl }
2747       { \MathMLintent { :silent } { \l__spintent_spmQ_wrap_parens_l_tl } }
2748   }
2749 }
2750 \cs_new_protected:Nn \__spintent_spmQ_stop_wrap_parens:
2751 {
2752   \bool_if:NT \l__spintent_spmQ_wrap_parens_bool
2753   {
2754     \__spintent_spmQ_set_wrap_parens_size:
2755     \bool_if:NTF \l__spintent_spmQ_read_parens_bool
2756       { \l__spintent_spmQ_wrap_parens_r_tl }
2757       { \MathMLintent { :silent } { \l__spintent_spmQ_wrap_parens_r_tl } }
2758   }
2759 }

```

(End of definition for `\__spintent_spmQ_start_wrap_parens:` and `\__spintent_spmQ_stop_wrap_parens:`)

```
\__spintent_spmQ_set_join_word:
```

The function `\__spintent_spmQ_set_join_word:` injects the grammatical connector between the integer part and the fraction by dispatching on the value stored in `\l__spintent_spmQ_join_word_str`, set by the key `join-word`. In all three cases the visual output is the Unicode invisible separator U+2063 via `\c__spintent_invisible_sep_tl`, which produces a `<mi>` node in the MathML structure; what differs is the `:intent` text attached to it via `\MathMLintent`, which NVDA/MathCAT will verbalize as «*entero*», «*enteros*» or «*y*» respectively.

```

2760 \cs_new_protected:Nn \__spintent_spmQ_set_join_word:
2761 {
2762   \str_case:Nn \l__spintent_spmQ_join_word_str
2763   {
2764     { entero } { \MathMLintent { _entero } { \c__spintent_invisible_sep_int } }
2765     { enteros } { \MathMLintent { _enteros } { \c__spintent_invisible_sep_int } }
2766     { y } { \MathMLintent { _y- } { \c__spintent_invisible_sep_int } }
2767   }
2768 }

```

(End of definition for `\__spintent_spmQ_set_join_word:`)

```
\__spintent_spmQ_print_scale_int:
```

The function `\__spintent_spmQ_print_scale_int:` prints the integer part of the “*mixed number*” scaled to match the visual height of the accompanying fraction. Before calling the Lua function `\__spintent_luafun_spmQ_scale_int:` it temporarily disables `luamml` by saving the current value of `\l__luamml_flag_int` into `\l__spintent_saved_luamml_flag_int` and setting it to `0` (skip completely). This prevents `luamml` from generating a spurious `<math>` wrapper around the `sub_box` node that holds the scaled integer, which is processed in isolation before `\frac` has expanded. Once the Lua function returns, the original flag value is restored via `\l__spintent_saved_luamml_flag_int`.

The Lua function receives the integer digits from `\l__spintent_luaset_part_int_tl` and the fraction command name (`\frac` or `\dfrac`, determined by the key `use-dfrac`) as its two arguments, and inserts the appropriately scaled `sub_box` node directly into the current horizontal list.

```

2769 \cs_new_protected:Nn \__spintent_spmQ_print_scale_int:
2770 {
2771   \int_set_eq:Nc \l__spintent_saved_luamml_flag_int { l__luamml_flag_int }
2772   \int_set:Nn \l__luamml_flag_int { 0 }
2773   \bool_if:NTF \l__spintent_spmQ_use_dfrac_bool
2774     { \__spintent_luafun_spmQ_scale_int:Nn \l__spintent_luaset_part_int_tl { dfrac } }
2775     { \__spintent_luafun_spmQ_scale_int:Nn \l__spintent_luaset_part_int_tl { frac } }

```

```

2776   \int_set_eq:cN { l__luamml_flag_int } \l__spintent_saved_luamml_flag_int
2777 }

```

(End of definition for `\l__spintent_spmQ_print_scale_int:`.)

`\l__spintent_spmQ_print_scale_int:n` The function `\l__spintent_spmQ_print_scale_int:n` takes the integer part of the “mixed number” passed in ‘#1’. It first calls `\l__spintent_luafun_clean_split_and_set:n` on ‘#1’ to parse and populate the shared Lua variables, then checks the status of `\l__spintent_luaset_has_integer_str`.

If the status is `true`, it inspects `\l__spintent_luaset_sign_tl`. When the sign is *empty*, it delegates directly to `\l__spintent_spmQ_print_scale_int:` to render the scaled integer. When a sign is present, it sets `\l__spintent_spmQ_intent_read_sign_str` to either `_más:prefix($e)` or `_menos:prefix($e)` and wraps the sign glyph together with a `MathML` argument (`$e`) containing the scaled integer inside a `\MathMLintent`, so that NVDA/MathCAT verbalizes the sign as a prefix before the integer value. An `\l__spintent_invisible_sep:` call between the sign and the `MathMLarg` forces the opening of a new `<mrow>` in the `MathML` structure, which is required for the intent tree to be well-formed.

If the status is `false`, the `spmQ-not-integer` “error” is raised immediately.

```

2778 \cs_new_protected:Nn \l__spintent_spmQ_print_scale_int:n
2779 {
2780   \l__spintent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2781   \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { true }
2782   {
2783     \tl_if_empty:NTF \l__spintent_luaset_sign_tl
2784     {
2785       \l__spintent_spmQ_print_scale_int:
2786     }
2787     {
2788       \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
2789       {
2790         \str_set:Nn
2791           \l__spintent_spmQ_intent_read_sign_str { _más:prefix($e) }
2792       }
2793       {
2794         \str_set:Nn
2795           \l__spintent_spmQ_intent_read_sign_str { _menos:prefix($e) }
2796       }
2797       \MathMLintent { \l__spintent_spmQ_intent_read_sign_str }
2798       {
2799         \l__spintent_luaset_sign_tl \c__spintent_invisible_sep_int % \l__spintent_invisible_s
2800         \MathMLarg { e } { \l__spintent_spmQ_print_scale_int: }
2801       }
2802     }
2803   }
2804   { \msg_error:nnn { spintent } { spmQ-not-integer } { #1 } }
2805 }

```

(End of definition for `\l__spintent_spmQ_print_scale_int:n`.)

`\l__spintent_spmQ_print_spnZ_dfrac:nn` The function `\l__spintent_spmQ_print_spnZ_dfrac:nn` typesets the fractional part of the “mixed number”. It takes the numerator in ‘#1’ and the denominator in ‘#2’. It first calls `\l__spintent_luafun_clean_split_and_set:` on ‘#1’ to validate that the numerator is an integer; if `\l__spintent_luaset_has_integer_str` is `false`, the `spmQ-not-integer` “error” is raised immediately.

If the validation passes, the function dispatches on the two boolean variables `\l__spintent_spmQ_use_spnZ_bool` and `\l__spintent_spmQ_use_dfrac_bool`, controlled respectively by the keys `use-spnZ` and `use-dfrac`. When `use-spnZ` is `true`, both the numerator and denominator are passed through `\spnZ` to gain `MathML` integer semantics; otherwise, the raw token lists ‘#1’ and ‘#2’ are used directly. In both branches, the fraction command is selected dynamically via `\use:c`: if `use-dfrac` is `true`, `dfrac` is assembled; otherwise, `frac` is used.

```

2806 \cs_new_protected:Nn \l__spintent_spmQ_print_spnZ_dfrac:nn
2807 {
2808   \l__spintent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2809   \str_if_eq:VnT \l__spintent_luaset_has_integer_str { false }
2810   {
2811     \msg_error:nnn { spintent } { spmQ-not-integer } { #1 }
2812   }
2813   \bool_if:NTF \l__spintent_spmQ_use_spnZ_bool
2814   {
2815     \use:c { \bool_if:NT \l__spintent_spmQ_use_dfrac_bool { d } frac }
2816     { \spnZ { #1 } } { \spnZ { #2 } }
2817   }

```

```

2818     {
2819         \use:c { \bool_if:NT \__spintent_spmQ_use_dfrac_bool { d } frac }
2820         { #1 } { #2 }
2821     }
2822 }

```

(End of definition for `\__spintent_spmQ_print_spmZ_dfrac:nn`.)

`\__spintent_spmQ_print:nnn`

The function `\__spintent_spmQ_print:nnn` is the main rendering orchestrator for `\spmQ`. It takes three arguments: the integer part ‘#1’, the numerator ‘#2’, and the denominator ‘#3’. It opens the optional parentheses via `\__spintent_spmQ_start_wrap_parens:`, then calls `\__spintent_spmQ_print_scale_int:n` to scale and render the integer part. Next, it injects the grammatical connector between the integer and the fraction via `\__spintent_spmQ_set_join_word:`. Then it calls `\__spintent_spmQ_print_spmZ_dfrac:nn` to typeset the fraction using the keys `use-dfrac` and `use-spmZ`. Finally, it closes the optional parentheses via `\__spintent_spmQ_stop_wrap_parens:`.

```

2823 \cs_new_protected:Nn \__spintent_spmQ_print:nnn
2824 {
2825     \__spintent_spmQ_start_wrap_parens:
2826     \__spintent_spmQ_print_scale_int:n { #1 }
2827     \__spintent_spmQ_set_join_word:
2828     \__spintent_spmQ_print_spmZ_dfrac:nn { #2 } { #3 }
2829     \__spintent_spmQ_stop_wrap_parens:
2830 }

```

(End of definition for `\__spintent_spmQ_print:nnn`.)

`\__spintent_spmQ_exec:nnnn`

The internal function `\__spintent_spmQ_exec:nnnn` takes four arguments: the optional *(keys)* ‘#1’, the integer ‘#2’, the numerator ‘#3’, and the denominator ‘#4’. First sets the optional *(keys)* from ‘#1’, then processes the denominator input in ‘#4’ using `\__spintent_luafun_clean_split_and_set:n` to validate it is a non-zero integer.

It checks the status of `\l__spintent_luaaset_has_integer_str`. If `true` and the integer part is exactly zero, it issues the `spmQ-zero-denominator` “error”. Otherwise, whether the status is `false` or the integer part is *non-zero*, it calls `\__spintent_spmQ_print:nnn` to format the “mixed number”.

```

2831 \cs_new_protected:Nn \__spintent_spmQ_exec:nnnn
2832 {
2833     \keys_set:nn { spintent / spmQ } { #1 }
2834     \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #4 } }
2835     \str_if_eq:VnTF \l__spintent_luaaset_has_integer_str { true }
2836     {
2837         \int_compare:nNnTF { \l__spintent_luaaset_part_int_tl } = { 0 }
2838         { \msg_error:nn { spintent } { spmQ-zero-denominator } }
2839         { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2840     }
2841     { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2842 }

```

(End of definition for `\__spintent_spmQ_exec:nnnn`.)

`\spmQ`

The command `\spmQ` is the public interface for formatting “mixed numbers”. It first checks if it is being used in *math mode*, issuing an “error” if not. Inside a local group, it calls `\__spintent_spmQ_exec:nnnn` to process the optional *(keys)* and the *{(arguments)}*.

```

2843 \NewDocumentCommand \spmQ { O{} m m m }
2844 {
2845     \mode_if_math:TF
2846     {
2847         \group_begin:
2848         \__spintent_spmQ_exec:nnnn { #1 } { #2 } { #3 } { #4 }
2849         \group_end:
2850     }
2851     { \msg_error:nnn { spintent } { need-math-mode } { \spmQ } }
2852 }

```

(End of definition for `\spmQ`. This function is documented on page 17.)

11.33 The commands of the geo family

The commands described in this section provide a semantic interface for geometric objects in Spanish mathematical contexts. They all delegate parsing to `spintent.lua`, sharing the variables `\l__spintent_geo_luaaset_error_str`, `\l__spintent_geo_luaaset_intent_str`, `\l__spintent_geo_luaaset_print_tl`, `\l__spintent_geo_luaaset_sub_str` and `\l__spintent_geo_luaaset_tipo_str` — constructing a valid MathCAT intent — and a single point is always annotated through `\__spintent_mathmlarg_point:nnn`. Every command also activates `\l__spintent_scope_attr_int` within its own group, so the shared queue never claims content typeset outside a geo command in the same formula.

11.33.1 Definition of shared variables

Internal variables shared across all commands of the family to avoid repeated calls into Lua.

```

\l__spintenc_geo_luaset_error_str
\l__spintenc_geo_luaset_intent_str
\l__spintenc_geo_luaset_print_tl
\l__spintenc_geo_luaset_is_name_str
\l__spintenc_geo_luaset_poly_sym_str
\l__spintenc_geo_luaset_angulo_case_str
\l__spintenc_geo_luaset_concept_str
\l__spintenc_geo_luaset_letra_word_str
\l__spintenc_geo_luaset_sub_spoken_str
\l__spintenc_geo_luaset_nombre_es_num_str
\l__spintenc_geo_luaset_letra_str
\l__spintenc_geo_luaset_sub_str
\l__spintenc_geo_luaset_tipo_str

2853 \str_new:N \l__spintenc_geo_luaset_error_str
2854 \str_new:N \l__spintenc_geo_luaset_intent_str
2855 \tl_new:N \l__spintenc_geo_luaset_print_tl
2856 \clist_new:N \l__spintenc_geo_luaset_points_clist
2857 \str_new:N \l__spintenc_geo_luaset_is_name_str
2858 \str_new:N \l__spintenc_geo_luaset_poly_sym_str
2859 \str_new:N \l__spintenc_geo_luaset_angulo_case_str
2860 \str_new:N \l__spintenc_geo_luaset_concept_str
2861 \str_new:N \l__spintenc_geo_luaset_letra_word_str
2862 \str_new:N \l__spintenc_geo_luaset_sub_spoken_str
2863 \str_new:N \l__spintenc_geo_luaset_nombre_es_num_str
2864 \str_new:N \l__spintenc_geo_luaset_letra_str
2865 \str_new:N \l__spintenc_geo_luaset_sub_str
2866 \str_new:N \l__spintenc_geo_luaset_tipo_str

```

(End of definition for `\l__spintenc_geo_luaset_error_str` and others.)

11.34 The command `\spcoord`

The user command `\spcoord` typesets an ordered pair (x,y) , with the comma (or semicolon) between them carrying its own `:infix` intent, and each side wrapped with `:prefix($vp)/:prefix($vn)` when it starts with $+/-$, so the sign is always read as más/menos — never positivo/negativo, which is not how MathCAT reads a bare sign in this position.

11.34.1 Definition of variables for `\spcoord`

Variables set by the function `\__spintenc_luafun_spcoord_parse_and_set:n`. The string variable `\l__spintenc_spcoord_luaset_sep_str` is `,` or `;`, whichever the argument actually used. For the abscissa and ordinate values, the variables `\l__spintenc_spcoord_luaset_ #1 _sign_str` is `+`, `-`, or empty; the variables `\l__spintenc_spcoord_luaset_ #1 _type_str` is `number` or `raw`; the variables `\l__spintenc_spcoord_luaset_ #1 _value_str` is the value with the sign already removed; and the variables `\l__spintenc_spcoord_luaset_ #1 _has_frac_str` is `none`, `frac`, or `dfrac`; and the variables `\l__spintenc_spcoord_luaset_ #1 _composite_str` is `true` when the value is a sole subscripted, superscripted or radical node, `false` otherwise.

```

\l__spintenc_spcoord_luaset_error_str
\l__spintenc_spcoord_luaset_sep_str
\l__spintenc_spcoord_luaset_x_sign_str
\l__spintenc_spcoord_luaset_x_type_str
\l__spintenc_spcoord_luaset_x_value_str
\l__spintenc_spcoord_luaset_x_has_frac_str
\l__spintenc_spcoord_luaset_y_sign_str
\l__spintenc_spcoord_luaset_y_type_str
\l__spintenc_spcoord_luaset_y_value_str
\l__spintenc_spcoord_luaset_y_has_frac_str

2867 \str_new:N \l__spintenc_spcoord_luaset_error_str
2868 \str_new:N \l__spintenc_spcoord_luaset_sep_str
2869 \clist_map_inline:nn { x , y }
2870 {
2871   \str_new:c { \l__spintenc_spcoord_luaset_ #1 _sign_str }
2872   \str_new:c { \l__spintenc_spcoord_luaset_ #1 _type_str }
2873   \str_new:c { \l__spintenc_spcoord_luaset_ #1 _value_str }
2874   \str_new:c { \l__spintenc_spcoord_luaset_ #1 _has_frac_str }
2875   \str_new:c { \l__spintenc_spcoord_luaset_ #1 _composite_str }
2876 }

```

(End of definition for `\l__spintenc_spcoord_luaset_error_str` and others.)

The variable `\l__spintenc_spcoord_raw_arg_tl` holds the `{\argument}` exactly as given. The variable `\l__spintenc_spcoord_intent_str` holds the `:infix` word for the separator `_coma:infix` or `_punto-y-coma:infix`. The variables `\l__spintenc_spcoord_wrap_parens_l_tl`/`\l__spintenc_spcoord_wrap_parens_r_tl` hold the opening/closing delimiter, sized by the function `\__spintenc_spcoord_set_wrap_parens_size:.`

```

2877 \tl_new:N \l__spintenc_spcoord_raw_arg_tl
2878 \str_new:N \l__spintenc_spcoord_intent_str
2879 \tl_new:N \l__spintenc_spcoord_wrap_parens_l_tl
2880 \tl_new:N \l__spintenc_spcoord_wrap_parens_r_tl

```

(End of definition for `\l__spintenc_spcoord_raw_arg_tl` and others.)

11.34.2 Definition of error messages for `\spcoord`

`spcoord-invalid-arg`

```

2881 \msg_new:nnn { spintenc } { spcoord-invalid-arg }
2882 {
2883   Invalid~argument~'#1'~for~\token_to_str:N \spcoord.
2884 }

```

(End of definition for `spcoord-invalid-arg`.)

11.34.3 Definition of keys for \spcoord

`silent` We define the key `silent`.

```

2885 \keys_define:nn { spintenc / spcoord }
2886 {
2887   silent .bool_set:N = \l__spintenc_spcoord_silent_bool,
2888   silent .initial:n = false,
2889   silent .value_required:n = true,
2890   unknown .code:n =
2891     {
2892       \str_set:Nn \l__spintenc_command_name_str { spcoord }
2893       \__spintenc_unknown_keys_cmd:n {#1}
2894     },
2895 }
```

(End of definition for `silent`.)

11.34.4 Internal functions for \spcoord

`\__spintenc_spcoord_set_wrap_parens_size:`

With neither side carrying a fraction, the parentheses stay unsized. Otherwise, `\mathstyle` picks `\Bigl/\Bigr` for display, `\bigl/\bigr` for text. When either side is `dfrac`, the parentheses are forced to `\Bigl/\Bigr` regardless of `\mathstyle`, since `\dfrac` always renders at display height.

```

2896 \cs_new_protected:Nn \__spintenc_spcoord_set_wrap_parens_size:
2897 {
2898   \tl_set:Nn \l__spintenc_spcoord_wrap_parens_l_tl { \lparen }
2899   \tl_set:Nn \l__spintenc_spcoord_wrap_parens_r_tl { \rparen }
2900   \bool_if:nT
2901     {
2902       \bool_lazy_or_p:nn
2903         { \bool_not_p:n { \str_if_eq_p:Vn \l__spintenc_spcoord_luaset_x_has_frac_str { none } } }
2904         { \bool_not_p:n { \str_if_eq_p:Vn \l__spintenc_spcoord_luaset_y_has_frac_str { none } } }
2905     }
2906     {
2907       \int_case:nnF { \mathstyle }
2908       {
2909         { 0 }
2910         {
2911           \tl_set:Nn \l__spintenc_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2912           \tl_set:Nn \l__spintenc_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2913         }
2914         { 1 }
2915         {
2916           \tl_set:Nn \l__spintenc_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2917           \tl_set:Nn \l__spintenc_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2918         }
2919         { 2 }
2920         {
2921           \tl_set:Nn \l__spintenc_spcoord_wrap_parens_l_tl { \bigl \lparen }
2922           \tl_set:Nn \l__spintenc_spcoord_wrap_parens_r_tl { \bigr \rparen }
2923         }
2924         { 3 }
2925         {
2926           \tl_set:Nn \l__spintenc_spcoord_wrap_parens_l_tl { \bigl \lparen }
2927           \tl_set:Nn \l__spintenc_spcoord_wrap_parens_r_tl { \bigr \rparen }
2928         }
2929       }
2930     }
2931   \bool_if:nT
2932     {
2933       \bool_lazy_or_p:nn
2934         { \str_if_eq_p:Vn \l__spintenc_spcoord_luaset_x_has_frac_str { dfraction } }
2935         { \str_if_eq_p:Vn \l__spintenc_spcoord_luaset_y_has_frac_str { dfraction } }
2936     }
2937     {
2938       \tl_set:Nn \l__spintenc_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2939       \tl_set:Nn \l__spintenc_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2940     }
2941 }
2942 }
```

(End of definition for `\__spintenc_spcoord_set_wrap_parens_size:`.)

`\__spintrent_spcoord_value:n` With `\__spintrent_spcoord_luaset_ #1 _type_str` equal to `number`, calls `\spnum` on `\__spintrent_spcoord_value:n`. Otherwise, typesets it as given — a `\frac/\dfrac` or a letter expression, untouched. Receives `x` or `y` in ‘`#1`’.

```

2943 \cs_new_protected:Nn \__spintrent_spcoord_value:n
2944 {
2945   \str_if_eq:eeTF
2946     { \str_use:c { \__spintrent_spcoord_luaset_ #1 _type_str } } { number }
2947     {
2948       \exp_args:Ne \spnum { \str_use:c { \__spintrent_spcoord_luaset_ #1 _value_str } }
2949     }
2950     { \str_use:c { \__spintrent_spcoord_luaset_ #1 _value_str } }
2951 }

```

(End of definition for `\__spintrent_spcoord_value:n`.)

`\__spintrent_spcoord_print:n` With `\__spintrent_spcoord_luaset_ #1 _sign_str` equal to `+` or `-`, wraps the sign and `\__spintrent_spcoord_value:n` together with `_más:prefix($vp)/_menos:prefix($vn)`, referencing the value through `read` as `más/menos`. With no sign, delegates directly to `\__spintrent_spcoord_value:n`. Receives `x` or `y` in ‘`#1`’.

```

2952 \cs_new_protected:Nn \__spintrent_spcoord_print:n
2953 {
2954   \str_case:enF { \str_use:c { \__spintrent_spcoord_luaset_ #1 _sign_str } }
2955   {
2956     { + }
2957     {
2958       \MathMLintent { _más:prefix ($vp) }
2959       {
2960         \str_use:c { \__spintrent_spcoord_luaset_ #1 _sign_str }
2961         \MathMLarg{ vp } { \__spintrent_spcoord_value:n { #1 } }
2962       }
2963     }
2964     { - }
2965     {
2966       \MathMLintent { _menos :prefix($vn) }
2967       {
2968         \str_use:c { \__spintrent_spcoord_luaset_ #1 _sign_str }
2969         \MathMLarg{ vn } { \__spintrent_spcoord_value:n { #1 } }
2970       }
2971     }
2972   }
2973   { \__spintrent_spcoord_value:n { #1 } }
2974 }

```

(End of definition for `\__spintrent_spcoord_print:n`.)

`\__spintrent_spcoord_inner_sep:n` Receives `x` or `y` in ‘`#1`’. With `\__spintrent_spcoord_luaset_ #1 _composite_str` equal to `true`, places `\__spintrent_inner_sep:` first, so that the group `\MathMLarg` wraps holds two children and keeps its `arg`: `luamml` drops it from a sole node that carries a subscript, a superscript or a radical.

```

2975 \cs_new_protected:Nn \__spintrent_spcoord_inner_sep:n
2976 {
2977   \str_if_eq:eeT
2978     { \str_use:c { \__spintrent_spcoord_luaset_ #1 _composite_str } } { true }
2979     { \__spintrent_inner_sep: }
2980 }

```

(End of definition for `\__spintrent_spcoord_inner_sep:n`.)

`\__spintrent_spcoord_intent_infix:` Sets `\__spintrent_spcoord_intent_str` to `_coma:infix` or `_punto-y-coma:infix`, per `\__spintrent_spcoord_intent_infix:` then wraps both sides — each referenced through with it, `$x/$y` standing for `\__spintrent_spcoord_print:n`’s own output.

```

2981 \cs_new_protected:Nn \__spintrent_spcoord_intent_infix:
2982 {
2983   \str_if_eq:VnTF \__spintrent_spcoord_luaset_sep_str { ; }
2984     { \str_set:Nn \__spintrent_spcoord_intent_str { _punto-y-coma:infix } }
2985     { \str_set:Nn \__spintrent_spcoord_intent_str { _coma:infix } }
2986   \MathMLintent{ \__spintrent_spcoord_intent_str ($x, $y)}
2987   {
2988     \MathMLarg{x}{ \__spintrent_spcoord_inner_sep:n { x } \__spintrent_spcoord_print:n { x } }
2989     \MathMLintent { :silent } { \__spintrent_spcoord_luaset_sep_str }
2990     \MathMLarg{y}{ \__spintrent_spcoord_inner_sep:n { y } \__spintrent_spcoord_print:n { y } }
2991   }
2992 }

```

(End of definition for `\__spintrent_spcoord_intent_infix:.`)

`\__spintent_spcoord_mathml:` With `silent=true`, everything — parentheses and argument alike — is wrapped in a single `_-`, reading nothing at all: `\l__spintent_spcoord_raw_arg_tl` is typeset exactly as given, untouched by `\__spintent_spcoord_print:n/\__spintent_spcoord_intent_infix:`. Otherwise, the parentheses are silenced individually and `\__spintent_spcoord_intent_infix:` provides the reading in between.

```

2993 \cs_new_protected:Nn \__spintent_spcoord_mathml:
2994 {
2995   \bool_if:NTF \l__spintent_spcoord_silent_bool
2996   {
2997     \MathMLintent { _- }
2998     {
2999       \l__spintent_spcoord_wrap_parens_l_tl
3000       \l__spintent_spcoord_raw_arg_tl
3001       \l__spintent_spcoord_wrap_parens_r_tl
3002     }
3003   }
3004   {
3005     \MathMLintent { :silent } { \l__spintent_spcoord_wrap_parens_l_tl }
3006     \__spintent_spcoord_intent_infix:
3007     \MathMLintent { :silent } { \l__spintent_spcoord_wrap_parens_r_tl }
3008   }
3009 }

```

(End of definition for `\__spintent_spcoord_mathml:`)

`\__spintent_spcoord_exec:nn` Sets the *(keys)* in ‘#1’ and `\l__spintent_spcoord_raw_arg_tl` to ‘#2’. Calls `\__spintent_luafun_spcoord_parse_and_set:n` either way — even with `silent=true`, since `\__spintent_spcoord_set_wrap_parens_size:` still needs `\l__spintent_spcoord_luaaset_ #1 _has_frac_str` — but with `silent=true`, forces `\l__spintent_spcoord_luaaset_error_str` back to `false` rather than reporting it: the raw *{(argument)}* is typeset regardless of whether it parsed. Otherwise, reports `spcoord-invalid-arg` on error. Either way, sizes the parentheses, then delegates to `\__spintent_spcoord_mathml:` unless there was an error.

```

3010 \cs_new_protected:Nn \__spintent_spcoord_exec:nn
3011 {
3012   \keys_set:nn { spintent / spcoord } { #1 }
3013   \tl_set:Nn \l__spintent_spcoord_raw_arg_tl { #2 }
3014   \bool_if:NTF \l__spintent_spcoord_silent_bool
3015   {
3016     \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n { #2 } }
3017     \str_set:Nn \l__spintent_spcoord_luaaset_error_str { false }
3018   }
3019   {
3020     \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n { #2 } }
3021     \str_if_eq:VnT \l__spintent_spcoord_luaaset_error_str { true }
3022     { \msg_error:nnn { spintent } { spcoord-invalid-arg } { \exp_not:n { #2 } } }
3023   }
3024   \__spintent_spcoord_set_wrap_parens_size:
3025   \str_if_eq:VnF \l__spintent_spcoord_luaaset_error_str { true }
3026   { \__spintent_spcoord_mathml: }
3027 }

```

(End of definition for `\__spintent_spcoord_exec:nn`)

`\spcoord` The public command enforces *math mode*, opens a local group, and delegates to `\__spintent_spcoord_exec:nn`.

```

3028 \NewDocumentCommand \spcoord { 0{} m }
3029 {
3030   \mode_if_math:TF
3031   {
3032     \group_begin:
3033     \attribute \l__spintent_scope_attr_int = 1
3034     \__spintent_spcoord_exec:nn { #1 } { #2 }
3035     \group_end:
3036   }
3037   { \msg_error:nnn { spintent } { need-math-mode } { \spcoord } }
3038 }

```

(End of definition for `\spcoord`. This function is documented on page 19.)

11.35 The function `\__spintent_mathmlarg_point:nnn`

Scratch variables for `\__spintent_mathmlarg_point:nnn`: the reference name (#1), and the calling command's own `read-cmd/silent` state, looked up by name (#2).

```

3039 \tl_new:N \__spintent_mathmlarg_point_ref_tl
3040 \str_new:N \__spintent_mathmlarg_point_mode_str
3041 \tl_new:N \__spintent_mathmlarg_point_kv_tl
3042 \bool_new:N \__spintent_mathmlarg_point_silent_bool

```

(End of definition for `\__spintent_mathmlarg_point_ref_tl` and others.)

Annotates one point (#3) with `arg` and, when applicable, a literal `intent`, without wrapping it in its own `<mrow>`. Concept `intent` is passed empty on purpose: the caller applies it externally, via a native `\MathMLintent`.

With `\__spintent_geo_luaset_tipo_str` equal to `sub`, for `read-cmd=school` the letter alone is tagged through `\__spintent_luafun_mrow_intent_arg:nn`, and the subscript is appended separately, silenced. A subscript of more than one digit is not annotated natively — the queue would hand its digits to the next point's claim — but typeset plain, with one extra silent claim per digit. For `read-cmd=mathcat`, letter and subscript are typeset together, tagged as a whole through a native `\MathMLarg`.

```

3043 \cs_new_protected:Nn \__spintent_mathmlarg_point:nnn
3044 {
3045   \tl_set:Nn \__spintent_mathmlarg_point_ref_tl {#1}
3046   \__spintent_luafun_geo_point_parse_and_set:n { \exp_not:n {#3} }
3047   \bool_set_eq:Nc \__spintent_mathmlarg_point_silent_bool { \__spintent_ #2 _silent_bool }
3048   \str_set_eq:Nc \__spintent_mathmlarg_point_mode_str { \__spintent_ #2 _mode_str }
3049   \__spintent_invisible_sep:
3050   \str_case:VnF \__spintent_geo_luaset_tipo_str
3051   {
3052     { sub }
3053     {
3054       \str_if_eq:VnTF \__spintent_mathmlarg_point_mode_str { school }
3055       {
3056         \tl_set:Nc \__spintent_mathmlarg_point_kv_tl
3057         { arg = "\__spintent_mathmlarg_point_ref_tl", intent = "_ \__spintent_geo_luas
3058         \__spintent_luafun_mrow_intent_arg:nn
3059         { } { \__spintent_mathmlarg_point_kv_tl }
3060         \int_compare:nNnTF { \str_count:N \__spintent_geo_luaset_sub_str } > { 1 }
3061         {
3062           \int_step_inline:nnnn { 1 } { 1 } { \str_count:N \__spintent_geo_luas
3063           { \__spintent_luafun_mrow_intent_arg:nn { } { intent = ":silent" } }
3064           {
3065             \__spintent_geo_luaset_letra_str
3066             \c_math_subscript_token { \__spintent_geo_luaset_sub_str }
3067           }
3068         }
3069       }
3070       {
3071         \__spintent_geo_luaset_letra_str
3072         \c_math_subscript_token
3073         { \MathMLintent { :silent } { \__spintent_geo_luaset_sub_str } }
3074       }
3075     }
3076   }
3077   {
3078     {
3079       \__spintent_inner_sep:
3080       \MathMLarg { \__spintent_mathmlarg_point_ref_tl }
3081       {
3082         {
3083           \__spintent_geo_luaset_letra_str
3084           \c_math_subscript_token { \__spintent_geo_luaset_sub_str }
3085         }
3086       }
3087     }
3088   }
3089 }

```

With `\__spintent_geo_luaset_tipo_str` equal to `prima`, the same distinction applies: `read-cmd=school` tags the letter with a literal `intent` and appends the mark unsilenced (a prime is not spoken on the base, so nothing to repeat); `read-cmd=mathcat` tags letter and mark together through a native `\MathMLarg`.

```

3090 { prima }

```

```

3091     {
3092       \str_if_eq:VnTF \l__spinttent_mathmlarg_point_mode_str { school }
3093       {
3094         \tl_set:Ne \l__spinttent_mathmlarg_point_kv_tl
3095           { arg = "\l__spinttent_mathmlarg_point_ref_tl", intent = "_ \l__spinttent_geo_lua
3096           \l__spinttent_luafun_mrow_intent_arg:nn
3097           { } { \l__spinttent_mathmlarg_point_kv_tl }
3098           { \l__spinttent_geo_luaaset_letra_str \l__spinttent_geo_luaaset_sub_str }
3099       }
3100       {
3101         {
3102           \l__spinttent_inner_sep:
3103           \MathMLarg { \l__spinttent_mathmlarg_point_ref_tl }
3104           {
3105             { \l__spinttent_geo_luaaset_letra_str \l__spinttent_geo_luaaset_sub_str }
3106           }
3107         }
3108       }
3109     }
3110 }

```

Otherwise (no subscript, no prime), the same distinction applies to the combined `\l__spinttent_geo_luaaset_print_tl` as a whole — the shared queue for `school` (or when `silent` is `true`), a native `\MathMLarg` for `mathcat`, never mixed within the same formula, since the queue does not recognize a native `\MathMLarg` claim as already taken.

```

3111     {
3112       \bool_if:NTF \l__spinttent_mathmlarg_point_silent_bool
3113       {
3114         \tl_set:Ne \l__spinttent_mathmlarg_point_kv_tl
3115           { arg = "\l__spinttent_mathmlarg_point_ref_tl", intent = "--" }
3116         \l__spinttent_luafun_mrow_intent_arg:nn
3117           { } % debe ir vacío
3118           { \l__spinttent_mathmlarg_point_kv_tl }
3119         { \l__spinttent_geo_luaaset_print_tl }
3120       }
3121       {
3122         \str_if_eq:VnTF \l__spinttent_mathmlarg_point_mode_str { school }
3123         {
3124           \tl_set:Ne \l__spinttent_mathmlarg_point_kv_tl
3125             { arg = "\l__spinttent_mathmlarg_point_ref_tl", intent = "_ \l__spinttent_geo_lua
3126             \l__spinttent_luafun_mrow_intent_arg:nn
3127             { } % debe ir vacío
3128             { \l__spinttent_mathmlarg_point_kv_tl }
3129             { \l__spinttent_geo_luaaset_print_tl }
3130         }
3131         {
3132           \MathMLarg { \l__spinttent_mathmlarg_point_ref_tl }
3133             { \l__spinttent_geo_luaaset_print_tl }
3134         }
3135       }
3136     }
3137 }
3138 \cs_generate_variant:Nn \l__spinttent_mathmlarg_point:nnn { nnV }

```

(End of definition for `\l__spinttent_mathmlarg_point:nnn`.)

11.36 The command `\spPoint`

The user-level command `\spPoint` provides a semantic interface for a single geometric point. It delegates argument validation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

11.36.1 Definition of variables for `\spPoint`

Internal variables holding the values set by `read-cmd` and `prefix`.

```

\l__spinttent_spPoint_mode_str
\l__spinttent_spPoint_prefix_tl
\l__spinttent_spPoint_intent_arg_tl
3139 \str_new:N \l__spinttent_spPoint_mode_str
3140 \tl_new:N \l__spinttent_spPoint_prefix_tl
3141 \tl_new:N \l__spinttent_spPoint_intent_arg_tl

```

(End of definition for `\l__spinttent_spPoint_mode_str`, `\l__spinttent_spPoint_prefix_tl`, and `\l__spinttent_spPoint_intent_arg_tl`.)

11.36.2 Definition of error messages for \spPoint

spPoint-invalid-point

The “error” spPoint-invalid-point is raised when the argument passed in ‘#1’ is not a single uppercase letter with an optional subscript or prime modifier.

```

3142 \msg_new:nnnn { spintent } { spPoint-invalid-point }
3143 {
3144   Invalid~point~format~in~\token_to_str:N \spPoint:~'#1'.
3145 }
3146 {
3147   The~argument~must~be~a~single~uppercase~letter~with~an~optional~
3148   subscript~(\_{...})~or~prime~(').\\
3149 }

```

(End of definition for spPoint-invalid-point.)

11.36.3 Definition of keys for \spPoint

read-cmd

We define the *(keys)* read-cmd, prefix, silent and concept.

prefix

silent

unknown

```

3150 \keys_define:nn { spintent / spPoint }
3151 {
3152   read-cmd                .choices:nn = { school , mathcat }
3153   {
3154     \int_case:nn { \l_keys_choice_int }
3155     {
3156       {1} { \str_set:Nn \l__spintent_spPoint_mode_str { school } }
3157       {2} { \str_set:Nn \l__spintent_spPoint_mode_str { mathcat } }
3158     }
3159   },
3160   read-cmd                .initial:n = school,
3161   read-cmd                .value_required:n = true,
3162   read-cmd / unknown      .code:n =
3163     \msg_error:nneee { spintent } { unknown-choice }
3164     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
3165   prefix                  .code:n =
3166     { \__spintent_sanitize_affix:Nn \l__spintent_spPoint_prefix_tl {#1} },
3167   prefix                  .initial:n = ,
3168   prefix                  .value_required:n = true,
3169   silent                  .bool_set:N = \l__spintent_spPoint_silent_bool,
3170   silent                  .initial:n = false,
3171   silent                  .value_required:n = true,
3172   concept                 .bool_set:N = \l__spintent_spPoint_concept_bool,
3173   concept                 .initial:n = true,
3174   concept                 .value_required:n = true,
3175   unknown                 .code:n =
3176     {
3177       \str_set:Nn \l__spintent_command_name_str { spPoint }
3178       \__spintent_unknown_keys_cmd:n {#1}
3179     },
3180 }

```

(End of definition for read-cmd and others.)

11.36.4 Internal functions for \spPoint

__spintent_spPoint_print:n

The internal function __spintent_spPoint_print:n processes the *{(argument)}* passed in ‘#1’. Calls __spintent_luafun_geo_point_parse_and_set:n, reporting spPoint-invalid-point on error. If silent is true or concept is false, sets \l__spintent_spPoint_intent_arg_tl to _(\$pt) — forced further to _- when silent is true, whatever read-cmd is: a composite point (one with a subscript) is not reliably silenced by the per-point :silent alone. Otherwise sets it to _punto:prefix(\$pt), with prefix placed after the leading _ when given. Finally calls __spintent_spPoint_mathml:n.

```

3181 \cs_new_protected:Nn \__spintent_spPoint_print:n
3182 {
3183   \__spintent_luafun_geo_point_parse_and_set:n { \exp_not:n {#1} }
3184   \str_if_eq:VnT \l__spintent_geo_luasets_error_str { true }
3185   {
3186     \msg_error:nne { spintent } { spPoint-invalid-point } { \exp_not:n {#1} }
3187   }
3188   \bool_if:nTF
3189   {
3190     \bool_lazy_or_p:nn
3191     { \l__spintent_spPoint_silent_bool }
3192     { ! \l__spintent_spPoint_concept_bool }
3193   }

```

```

3194     {
3195         \tl_set:Nn \l__spintent_spPoint_intent_arg_tl { _($pt) }
3196         \bool_if:NT \l__spintent_spPoint_silent_bool
3197         {
3198             \tl_set:Nn \l__spintent_spPoint_intent_arg_tl { _- }
3199         }
3200     }
3201     {
3202         \tl_set:Ne \l__spintent_spPoint_intent_arg_tl
3203         {
3204             -
3205             \tl_if_empty:NF \l__spintent_spPoint_prefix_tl
3206             {
3207                 \tl_use:N \l__spintent_spPoint_prefix_tl -
3208             }
3209             punto:prefix($pt)
3210         }
3211     }
3212     \__spintent_spPoint_mathml:n { #1 }
3213 }

```

(End of definition for `\__spintent_spPoint_print:n`.)

`\__spintent_spPoint_mathml:n`

The function `\__spintent_spPoint_mathml:n` calls `\__spintent_invisible_sep:` first, then wraps `\__spintent_mathmlarg_point:nnn` — which parses ‘`#1`’, tags it `arg="pt"`, and silences a real subscript or prime on its own — inside a native `\MathMLintent` carrying `\l__spintent_spPoint_intent_arg_tl` as the concept `intent`, with `\__spintent_inner_sep:` first inside that group.

```

3214 \cs_new_protected:Nn \__spintent_spPoint_mathml:n
3215 {
3216     \__spintent_invisible_sep:
3217     \MathMLintent { \l__spintent_spPoint_intent_arg_tl }
3218     {
3219         \__spintent_inner_sep:
3220         \__spintent_mathmlarg_point:nnn { pt } { spPoint } { #1 }
3221     }
3222 }

```

(End of definition for `\__spintent_spPoint_mathml:n`.)

`\__spintent_spPoint_exec:nnn`

The function `\__spintent_spPoint_exec:nnn` receives the optional *⟨keys⟩* in ‘`#1`’ and the point *⟨argument⟩* in ‘`#2`’, delegating to `\__spintent_spPoint_print:n`. With ‘`#3`’ given (the optional coordinate), silences the gap between the point and the coordinate first (otherwise read as an implicit product), then temporarily aliases `\l__spintent_spcoord_silent_bool` to `\l__spintent_spPoint_silent_bool` and calls `\__spintent_luafun_spcoord_parse_and_set:n` directly, reporting `spcoord-invalid-arg` on error unless silenced, then `\__spintent_spcoord_set_wrap_parens_size:` and `\__spintent_spcoord_mathml:`.

```

3223 \cs_new_protected:Nn \__spintent_spPoint_exec:nnn
3224 {
3225     \keys_set:nn { spintent / spPoint } { #1 }
3226     \__spintent_spPoint_print:n { #2 }
3227     \tl_if_novalue:nF {#3}
3228     {
3229         \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % silencia el "por"
3230         \bool_set_eq:NN \l__spintent_spcoord_silent_bool \l__spintent_spPoint_silent_bool
3231         \tl_set:Nn \l__spintent_spcoord_raw_arg_tl {#3}
3232         \bool_if:NTF \l__spintent_spcoord_silent_bool
3233         {
3234             \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
3235             \str_set:Nn \l__spintent_spcoord_luaset_error_str { false }
3236         }
3237         {
3238             \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
3239             \str_if_eq:VnT \l__spintent_spcoord_luaset_error_str { true }
3240             { \msg_error:nnn { spintent } { spcoord-invalid-arg } { \exp_not:n {#3} } }
3241         }
3242         \__spintent_spcoord_set_wrap_parens_size:
3243         \str_if_eq:VnF \l__spintent_spcoord_luaset_error_str { true }
3244         { \__spintent_spcoord_mathml: }
3245     }
3246 }

```

(End of definition for `\__spintent_spPoint_exec:nnn`.)

`\spPoint` The public command `\spPoint` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spPoint_exec:nnn`.

```

3247 \NewDocumentCommand \spPoint { 0{} m d() }
3248 {
3249   \mode_if_math:TF
3250   {
3251     \group_begin:
3252     \attribute \__spintent_scope_attr_int = 1
3253     \__spintent_spPoint_exec:nnn { #1 } { #2 } { #3 }
3254     \group_end:
3255   }
3256   { \msg_error:nnn { spintent } { need-math-mode } { \spPoint } }
3257 }

```

(End of definition for `\spPoint`. This function is documented on page 19.)

11.37 The command `\spvec`

The user command `\spvec` supports two types of notation for the input $\langle argument \rangle$: a pair of points (with `\overrightarrow` applied), or a one-letter name (with `\vec` applied instead). Both notations are automatically detected by the Lua module. An optional coordinate, in parentheses, may follow the $\langle argument \rangle$.

11.37.1 Definition of variables for `\spvec`

The variable `\__spintent_spvec_mode_str` stores the value set by the key `read-cmd` and the variable `\__spintent_spvec_prefix_tl` stores the value set by the key `prefix`.

```

\__spintent_spvec_mode_str
\__spintent_spvec_prefix_tl
\__spintent_spvec_intent_arg_tl
\__spintent_spvec_kv_tl
\__spintent_spvec_pt_seq
\__spintent_spvec_refs_tl
\__spintent_spvec_pt_idx_int
3258 \str_new:N \__spintent_spvec_mode_str
3259 \tl_new:N \__spintent_spvec_prefix_tl
3260 \tl_new:N \__spintent_spvec_intent_arg_tl
3261 \tl_new:N \__spintent_spvec_kv_tl
3262 \seq_new:N \__spintent_spvec_pt_seq
3263 \tl_new:N \__spintent_spvec_refs_tl
3264 \int_new:N \__spintent_spvec_pt_idx_int

```

(End of definition for `\__spintent_spvec_mode_str` and others.)

11.37.2 Definition of error messages for `\spvec`

`spvec-invalid-arg` The “error” `spvec-invalid-arg` is raised when the argument fails to parse, when a one-letter name carries a prime, or when the point list is not exactly two points.

```

3265 \msg_new:nnn { spintent } { spvec-invalid-arg }
3266 {
3267   Invalid~argument~'!#1'~for~\token_to_str:N \spvec.
3268 }

```

(End of definition for `spvec-invalid-arg`.)

11.37.3 Definition of keys for `\spvec`

`silent` We define the $\langle keys \rangle$ `silent`, `concept`, `autofit`, `read-cmd` and `prefix`.

```

concept
autofit
read-cmd
prefix
unknown
3269 \keys_define:nn { spintent / spvec }
3270 {
3271   silent      .bool_set:N = \__spintent_spvec_silent_bool,
3272   silent      .initial:n = false,
3273   silent      .value_required:n = true,
3274   concept     .bool_set:N = \__spintent_spvec_concept_bool,
3275   concept     .initial:n = true,
3276   concept     .value_required:n = true,
3277   autofit     .bool_set:N = \__spintent_spvec_autofit_bool,
3278   autofit     .initial:n = false,
3279   autofit     .value_required:n = true,
3280   read-cmd    .choices:nn = { school , mathcat }
3281   {
3282     \int_case:nn { \l_keys_choice_int }
3283     {
3284       {1} { \str_set:Nn \__spintent_spvec_mode_str { school } }
3285       {2} { \str_set:Nn \__spintent_spvec_mode_str { mathcat } }
3286     }
3287   },
3288   read-cmd     .initial:n = school,
3289   read-cmd     .value_required:n = true,
3290   read-cmd / unknown .code:n =
3291     \msg_error:nnee { spintent } { unknown-choice }
3292     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3293   prefix       .code:n =

```

```

3294     { \__spintent_sanitize_affix:Nn \__spintent_svec_prefix_tl {#1} },
3295     prefix      .initial:n = { },
3296     prefix      .value_required:n = true,
3297     unknown     .code:n =
3298     {
3299         \str_set:Nn \__spintent_command_name_str { svec }
3300         \__spintent_unknown_keys_cmd:n {#1}
3301     },
3302 }

```

(End of definition for *silent* and others.)

11.37.4 Internal functions for \svec

`\__spintent_svec_print:n` The internal function `\__spintent_svec_print:n` processes the $\langle argument \rangle$ passed in ‘#1’. Calls `\__spintent_luafun_geo_parse_and_set:n`, reporting `svec-invalid-arg` on error, on a proper name with `\__spintent_geo_luaset_tipo_str` equal to `prima` (a one-letter vector cannot carry a prime), or on a point list that is not exactly two points. Otherwise calls `\__spintent_svec_mathml:n`.

```

3303 \cs_new_protected:Nn \__spintent_svec_print:n
3304 {
3305     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
3306     \str_if_eq:VnT \__spintent_geo_luaset_is_name_str { true }
3307     {
3308         \str_if_eq:VnT \__spintent_geo_luaset_tipo_str { prima }
3309         { \str_set:Nn \__spintent_geo_luaset_error_str { true } }
3310     }
3311     \str_if_eq:VnT \__spintent_geo_luaset_is_name_str { false }
3312     {
3313         \int_compare:nT { \clist_count:N \__spintent_geo_luaset_points_clist != 2 }
3314         { \str_set:Nn \__spintent_geo_luaset_error_str { true } }
3315     }
3316     \str_if_eq:VnTF \__spintent_geo_luaset_error_str { true }
3317     { \msg_error:nne { spintent } { svec-invalid-arg } { \exp_not:n {#1} } }
3318     { \__spintent_svec_mathml:n { #1 } }
3319 }

```

(End of definition for `\__spintent_svec_print:n`.)

`\__spintent_svec_mathml:n` The function `\__spintent_svec_mathml:n` branches on `\__spintent_geo_luaset_is_name_str`. For a proper name (a single letter, no prime), the concept `intent` is `_ $\$pt$` when `silent` is `true` or `concept` is `false` — forced further to `_-` when `silent` is `true`, whatever `read-cmd` is; otherwise `_vector:prefix( $\$pt$ )`, with `prefix` placed after the leading `_` when given. With `\__spintent_geo_luaset_tipo` equal to `sub`, `read-cmd=school` tags `\vec{letra}` through `\__spintent_luafun_mrow_intent_arg:nn` with a literal kv, the subscript appended and silenced; `read-cmd=mathcat` tags `\vec{...}` through a native `\MathMLarg` instead, the subscript silenced only under `silent`. `autofit` inserts a `\vphantom{b}` inside `\vec{...}` to normalize its height. Otherwise (no subscript), the same kv logic wraps `\vec{print\_tl}` as a whole.

```

3320 \cs_new_protected:Nn \__spintent_svec_mathml:n
3321 {
3322     \str_if_eq:VnTF \__spintent_geo_luaset_is_name_str { true }
3323     {
3324         \bool_if:nTF
3325         {
3326             \bool_lazy_or_p:nn
3327             { \__spintent_svec_silent_bool }
3328             { ! \__spintent_svec_concept_bool }
3329         }
3330         {
3331             \tl_set:Nn \__spintent_svec_intent_arg_tl { _( $\$pt$ ) }
3332             \bool_if:NT \__spintent_svec_silent_bool
3333             { \tl_set:Nn \__spintent_svec_intent_arg_tl { _- } }
3334         }
3335         {
3336             \tl_set:Ne \__spintent_svec_intent_arg_tl
3337             {
3338                 -
3339                 \tl_if_empty:NF \__spintent_svec_prefix_tl
3340                 { \tl_use:N \__spintent_svec_prefix_tl - }
3341                 vector:prefix( $\$pt$ )
3342             }
3343         }
3344         \str_case:VnF \__spintent_geo_luaset_tipo_str

```

```

3345 {
3346   { sub }
3347   {
3348     \str_if_eq:VnTF \l__spintent_sspec_mode_str { school }
3349     {
3350       \bool_if:NTF \l__spintent_sspec_silent_bool
3351       { \tl_set:Nn \l__spintent_sspec_kv_tl { arg = "pt", intent = "-_" } }
3352       {
3353         \tl_set:Ne \l__spintent_sspec_kv_tl
3354         { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3355       }
3356       \__spintent_invisible_sep:
3357       \MathMLintent { \l__spintent_sspec_intent_arg_tl }
3358       {
3359         \__spintent_inner_sep:
3360         \__spintent_luafun_mrow_intent_arg:nn
3361         { } { \l__spintent_sspec_kv_tl }
3362         {
3363           \vec
3364           {
3365             \bool_if:NT \l__spintent_sspec_autofit_bool
3366             { \vphantom{ b } }
3367             \l__spintent_geo_luaset_letra_str
3368           }
3369           \c_math_subscript_token
3370           { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
3371         }
3372       }
3373     }
3374   }
3375   \__spintent_invisible_sep:
3376   \MathMLintent { \l__spintent_sspec_intent_arg_tl }
3377   {
3378     \__spintent_inner_sep:
3379     \vec
3380     {
3381       \bool_if:NT \l__spintent_sspec_autofit_bool
3382       { \vphantom{ b } }
3383       \MathMLarg{ pt } { \l__spintent_geo_luaset_letra_str }
3384     }
3385     \bool_if:NT \l__spintent_sspec_silent_bool
3386     { \c_math_subscript_token { \l__spintent_geo_luaset_sub_str } }
3387   }
3388   \bool_if:NF \l__spintent_sspec_silent_bool
3389   { \c_math_subscript_token { \l__spintent_geo_luaset_sub_str } }
3390 }
3391 }
3392 }
3393 {
3394   \bool_if:NTF \l__spintent_sspec_silent_bool
3395   { \tl_set:Nn \l__spintent_sspec_kv_tl { arg = "pt", intent = "-_" } }
3396   {
3397     \tl_set:Ne \l__spintent_sspec_kv_tl
3398     { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3399   }
3400   \__spintent_invisible_sep:
3401   \MathMLintent { \l__spintent_sspec_intent_arg_tl }
3402   {
3403     \__spintent_inner_sep:
3404     \__spintent_luafun_mrow_intent_arg:nn
3405     { } { \l__spintent_sspec_kv_tl }
3406     {
3407       \vec
3408       {
3409         \bool_if:NT \l__spintent_sspec_autofit_bool
3410         { \vphantom{ b } }
3411         \l__spintent_geo_luaset_print_tl
3412       }
3413     }
3414   }
3415 }

```

```
3416     }
```

For a pair of points, `\__spintent_geo_luaset_points_clist` is split into `\__spintent_svec_pt_seq` and walked to build `\__spintent_svec_refs_tl`, the $\$pts1$, $\$pts2$ reference list. The concept `intent` is `_` when `silent` is `true`, `_`(refs) when `concept` is `false` or `_vector:prefix(refs)` otherwise, applied with a native `\MathMLintent` wrapping `\overrightarrow`. The sequence is walked again, calling `\__spintent_mathmlarg_point:nnn` once per point.

```
3417     {
3418       \seq_set_from_clist:NN \__spintent_svec_pt_seq \__spintent_geo_luaset_points_clist
3419       \tl_clear:N \__spintent_svec_refs_tl
3420       \int_zero:N \__spintent_svec_pt_idx_int
3421       \seq_map_inline:Nn \__spintent_svec_pt_seq
3422       {
3423         \int_incr:N \__spintent_svec_pt_idx_int
3424         \tl_if_empty:NF \__spintent_svec_refs_tl
3425         { \tl_put_right:Nn \__spintent_svec_refs_tl { , } }
3426         \tl_put_right:Ne \__spintent_svec_refs_tl { $ pts \int_use:N \__spintent_svec_pt_
3427       }
3428       \bool_if:nTF
3429       {
3430         \bool_lazy_or:nn
3431         { \__spintent_svec_silent_bool }
3432         { ! \__spintent_svec_concept_bool }
3433       }
3434       {
3435         \tl_set:Ne \__spintent_svec_intent_arg_tl { _ ( \__spintent_svec_refs_tl ) }
3436         \bool_if:NT \__spintent_svec_silent_bool
3437         { \tl_set:Nn \__spintent_svec_intent_arg_tl { _- } }
3438       }
3439       {
3440         \tl_set:Ne \__spintent_svec_intent_arg_tl
3441         {
3442           -
3443           \tl_if_empty:NF \__spintent_svec_prefix_tl
3444           { \tl_use:N \__spintent_svec_prefix_tl - }
3445           vector:prefix( \__spintent_svec_refs_tl )
3446         }
3447       }
3448       \__spintent_invisible_sep:
3449       \MathMLintent { \__spintent_svec_intent_arg_tl }
3450       {
3451         \__spintent_inner_sep:
3452         \overrightarrow
3453         {
3454           \int_zero:N \__spintent_svec_pt_idx_int
3455           \seq_map_inline:Nn \__spintent_svec_pt_seq
3456           {
3457             \int_incr:N \__spintent_svec_pt_idx_int
3458             \__spintent_mathmlarg_point:nnn
3459             { pts \int_use:N \__spintent_svec_pt_idx_int }
3460             { svec }
3461             { ##1 }
3462           }
3463         }
3464       }
3465     }
3466   }
```

(End of definition for `\__spintent_svec_mathml:n`)

```
\__spintent_svec_exec:nnn
```

The function `\__spintent_svec_exec:nnn` receives the optional *(keys)* in ‘#1’ and the *(argument)* in ‘#2’, delegating to `\__spintent_svec_print:n`. With ‘#3’ given (the optional coordinate), aliases `\__spintent_spcord_silent_bool` to `\__spintent_svec_silent_bool` — reverted when the calling group closes — and calls `\__spintent_luafun_spcord_parse_and_set:n` directly, reporting `spcord-invalid-arg` on error unless silenced, then `\__spintent_spcord_set_wrap_parens_size:` and `\__spintent_spcord_mathml:` — the same rendering functions `\spcord` itself uses.

```
3467 \cs_new_protected:Nn \__spintent_svec_exec:nnn
3468 {
3469   \keys_set:nn { spintent / svec } { #1 }
3470   \__spintent_svec_print:n { #2 }
3471   \tl_if_no_value:NF {#3}
```

```

3472 {
3473   \bool_if:NTF \l__spintent_sspec_silent_bool
3474   { \MathMLintent { :silent } { \equal } }
3475   { \equal } % notación de vector =( , )
3476   \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % previene lectura de intervalo
3477   \bool_set_eq:NN \l__spintent_spcoord_silent_bool \l__spintent_sspec_silent_bool
3478   \tl_set:Nn \l__spintent_spcoord_raw_arg_tl {#3}
3479   \bool_if:NTF \l__spintent_sspec_silent_bool
3480   {
3481     \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
3482     \str_set:Nn \l__spintent_spcoord_luaerror_str { false }
3483   }
3484   {
3485     \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
3486     \str_if_eq:VnT \l__spintent_spcoord_luaerror_str { true }
3487     { \msg_error:nnn { spintent } { spcoord-invalid-arg } { \exp_not:n {#3} } }
3488   }
3489   \__spintent_spcoord_set_wrap_parens_size:
3490   \str_if_eq:VnF \l__spintent_spcoord_luaerror_str { true }
3491   { \__spintent_spcoord_mathml: }
3492 }
3493 }

```

(End of definition for `\__spintent_sspec_exec:nnn`.)

\spec The public command `\spec` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sspec_exec:nnn`.

```

3494 \NewDocumentCommand \spec { O{} m d() }
3495 {
3496   \mode_if_math:TF
3497   {
3498     \group_begin:
3499     \attribute \l__spintent_scope_attr_int = 1
3500     \__spintent_sspec_exec:nnn { #1 } { #2 } { #3 }
3501     \group_end:
3502   }
3503   { \msg_error:nnn { spintent } { need-math-mode } { \spec } }
3504 }

```

(End of definition for `\spec`. This function is documented on page 20.)

11.38 The command `\sprecta`

The user command `\sprecta` supports two types of notation for the input $\langle argument \rangle$: when the argument contains two points (separated by an optional comma), `\overleftarrow{\rightarrow}` is applied; when the argument is a single point (without comma) or a lowercase letter, it is formatted without any visual decorator. Both notations are automatically detected by the Lua module.

11.38.1 Definition of variables for `\sprecta`

The variable `\l__spintent_sprecta_mode_str` store the values set by the key `read-cmd` and the variable `\l__spintent_sprecta_prefix_tl` store the value set by the key `prefix`.

```

3505 \str_new:N \l__spintent_sprecta_mode_str
3506 \tl_new:N \l__spintent_sprecta_prefix_tl
3507 \seq_new:N \l__spintent_sprecta_pt_seq
3508 \tl_new:N \l__spintent_sprecta_refs_tl
3509 \tl_new:N \l__spintent_sprecta_intent_arg_tl
3510 \tl_new:N \l__spintent_sprecta_kv_tl
3511 \int_new:N \l__spintent_sprecta_pt_idx_int

```

(End of definition for `\l__spintent_sprecta_mode_str` and others.)

11.38.2 Definition of error messages for `\sprecta`

`sprecta-invalid-points`

The “error” `sprecta-invalid-points` is raised when the argument matches neither the two-point notation nor the name notation.

```

3512 \msg_new:nnnn { spintent } { sprecta-invalid-points }
3513 {
3514   Invalid~format~in~\token_to_str:N \sprecta:~'~#1'~\msg_line_context:.
3515 }
3516 {
3517   See~the~documentation~of~\token_to_str:N \sprecta~for~the~accepted~argument~forms.
3518 }

```

(End of definition for `sprecta-invalid-points`.)

11.38.3 Definition of keys for \sprecta

read-cmd We define the *(keys)* read-cmd, prefix and silent.

```

prefix 3519 \keys_define:nn { spintent / sprecta }
silent 3520 {
unknown 3521   read-cmd           .choices:nn = { school , mathcat }
          3522   {
          3523     \int_case:nn { \l_keys_choice_int }
          3524     {
          3525       {1} { \str_set:Nn \l__spintent_sprecta_mode_str { school } }
          3526       {2} { \str_set:Nn \l__spintent_sprecta_mode_str { mathcat } }
          3527     }
          3528   },
          3529   read-cmd           .initial:n = school,
          3530   read-cmd           .value_required:n = true,
          3531   read-cmd / unknown .code:n =
          3532     \msg_error:nneee { spintent } { unknown-choice }
          3533     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
          3534   prefix           .code:n =
          3535     { \__spintent_sanitize_affix:Nn \l__spintent_sprecta_prefix_tl {#1} },
          3536   prefix           .initial:n = ,
          3537   prefix           .value_required:n = true,
          3538   silent            .bool_set:N = \l__spintent_sprecta_silent_bool,
          3539   silent            .initial:n = false,
          3540   silent            .value_required:n = true,
          3541   concept           .bool_set:N = \l__spintent_sprecta_concept_bool,
          3542   concept           .initial:n = true,
          3543   concept           .value_required:n = true,
          3544   unknown           .code:n =
          3545     {
          3546       \str_set:Nn \l__spintent_command_name_str { sprecta }
          3547       \__spintent_unknown_keys_cmd:n {#1}
          3548     },
          3549   }

```

(End of definition for read-cmd and others.)

11.38.4 Internal functions for \sprecta

__spintent_sprecta_print:n

The internal function `\__spintent_sprecta_print:n` processes the *{(argument)}* passed in ‘#1’. Calls `\__spintent_luafun_geo_parse_and_set:n`, reporting sprecta-invalid-points on error, then `\__spintent_`

```

3550 \cs_new_protected:Nn \__spintent_sprecta_print:n
3551 {
3552   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
3553   \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
3554   {
3555     \msg_error:nne { spintent } { sprecta-invalid-points } { \exp_not:n {#1} }
3556   }
3557   \__spintent_sprecta_mathml:n { #1 }
3558 }

```

(End of definition for __spintent_sprecta_print:n.)

__spintent_sprecta_mathml:n

The function `\__spintent_sprecta_mathml:n` branches on `\l__spintent_geo_luaset_is_name_str`. For a proper name (m, l, ...), the concept intent is `_( $pt)` when `silent` is `true` or `concept` is `false` — forced further to `_-` when `silent` is `true` and `read-cmd` is not `school`; otherwise `_recta:prefix( $pt)`, with `prefix` placed after the leading `_` when given. With `read-cmd=school`, the sub and prima cases tag the letter with a literal kv that is `_-` when `silent` is `true`.

With `\l__spintent_geo_luaset_tipo_str` equal to `sub`, `read-cmd=school` tags the base through `\__spintent_luafun_mrow_intent_arg:nn` with a literal kv, the subscript appended and silenced; `read-cmd=mathcat` tags base and subscript together through a native `\MathMLarg` instead.

```

3559 \cs_new_protected:Nn \__spintent_sprecta_mathml:n
3560 {
3561   \str_if_eq:VnTF \l__spintent_geo_luaset_is_name_str { true }
3562   {
3563     \__spintent_invisible_sep:
3564     \bool_if:nTF
3565     {
3566       \bool_lazy_or_p:nn
3567       { \l__spintent_sprecta_silent_bool }
3568       { ! \l__spintent_sprecta_concept_bool }
3569     }

```

```

3570     {
3571         \tl_set:Nn \l__spintent_sprecta_intent_arg_tl { _($pt) }
3572     \bool_if:NT \l__spintent_sprecta_silent_bool
3573     {
3574         \str_if_eq:VnF \l__spintent_sprecta_mode_str { school }
3575         { \tl_set:Nn \l__spintent_sprecta_intent_arg_tl { _- } }
3576     }
3577     }
3578     {
3579         \tl_set:Ne \l__spintent_sprecta_intent_arg_tl
3580         {
3581             -
3582             \tl_if_empty:NF \l__spintent_sprecta_prefix_tl
3583             { \tl_use:N \l__spintent_sprecta_prefix_tl - }
3584             recta:prefix($pt)
3585         }
3586     }
3587     \str_case:VnF \l__spintent_geo_luaset_tipo_str
3588     {
3589         { sub }
3590         {
3591             \str_if_eq:VnTF \l__spintent_sprecta_mode_str { school }
3592             {
3593                 \bool_if:NTF \l__spintent_sprecta_silent_bool
3594                 { \tl_set:Nn \l__spintent_sprecta_kv_tl { arg = "pt", intent = "_-" } }
3595                 {
3596                     \tl_set:Ne \l__spintent_sprecta_kv_tl
3597                     { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3598                 }
3599                 \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3600                 {
3601                     \__spintent_inner_sep:
3602                     \__spintent_luafun_mrow_intent_arg:nn
3603                     { } { \l__spintent_sprecta_kv_tl }
3604                     {
3605                         \l__spintent_geo_luaset_letra_str
3606                         \c_math_subscript_token
3607                         { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
3608                     }
3609                 }
3610             }
3611         }

```

For `read-cmd=mathcat`, the same sub case tags base and subscript together through a native `\MathMLarg` instead — never mixed with the shared queue, since the queue does not recognize a native `\MathMLarg` claim as already taken.

```

3611     {
3612         \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3613         {
3614             {
3615                 \__spintent_inner_sep:
3616                 \MathMLarg { pt }
3617                 {
3618                     {
3619                         \l__spintent_geo_luaset_letra_str
3620                         \c_math_subscript_token { \l__spintent_geo_luaset_sub_str }
3621                     }
3622                 }
3623             }
3624         }
3625     }
3626 }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `prima`, the same distinction applies: `read-cmd=school` tags the base with the literal kv and appends the mark, unsilenced (a prime is not spelled out on the base, so nothing to repeat); `read-cmd=mathcat` tags base and mark together through a native `\MathMLarg`.

```

3627     { prima }
3628     {
3629         \str_if_eq:VnTF \l__spintent_sprecta_mode_str { school }
3630         {
3631             \bool_if:NTF \l__spintent_sprecta_silent_bool
3632             { \tl_set:Nn \l__spintent_sprecta_kv_tl { arg = "pt", intent = "_-" } }
3633             {

```

```

3634         \tl_set:Nx \l__spintent_spectra_kv_tl
3635         { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3636     }
3637     \MathMLintent { \l__spintent_spectra_intent_arg_tl }
3638     {
3639         \__spintent_inner_sep:
3640         \__spintent_luafun_mrow_intent_arg:nn
3641         { } { \l__spintent_spectra_kv_tl }
3642         { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
3643     }
3644 }
3645 {
3646     \MathMLintent { \l__spintent_spectra_intent_arg_tl }
3647     {
3648         {
3649             \__spintent_inner_sep:
3650             \MathMLarg { pt }
3651             {
3652                 { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
3653             }
3654         }
3655     }
3656 }
3657 }
3658 }

```

Otherwise (no subscript, no prime), the same kv logic wraps the combined `\l__spintent_geo_luaset_print_tl` as a whole — the shared queue for `school`, its own `arg` alone for `mathcat`.

```

3659 {
3660     \bool_if:NTF \l__spintent_spectra_silent_bool
3661     {
3662         \tl_set:Nn \l__spintent_spectra_kv_tl { arg = "pt", intent = "--" }
3663     }
3664     {
3665         \str_if_eq:VnTF \l__spintent_spectra_mode_str { school }
3666         {
3667             \tl_set:Nx \l__spintent_spectra_kv_tl
3668             { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3669         }
3670         {
3671             \tl_set:Nn \l__spintent_spectra_kv_tl { arg = "pt" }
3672         }
3673     }
3674     \__spintent_luafun_mrow_intent_arg:nn
3675     { \l__spintent_spectra_intent_arg_tl }
3676     { \l__spintent_spectra_kv_tl }
3677     { \l__spintent_geo_luaset_print_tl }
3678 }
3679 }

```

For a list of points, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_spectra_pt_seq` and walked to build `\l__spintent_spectra_refs_tl`, the `$pt1, $pt2, ...` reference list. The concept intent is `_` when `silent` is true, `_` (refs) when `concept` is false or `_recta:prefix(refs)` otherwise, applied with a native `\MathMLintent` wrapping `\overlefttrightarrow`. The sequence is walked again, calling `\__spintent_mathmlarg_point:nnn` once per point.

```

3680 {
3681     \seq_set_from_clist:NN \l__spintent_spectra_pt_seq \l__spintent_geo_luaset_points_clist
3682     \tl_clear:N \l__spintent_spectra_refs_tl
3683     \int_zero:N \l__spintent_spectra_pt_idx_int
3684     \seq_map_inline:Nn \l__spintent_spectra_pt_seq
3685     {
3686         \int_incr:N \l__spintent_spectra_pt_idx_int
3687         \tl_if_empty:NF \l__spintent_spectra_refs_tl
3688         { \tl_put_right:Nn \l__spintent_spectra_refs_tl { , } }
3689         \tl_put_right:Nx \l__spintent_spectra_refs_tl { $ pt \int_use:N \l__spintent_spectra_
3690     }
3691     \bool_if:NTF
3692     {
3693         \bool_lazy_or_p:nn
3694         { \l__spintent_spectra_silent_bool }
3695         { ! \l__spintent_spectra_concept_bool }
3696     }

```

```

3697     {
3698       \tl_set:Nx \l__spintent_sprecta_intent_arg_tl { _( \l__spintent_sprecta_refs_tl ) }
3699       \bool_if:NT \l__spintent_sprecta_silent_bool
3700         { \tl_set:Nn \l__spintent_sprecta_intent_arg_tl { _- } }
3701     }
3702     {
3703       \tl_set:Nx \l__spintent_sprecta_intent_arg_tl
3704         {
3705           -
3706           \tl_if_empty:NF \l__spintent_sprecta_prefix_tl
3707             { \tl_use:N \l__spintent_sprecta_prefix_tl - }
3708           recta:prefix( \l__spintent_sprecta_refs_tl )
3709         }
3710     }
3711     \__spintent_invisible_sep:
3712     \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3713     {
3714       \__spintent_inner_sep:
3715       \overleftrightharrow
3716       {
3717         \int_zero:N \l__spintent_sprecta_pt_idx_int
3718         \seq_map_inline:Nn \l__spintent_sprecta_pt_seq
3719           {
3720             \int_incr:N \l__spintent_sprecta_pt_idx_int
3721             \__spintent_mathmlarg_point:nnn
3722               { pt \tl_use:N \l__spintent_sprecta_pt_idx_int }
3723               { sprecta }
3724               { ##1 }
3725           }
3726       }
3727     }
3728   }
3729 }

```

(End of definition for `\__spintent_sprecta_mathml:n`.)

`\__spintent_sprecta_exec:nn`

The function `\__spintent_sprecta_exec:nn` receives the optional *(keys)* passed in ‘#1’ and the *(argument)* passed in ‘#2’ delegating the rendering of the latter to `\__spintent_sprecta_print:n`.

```

3730 \cs_new_protected:Nn \__spintent_sprecta_exec:nn
3731 {
3732   \keys_set:nn { spintent / sprecta } { #1 }
3733   \__spintent_sprecta_print:n { #2 }
3734 }

```

(End of definition for `\__spintent_sprecta_exec:nn`.)

`\sprecta`

The public command `\sprecta` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sprecta_exec:nn`.

```

3735 \NewDocumentCommand \sprecta { 0{} m }
3736 {
3737   \mode_if_math:TF
3738   {
3739     \group_begin:
3740     \attribute \l__spintent_scope_attr_int = 1
3741     \__spintent_sprecta_exec:nn { #1 } { #2 }
3742     \group_end:
3743   }
3744   { \msg_error:nnn { spintent } { need-math-mode } { \sprecta } }
3745 }

```

(End of definition for `\sprecta`. This function is documented on page 20.)

11.39 The command `\sprayo`

The user command `\sprayo` provides a semantic interface for a ray. The key `read-sty` selects the verbalized term, *rayo* or *semirrecta*, both rendered identically with `\overrightarrow`.

11.39.1 Definition of variables for \sprayo

The variable `\l__spinttent_sprayo_mode_str` store the values set by the key `read-cmd`; the variable `\l__spinttent_sprayo_prefix_tl` store the value set by the key `prefix` and the variable `\l__spinttent_sprayo_read_sty_str` stores the value set by the key `read-sty`.

- The variables defined here are assigned using `.choices:nn` or `.code:n` within `\keys_define:nn`.

```
3746 \str_new:N \l__spinttent_sprayo_mode_str
3747 \tl_new:N \l__spinttent_sprayo_prefix_tl
3748 \str_new:N \l__spinttent_sprayo_read_sty_str
```

(End of definition for `\l__spinttent_sprayo_mode_str`, `\l__spinttent_sprayo_prefix_tl`, and `\l__spinttent_sprayo_read_sty_str`.)

11.39.2 Definition of error messages for \sprayo

`sprayo-invalid-points` Raised when the argument is not a comma-separated pair of uppercase letters.

```
3749 \msg_new:nnnn { spintent } { sprayo-invalid-points }
3750 {
3751   Invalid~point~format~in~\token_to_str:N \sprayo:~'#1'~\msg_line_context:.
3752 }
3753 {
3754   See~the~documentation~of~\token_to_str:N \sprayo~for~the~accepted~point~formats.
3755 }
```

(End of definition for `sprayo-invalid-points`.)

11.39.3 Definition of keys for \sprayo

We define the *(keys)* `read-cmd`, `prefix`, `silent`, `concept` and `read-sty`.

```
read-cmd 3756 \keys_define:nn { spintent / sprayo }
prefix   3757 {
silent   3758   read-cmd           .choices:nn = { school , mathcat }
read-sty 3759   {
unknown  3760     \int_case:nn { \l_keys_choice_int }
          3761     {
          3762       {1} { \str_set:Nn \l__spinttent_sprayo_mode_str { school } }
          3763       {2} { \str_set:Nn \l__spinttent_sprayo_mode_str { mathcat } }
          3764     }
          3765   },
          3766   read-cmd           .initial:n = school,
          3767   read-cmd           .value_required:n = true,
          3768   read-cmd / unknown .code:n =
          3769     \msg_error:nneee { spintent } { unknown-choice }
          3770     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
          3771   prefix           .code:n =
          3772     { \__spinttent_sanitiz_affix:Nn \l__spinttent_sprayo_prefix_tl {#1} },
          3773   prefix           .initial:n = ,
          3774   prefix           .value_required:n = true,
          3775   silent           .bool_set:N = \l__spinttent_sprayo_silent_bool,
          3776   silent           .initial:n = false,
          3777   silent           .value_required:n = true,
          3778   concept          .bool_set:N = \l__spinttent_sprayo_concept_bool,
          3779   concept          .initial:n = true,
          3780   concept          .value_required:n = true,
          3781   read-sty          .choices:nn = { rayo , semirrecta }
          3782   {
          3783     \int_case:nn { \l_keys_choice_int }
          3784     {
          3785       {1} { \str_set:Nn \l__spinttent_sprayo_read_sty_str { rayo } }
          3786       {2} { \str_set:Nn \l__spinttent_sprayo_read_sty_str { semirrecta } }
          3787     }
          3788   },
          3789   read-sty          .initial:n = rayo,
          3790   read-sty          .value_required:n = true,
          3791   read-sty / unknown .code:n =
          3792     \msg_error:nneee { spintent } { unknown-choice }
          3793     { read-sty } { rayo,~semirrecta } { \exp_not:n {#1} },
          3794   unknown           .code:n =
          3795   {
          3796     \str_set:Nn \l__spinttent_command_name_str { sprayo }
          3797     \__spinttent_unknown_keys_cmd:n {#1}
          3798   },
          3799 }
```

(End of definition for `read-cmd` and others.)

11.39.4 Internal functions for \spray

Auxiliary variables used by `\__spintent_sprayo_mathml:` to split `\l__spintent_geo_luaset_points_clist` and build the `$pts1, $pts2, ...` reference list.

```

3800 \seq_new:N \l__spintent_sprayo_pt_seq
3801 \tl_new:N \l__spintent_sprayo_refs_tl
3802 \tl_new:N \l__spintent_sprayo_intent_arg_tl
3803 \int_new:N \l__spintent_sprayo_pt_idx_int

```

(End of definition for `\l__spintent_sprayo_pt_seq` and others.)

The internal function `\__spintent_sprayo_print:n` process the `{\langle argument \rangle}` passed in ‘#1’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process the `{\langle argument \rangle}`, returning an “error” if the input is invalid, then calls `\__spintent_sprayo_mathml:`. Unlike `\sprecta`, `\spray` has no proper-name argument: it always takes a list of points, so no `intent` is built here.

```

3804 \cs_new_protected:Nn \__spintent_sprayo_print:n
3805 {
3806   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
3807   \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
3808   {
3809     \msg_error:nne { spintent } { sprayo-invalid-points } { \exp_not:n {#1} }
3810   }
3811   \__spintent_sprayo_mathml:
3812 }

```

(End of definition for `\__spintent_sprayo_print:n`.)

The function `\__spintent_sprayo_mathml:` splits `\l__spintent_geo_luaset_points_clist` (set by Lua, one raw point per element) into `\l__spintent_sprayo_pt_seq` and walks it once to build `\l__spintent_sprayo_refs_tl`, the comma-separated `$pts1, $pts2, ...` reference list, naming each point by position.

The concept `intent` is `_` when `silent` is `true`, `_(refs)` when `concept` is `false`; otherwise it is `_\l__spintent_sprayo_read_sty_str:prefix(refs)` — the concept word itself (`rayo` or `semirrecta`) comes from the `read-sty` key, not a literal, since `\spray` verbalizes under two names — prefixed with `prefix` when given. It is applied with a native `\MathMLintent` wrapping `\overrightarrow`, preceded by `\__spintent_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`). The sequence is walked a second time, calling `\__spintent_mathmlarg_point:nnn` once per point with its `ptsn` reference name and `silent/read-cmd` looked up under `sprayo`: it alone decides `arg/literal intent` per point, so `concept` plays no role at that level.

```

3813 \cs_new_protected:Nn \__spintent_sprayo_mathml:
3814 {
3815   \seq_set_from_clist:NN \l__spintent_sprayo_pt_seq \l__spintent_geo_luaset_points_clist
3816   \tl_clear:N \l__spintent_sprayo_refs_tl
3817   \int_zero:N \l__spintent_sprayo_pt_idx_int
3818   \seq_map_inline:Nn \l__spintent_sprayo_pt_seq
3819   {
3820     \int_incr:N \l__spintent_sprayo_pt_idx_int
3821     \tl_if_empty:NF \l__spintent_sprayo_refs_tl
3822     { \tl_put_right:Nn \l__spintent_sprayo_refs_tl { , } }
3823     \tl_put_right:Ne \l__spintent_sprayo_refs_tl { $ pts \int_use:N \l__spintent_sprayo_pt_idx_int }
3824   }
3825   \bool_if:nTF
3826   {
3827     \bool_lazy_or:p:nn
3828     { \l__spintent_sprayo_silent_bool }
3829     { ! \l__spintent_sprayo_concept_bool }
3830   }
3831   {
3832     \tl_set:Ne \l__spintent_sprayo_intent_arg_tl { _ ( \l__spintent_sprayo_refs_tl ) }
3833     \bool_if:NT \l__spintent_sprayo_silent_bool
3834     { \tl_set:Nn \l__spintent_sprayo_intent_arg_tl { _- } }
3835   }
3836   {
3837     \tl_set:Ne \l__spintent_sprayo_intent_arg_tl
3838     {
3839       -
3840       \tl_if_empty:NF \l__spintent_sprayo_prefix_tl
3841       { \tl_use:N \l__spintent_sprayo_prefix_tl - }
3842       \l__spintent_sprayo_read_sty_str :prefix( \l__spintent_sprayo_refs_tl )
3843     }
3844   }

```

```

3845 \__spintent_invisible_sep:
3846 \MathMLint { \__spintent_sprayo_intent_arg_tl }
3847 {
3848   \__spintent_inner_sep:
3849   \overrightarrow
3850   {
3851     \int_zero:N \__spintent_sprayo_pt_idx_int
3852     \seq_map_inline:Nn \__spintent_sprayo_pt_seq
3853     {
3854       \int_incr:N \__spintent_sprayo_pt_idx_int
3855       \__spintent_mathmlarg_point:nnn
3856       { pts \int_use:N \__spintent_sprayo_pt_idx_int }
3857       { sprayo }
3858       { ##1 }
3859     }
3860   }
3861 }
3862 }

```

(End of definition for `\__spintent_sprayo_mathml:`.)

`\__spintent_sprayo_exec:nn` The function `\__spintent_sprayo_exec:nn` receives the optional $\langle keys \rangle$ passed in ‘#1’ and the $\{\langle argument \rangle\}$ passed in ‘#2’ delegating the rendering of the latter to `\__spintent_sprayo_print:n`.

```

3863 \cs_new_protected:Nn \__spintent_sprayo_exec:nn
3864 {
3865   \keys_set:nn { spintent / sprayo } { #1 }
3866   \__spintent_sprayo_print:n { #2 }
3867 }

```

(End of definition for `\__spintent_sprayo_exec:nn`.)

`\sprayo` The public command `\sprayo` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sprayo_exec:nn`.

```

3868 \NewDocumentCommand \sprayo { 0{} m }
3869 {
3870   \mode_if_math:TF
3871   {
3872     \group_begin:
3873     \attribute \__spintent_scope_attr_int = 1
3874     \__spintent_sprayo_exec:nn { #1 } { #2 }
3875     \group_end:
3876   }
3877   { \msg_error:nnn { spintent } { need-math-mode } { \sprayo } }
3878 }

```

(End of definition for `\sprayo`. This function is documented on page 21.)

11.40 The commands `\spside` and `\spmside`

The user commands `\spside` and `\spmside` provide a semantic interface for a side, segment or trace determined by two points. The key `read-sty` selects the verbalized term — *lado*, *segmento*, or *trazo*, the latter used in straightedge-and-compass constructions — and the key `read-arg` overrides that term entirely with a free-form string. Additionally, the command `\spmside` provides the key `print-m`, controlling whether `\operatorname{m}` is typeset before the argument.

11.40.1 Definition of variables for `\spside` and `\spmside`

`\__spintent_spside_mode_str` Each pair of variables is declared once for both `\spside` and `\spmside` via `\clist_map_inline:nn`. The variables `\__spintent_spside_mode_str` and `\__spintent_spside_mode_str` store the values set by the key `read-cmd`; the variables `\__spintent_spside_prefix_tl` and `\__spintent_spmside_prefix_tl` store the values set by the key `prefix`.

`\__spintent_spside_read_sty_str` The variables `\__spintent_spside_read_sty_str` and `\__spintent_spmside_read_sty_str` stores the value set by the key `read-sty`.

`\__spintent_spside_read_arg_tl` The variables `\__spintent_spside_read_arg_tl` and `\__spintent_spmside_read_arg_tl` stores the value set by the key `read-arg`.

• The variables are defined here as they are assigned using `.choices:nn` or `.code:n` rather than a direct property within `\keys_define:nn`.

```

3879 \clist_map_inline:nn { side , mside }
3880 {
3881   \str_new:c { \__spintent_sp #1 _mode_str }
3882   \tl_new:c { \__spintent_sp #1 _prefix_tl }
3883   \str_new:c { \__spintent_sp #1 _read_sty_str }

```

```

3884     \tl_new:c { l__spintent_sp #1 _read_arg_tl }
3885 }

```

(End of definition for `l__spintent_spside_mode_str` and others.)

11.40.2 Definition of error messages for `\spside` and `\spmside`

spside-invalid-points
spmside-invalid-points

We define an error message for the commands `\spside` and `\spmside`. The “error” is triggered if the user includes a invalid “point” in the $\langle\textit{argument}\rangle$ passed in ‘#1’ to the command.

```

3886 \clist_map_inline:nn { side , mside }
3887 {
3888     \msg_new:nnnn { spintent } { sp #1 -invalid-points }
3889     {
3890         Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
3891     }
3892     {
3893         See~the~documentation~of~##1~for~the~accepted~point~formats.
3894     }
3895 }

```

(End of definition for `spside-invalid-points` and `spmside-invalid-points`.)

11.40.3 Definition of keys for `\spside` and `\spmside`

read-cmd
prefix
silent
concept
read-sty
read-arg
print-m
unknown

We define the shared $\langle\textit{keys}\rangle$ `read-cmd`, `prefix`, `silent`, `concept`, `read-sty`, `read-arg` and `print-m`.

```

3896 \clist_map_inline:nn { side , mside }
3897 {
3898     \keys_define:nn { spintent / sp #1 }
3899     {
3900         read-cmd .choices:nn = { school , mathcat }
3901         {
3902             \int_case:nn { \l_keys_choice_int }
3903             {
3904                 {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
3905                 {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
3906             }
3907         },
3908         read-cmd .initial:n = school,
3909         read-cmd .value_required:n = true,
3910         read-cmd / unknown .code:n =
3911             \msg_error:nnee { spintent } { unknown-choice }
3912             { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3913         prefix .code:n =
3914             { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
3915         prefix .initial:n = ,
3916         prefix .value_required:n = true,
3917         silent .bool_set:c = { l__spintent_sp #1 _silent_bool },
3918         silent .initial:n = false,
3919         silent .value_required:n = true,
3920         concept .bool_set:c = { l__spintent_sp #1 _concept_bool },
3921         concept .initial:n = true,
3922         concept .value_required:n = true,
3923         read-sty .choices:nn = { lado , segmento , trazo }
3924         {
3925             \int_case:nn { \l_keys_choice_int }
3926             {
3927                 {1} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { lado } }
3928                 {2} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { segmento } }
3929                 {3} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { trazo } }
3930             }
3931         },
3932         read-sty .initial:n = lado,
3933         read-sty .value_required:n = true,
3934         read-sty / unknown .code:n =
3935             \msg_error:nnee { spintent } { unknown-choice }
3936             { read-sty } { lado,~segmento,~trazo } { \exp_not:n {##1} },
3937         read-arg .code:n =
3938         {
3939             \__spintent_sanitize_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1}
3940         },
3941         read-arg .initial:n = ,
3942         read-arg .value_required:n = true,
3943         print-m .bool_set:c = { l__spintent_sp #1 _print_m_bool },

```

```

3944     print-m                .initial:n = false,
3945     print-m                .value_required:n = true,
3946     unknown                .code:n =
3947     {
3948         \str_set:Nn \l__spintent_command_name_str { sp #1 }
3949         \__spintent_unknown_keys_cmd:n {##1}
3950     },
3951 }
3952 }

```

• The keys `print-m` and `concept` are defined for both commands for simplicity, but has no effect on `\spside`.

(End of definition for `read-cmd` and others.)

The variable `\l__spintent_spside_intent_tl` holds the concept word (`read-arg` or `read-sty`). The rest are scratch variables shared by both commands (the function itself is shared) to split `\l__spintent_geo_luaset_point` and build the `$pts1, $pts2, ...` reference list.

```

\l__spintent_spside_intent_tl
\l__spintent_spside_pt_seq
\l__spintent_spside_refs_tl
\l__spintent_spside_intent_arg_tl
\l__spintent_spside_pt_idx_int
3953 \tl_new:N \l__spintent_spside_intent_tl
3954 \seq_new:N \l__spintent_spside_pt_seq
3955 \tl_new:N \l__spintent_spside_refs_tl
3956 \tl_new:N \l__spintent_spside_intent_arg_tl
3957 \int_new:N \l__spintent_spside_pt_idx_int

```

(End of definition for `\l__spintent_spside_intent_tl` and others.)

The internal function `\__spintent_spside_process:nn` receives the short name of the command (`side` or `mside`) in ‘#1’ and the `{\argument}` in ‘#2’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process ‘#2’, returning an “error” if invalid, then builds `\l__spintent_spside_intent_tl`. When `read-arg` is empty, it is set from `read-sty`, prefixed with ‘medida-del-’ for `\spmside`. When `read-arg` is given, it is used as is, and ‘medida-del-’ is prepended only if `concept` is `true`: with `concept=false`, `read-arg` is the verbatim escape hatch for a full user-written phrase (e.g. `read-arg={medida de la hipotenusa}`), gender and article left to the user. No `intent` is built here: it needs the reference list, which `\__spintent_spside_math` builds after splitting the points.

```

3958 \cs_new_protected:Nn \__spintent_spside_process:nn
3959 {
3960     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
3961     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
3962     {
3963         \msg_error:nnee { spintent } { sp #1 -invalid-points }
3964         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3965     }
3966     \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
3967     {
3968         \tl_set:Ne \l__spintent_spside_intent_tl { \str_use:c { l__spintent_sp #1 _read_sty_str } }
3969         \str_if_eq:eeT {#1} { mside }
3970         {
3971             \tl_put_left:Nn \l__spintent_spside_intent_tl { medida-del- }
3972         }
3973     }
3974     {
3975         \tl_set:Ne \l__spintent_spside_intent_tl { \tl_use:c { l__spintent_sp #1 _read_arg_tl } }
3976         \bool_if:cT { l__spintent_sp #1 _concept_bool }
3977         {
3978             \str_if_eq:eeT {#1} { mside }
3979             {
3980                 \tl_put_left:Nn \l__spintent_spside_intent_tl { medida-del- }
3981             }
3982         }
3983     }
3984     \__spintent_spside_mathml:n { #1 }
3985 }

```

(End of definition for `\__spintent_spside_process:nn`)

The function `\__spintent_spside_mathml:n` receives the short command name in ‘#1’. It splits `\l__spintent_geo_luaset_points_clist` into `\l__spintent_spside_pt_seq` and walks it once to build `\l__spintent_spside_refs_tl`, the comma-separated `$pts1, $pts2, ...` reference list, naming each point by position.

```

3986 \cs_new_protected:Nn \__spintent_spside_mathml:n
3987 {
3988     \seq_set_from_clist:NN \l__spintent_spside_pt_seq \l__spintent_geo_luaset_points_clist

```

```

3989 \tl_clear:N \l__spintent_spside_refs_tl
3990 \int_zero:N \l__spintent_spside_pt_idx_int
3991 \seq_map_inline:Nn \l__spintent_spside_pt_seq
3992 {
3993   \int_incr:N \l__spintent_spside_pt_idx_int
3994   \tl_if_empty:NF \l__spintent_spside_refs_tl
3995   { \tl_put_right:Nn \l__spintent_spside_refs_tl { , } }
3996   \tl_put_right:Ne \l__spintent_spside_refs_tl { $ pts \int_use:N \l__spintent_spside_pt_idx_int }
3997 }

```

The concept `intent` is `_` when `silent` is `true`; otherwise, if `read-arg` is empty, it is `_(refs)` when `concept` is `false` or `\l__spintent_spside_intent_tl:prefix(refs)` when `true`; if `read-arg` is given, it is always `\l__spintent_spside_intent_tl:prefix(refs)` — `concept` never silences a user-given `read-arg` (see `\__spintent_spside_process:nn`). The `:prefix(...)` form is prefixed with `prefix` when given.

```

3998 \bool_if:cTF { \l__spintent_sp #1 _silent_bool }
3999 {
4000   \tl_set:Nn \l__spintent_spside_intent_arg_tl { _- }
4001 }
4002 {
4003   \tl_if_empty:cTF { \l__spintent_sp #1 _read_arg_tl }
4004   {
4005     \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
4006     {
4007       \tl_set:Ne \l__spintent_spside_intent_arg_tl
4008       {
4009         -
4010         \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4011         { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4012         \l__spintent_spside_intent_tl :prefix( \l__spintent_spside_refs_tl )
4013       }
4014     }
4015     {
4016       \tl_set:Ne \l__spintent_spside_intent_arg_tl { _ ( \l__spintent_spside_refs_tl ) }
4017     }
4018   }
4019   {
4020     \tl_set:Ne \l__spintent_spside_intent_arg_tl
4021     {
4022       -
4023       \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4024       { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4025       \l__spintent_spside_intent_tl :prefix( \l__spintent_spside_refs_tl )
4026     }
4027   }
4028 }

```

Applied with a native `\MathMLintent`, preceded by `\__spintent_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`). Inside, for `#1=side` the points are always wrapped in `\overline`; for `#1=mside`, when `print-m` is `true` a silent `\operatorname{m}` precedes the `\overline`-wrapped points; when `false`, the points are typeset bare, with no `\overline` at all — the recursive lookup in the shared `mathml_filter` callback finds each point either way, whether nested inside `\overline` or sitting directly in the `\MathMLintent` group. In every case the sequence is walked a second time, calling `\__spintent_mathmlarg_point:nnn` once per point with its `pts`/`n` reference name and `silent/read-cmd` looked up under `sp#1`: it alone decides `arg/literal intent` per point, so `concept` plays no role at that level.

```

4029 \__spintent_invisible_sep: % (luamml issues #26)
4030 \MathMLintent { \l__spintent_spside_intent_arg_tl }
4031 {
4032   \__spintent_inner_sep:
4033   \str_if_eq:eeTF {#1} { side }
4034   {
4035     \overline { \__spintent_spside_points_loop:n {#1} }
4036   }
4037   {
4038     \bool_if:cTF { \l__spintent_sp #1 _print_m_bool }
4039     {
4040       \__spintent_luafun_mrow_intent_arg:nn
4041       { } { intent = ":silent" }
4042       { \operatorname { m } }
4043       \overline { \__spintent_spside_points_loop:n {#1} }
4044     }
4045   }

```

```

4045         {
4046         \__spintent_spside_points_loop:n {#1}
4047         }
4048     }
4049 }
4050 }

```

(End of definition for `\__spintent_spside_mathml:n`.)

`\__spintent_spside_points_loop:n` Walks `\__spintent_spside_pt_seq` a second time, calling `\__spintent_mathmlarg_point:nnn` once per point. Factored out of `\__spintent_spside_mathml:n` since it is needed identically in three places there (`side`, `mside` with and without `print-m`); it takes the short command name in ‘#1’ again since a separate macro does not inherit its caller’s own #1.

```

4051 \cs_new_protected:Nn \__spintent_spside_points_loop:n
4052 {
4053     \int_zero:N \__spintent_spside_pt_idx_int
4054     \seq_map_inline:Nn \__spintent_spside_pt_seq
4055     {
4056         \int_incr:N \__spintent_spside_pt_idx_int
4057         \__spintent_mathmlarg_point:nnn
4058         { pts \int_use:N \__spintent_spside_pt_idx_int }
4059         { sp #1 }
4060         { ##1 }
4061     }
4062 }

```

(End of definition for `\__spintent_spside_points_loop:n`.)

`\__spintent_spside_exec:nnn` The function `\__spintent_spside_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *keys* passed in ‘#2’ and the *{argument}* passed in ‘#3’, delegating the rendering to `\__spintent_spside_process:nn`.

```

4063 \cs_new_protected:Nn \__spintent_spside_exec:nnn
4064 {
4065     \keys_set:nn { spintent / sp #1 } { #2 }
4066     \__spintent_spside_process:nn { #1 } { #3 }
4067 }

```

(End of definition for `\__spintent_spside_exec:nnn`.)

`\spside` Both public commands `\spside` and `\spmside` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spside_exec:nnn`.

```

4068 \NewDocumentCommand \spside { O{} m }
4069 {
4070     \mode_if_math:TF
4071     {
4072         \group_begin:
4073         \attribute \__spintent_scope_attr_int = 1
4074         \__spintent_spside_exec:nnn { side } { #1 } { #2 }
4075         \group_end:
4076     }
4077     { \msg_error:nnn { spintent } { need-math-mode } { \spside } }
4078 }
4079 \NewDocumentCommand \spmside { O{} m }
4080 {
4081     \mode_if_math:TF
4082     {
4083         \group_begin:
4084         \attribute \__spintent_scope_attr_int = 1
4085         \__spintent_spside_exec:nnn { mside } { #1 } { #2 }
4086         \group_end:
4087     }
4088     { \msg_error:nnn { spintent } { need-math-mode } { \spmside } }
4089 }

```

(End of definition for `\spside` and `\spmside`. These functions are documented on page 21.)

11.41 The commands `\spangle` and `\spmangle`

The user commands `\spangle` and `\spmangle`, provide a semantic interface for angles, following the same generic pattern as `\spside`/`\spmside`. The key `print-m` chooses between the `\angle`/`\rightanglesqr` pair, typeset together with an explicit `\operatorname{m}`, and the `\measuredangle`/`\measuredrightangle` pair, which already conveys the notion of measure on its own; the key `recto` independently chooses the right-angle variant of whichever pair is active.

11.41.1 Definition of variables for \spangle and \spmangle

```
\__spintent_spangle_mode_str
\__spintent_spmangle_mode_str
\__spintent_spangle_prefix_tl
\__spintent_spmangle_prefix_tl
```

Each pair of variables is declared once for both `\spangle` and `\spmangle` via `\clist_map_inline:nn`. The variables `\__spintent_spangle_mode_str` and `\__spintent_spmangle_mode_str` store the values set by the key `read-cmd`; the variables `\__spintent_spangle_prefix_tl` and `\__spintent_spmangle_prefix_tl` store the values set by the key `prefix`.

• The variables defined here are assigned using `.choices:nn` or `.code:n` within `\keys_define:nn`.

```
4090 \clist_map_inline:nn { angle , mangle }
4091 {
4092   \str_new:c { \__spintent_sp #1 _mode_str }
4093   \tl_new:c { \__spintent_sp #1 _prefix_tl }
4094 }
```

(End of definition for `\__spintent_spangle_mode_str` and others.)

11.41.2 Definition of error messages for \spangle and \spmangle

```
spangle-invalid-arg
spmangle-invalid-arg
```

We define an “error” message for the commands `\spangle` and `\spmangle`. The “error” is triggered if the user includes a invalid `{⟨argument⟩}` passed in ‘#1’ to the command.

```
4095 \clist_map_inline:nn { angle , mangle }
4096 {
4097   \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
4098   {
4099     Invalid~format~in~##1:~'##2'~\msg_line_context:.
4100   }
4101   {
4102     See~the~documentation~of~##1~for~the~accepted~argument~forms.
4103   }
4104 }
```

(End of definition for `spangle-invalid-arg` and `spmangle-invalid-arg`.)

11.41.3 Definition of keys for \spangle and \spmangle

We define the shared `⟨keys⟩` `read-cmd`, `prefix`, `recto`, `print-m`, `silent` and `concept`,

```
read-cmd
prefix
recto
print-m
silent
concept
unknown
4105 \clist_map_inline:nn { angle , mangle }
4106 {
4107   \keys_define:nn { spintent / sp #1 }
4108   {
4109     read-cmd .choices:nn = { school , mathcat }
4110     {
4111       \int_case:nn { \_keys_choice_int }
4112       {
4113         {1} { \str_set:cn { \__spintent_sp #1 _mode_str } { school } }
4114         {2} { \str_set:cn { \__spintent_sp #1 _mode_str } { mathcat } }
4115       }
4116     },
4117     read-cmd .initial:n = school,
4118     read-cmd .value_required:n = true,
4119     read-cmd / unknown .code:n =
4120     \msg_error:nneee { spintent } { unknown-choice }
4121     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
4122     prefix .code:n =
4123     { \__spintent_sanitize_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
4124     prefix .initial:n = ,
4125     prefix .value_required:n = true,
4126     recto .bool_set:c = { \__spintent_sp #1 _recto_bool },
4127     recto .initial:n = false,
4128     recto .value_required:n = true,
4129     print-m .bool_set:c = { \__spintent_sp #1 _print_m_bool },
4130     print-m .initial:n = false,
4131     print-m .value_required:n = true,
4132     silent .bool_set:c = { \__spintent_sp #1 _silent_bool },
4133     silent .initial:n = false,
4134     silent .value_required:n = true,
4135     concept .bool_set:c = { \__spintent_sp #1 _concept_bool },
4136     concept .initial:n = true,
4137     concept .value_required:n = true,
4138     unknown .code:n =
4139     {
4140       \str_set:Nn \__spintent_command_name_str { sp #1 }
4141       \__spintent_unknown_keys_cmd:n {##1}
4142     },
```

```

4143     }
4144 }

```

• The keys `print-m` and `concept` are defined for both commands for simplicity, but has no effect on `\spangle`.

(End of definition for `read-cmd` and others.)

11.41.4 Internal functions for `\spangle` and `\spmangle`

```
\__spintent_spangle_sym:n
```

The internal function `\__spintent_spangle_sym:n` dispatches on `#1`, `print-m` and `recto` to select one of four symbols: `\angle/\rightanglesqr` when `print-m` is true; `\measuredangle/\measuredrightangle` otherwise. When `#1` is `angle`, only `recto` applies, choosing between `\angle` and `\rightangle`.

```

4145 \cs_new_protected:Nn \__spintent_spangle_sym:n
4146 {
4147   \str_if_eq:eeTF {#1} { mangle }
4148   {
4149     \bool_if:cTF { \__spintent_sp #1 _print_m_bool }
4150     {
4151       \bool_if:cTF { \__spintent_sp #1 _recto_bool }
4152       { \__spintent_right_angle_sqr: } { \angle }
4153     }
4154     {
4155       \bool_if:cTF { \__spintent_sp #1 _recto_bool }
4156       { \measuredrightangle } { \measuredangle }
4157     }
4158   }
4159   {
4160     \bool_if:cTF { \__spintent_sp #1 _recto_bool }
4161     { \__spintent_right_angle_sqr: } { \angle }
4162   }
4163 }

```

(End of definition for `\__spintent_spangle_sym:n`.)

```

\l__spintent_spangle_intent_tl
\l__spintent_spangle_pt_seq
\l__spintent_spangle_refs_tl
\l__spintent_spangle_intent_arg_tl
\l__spintent_spangle_pt_idx_int
\l__spintent_spangle_print_m_now_bool

```

The variable `\l__spintent_spangle_intent_tl` holds the concept word (ángulo or medida-del-ángulo, ‘-recto’-suffixed when `recto` is true). The rest are scratch variables shared by both commands to split `\l__spintent_geo_luaset_points_clist` and build the reference list — used only in the tres-puntos and un-punto cases (see `\__spintent_spangle_mathml:n`).

```

4164 \tl_new:N \l__spintent_spangle_intent_tl
4165 \seq_new:N \l__spintent_spangle_pt_seq
4166 \tl_new:N \l__spintent_spangle_refs_tl
4167 \tl_new:N \l__spintent_spangle_intent_arg_tl
4168 \int_new:N \l__spintent_spangle_pt_idx_int
4169 \str_new:N \l__spintent_spangle_kv_tl
4170 \bool_new:N \l__spintent_spangle_print_m_now_bool

```

(End of definition for `\l__spintent_spangle_intent_tl` and others.)

```
\__spintent_spangle_process:nn
```

The internal function `\__spintent_spangle_process:nn` receives the short name of the command (`angle` or `mangle`) in `#1` and the `{⟨argument⟩}` in `#2`. Calls `\__spintent_luafun_geo_angulo_parse_and_set:n`, reporting `sp#1-invalid-arg` on error. For a proper name that is a single letter (not numeric), also reports the same error when that letter is lowercase — `angle` names take an uppercase letter, number, or Greek letter, never a lowercase Latin one; the length check keeps a Greek letter command such as `\alpha` out of this comparison. Then sets `\l__spintent_spangle_intent_tl` to `ángulo` (or `medida-del-ángulo` for `\spmangle`), suffixed with `-recto` when `recto` is true, and calls `\__spintent_spangle_mathml:n`.

```

4171 \cs_new_protected:Nn \__spintent_spangle_process:nn
4172 {
4173   \__spintent_luafun_geo_angulo_parse_and_set:n { \exp_not:n {#2} }
4174   \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
4175   {
4176     \msg_error:nnee { spintent } { sp #1 -invalid-arg }
4177     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4178   }
4179   \str_if_eq:VnT \l__spintent_geo_luaset_angulo_case_str { nombre }
4180   {
4181     \str_if_eq:VnF \l__spintent_geo_luaset_nombre_es_num_str { true }
4182     {
4183       \int_compare:nT { \str_count:N \l__spintent_geo_luaset_letra_str = 1 }
4184       {
4185         \str_if_eq:eeF { \l__spintent_geo_luaset_letra_str } { \str_uppercase:n { \l__spintent_geo_luaset_letra_str } }
4186         {
4187           \msg_error:nnee { spintent } { sp #1 -invalid-arg }

```

```

4188         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4189     }
4190 }
4191 }
4192 }
4193 \str_if_eq:eeTF {#1} { mangle }
4194 {
4195     \tl_set:Nn \l__spintent_spangle_intent_tl { medida-del-ángulo }
4196 }
4197 {
4198     \tl_set:Nn \l__spintent_spangle_intent_tl { ángulo }
4199 }
4200 \bool_if:cT { l__spintent_sp #1 _recto_bool }
4201 {
4202     \tl_put_right:Nn \l__spintent_spangle_intent_tl { -recto }
4203 }
4204 \__spintent_spangle_mathml:n {#1}
4205 }

```

(End of definition for `\__spintent_spangle_process:nn`.)

`\__spintent_spangle_mathml:n`

The function `\__spintent_spangle_mathml:n` branches on `\l__spintent_geo_luaset_angulo_case_str`. For `nombre`, the symbol (`\__spintent_spangle_sym:n`) is tagged through `\__spintent_luafun_mrow_intent_arg` on its own, with `\l__spintent_spangle_intent_tl` alone (bare, no $\$$ -reference: `nombre` never refers to more than one thing) — `_` — when `silent` is `true` or `concept` is `false`. With `silent` `true`, the name that follows is silenced as a whole.

```

4206 \cs_new_protected:Nn \__spintent_spangle_mathml:n
4207 {
4208     \__spintent_invisible_sep: % (luamml issues #26)
4209     \str_if_eq:VnTF \l__spintent_geo_luaset_angulo_case_str { nombre }
4210     {
4211         \bool_if:cTF { l__spintent_sp #1 _silent_bool }
4212         {
4213             \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
4214         }
4215         {
4216             \bool_if:cTF { l__spintent_sp #1 _concept_bool }
4217             {
4218                 \tl_set:Ne \l__spintent_spangle_intent_arg_tl
4219                 {
4220                     -
4221                     \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
4222                     { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
4223                     \l__spintent_spangle_intent_tl
4224                 }
4225             }
4226             {
4227                 \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
4228             }
4229         }
4230         \tl_set:Ne \l__spintent_spangle_kv_tl
4231         { intent = "\l__spintent_spangle_intent_arg_tl" }
4232         \__spintent_luafun_mrow_intent_arg:nn
4233         { } { \l__spintent_spangle_kv_tl }
4234         { \__spintent_spangle_sym:n {#1} }
4235         \bool_if:cTF { l__spintent_sp #1 _silent_bool }
4236         {
4237             \__spintent_luafun_mrow_intent_arg:nn { } { intent = "_-" }
4238             { \l__spintent_geo_luaset_print_tl }
4239         }

```

Otherwise, a numeric name is tagged as a whole, through `\__spintent_luafun_mrow_intent_arg:nn`, with a literal `intent` built from `\l__spintent_geo_luaset_intent_str`.

```

4240     {
4241         \str_if_eq:VnTF \l__spintent_geo_luaset_nombre_es_num_str { true }
4242         {
4243             \tl_set:Ne \l__spintent_spangle_kv_tl
4244             { intent = "_ \l__spintent_geo_luaset_intent_str -" }
4245             \__spintent_luafun_mrow_intent_arg:nn
4246             { } { \l__spintent_spangle_kv_tl }
4247             { \l__spintent_geo_luaset_print_tl }
4248         }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `sub`, `read-cmd=school` tags the letter through the same function, with a literal `intent`, and the subscript is appended and silenced; `read-cmd=mathcat` leaves both letter and subscript untouched, for MathCAT to read the whole `<msub>` natively (no `arg` is set in either case: there is no `$`-reference to resolve).

```

4249         {
4250         \str_case:VnF \l__spintent_geo_luaset_tipo_str
4251         {
4252         { sub }
4253         {
4254         \str_if_eq:eeTF { \use:c { \l__spintent_sp #1 _mode_str } } { school }
4255         {
4256         \tl_set:Ne \l__spintent_spangle_kv_tl
4257         { intent = "_ \l__spintent_geo_luaset_intent_str -" }
4258         \__spintent_luafun_mrow_intent_arg:nn
4259         { } { \l__spintent_spangle_kv_tl }
4260         {
4261         \l__spintent_geo_luaset_letra_str
4262         \c_math_subscript_token
4263         { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
4264         }
4265         }
4266         {
4267         \l__spintent_geo_luaset_letra_str
4268         \c_math_subscript_token { \l__spintent_geo_luaset_sub_str }
4269         }
4270         }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `prima`, the prime is always read — spoken as part of the literal for `read-cmd=school`, native for `read-cmd=mathcat` — since a prime is not a repeated word to silence.

```

4271         { prima }
4272         {
4273         \str_if_eq:eeTF { \use:c { \l__spintent_sp #1 _mode_str } } { school }
4274         {
4275         \tl_set:Ne \l__spintent_spangle_kv_tl
4276         { intent = "_ \l__spintent_geo_luaset_intent_str -" }
4277         \__spintent_luafun_mrow_intent_arg:nn
4278         { } { \l__spintent_spangle_kv_tl }
4279         { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
4280         }
4281         {
4282         \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str
4283         }
4284         }
4285         }
4286         { \l__spintent_geo_luaset_print_tl }
4287         }
4288     }
4289 }

```

For `tres-puntos/un-punto`, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_spangle_pt_seq` and walked to build `\l__spintent_spangle_refs_tl`, the `$pt1`, `$pt2`, ... reference list. A single native `\MathMLintent` wraps the symbol and the points together, with `\l__spintent_spangle_intent_tl:prefix(ref` — `_` — when `silent` is `true`, `_(refs)` when `concept` is `false` alone. For `\spmangle` with `print-m true`, a silent `\operatorname{m}` precedes `\__spintent_spangle_sym:n` inside the same `\MathMLintent`. The sequence is walked again, calling `\__spintent_mathmlarg_point:nnn` once per point.

```

4290     {
4291     \seq_set_from_clist:NN \l__spintent_spangle_pt_seq \l__spintent_geo_luaset_points_clist
4292     \tl_clear:N \l__spintent_spangle_refs_tl
4293     \int_zero:N \l__spintent_spangle_pt_idx_int
4294     \seq_map_inline:Nn \l__spintent_spangle_pt_seq
4295     {
4296     \int_incr:N \l__spintent_spangle_pt_idx_int
4297     \tl_if_empty:NF \l__spintent_spangle_refs_tl
4298     { \tl_put_right:Nn \l__spintent_spangle_refs_tl { , } }
4299     \tl_put_right:Ne \l__spintent_spangle_refs_tl { $ pt \int_use:N \l__spintent_spangle_pt_idx_int }
4300     }
4301     \bool_if:cTF { \l__spintent_sp #1 _silent_bool }
4302     {
4303     \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
4304     }

```

```

4305     {
4306       \bool_if:cTF { \__spintent_sp #1 _concept_bool }
4307       {
4308         \tl_set:Ne \__spintent_spangle_intent_arg_tl
4309         {
4310           -
4311           \tl_if_empty:cF { \__spintent_sp #1 _prefix_tl }
4312           { \tl_use:c { \__spintent_sp #1 _prefix_tl } - }
4313           \__spintent_spangle_intent_tl :prefix( \__spintent_spangle_refs_tl )
4314         }
4315       }
4316       {
4317         \tl_set:Ne \__spintent_spangle_intent_arg_tl { _ ( \__spintent_spangle_refs_tl )
4318       }
4319     }
4320     \bool_set_false:N \__spintent_spangle_print_m_now_bool
4321     \str_if_eq:eeT {#1} { mangle }
4322     {
4323       \bool_if:cT { \__spintent_sp #1 _print_m_bool }
4324       { \bool_set_true:N \__spintent_spangle_print_m_now_bool }
4325     }
4326     \MathMLIntent { \__spintent_spangle_intent_arg_tl }
4327     {
4328       \__spintent_inner_sep:
4329       \bool_if:NT \__spintent_spangle_print_m_now_bool
4330       {
4331         \__spintent_luafun_mrow_intent_arg:nn
4332         { } { intent = ":silent" }
4333         { \operatorname { m } } }
4334       }
4335       \__spintent_luafun_mrow_intent_arg:nn
4336       { } { intent = ":silent" }
4337       { \__spintent_spangle_sym:n {#1} }
4338       \int_zero:N \__spintent_spangle_pt_idx_int
4339       \seq_map_inline:Nn \__spintent_spangle_pt_seq
4340       {
4341         \int_incr:N \__spintent_spangle_pt_idx_int
4342         \__spintent_mathmlarg_point:nnn
4343         { pt \int_use:N \__spintent_spangle_pt_idx_int
4344         { sp#1 }
4345         { ##1 }
4346       }
4347     }
4348   }
4349 }

```

(End of definition for __spintent_spangle_mathml:n.)

__spintent_spangle_exec:nnn

The function `\__spintent_spangle_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *⟨keys⟩* passed in ‘#2’ and the *⟨argument⟩* passed in ‘#3’, delegating the rendering to `\__spintent_spangle_process:nn`.

```

4350 \cs_new_protected:Nn \__spintent_spangle_exec:nnn
4351 {
4352   \keys_set:nn { spintent / sp #1 } { #2 }
4353   \__spintent_spangle_process:nn { #1 } { #3 }
4354 }

```

(End of definition for __spintent_spangle_exec:nnn.)

\spangle
\spmangle

Both public commands `\spangle` and `\spmangle` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spangle_exec:nnn`.

```

4355 \NewDocumentCommand \spangle { 0{} m }
4356 {
4357   \mode_if_math:TF
4358   {
4359     \group_begin:
4360     \attribute \__spintent_scope_attr_int = 1
4361     \__spintent_spangle_exec:nnn { angle } { #1 } { #2 }
4362     \group_end:
4363   }
4364   { \msg_error:nnn { spintent } { need-math-mode } { \spangle } }

```

```

4365 }
4366 \NewDocumentCommand \spmangle { 0{} m }
4367 {
4368   \mode_if_math:TF
4369   {
4370     \group_begin:
4371     \attribute \l__spintent_scope_attr_int = 1
4372     \__spintent_spangle_exec:nnn { mangle } { #1 } { #2 }
4373     \group_end:
4374   }
4375   { \msg_error:nnn { spintent } { need-math-mode } { \spmangle } }
4376 }

```

(End of definition for `\spangle` and `\spmangle`. These functions are documented on page 23.)

11.42 The commands `\sparc` and `\spmarc`

The user commands `\sparc` and `\spmarc` provide a semantic interface for arcs. Unlike `\spmside` and `\spmangle`, `\spmarc` has no `print-m` key: `\operatorname{m}` is always present, since $m\widehat{AB}$ is the only standard notation for the measure of an arc.

11.42.1 Definition of variables for `\sparc` and `\spmarc`

Each pair of variables is declared once for both `\sparc` and `\spmarc` via `\clist_map_inline:nn`. The variables `\l__spintent_sparc_mode_str` and `\l__spintent_spmarc_mode_str` store the values set by the key `read-cmd`; the variables `\l__spintent_sparc_prefix_tl` and `\l__spintent_spmarc_prefix_tl` store the values set by the key `prefix`.

```

4377 \clist_map_inline:nn { arc , marc }
4378 {
4379   \str_new:c { l__spintent_sp #1 _mode_str }
4380   \tl_new:c { l__spintent_sp #1 _prefix_tl }
4381 }

```

(End of definition for `\l__spintent_sparc_mode_str` and others.)

11.42.2 Definition of error messages for `\sparc` and `\spmarc`

We define an “error” message for the commands `\sparc` and `\spmarc`. The “error” is triggered if the user includes a invalid `{⟨argument⟩}` passed in ‘#1’ to the command.

```

4382 \clist_map_inline:nn { arc , marc }
4383 {
4384   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
4385   {
4386     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
4387   }
4388   {
4389     See~the~documentation~of~##1~for~the~accepted~point~formats.
4390   }
4391 }

```

(End of definition for `sparc-invalid-points` and `spmarc-invalid-points`.)

11.42.3 Definition of keys for `\sparc` and `\spmarc`

We define the shared `⟨keys⟩` `read-cmd`, `prefix`, `silent` and `concept`.

```

read-cmd 4392 \clist_map_inline:nn { arc , marc }
prefix   4393 {
silent   4394   \keys_define:nn { spintent / sp #1 }
concept  4395   {
unknown  4396     read-cmd .choices:nn = { school , mathcat }
          4397     {
          4398       \int_case:nn { \l_keys_choice_int }
          4399       {
          4400         {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
          4401         {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
          4402       }
          4403     },
          4404     read-cmd .initial:n = school,
          4405     read-cmd .value_required:n = true,
          4406     read-cmd / unknown .code:n =
          4407     \msg_error:nneee { spintent } { unknown-choice }
          4408     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
          4409     prefix .code:n =
          4410     { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
          4411     prefix .initial:n = ,

```

```

4412     prefix                .value_required:n = true,
4413     silent                .bool_set:c = { l__spintent_sp #1 _silent_bool },
4414     silent                .initial:n = false,
4415     silent                .value_required:n = true,
4416     concept              .bool_set:c = { l__spintent_sp #1 _concept_bool },
4417     concept              .initial:n = true,
4418     concept              .value_required:n = true,
4419     unknown              .code:n =
4420     {
4421         \str_set:Nn \l__spintent_command_name_str { sp #1 }
4422         \__spintent_unknown_keys_cmd:n {##1}
4423     },
4424 }
4425 }

```

(End of definition for read-cmd and others.)

11.42.4 Internal functions for \sparc and \spmarc

__spintent_sparc_wide_arc:n

The function `\__spintent_sparc_wide_arc:n` acts as a fallback when the mathematical font used in the document does not provide the command `\widearc`. The definition of this function is a direct adaptation of a response given by Clea F. Rees (@cfr) to a query in the chat room of [TeX-SX](#).

```

4426 \cs_new_protected_nopar:Nn \__spintent_sparc_wide_arc:n
4427 {
4428     \Umathaccent 7 \symoperators "023DC \scan_stop: {#1}
4429 }

```

(End of definition for __spintent_sparc_wide_arc:n.)

__spintent_sparc_print_arc:n

The function `\__spintent_sparc_print_arc:n` applies the command `\widearc` to the argument passed to `\sparc` or `\spmarc`. If the mathematical font set in the document does not provide the command `\widearc`, we will use the built-in function `\__spintent_sparc_wide_arc:n`.

```

4430 \cs_new_protected:Nn \__spintent_sparc_print_arc:n
4431 {
4432     \cs_if_exist:NTF \widearc
4433     { \widearc {#1} }
4434     { \__spintent_sparc_wide_arc:n {#1} }
4435 }

```

(End of definition for __spintent_sparc_print_arc:n.)

\l__spintent_sparc_intent_tl

The variable `\l__spintent_sparc_intent_tl` holds the concept word (arco or medida-del-arco). The rest are scratch variables to split `\l__spintent_geo_luaset_points_clist` and build the `$pt1, $pt2, ...` reference list used in the concept intent.

```

4436 \tl_new:N \l__spintent_sparc_intent_tl
4437 \seq_new:N \l__spintent_sparc_pt_seq
4438 \tl_new:N \l__spintent_sparc_refs_tl
4439 \tl_new:N \l__spintent_sparc_intent_arg_tl
4440 \int_new:N \l__spintent_sparc_pt_idx_int

```

(End of definition for \l__spintent_sparc_intent_tl and others.)

__spintent_sparc_process:nn

The internal function `\__spintent_sparc_process:nn` receives the short name of the command (arc or marc) in ‘#1’ and the `{(argument)}` in ‘#2’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process ‘#2’, returning an “error” if invalid, then sets `\l__spintent_sparc_intent_tl` to arco (or medida-del-arco for `\spmarc`). No intent is built here: it needs the reference list, which `\__spintent_sparc_mathl` builds after splitting the points.

```

4441 \cs_new_protected:Nn \__spintent_sparc_process:nn
4442 {
4443     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
4444     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
4445     {
4446         \msg_error:nnee { spintent } { sp #1 -invalid-points }
4447         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4448     }
4449     \str_if_eq:eeTF {#1} { marc }
4450     { \tl_set:Nn \l__spintent_sparc_intent_tl { medida-del-arco } }
4451     { \tl_set:Nn \l__spintent_sparc_intent_tl { arco } }
4452     \__spintent_sparc_mathl:n {#1}
4453 }

```

(End of definition for __spintent_sparc_process:nn.)

`\__spintent_sparc_mathml:n`

The function `\__spintent_sparc_mathml:n` receives the short command name in ‘#1’. It splits `\l__spintent_geo_` into `\l__spintent_sparc_pt_seq` and walks it once to build `\l__spintent_sparc_refs_tl`, the comma-separated `$pt1,$pt2,...` reference list, naming each point by position.

```

4454 \cs_new_protected:Nn \__spintent_sparc_mathml:n
4455 {
4456   \seq_set_from_clist:NN \l__spintent_sparc_pt_seq \l__spintent_geo_luaseta_points_clist
4457
4458   \tl_clear:N \l__spintent_sparc_refs_tl
4459   \int_zero:N \l__spintent_sparc_pt_idx_int
4460   \seq_map_inline:Nn \l__spintent_sparc_pt_seq
4461   {
4462     \int_incr:N \l__spintent_sparc_pt_idx_int
4463     \tl_if_empty:NF \l__spintent_sparc_refs_tl
4464     { \tl_put_right:Nn \l__spintent_sparc_refs_tl { , } }
4465     \tl_put_right:Ne \l__spintent_sparc_refs_tl { $ pt \int_use:N \l__spintent_sparc_pt_idx_int }
4466   }

```

The concept `intent` is `_` - when `silent` is `true`, `_` (refs) when `concept` is `false`; otherwise `\l__spintent_sparc_` with `prefix` placed after the leading `_` when given.

```

4467   \bool_if:cTF { l__spintent_sp #1 _silent_bool }
4468   {
4469     \tl_set:Nn \l__spintent_sparc_intent_arg_tl { _- }
4470   }
4471   {
4472     \bool_if:cTF { l__spintent_sp #1 _concept_bool }
4473     {
4474       \tl_set:Ne \l__spintent_sparc_intent_arg_tl
4475       {
4476         -
4477         \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
4478         { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
4479         \l__spintent_sparc_intent_tl :prefix( \l__spintent_sparc_refs_tl )
4480       }
4481     }
4482     {
4483       \tl_set:Ne \l__spintent_sparc_intent_arg_tl { _ ( \l__spintent_sparc_refs_tl ) }
4484     }
4485   }

```

It is applied with a single native `\MathMLintent` wrapping `\__spintent_sparc_print_arc:n` directly (same shape as `\__spintent_spside_mathml:n`), preceded by `\__spintent_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`, [luamml issue #26](#)). A silent `\operatorname{m}` precedes it for `\spmarc` (always, no `print-m` key here). The sequence is walked a second time, calling `\__spintent_mathmlarg_point:nnn` once per point with its `ptn` reference name, looked up under `sp#1`.

```

4486   \__spintent_invisible_sep: % need by MathCAT (bug in luamml issues #26)
4487   \MathMLintent { \l__spintent_sparc_intent_arg_tl }
4488   {
4489     \__spintent_inner_sep:
4490     \str_if_eq:eeT {#1} { marc }
4491     {
4492       \__spintent_luafun_mrow_intent_arg:nn
4493       { } { intent = ":silent" }
4494       { \operatorname { m } } }
4495     }
4496     \__spintent_sparc_print_arc:n
4497     {
4498       \int_zero:N \l__spintent_sparc_pt_idx_int
4499       \seq_map_inline:Nn \l__spintent_sparc_pt_seq
4500       {
4501         \int_incr:N \l__spintent_sparc_pt_idx_int
4502         \__spintent_mathmlarg_point:nnn
4503         { pt \int_use:N \l__spintent_sparc_pt_idx_int }
4504         { sp #1 }
4505         { ##1 }
4506       }
4507     }
4508   }
4509 }

```

(End of definition for `\__spintent_sparc_mathml:n`.)

`\__spintent_sparc_exec:nnn` The function `\__spintent_sparc_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *keys* passed in ‘#2’ and the *{argument}* passed in ‘#3’, delegating the rendering to `\__spintent_sparc_process:nn`.

```

4510 \cs_new_protected:Nn \__spintent_sparc_exec:nnn
4511 {
4512     \keys_set:nn { spintent / sp #1 } { #2 }
4513     \__spintent_sparc_process:nn { #1 } { #3 }
4514 }

```

(End of definition for `\__spintent_sparc_exec:nnn`.)

`\sparc` Both public commands `\sparc` and `\spmarc` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sparc_exec:nnn`.

```

4515 \NewDocumentCommand \sparc { O{} m }
4516 {
4517     \mode_if_math:TF
4518     {
4519         \group_begin:
4520         \attribute \__spintent_scope_attr_int = 1
4521         \__spintent_sparc_exec:nnn { arc } { #1 } { #2 }
4522         \group_end:
4523     }
4524     { \msg_error:nnn { spintent } { need-math-mode } { \sparc } }
4525 }
4526 \NewDocumentCommand \spmarc { O{} m }
4527 {
4528     \mode_if_math:TF
4529     {
4530         \group_begin:
4531         \attribute \__spintent_scope_attr_int = 1
4532         \__spintent_sparc_exec:nnn { marc } { #1 } { #2 }
4533         \group_end:
4534     }
4535     { \msg_error:nnn { spintent } { need-math-mode } { \spmarc } }
4536 }

```

(End of definition for `\sparc` and `\spmarc`. These functions are documented on page 25.)

11.43 The command `\spPoly`

The user command `\spPoly` typesets a polygon defined by its vertices. The figure type and its visual symbol are inferred from the vertex count: three points produce a triangle, four a square, five or more a generic polygon with no symbol. It never sends a boolean to Lua: `\__spintent_geo_luaset_intent_str` (the full *intent*, with the figure name).

11.43.1 Definition of variables for `\spPoly`

`\__spintent_spPoly_mode_str` The variable `\__spintent_spPoly_mode_str` stores the value set by the key `read-cmd`; the variable `\__spintent_spPoly_prefix_tl` stores the value set by the key `prefix`.

```

4537 \str_new:N \__spintent_spPoly_mode_str
4538 \tl_new:N \__spintent_spPoly_prefix_tl

```

(End of definition for `\__spintent_spPoly_mode_str` and `\__spintent_spPoly_prefix_tl`.)

11.43.2 Definition of error messages for `\spPoly`

`spPoly-invalid-points` Raised when the argument is not a comma-separated list of at least three uppercase letters.

```

4539 \msg_new:nnnn { spintent } { spPoly-invalid-points }
4540 {
4541     Invalid~point~format~in~\token_to_str:N \spPoly:~'#1'~\msg_line_context:.
4542 }
4543 {
4544     See~the~documentation~of~\token_to_str:N \spPoly~for~the~accepted~point~formats.
4545 }

```

(End of definition for `spPoly-invalid-points`.)

11.43.3 Definition of keys for \spPoly

We define the *(keys)* read-cmd, prefix, print-sym, silent and concept.

```

read-cmd 4546 \keys_define:nn { spintent / spPoly }
prefix   4547 {
print-sym 4548   read-cmd           .choices:nn = { school , mathcat }
silent    4549   {
concept   4550     \int_case:nn { \l_keys_choice_int }
unknown   4551     {
4552       {1} { \str_set:Nn \l__spintent_spPoly_mode_str { school } }
4553       {2} { \str_set:Nn \l__spintent_spPoly_mode_str { mathcat } }
4554     }
4555   },
4556   read-cmd           .initial:n = school,
4557   read-cmd           .value_required:n = true,
4558   read-cmd / unknown .code:n =
4559     \msg_error:nneee { spintent } { unknown-choice }
4560     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
4561   prefix             .code:n =
4562     { \__spintent_sanitize_affix:Nn \l__spintent_spPoly_prefix_tl {#1} },
4563   prefix             .initial:n = ,
4564   prefix             .value_required:n = true,
4565   print-sym          .bool_set:N = \l__spintent_spPoly_print_sym_bool,
4566   print-sym          .initial:n = true,
4567   print-sym          .value_required:n = true,
4568   silent             .bool_set:N = \l__spintent_spPoly_silent_bool,
4569   silent             .initial:n = false,
4570   silent             .value_required:n = true,
4571   concept            .bool_set:N = \l__spintent_spPoly_concept_bool,
4572   concept            .initial:n = true,
4573   concept            .value_required:n = true,
4574   unknown            .code:n =
4575     {
4576       \str_set:Nn \l__spintent_command_name_str { spPoly }
4577       \__spintent_unknown_keys_cmd:n {#1}
4578     },
4579   }

```

(End of definition for read-cmd and others.)

11.43.4 Internal functions for \spPoly

Scratch variables to split \l__spintent_geo_luaset_points_clist and build the \$pt1, \$pt2, ... reference list, and to hold the final concept intent once built.

```

\l__spintent_spPoly_pt_seq 4580 \seq_new:N \l__spintent_spPoly_pt_seq
\l__spintent_spPoly_refs_tl 4581 \tl_new:N \l__spintent_spPoly_refs_tl
\l__spintent_spPoly_intent_arg_tl 4582 \tl_new:N \l__spintent_spPoly_intent_arg_tl
\l__spintent_spPoly_pt_idx_int 4583 \int_new:N \l__spintent_spPoly_pt_idx_int

```

(End of definition for \l__spintent_spPoly_pt_seq and others.)

__spintent_spPoly_sym: Unchanged: reads \l__spintent_geo_luaset_poly_sym_str and typesets the corresponding symbol.

```

4584 \cs_new_protected:Nn \__spintent_spPoly_sym:
4585 {
4586   \str_case:Vn \l__spintent_geo_luaset_poly_sym_str
4587   {
4588     { triangle } { \triangle }
4589     { square } { \mdlgwhtsquare }
4590   }
4591 }

```

(End of definition for __spintent_spPoly_sym:.)

__spintent_spPoly_build_refs: Splits \l__spintent_geo_luaset_points_clist into \l__spintent_spPoly_pt_seq and walks it once to build \l__spintent_spPoly_refs_tl, the comma-separated \$pt1, \$pt2, ... reference list, naming each vertex by position.

```

4592 \cs_new_protected:Nn \__spintent_spPoly_build_refs:
4593 {
4594   \seq_set_from_clist:NN \l__spintent_spPoly_pt_seq \l__spintent_geo_luaset_points_clist
4595   \tl_clear:N \l__spintent_spPoly_refs_tl
4596   \int_zero:N \l__spintent_spPoly_pt_idx_int
4597   \seq_map_inline:Nn \l__spintent_spPoly_pt_seq
4598   {

```

```

4599         \int_incr:N \__spintent_spPoly_pt_idx_int
4600         \tl_if_empty:NF \__spintent_spPoly_refs_tl
4601         { \tl_put_right:Nn \__spintent_spPoly_refs_tl { , } }
4602         \tl_put_right:Ne \__spintent_spPoly_refs_tl { $ pt \int_use:N \__spintent_spPoly_pt_idx_int }
4603     }
4604 }

```

(End of definition for `\__spintent_spPoly_build_refs:`)

`\__spintent_spPoly_build_concept:`

Builds `\__spintent_spPoly_intent_arg_tl`, independently of `print-sym: _-` when `silent` is `true`, `_ (refs)` when `concept` is `false`; otherwise `\__spintent_geo_luaset_concept_str:prefix(refs)`, with `prefix` placed after the leading `_` when given. Must be called after `\__spintent_spPoly_build_refs:`.

```

4605 \cs_new_protected:Nn \__spintent_spPoly_build_concept:
4606 {
4607     \bool_if:NTF \__spintent_spPoly_silent_bool
4608     {
4609         \tl_set:Nn \__spintent_spPoly_intent_arg_tl { _- }
4610     }
4611     {
4612         \bool_if:NTF \__spintent_spPoly_concept_bool
4613         {
4614             \tl_set:Ne \__spintent_spPoly_intent_arg_tl
4615             {
4616                 -
4617                 \tl_if_empty:NF \__spintent_spPoly_prefix_tl
4618                 { \tl_use:N \__spintent_spPoly_prefix_tl - }
4619                 \__spintent_geo_luaset_concept_str :prefix( \__spintent_spPoly_refs_tl )
4620             }
4621         }
4622         {
4623             \tl_set:Ne \__spintent_spPoly_intent_arg_tl { _ ( \__spintent_spPoly_refs_tl ) }
4624         }
4625     }
4626 }

```

(End of definition for `\__spintent_spPoly_build_concept:`)

`\__spintent_spPoly_points_loop:`

Walks `\__spintent_spPoly_pt_seq` a second time, calling `\__spintent_mathmlarg_point:nnn` once per vertex with its `ptn` reference name, looked up under `spPoly:` it alone decides `arg/literal intent` per vertex — replacing the single combined `\MathMLarg{pts}{...}` that lost `arg` on any vertex with a subscript or prime (the same `annotate_attribute/sub-sup` limitation already found and worked around for `\spPoint`).

```

4627 \cs_new_protected:Nn \__spintent_spPoly_points_loop:
4628 {
4629     \int_zero:N \__spintent_spPoly_pt_idx_int
4630     \seq_map_inline:Nn \__spintent_spPoly_pt_seq
4631     {
4632         \int_incr:N \__spintent_spPoly_pt_idx_int
4633         \__spintent_mathmlarg_point:nnn
4634         { pt \int_use:N \__spintent_spPoly_pt_idx_int }
4635         { spPoly }
4636         { ##1 }
4637     }
4638 }

```

(End of definition for `\__spintent_spPoly_points_loop:`)

`\__spintent_spPoly_process:n`

Calls `\__spintent_luafun_geo_poly_parse_and_set:n`, reports `spPoly-invalid-points` on failure, then delegates to `\__spintent_spPoly_mathml:`. No `intent` is built here: it needs the reference list, which `\__spintent_spPoly_mathml:` builds after splitting the vertices.

```

4639 \cs_new_protected:Nn \__spintent_spPoly_process:n
4640 {
4641     \__spintent_luafun_geo_poly_parse_and_set:n { \exp_not:n {#1} }
4642     \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
4643     {
4644         \msg_error:nne { spintent } { spPoly-invalid-points }
4645         { \exp_not:n {#1} }
4646     }
4647     \__spintent_spPoly_mathml:
4648 }

```

(End of definition for `\__spintent_spPoly_process:n`)

`\__spintent_spPoly_mathml:` Builds the reference list and the concept `\__spintent_spPoly_build_refs:`, `\__spintent_spPoly_build_refs:` then wraps everything in a single native `\MathMLintent`, preceded by `\__spintent_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`, `luamml` issue #26).
 The keys `print-sym` and `concept/silent` are independent: the former only decides whether the glyph (`\__spintent_spPoly_sym:`, silenced with `\__spintent_luafun_mrow_intent_arg:nn`) is attempted — it is skipped anyway when `\l__spintent_geo_luaset_poly_sym_str` is empty (no symbol defined for that vertex count, e.g. pentagon and above) — and never affects whether the concept word itself is spoken.

```

4649 \cs_new_protected:Nn \__spintent_spPoly_mathml:
4650 {
4651   \__spintent_invisible_sep: % (luamml issues #26)
4652   \__spintent_spPoly_build_refs:
4653   \__spintent_spPoly_build_concept:
4654   \MathMLintent { \__spintent_spPoly_intent_arg_tl }
4655   {
4656     \__spintent_inner_sep:
4657     \bool_if:NT \l__spintent_spPoly_print_sym_bool
4658     {
4659       \str_if_empty:NF \l__spintent_geo_luaset_poly_sym_str
4660       {
4661         \__spintent_luafun_mrow_intent_arg:nn
4662         { } { intent = ":silent" }
4663         { \__spintent_spPoly_sym: }
4664       }
4665     }
4666     \__spintent_spPoly_points_loop:
4667   }
4668 }

```

(End of definition for `\__spintent_spPoly_mathml:`.)

`\__spintent_spPoly_exec:nn` The function `\__spintent_spPoly_exec:nn` receives the optional keys in #1 and the argument in #2, delegating the rendering to `\__spintent_spPoly_process:n`.

```

4669 \cs_new_protected:Nn \__spintent_spPoly_exec:nn
4670 {
4671   \keys_set:nn { spintent / spPoly } { #1 }
4672   \__spintent_spPoly_process:n { #2 }
4673 }

```

(End of definition for `\__spintent_spPoly_exec:nn`.)

`\spPoly` The public command `\spPoly` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spPoly_exec:nn`.

```

4674 \NewDocumentCommand \spPoly { O{} m }
4675 {
4676   \mode_if_math:TF
4677   {
4678     \group_begin:
4679     \attribute \l__spintent_scope_attr_int = 1
4680     \__spintent_spPoly_exec:nn { #1 } { #2 }
4681     \group_end:
4682   }
4683   { \msg_error:nnn { spintent } { need-math-mode } { \spPoly } }
4684 }

```

(End of definition for `\spPoly`. This function is documented on page 26.)

11.44 The commands `\spAfig` and `\spPfig`

The user commands `\spAfig` and `\spPfig` typeset the area and the perimeter of a polygonal figure, using the delimited-optional argument syntax `d()`. The key `print-sym` is never sent to Lua: the symbol type is returned unconditionally in `\l__spintent_geo_luaset_poly_sym_str`, and `expl3` decides whether to wrap it in `\MathMLintent{[:silent]}`.

11.44.1 Definition of variables for `\spAfig` and `\spPfig`

`\l__spintent_geo_final_print_tl` Scratch variables used by `\__spintent_spfig_process:nn` to assemble the visual and the `intent`. The variable `\l__spintent_geo_final_print_tl` holds the final visual placed under `\mathcal`'s subscript. The final literal `intent` and the concept forwarded when `read-cmd` is `mathcat`.

```

4685 \tl_new:N \l__spintent_geo_final_print_tl

```

(End of definition for `\l__spintent_geo_final_print_tl`.)

```

\__spintent_spAfig_mode_str
\__spintent_spPfig_mode_str
  \__spintent_spAfig_print_sym_str
  \__spintent_spPfig_print_sym_str
    \__spintent_spAfig_prefix_tl
    \__spintent_spPfig_prefix_tl
  \__spintent_spAfig_read_arg_tl
  \__spintent_spPfig_read_arg_tl

```

These variables store the value set by each corresponding key: `read-cmd`, `print-sym`, `prefix` and `read-arg`, respectively. The value set by `read-arg` overrides the figure name inferred from the vertex count, independently of `print-sym`.

```

4686 \clist_map_inline:nn { Afig , Pfig }
4687 {
4688   \str_new:c { \__spintent_sp #1 _mode_str      }
4689   \str_new:c { \__spintent_sp #1 _print_sym_str }
4690   \tl_new:c  { \__spintent_sp #1 _prefix_tl     }
4691   \tl_new:c  { \__spintent_sp #1 _read_arg_tl   }
4692 }

```

(End of definition for `\__spintent_spAfig_mode_str` and others.)

11.44.2 Definition of error messages for `\spAfig` and `\spPfig`

All eight messages are generated by the same map, using the same `##1/##2` doubling convention as `\spside/\spmside`. `sp #1-invalid-points` is currently unused by `\__spintent_spfig_process:nn`, which raises only `sp #1-invalid-arg` for any parse failure, regardless of whether a comma was present. `sp #1-sym-letra-conflict` and `sp #1-sym-cardinality-conflict` are raised when `print-sym` is forced to a value incompatible with the actual argument.

```

4693 \clist_map_inline:nn { Afig , Pfig }
4694 {
4695   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
4696   {
4697     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
4698   }
4699   {
4700     See~the~documentation~of~##1~for~the~accepted~point~formats.
4701   }
4702   \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
4703   {
4704     Invalid~argument~format~in~##1:~'##2'~\msg_line_context:.
4705   }
4706   {
4707     See~the~documentation~of~##1~for~the~accepted~argument~forms.
4708   }
4709   \msg_new:nnnn { spintent } { sp #1 -sym-letra-conflict }
4710   {
4711     print-sym~cannot~be~combined~with~a~lettered~argument~in~'##1'~\msg_line_context:.
4712   }
4713   {
4714     See~the~documentation~of~##1~for~details~on~print-sym.
4715   }
4716   \msg_new:nnnn { spintent } { sp #1 -sym-cardinality-conflict }
4717   {
4718     print-sym~does~not~match~the~vertex~count~in~'##1'~\msg_line_context:.
4719   }
4720   {
4721     See~the~documentation~of~##1~for~details~on~print-sym.
4722   }
4723   \msg_new:nnnn { spintent } { sp #1 -read-arg-key }
4724   {
4725     read-arg~has~no~effect~with~a~single~argument~in~'##1'~\msg_line_context:.
4726   }
4727   {
4728     See~the~documentation~of~##1~for~details~on~read-arg.
4729   }
4730 }

```

(End of definition for `spAfig-invalid-points` and others.)

11.44.3 Definition of keys for `\spAfig` and `\spPfig`

```

read-cmd
prefix
print-sym
read-arg
read-sub
silent
concept
concept*
unknown

```

The key `print-sym` chooses the visual symbol: `auto` (default) infers it from the vertex count (three points give `\triangle`, four give `\mdlgwhtsquare`, five or more give none); `triangle/square` force one, requiring the vertex count to match; `none` omits it, even with vertices. With a bare number/letter argument, `auto` and `none` behave the same (no symbol); `triangle/square` nest the argument under the forced symbol instead, and cannot be combined with a table letter (F/P/B/L). The key `read-arg` overrides the spoken figure name for real vertices, independently of `print-sym`. The key `read-sub` controls whether the bare number/letter argument is read with the word «sub» or as a plain number.

```

4731 \clist_map_inline:nn { Afig , Pfig }
4732 {

```

```

4733 \keys_define:nn { spintent / sp #1 }
4734 {
4735   read-cmd          .choices:nn = { school , mathcat }
4736   {
4737     \int_case:nn { \l_keys_choice_int }
4738     {
4739       {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
4740       {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
4741     }
4742   },
4743   read-cmd          .initial:n = school,
4744   read-cmd          .value_required:n = true,
4745   read-cmd / unknown .code:n =
4746     \msg_error:nneee { spintent } { unknown-choice }
4747     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
4748   prefix            .code:n =
4749     { \__spintent_sanitiz_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
4750   prefix            .initial:n = ,
4751   prefix            .value_required:n = true,
4752   print-sym         .choices:nn = { auto , triangle , square , none }
4753   {
4754     \int_case:nn { \l_keys_choice_int }
4755     {
4756       {1} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { auto } }
4757       {2} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { triangle } }
4758       {3} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { square } }
4759       {4} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { none } }
4760     }
4761   },
4762   print-sym         .initial:n = auto,
4763   print-sym         .value_required:n = true,
4764   print-sym / unknown .code:n =
4765     \msg_error:nneee { spintent } { unknown-choice }
4766     { print-sym } { auto,~triangle,~square,~none } { \exp_not:n {##1} },
4767   read-arg          .code:n =
4768     { \__spintent_sanitiz_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1} },
4769   read-arg          .initial:n = ,
4770   read-arg          .value_required:n = true,
4771   read-sub          .bool_set:c = { l__spintent_sp #1 _read_sub_bool },
4772   read-sub          .initial:n = false,
4773   read-sub          .value_required:n = true,
4774   silent            .bool_set:c = { l__spintent_sp #1 _silent_bool },
4775   silent            .initial:n = false,
4776   silent            .value_required:n = true,
4777   concept           .bool_set:c = { l__spintent_sp #1 _concept_bool },
4778   concept           .initial:n = true,
4779   concept           .value_required:n = true,
4780   concept*          .meta:n =
4781   {
4782     concept = false,
4783     read-arg = { \str_if_eq:eeTF {#1} { Afig } { _área } { _perímetro } },
4784   },
4785   concept*          .value_forbidden:n = true,
4786   unknown           .code:n =
4787   {
4788     \str_set:Nn \l__spintent_command_name_str { sp #1 }
4789     \__spintent_unknown_keys_cmd:n {##1}
4790   },
4791 }
4792 }

```

(End of definition for read-cmd and others.)

11.44.4 Internal functions for \spAfig and \spPfig

`\__spintent_spfig_sym:` The function `\__spintent_spfig_sym:` reads `\l__spintent_geo_print_sym_str` and typesets the corresponding symbol: `\triangle` for the value `triangle`, `\mdlgwhtsquare` for the value `square`.

```

4793 \cs_new_protected:Nn \__spintent_spfig_sym:
4794 {
4795   \str_case:Vn \l__spintent_geo_print_sym_str
4796   {
4797     { triangle } { \triangle }

```

```

4798         { square } { \mdlgwhtsquare }
4799     }
4800 }

```

(End of definition for `\__spintent_spfig_sym:`)

The variable `\__spintent_spfig_op_tl` holds área or perímetro. The variable `\__spintent_spfig_intent_tl` holds the concept word for the vertices case (`read-arg` or the auto-detected figure name). The rest are scratch variables to split `\__spintent_geo_luaset_points_clist` and build the `$pt1, $pt2, ...` reference list.

```

4801 \tl_new:N \__spintent_spfig_op_tl
4802 \tl_new:N \__spintent_spfig_intent_tl
4803 \tl_new:N \__spintent_spfig_desc_tl
4804 \tl_new:N \__spintent_spfig_symbol_tl
4805 \seq_new:N \__spintent_spfig_pt_seq
4806 \tl_new:N \__spintent_spfig_refs_tl
4807 \tl_new:N \__spintent_spfig_intent_arg_tl
4808 \int_new:N \__spintent_spfig_pt_idx_int
4809 \str_new:N \__spintent_geo_print_sym_str

```

(End of definition for `\__spintent_spfig_op_tl` and others.)

Receives the short command name (Afig or Pfig) in ‘#1’ and the $\langle argument \rangle$ in ‘#2’. Calls `\__spintent_luafun_geo` or `\__spintent_luafun_geo_pfig_parse_and_set:n`, reports `sp#1-invalid-arg` on failure, and `sp#1-read-arg` when `read-arg` is given for a bare number/letter argument (no vertices), where it has no effect. Sets `\__spintent_spfig_op_tl` to área/perímetro, then delegates to `\__spintent_spfig_mathml:n`.

```

4810 \cs_new_protected:Nn \__spintent_spfig_process:nn
4811 {
4812     \str_if_eq:eeTF {#1} { Afig }
4813     { \__spintent_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
4814     { \__spintent_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
4815     \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
4816     {
4817         \msg_error:nnee { spintent } { sp #1 -invalid-arg }
4818         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4819     }
4820     \clist_if_empty:NT \__spintent_geo_luaset_points_clist
4821     {
4822         \tl_if_empty:cF { \__spintent_sp #1 _read_arg_tl }
4823         {
4824             \msg_error:nne { spintent } { sp #1 -read-arg-key }
4825             { \c_backslash_str sp #1 }
4826         }
4827     }
4828     \str_if_eq:eeTF {#1} { Afig }
4829     { \tl_set:Nn \__spintent_spfig_op_tl { área } }
4830     { \tl_set:Nn \__spintent_spfig_op_tl { perímetro } }
4831     \str_set:Ne \__spintent_geo_print_sym_str { \str_use:c { \__spintent_sp #1 _print_sym_str } }
4832     \__spintent_spfig_mathml:n {#1}
4833 }

```

(End of definition for `\__spintent_spfig_process:nn`.)

Handles the argument-less, table-letter and bare number/letter cases (no vertices). Builds `\__spintent_geo_final_print` (the visual placed under `\mathcal`’s subscript) and `\__spintent_spfig_desc_tl` (the descriptive part, without área/perímetro). Receives the short command name in ‘#1’.

With an empty argument, both are cleared for `print-sym auto/ none`; a forced `print-sym` instead places the symbol alone, with its own name as the description.

```

4834 \cs_new_protected:Nn \__spintent_spfig_build_simple:n
4835 {
4836     \tl_if_empty:nTF { \__spintent_geo_luaset_print_tl }
4837     {
4838         \str_case:VnF \__spintent_geo_print_sym_str
4839         {
4840             { auto }
4841             {
4842                 \tl_clear:N \__spintent_geo_final_print_tl
4843                 \tl_clear:N \__spintent_spfig_desc_tl
4844             }
4845             { none }
4846             {
4847                 \tl_clear:N \__spintent_geo_final_print_tl

```

```

4848         \tl_clear:N \l__spintent_spfig_desc_tl
4849     }
4850 }
4851 {
4852     \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_spfig_sym: }
4853     \tl_set:Ne \l__spintent_spfig_desc_tl
4854     {
4855         \str_if_eq:VnTF \l__spintent_geo_print_sym_str { triangle }
4856         { triángulo } { cuadrado }
4857     }
4858 }
4859 }

```

With a bare number/letter (empty `\l__spintent_geo_luaset_letra_word_str`), the value itself becomes the visual for `print-sym auto/none`, or its own subscript under a forced symbol otherwise; the description is the spoken value, prefixed with «sub-» when `read-sub` is `true` (further prefixed with the symbol's own name for a forced `print-sym`).

```

4860     {
4861         \str_if_empty:NTF \l__spintent_geo_luaset_letra_word_str
4862         {
4863             \str_case:VnF \l__spintent_geo_print_sym_str
4864             {
4865                 { auto } { \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_p
4866                 { none } { \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_p
4867             }
4868             {
4869                 \tl_set:Nn \l__spintent_geo_final_print_tl
4870                 {
4871                     \l__spintent_spfig_sym:
4872                     \c_math_subscript_token { \l__spintent_geo_luaset_print_tl }
4873                 }
4874             }
4875             \str_case:VnF \l__spintent_geo_print_sym_str
4876             {
4877                 { auto }
4878                 {
4879                     \tl_set:Ne \l__spintent_spfig_desc_tl
4880                     {
4881                         \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
4882                         \l__spintent_geo_luaset_sub_spoken_str
4883                     }
4884                 }
4885                 { none }
4886                 {
4887                     \tl_set:Ne \l__spintent_spfig_desc_tl
4888                     {
4889                         \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
4890                         \l__spintent_geo_luaset_sub_spoken_str
4891                     }
4892                 }
4893             }
4894         }
4895         \tl_set:Ne \l__spintent_spfig_desc_tl
4896         {
4897             del-
4898             \str_if_eq:VnTF \l__spintent_geo_print_sym_str { triangle }
4899             { triángulo } { cuadrado }
4900             -
4901             \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
4902             \l__spintent_geo_luaset_sub_spoken_str
4903         }
4904     }
4905 }

```

With a table letter (F/P/B/L/T, non-empty `\l__spintent_geo_luaset_letra_word_str`), a forced `print-sym` raises `sp#1-sym-letra-conflict` — a table letter is never nested under a symbol. The visual is the value as-is; the description is its own word, connected as `\l__spintent_geo_luaset_letra_conector_str` requires, with the same spoken subscript as above when present.

```

4906     {
4907         \str_case:VnF \l__spintent_geo_print_sym_str
4908         {
4909             { auto } { }

```

```

4910         { none } { }
4911     }
4912     {
4913         \msg_error:nne { spintent } { sp #1 -sym-letra-conflict }
4914         { \c_backslash_str sp #1 }
4915     }
4916     \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_print_tl }
4917     \tl_set:Ne \l__spintent_spfig_desc_tl
4918     {
4919         \str_if_empty:NF \l__spintent_geo_luaset_letra_conector_str
4920         { \l__spintent_geo_luaset_letra_conector_str - }
4921         \l__spintent_geo_luaset_letra_word_str
4922         \str_if_empty:NF \l__spintent_geo_luaset_sub_spoken_str
4923         {
4924             -
4925             \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
4926             \l__spintent_geo_luaset_sub_spoken_str
4927         }
4928     }
4929 }
4930 }

```

The variable `\l__spintent_spfig_intent_tl` is always the word- shape, without the leading `_`: `\l__spintent_spfig_intent_tl` alone when `\l__spintent_spfig_desc_tl` is empty, both joined by a dash otherwise. `silent` overrides `\l__spintent_spfig_intent_arg_tl` with `_`; with `silent=false` and `concept false`, it becomes `_l__spintent_spfig_desc_tl-` (the descriptive part alone, without *área/perímetro* nor, for a table letter, its connector *de-la/del*); with `concept true` it is `_`, then `prefix` when given, and then `\l__spintent_spfig_intent_tl`.

```

4931     \tl_set:Ne \l__spintent_spfig_intent_tl
4932     {
4933         \tl_if_empty:NTF \l__spintent_spfig_desc_tl
4934         { \l__spintent_spfig_op_tl - }
4935         { \l__spintent_spfig_op_tl - \l__spintent_spfig_desc_tl - }
4936     }
4937     \bool_if:cTF { \l__spintent_sp #1 _silent_bool }
4938     { \tl_set:Nn \l__spintent_spfig_intent_arg_tl { _- } }
4939     {
4940         \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
4941         {
4942             \tl_set:Ne \l__spintent_spfig_intent_arg_tl
4943             {
4944                 -
4945                 \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4946                 { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4947                 \l__spintent_spfig_intent_tl
4948             }
4949         }
4950         {
4951             \tl_set_eq:NN \l__tmpa_tl \l__spintent_spfig_desc_tl
4952             \str_if_empty:NF \l__spintent_geo_luaset_letra_conector_str
4953             {
4954                 \tl_replace_once:Nee \l__tmpa_tl
4955                 { \l__spintent_geo_luaset_letra_conector_str - } { }
4956             }
4957             \tl_set:Ne \l__spintent_spfig_intent_arg_tl { _ \l__tmpa_tl - }
4958         }
4959     }
4960 }

```

(End of definition for `\__spintent_spfig_build_simple:n`)

`\__spintent_spfig_build_refs:` Splits `\l__spintent_geo_luaset_points_clist` into `\l__spintent_spfig_pt_seq` and walks it once to build `\l__spintent_spfig_refs_tl`, the comma-separated `$pt1,$pt2,...` reference list, naming each vertex by position.

```

4961 \cs_new_protected:Nn \__spintent_spfig_build_refs:
4962 {
4963     \seq_set_from_clist:NN \l__spintent_spfig_pt_seq \l__spintent_geo_luaset_points_clist
4964     \tl_clear:N \l__spintent_spfig_refs_tl
4965     \int_zero:N \l__spintent_spfig_pt_idx_int
4966     \seq_map_inline:Nn \l__spintent_spfig_pt_seq
4967     {

```

```

4968     \int_incr:N \__spintent_sfig_pt_idx_int
4969     \tl_if_empty:NF \__spintent_sfig_refs_tl
4970     { \tl_put_right:Nn \__spintent_sfig_refs_tl { , } }
4971     \tl_put_right:Ne \__spintent_sfig_refs_tl { $ pt \int_use:N \__spintent_sfig_pt_idx_int
4972   }
4973 }

```

(End of definition for `\__spintent_sfig_build_refs:`.)

`\__spintent_sfig_build_concept_vertices:n` Handles the vertices case. Receives the short command name in ‘#1’. Must be called after `\__spintent_sfig_build_refs:n`. The key `read-arg`, when given, overrides the auto-detected figure name in `\__spintent_sfig_intent_tl`: with `concept true` it is still prefixed with `área-del-/perímetro-del-`, with `concept false` it is used verbatim. With `read-arg` empty, `\__spintent_sfig_intent_tl` is simply `\__spintent_geo_luaset_concept_`.

```

4974 \cs_new_protected:Nn \__spintent_sfig_build_concept_vertices:n
4975 {
4976   \tl_if_empty:cTF { \__spintent_sp #1 _read_arg_tl }
4977   { \tl_set:Ne \__spintent_sfig_intent_tl { \__spintent_geo_luaset_concept_str } }
4978   {
4979     \bool_if:cTF { \__spintent_sp #1 _concept_bool }
4980     {
4981       \tl_set:Ne \__spintent_sfig_intent_tl
4982       {
4983         \__spintent_sfig_op_tl -del-
4984         \tl_use:c { \__spintent_sp #1 _read_arg_tl }
4985       }
4986     }
4987     {
4988       \tl_set:Ne \__spintent_sfig_intent_tl { \tl_use:c { \__spintent_sp #1 _read_arg_tl } }
4989     }
4990   }

```

Determines `\__spintent_sfig_symbol_tl` (empty when none applies): for `print-sym=auto`, the symbol matching `\__spintent_geo_luaset_poly_sym_str` (`\triangle`/`\mdlgwhtsquare`), or none when that is empty; `print-sym=none` always clears it; a forced `print-sym` requires it to match the actual vertex count, reporting `sp#1-sym-cardinality-conflict` otherwise.

```

4991   \str_case:VnF \__spintent_geo_print_sym_str
4992   {
4993     { auto }
4994     {
4995       \str_case:Vn \__spintent_geo_luaset_poly_sym_str
4996       {
4997         { triangle } { \tl_set:Nn \__spintent_sfig_symbol_tl { \triangle } }
4998         { square } { \tl_set:Nn \__spintent_sfig_symbol_tl { \mdlgwhtsquare } }
4999       }
5000       \str_if_empty:NT \__spintent_geo_luaset_poly_sym_str
5001       { \tl_clear:N \__spintent_sfig_symbol_tl }
5002     }
5003     { none }
5004     { \tl_clear:N \__spintent_sfig_symbol_tl }
5005   }
5006   {
5007     \str_if_eq:VVTf \__spintent_geo_print_sym_str \__spintent_geo_luaset_poly_sym_str
5008     { \tl_set:Nn \__spintent_sfig_symbol_tl { \__spintent_sfig_sym: } }
5009     {
5010       \msg_error:nne { spintent } { sp #1 -sym-cardinality-conflict }
5011       { \c_backslash_str sp #1 }
5012     }
5013   }

```

The key `silent` sets `\__spintent_sfig_intent_arg_tl` to `_-`. Otherwise, with `read-arg` empty, `concept` chooses between `\__spintent_sfig_intent_tl:prefix(refs)` (with `prefix` placed after the leading `_` when given) and `_(refs)` alone; with `read-arg` given, it is always `\__spintent_sfig_intent_tl:prefix(refs)` — `read-arg` is never silenced by `concept` alone, only by `silent`.

```

5014   \bool_if:cTF { \__spintent_sp #1 _silent_bool }
5015   {
5016     \tl_set:Nn \__spintent_sfig_intent_arg_tl { _- }
5017   }
5018   {
5019     \tl_if_empty:cTF { \__spintent_sp #1 _read_arg_tl }
5020     {
5021       \bool_if:cTF { \__spintent_sp #1 _concept_bool }

```

```

5022         {
5023             \tl_set:Ne \l__spintent_sfig_intent_arg_tl
5024             {
5025                 -
5026                 \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
5027                 { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
5028                 \l__spintent_sfig_intent_tl :prefix( \l__spintent_sfig_refs_tl )
5029             }
5030         }
5031         {
5032             \tl_set:Ne \l__spintent_sfig_intent_arg_tl { _( \l__spintent_sfig_refs_tl ) }
5033         }
5034     }
5035     {
5036         \tl_set:Ne \l__spintent_sfig_intent_arg_tl
5037         {
5038             -
5039             \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
5040             { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
5041             \l__spintent_sfig_intent_tl :prefix( \l__spintent_sfig_refs_tl )
5042         }
5043     }
5044 }
5045 }

```

(End of definition for `\__spintent_sfig_build_concept_vertices:n`)

`\__spintent_sfig_points_loop:n`

Walks `\__spintent_sfig_pt_seq` a second time, calling `\__spintent_mathmlarg_point:nnn` once per vertex with its `ptn` reference name, looked up under `sp#1`. Receives the short command name in ‘`#1`’.

```

5046 \cs_new_protected:Nn \__spintent_sfig_points_loop:n
5047 {
5048     \int_zero:N \l__spintent_sfig_pt_idx_int
5049     \seq_map_inline:Nn \l__spintent_sfig_pt_seq
5050     {
5051         \int_incr:N \l__spintent_sfig_pt_idx_int
5052         \__spintent_mathmlarg_point:nnn
5053         { pt \int_use:N \l__spintent_sfig_pt_idx_int }
5054         { sp #1 }
5055         { ##1 }
5056     }
5057 }

```

(End of definition for `\__spintent_sfig_points_loop:n`)

`\__spintent_sfig_mathml:n`

Receives the short command name in ‘`#1`’. Branches on whether `\__spintent_geo_luaseta_points_clist` is empty.

Without vertices, `\__spintent_sfig_build_simple:n` builds the content; a `\MathMLintent` wraps it only when `concept` is `true` — with `concept=false` the content is typeset directly, with no intent at all. With vertices, `\__spintent_sfig_build_refs:` then `\__spintent_sfig_build_concept_vertices:n` build the reference list and the concept intent, applied with a single native `\MathMLintent`. Inside, `\__spintent_mathcal:n{⟨A | P⟩}` is silenced with `\__spintent_luafun_mrow_intent_arg:nn` (it would otherwise be claimed by the queue meant for the symbol/vertices, the same ordering issue already found and fixed for `\spangle`’s own symbol); `\__spintent_sfig_symbol_tl` is silenced the same way when non-empty; the sequence is then walked by `\__spintent_sfig_points_loop:n`. All of this sits nested inside a subscript (`\c_math_subscript_token`), reached by the shared `mathml_filter` callback’s recursive lookup through a noad’s sub field, not just its nucleus.

The function `\__spintent_invisible_sep:` precedes both branches (a second sibling so the sole `<mrow>` does not collapse into `<math>`).

```

5058 \cs_new_protected:Nn \__spintent_sfig_mathml:n
5059 {
5060     \__spintent_invisible_sep: % (luamml issues #26)
5061     \clist_if_empty:NTF \l__spintent_geo_luaseta_points_clist
5062     {
5063         \__spintent_sfig_build_simple:n {#1}
5064         \MathMLintent { \l__spintent_sfig_intent_arg_tl }
5065         {
5066             \__spintent_inner_sep:
5067             \str_if_eq:eeTF {#1} { Afig } { \__spintent_mathcal:n { A } } { \__spintent_mathcal:n
5068             \tl_if_empty:NF \l__spintent_geo_final_print_tl
5069             {

```

```

5070         \c_math_subscript_token { \l__spintenz_geo_final_print_tl }
5071     }
5072 }
5073 }
5074 {
5075     \l__spintenz_spfig_build_refs:
5076     \l__spintenz_spfig_build_concept_vertices:n {#1}
5077     \MathMLintent { \l__spintenz_spfig_intent_arg_tl }
5078     {
5079         \l__spintenz_inner_sep:
5080         \l__spintenz_luafun_mrow_intent_arg:nn
5081         { } { intent = ":silent" }
5082     }
5083     \str_if_eq:eeTF {#1} { Afig } { \l__spintenz_mathcal:n { A } } { \l__spintenz_mathcal:n { A } }
5084 }
5085     \c_math_subscript_token
5086     {
5087         \tl_if_empty:NF \l__spintenz_spfig_symbol_tl
5088         {
5089             \l__spintenz_luafun_mrow_intent_arg:nn
5090             { } { intent = ":silent" }
5091             { \tl_use:N \l__spintenz_spfig_symbol_tl }
5092         }
5093         \l__spintenz_spfig_points_loop:n {#1}
5094     }
5095 }
5096 }
5097 }

```

(End of definition for `\l__spintenz_spfig_mathml:n`.)

`\l__spintenz_spfig_exec:nnn` The function `\l__spintenz_spfig_exec:nnn` receives the short name of the command in `#1`, the optional keys in `#2`, and the argument in `#3`, delegating the rendering to `\l__spintenz_spfig_process:nn`.

```

5098 \cs_new_protected:Nn \l__spintenz_spfig_exec:nnn
5099 {
5100     \keys_set:nn { spintenz / sp #1 } { #2 }
5101     \l__spintenz_spfig_process:nn { #1 } { #3 }
5102 }

```

(End of definition for `\l__spintenz_spfig_exec:nnn`.)

`\spAfig` Both public commands `\spAfig` and `\spPfig` enforce math mode, open a local group, and delegate execution to `\l__spintenz_spfig_exec:nnn`.

`\spPfig`

```

5103 \NewDocumentCommand \spAfig { 0{} d() }
5104 {
5105     \mode_if_math:TF
5106     {
5107         \group_begin:
5108         \attribute \l__spintenz_scope_attr_int = 1
5109         \tl_if_novalue:nTF {#2}
5110         { \l__spintenz_spfig_exec:nnn { Afig } { #1 } { } }
5111         { \l__spintenz_spfig_exec:nnn { Afig } { #1 } { #2 } }
5112     }
5113     \group_end:
5114     { \msg_error:nnn { spintenz } { need-math-mode } { \spAfig } }
5115 }
5116 \NewDocumentCommand \spPfig { 0{} d() }
5117 {
5118     \mode_if_math:TF
5119     {
5120         \group_begin:
5121         \attribute \l__spintenz_scope_attr_int = 1
5122         \tl_if_novalue:nTF {#2}
5123         { \l__spintenz_spfig_exec:nnn { Pfig } { #1 } { } }
5124         { \l__spintenz_spfig_exec:nnn { Pfig } { #1 } { #2 } }
5125     }
5126     \group_end:
5127     { \msg_error:nnn { spintenz } { need-math-mode } { \spPfig } }
5128 }

```

(End of definition for `\spAfig` and `\spPfig`. These functions are documented on page 26.)

11.45 The commands \sprad and \spdiam

The user-level commands `\sprad` and `\spdiam` provide a semantic interface for the radius and diameter of a circle.

11.45.1 Definition of variables for \sprad and \spdiam

The variables `\l__spinttent_sp_rad_diam_letra_tl` and `\l__spinttent_sp_rad_diam_word_tl` are scratch variables used by `\__spinttent_sp_rad_diam_print:n`. `\l__spinttent_sp_rad_diam_read_arg_full_tl` is used by `\__spinttent_sp_rad_diam_read_arg_print:n` to hold the text to voice when `read-arg` is given.

```
5129 \tl_new:N \l__spinttent_sp_rad_diam_letra_tl
5130 \tl_new:N \l__spinttent_sp_rad_diam_word_tl
5131 \tl_new:N \l__spinttent_sp_rad_diam_read_arg_full_tl
```

(End of definition for `\l__spinttent_sp_rad_diam_letra_tl`, `\l__spinttent_sp_rad_diam_word_tl`, and `\l__spinttent_sp_rad_diam_read_arg_full_tl`.)

11.45.2 Definition of keys for \sprad and \spdiam

We define the shared `(keys)` `read-cmd`, `upper-case`, `sub-index`, `read-sub`, `read-arg`, `concept`, `silent` and `print-sym`.

```
5132 \clist_map_inline:nn { rad , diam }
5133 {
5134   \keys_define:nn { spinttent / sp #1 }
5135   {
5136     read-cmd .choices:nn = { school , mathcat }
5137     {
5138       \int_case:nn { \l_keys_choice_int }
5139       {
5140         {1} { \str_set:cn { l__spinttent_sp #1 _mode_str } { school } }
5141         {2} { \str_set:cn { l__spinttent_sp #1 _mode_str } { mathcat } }
5142       }
5143     },
5144     read-cmd .initial:n = school,
5145     read-cmd .value_required:n = true,
5146     read-cmd / unknown .code:n =
5147       \msg_error:nneee { spinttent } { unknown-choice }
5148       { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
5149     upper-case .bool_set:c = { l__spinttent_sp #1 _upper_case_bool },
5150     upper-case .initial:n = false,
5151     upper-case .value_required:n = true,
5152     sub-index .code:n =
5153       { \str_set:cn { l__spinttent_sp #1 _sub_index_str } {##1} },
5154     sub-index .initial:n = ,
5155     sub-index .value_required:n = true,
5156     read-sub .bool_set:c = { l__spinttent_sp #1 _read_sub_bool },
5157     read-sub .initial:n = false,
5158     read-sub .value_required:n = true,
5159     read-arg .code:n =
5160       { \__spinttent_sanitiz_eaffix:cn { l__spinttent_sp #1 _read_arg_tl } {##1} },
5161     read-arg .initial:n = ,
5162     read-arg .value_required:n = true,
5163     concept .bool_set:c = { l__spinttent_sp #1 _concept_bool },
5164     concept .initial:n = true,
5165     concept .value_required:n = true,
5166     silent .bool_set:c = { l__spinttent_sp #1 _silent_bool },
5167     silent .initial:n = false,
5168     silent .value_required:n = true,
5169     print-sym .bool_set:c = { l__spinttent_sp #1 _print_sym_bool },
5170     print-sym .initial:n = false,
5171     print-sym .value_required:n = true,
5172     unknown .code:n =
5173       {
5174         \str_set:Nn \l__spinttent_command_name_str { sp #1 }
5175         \__spinttent_unknown_keys_cmd:n {##1}
5176       },
5177   }
5178 }
```

🌱 The keys `print-sym` are defined for both commands for simplicity, but has no effect on `\sprad`.

(End of definition for `read-cmd` and others.)

11.45.3 Internal functions for \sprad and \spdiam

_spinttent_sp_rad_diam_print:n

The function `\_spinttent_sp_rad_diam_print:n` sets `\_l\_spinttent_sp_rad_diam_letra_tl` to the bare letter — `r/R` for `\sprad`, `d/D` for `\spdiam` — per upper-case, and, for `\spdiam` with `print-sym true`, silently typesets `\diameter` first. Receives the short command name in `'#1'` (`rad/diam`).

```

5179 \cs_new_protected:Nn \_spinttent_sp_rad_diam_print:n
5180 {
5181   \bool_if:cTF { \_l\_spinttent_sp #1_upper_case_bool }
5182   {
5183     \str_if_eq:eeTF {#1} { rad }
5184     { \tl_set:Nn \_l\_spinttent_sp_rad_diam_letra_tl { R } }
5185     { \tl_set:Nn \_l\_spinttent_sp_rad_diam_letra_tl { D } }
5186   }
5187   {
5188     \str_if_eq:eeTF {#1} { rad }
5189     { \tl_set:Nn \_l\_spinttent_sp_rad_diam_letra_tl { r } }
5190     { \tl_set:Nn \_l\_spinttent_sp_rad_diam_letra_tl { d } }
5191   }
5192   \_spinttent_invisible_sep:
5193   \str_if_eq:eeT {#1} { diam }
5194   {
5195     \bool_if:NT \_l\_spinttent_spdiam_print_sym_bool
5196     { \MathMLintent { :silent } { \diameter } }
5197   }

```

With `silent true`, the intent is `_` — the letter alone through the shared queue when there is no sub-index, or letter and subscript together in one native `\MathMLintent` (subscript silenced) when there is.

```

5198   \bool_if:cTF { \_l\_spinttent_sp #1_silent_bool }
5199   {
5200     \str_if_empty:cTF { \_l\_spinttent_sp #1_sub_index_str }
5201     {
5202       \_spinttent_luafun_mrow_intent_arg:nn
5203       { } { intent = "_-" }
5204       { \_l\_spinttent_sp_rad_diam_letra_tl }
5205     }
5206     {
5207       \MathMLintent { _- }
5208       {
5209         \_spinttent_inner_sep:
5210         \_l\_spinttent_sp_rad_diam_letra_tl
5211         \c_math_subscript_token
5212         {
5213           \MathMLintent { :silent }
5214           { \str_use:c { \_l\_spinttent_sp #1_sub_index_str } }
5215         }
5216       }
5217     }
5218   }

```

Otherwise, with `concept true` (the default), it verbalizes radio/diámetro (`read-arg` replaces this word when given), with `sub-index`, if present, appended right after (`read-sub` choosing sub-`<value>` or `<value>` alone) — the letter itself is never read here. With no `sub-index`, the shared queue tags the letter alone; with one, letter and subscript are wrapped together in one native `\MathMLintent` for `read-cmd=school` (subscript silenced, already spoken), or the shared queue tags the letter alone with the subscript left bare for `read-cmd=mathcat`.

```

5219   {
5220     \tl_if_empty:cTF { \_l\_spinttent_sp #1_read_arg_tl }
5221     {
5222       \bool_if:cTF { \_l\_spinttent_sp #1_concept_bool }
5223       {
5224         \str_if_eq:eeTF {#1} { rad }
5225         { \tl_set:Nn \_l\_spinttent_sp_rad_diam_word_tl { radio } }
5226         { \tl_set:Nn \_l\_spinttent_sp_rad_diam_word_tl { diámetro } }
5227         \str_if_empty:cTF { \_l\_spinttent_sp #1_sub_index_str }
5228         {
5229           \_spinttent_luafun_mrow_intent_arg:nn
5230           { } { intent = "_ \_l\_spinttent_sp_rad_diam_word_tl :nofix" }
5231           { \_l\_spinttent_sp_rad_diam_letra_tl }
5232         }
5233         {
5234           \str_if_eq:eeTF
5235           { \str_use:c { \_l\_spinttent_sp #1_mode_str } } { school }

```

```

5236 {
5237   \MathMLintent
5238   {
5239     _ \l__spintent_sp_rad_diam_word_tl -
5240     \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
5241     \str_use:c { l__spintent_sp #1 _sub_index_str }
5242   }
5243   {
5244     \__spintent_inner_sep:
5245     \l__spintent_sp_rad_diam_letra_tl
5246     \c_math_subscript_token
5247     {
5248       \MathMLintent { :silent }
5249       { \str_use:c { l__spintent_sp #1 _sub_index_str } }
5250     }
5251   }
5252 }
5253 {
5254   \__spintent_luafun_mrow_intent_arg:nn
5255   { } { intent = "_ \l__spintent_sp_rad_diam_word_tl :nofix" }
5256   { \l__spintent_sp_rad_diam_letra_tl }
5257   \c_math_subscript_token
5258   { \str_use:c { l__spintent_sp #1 _sub_index_str } }
5259 }
5260 }
5261 }

```

With `concept=false`, the letter itself is read instead — but only when `read-cmd` is `school`, built the same way from the letter (with `sub-index`, letter and subscript wrapped together, subscript silenced). With `read-cmd mathcat`, the intent is left empty: MathCAT applies its own rules to the letter and its subscript, unmarked by us.

```

5262 {
5263   \str_if_empty:cTF { l__spintent_sp #1 _sub_index_str }
5264   {
5265     \str_if_eq:eeTF
5266     { \str_use:c { l__spintent_sp #1 _mode_str } } { school }
5267     {
5268       \__spintent_luafun_mrow_intent_arg:nn
5269       { } { intent = "_ \l__spintent_sp_rad_diam_letra_tl -" }
5270       { \l__spintent_sp_rad_diam_letra_tl }
5271     }
5272     { \l__spintent_sp_rad_diam_letra_tl }
5273   }
5274   {
5275     \str_if_eq:eeTF
5276     { \str_use:c { l__spintent_sp #1 _mode_str } } { school }
5277     {
5278       \MathMLintent
5279       {
5280         _ \l__spintent_sp_rad_diam_letra_tl -
5281         \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
5282         \str_use:c { l__spintent_sp #1 _sub_index_str }
5283       }
5284       {
5285         \__spintent_inner_sep:
5286         \l__spintent_sp_rad_diam_letra_tl
5287         \c_math_subscript_token
5288         {
5289           \MathMLintent { :silent }
5290           { \str_use:c { l__spintent_sp #1 _sub_index_str } }
5291         }
5292       }
5293     }
5294     {
5295       \l__spintent_sp_rad_diam_letra_tl
5296       \c_math_subscript_token
5297       { \str_use:c { l__spintent_sp #1 _sub_index_str } }
5298     }
5299   }
5300 }
5301 }

```

```

5302         { \l__spintent_sp_rad_diam_read_arg_print:n {#1} }
5303     }
5304 }

```

(End of definition for \l__spintent_sp_rad_diam_print:n.)

\l__spintent_sp_rad_diam_read_arg_print:n

With `read-arg` given, `sub-index/read-sub` are ignored for the *spoken* content (the subscript, if any, still displays, silenced). With `concept true` (the default), `\l__spintent_sp_rad_diam_read_arg_full_tl` becomes `radio/diámetro` followed by `read-arg`'s text; with `false`, `read-arg`'s text alone. Combined natively, regardless of `read-cmd`: with no `sub-index`, the shared queue tags the letter alone; with one, letter and subscript are wrapped together in one native `\MathMLintent`.

```

5305 \cs_new_protected:Nn \l__spintent_sp_rad_diam_read_arg_print:n
5306 {
5307     \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
5308     {
5309         \str_if_eq:eeTF {#1} { rad }
5310         {
5311             \tl_set:Nc \l__spintent_sp_rad_diam_read_arg_full_tl
5312             { radio- \tl_use:c { \l__spintent_sp #1 _read_arg_tl } }
5313         }
5314         {
5315             \tl_set:Nc \l__spintent_sp_rad_diam_read_arg_full_tl
5316             { diámetro- \tl_use:c { \l__spintent_sp #1 _read_arg_tl } }
5317         }
5318     }
5319     {
5320         \tl_set_eq:Nc \l__spintent_sp_rad_diam_read_arg_full_tl
5321         { \l__spintent_sp #1 _read_arg_tl }
5322     }
5323     \str_if_empty:cTF { \l__spintent_sp #1 _sub_index_str }
5324     {
5325         \l__spintent_luafun_mrow_intent_arg:nn
5326         { } { intent = "_ \l__spintent_sp_rad_diam_read_arg_full_tl :nofix" }
5327         { \l__spintent_sp_rad_diam_letra_tl }
5328     }
5329     {
5330         \MathMLintent
5331         { _ \l__spintent_sp_rad_diam_read_arg_full_tl :nofix }
5332         {
5333             \l__spintent_inner_sep:
5334             \l__spintent_sp_rad_diam_letra_tl
5335             \c_math_subscript_token
5336             {
5337                 \MathMLintent { :silent }
5338                 { \str_use:c { \l__spintent_sp #1 _sub_index_str } }
5339             }
5340         }
5341     }
5342 }

```

(End of definition for \l__spintent_sp_rad_diam_read_arg_print:n.)

\l__spintent_sp_rad_diam_exec:nn

The function `\l__spintent_sp_rad_diam_exec:nn` receives the *short name* of the command passed in ‘#1’ and the optional (*keys*) passed in ‘#2’ then delegating the rendering to `\l__spintent_sp_rad_diam_print:n`.

```

5343 \cs_new_protected:Nn \l__spintent_sp_rad_diam_exec:nn
5344 {
5345     \keys_set:nn { spintent / sp #1 } {#2}
5346     \l__spintent_sp_rad_diam_print:n {#1}
5347 }

```

(End of definition for \l__spintent_sp_rad_diam_exec:nn.)

`\sprad`
`\spdiam`

Both public commands `\sprad` and `\spdiam` enforces *math mode*, opens a local group, and delegates execution to `\l__spintent_sp_rad_diam_exec:nn`.

```

5348 \NewDocumentCommand \sprad { O{} }
5349 {
5350     \mode_if_math:TF
5351     {
5352         \group_begin:
5353         \attribute \l__spintent_scope_attr_int = 1
5354         \l__spintent_sp_rad_diam_exec:nn { rad } {#1}

```

```

5355     \group_end:
5356   }
5357   { \msg_error:nnn { spintent } { need-math-mode } { \sprad } }
5358 }
5359 \NewDocumentCommand \spdiam { 0{ } }
5360 {
5361   \mode_if_math:TF
5362   {
5363     \group_begin:
5364     \attribute \l__spintent_scope_attr_int = 1
5365     \__spintent_sp_rad_diam_exec:nn { diam } {#1}
5366     \group_end:
5367   }
5368   { \msg_error:nnn { spintent } { need-math-mode } { \spdiam } }
5369 }

```

(End of definition for `\sprad` and `\spdiam`. These functions are documented on page 29.)

11.46 The command `\spcirc`

The user command `\spcirc` provides a semantic interface for a circle or a circumference, using the delimited-optional argument syntax `d()`. The argument holds a required center point and an optional radius, separated by a comma. The radius is never required to declare its own kind via a key: its shape alone decides whether it is a pure number, a measure (number + unit), a bare letter, or a two-letter segment — reusing, respectively, `\spnum`, `\spunit`, and `\spside`/`\spmside` directly as sub-commands, rather than reimplementing any of their parsing.

11.46.1 Definition of variables for `\spcirc`

Shared outputs of `\__spintent_luafun_geo_circ_parse_and_set:n`. The variable `\l__spintent_geo_luaset_circ_radio_type_str` holds one of `number`, `measure`, `label`, `segment`, or an empty string when no radius was given. `\l__spintent_geo_luaset_circ_radio_raw_str` holds the corresponding raw text, ready to be forwarded to the sub-command that renders it.

```

5370 \str_new:N \l__spintent_geo_luaset_circ_radio_type_str
5371 \str_new:N \l__spintent_geo_luaset_circ_radio_raw_str

```

(End of definition for `\l__spintent_geo_luaset_circ_radio_type_str` and `\l__spintent_geo_luaset_circ_radio_raw_str`.)

The variable `\l__spintent_spcirc_mode_str` stores the value set by the key `read-cmd`; the variable `\l__spintent_spcirc_prefix_tl` stores the value set by the key `prefix`. The variable `\l__spintent_spcirc_print_sym_str` stores the value set by the key `print-sym` (`usual` or `odot`). The variable `\l__spintent_spcirc_read_sty_str` stores the value set by the key `read-sty` (`circun` or `circle`). The variable `\l__spintent_spcirc_radio_sty_str` stores the value set by the key `radio-sty`, only meaningful when the radius is a segment. The variable `\l__spintent_spcirc_sub_index_str` stores the value set by the key `sub-index`. The variable `\l__spintent_spcirc_articulo_str` is set internally by `\__spintent_spcirc_word_and_article:` to `la` or `el`, matching the grammatical gender of the chosen word.

```

5372 \str_new:N \l__spintent_spcirc_mode_str
5373 \tl_new:N \l__spintent_spcirc_prefix_tl
5374 \str_new:N \l__spintent_spcirc_print_sym_str
5375 \str_new:N \l__spintent_spcirc_read_sty_str
5376 \str_new:N \l__spintent_spcirc_radio_sty_str
5377 \str_new:N \l__spintent_spcirc_sub_index_str
5378 \str_new:N \l__spintent_spcirc_articulo_str
5379 \tl_new:N \l__spintent_spcirc_read_arg_tl

```

(End of definition for `\l__spintent_spcirc_mode_str` and others.)

`\l__spintent_spcirc_word_tl` Scratch variables used by `\__spintent_spcirc_process:n` to assemble the chosen word (`circunferencia/círculo` or the `read-arg` override) and the final literal `intent` string.

```

5380 \tl_new:N \l__spintent_spcirc_word_tl

```

(End of definition for `\l__spintent_spcirc_word_tl`.)

11.46.2 Definition of error messages for `\spcirc`

`spcirc-invalid-arg`

Raised when the argument does not match a single center point, optionally followed by a comma and a valid radius shape.

```

5381 \msg_new:nnnn { spintent } { spcirc-invalid-arg }
5382 {
5383   Invalid~argument~format~in~\token_to_str:N \spcirc:~'!#1'~\msg_line_context:.
5384 }
5385 {
5386   See~the~documentation~of~\token_to_str:N \spcirc~for~the~accepted~argument~forms.
5387 }

```

(End of definition for `spcirc-invalid-arg`.)

11.46.3 Definition of keys for \spcirc

The key `print-sym` chooses the visual symbol: `usual` for a plain C, `odot` (default) for `\odot`. The key `read-sty` chooses the verbalized word and its article: `circun` (default) for «*la circunferencia*», `circle` for «*el círculo*». The key `radio-sty` chooses, only when the radius is a two-letter segment, whether it is rendered as the *object* (`side`, via `\spside`) or its *measure* (`mside`, default, via `\spmside`). The key `sub-index` sets a subscript on the symbol; the key `read-sub` controls whether it is read with the word «sub» or as a plain number, matching the same convention as `\spAfig`.

```

5388 \keys_define:nn { spintent / spcirc }
5389 {
5390   read-cmd      .choices:nn = { school , mathcat }
5391   {
5392     \int_case:nn { \l_keys_choice_int }
5393     {
5394       {1}
5395       {
5396         \str_set:Nn \l__spintent_spcirc_mode_str { school }
5397         \keys_set:nn { spintent / spside } { read-cmd = school }
5398         \keys_set:nn { spintent / spmside } { read-cmd = school }
5399       }
5400     } {2}
5401     {
5402       \str_set:Nn \l__spintent_spcirc_mode_str { mathcat }
5403       \keys_set:nn { spintent / spside } { read-cmd = mathcat }
5404       \keys_set:nn { spintent / spmside } { read-cmd = mathcat }
5405     }
5406   }
5407 },
5408 read-cmd      .initial:n = school,
5409 read-cmd      .value_required:n = true,
5410 prefix        .code:n =
5411 { \__spintent_sanitizate_affix:Nn \l__spintent_spcirc_prefix_tl {#1} },
5412 prefix        .initial:n = ,
5413 prefix        .value_required:n = true,
5414 print-sym     .choices:nn = { usual , odot }
5415 {
5416   \int_case:nn { \l_keys_choice_int }
5417   {
5418     {1} { \str_set:Nn \l__spintent_spcirc_print_sym_str { usual } }
5419     {2} { \str_set:Nn \l__spintent_spcirc_print_sym_str { odot } }
5420   }
5421 },
5422 print-sym     .initial:n = odot,
5423 print-sym     .value_required:n = true,
5424 read-sty      .choices:nn = { circun , circle }
5425 {
5426   \int_case:nn { \l_keys_choice_int }
5427   {
5428     {1} { \str_set:Nn \l__spintent_spcirc_read_sty_str { circun } }
5429     {2} { \str_set:Nn \l__spintent_spcirc_read_sty_str { circle } }
5430   }
5431 },
5432 read-sty      .initial:n = circun,
5433 read-sty      .value_required:n = true,
5434 diameter      .bool_set:N = \l__spintent_spcirc_diameter_bool,
5435 diameter      .initial:n = false,
5436 diameter      .value_required:n = true,
5437 radio-sty     .choices:nn = { side , mside }
5438 {
5439   \int_case:nn { \l_keys_choice_int }
5440   {
5441     {1} { \str_set:Nn \l__spintent_spcirc_radio_sty_str { side } }
5442     {2} { \str_set:Nn \l__spintent_spcirc_radio_sty_str { mside } }
5443   }
5444 },
5445 radio-sty     .initial:n = mside,
5446 radio-sty     .value_required:n = true,
5447 sub-index     .code:n =
5448 { \str_set:Nn \l__spintent_spcirc_sub_index_str {#1} },
5449 sub-index     .initial:n = ,
5450 sub-index     .value_required:n = true,

```

```

5451   read-sub      .bool_set:N = \l__spintenc_spcirc_read_sub_bool,
5452   read-sub      .initial:n = false,
5453   read-sub      .value_required:n = true,
5454   read-arg      .code:n =
5455     { \__spintenc_sanitizc_affix:Nn \l__spintenc_spcirc_read_arg_tl {#1} },
5456   read-arg      .initial:n = ,
5457   read-arg      .value_required:n = true,
5458   silent        .bool_set:N = \l__spintenc_spcirc_silent_bool,
5459   silent        .initial:n = false,
5460   silent        .value_required:n = true,
5461   concept       .bool_set:N = \l__spintenc_spcirc_concept_bool,
5462   concept       .initial:n = true,
5463   concept       .value_required:n = true,
5464   spside        .meta:nn = { spintenc / spside } { #1 },
5465   spside        .value_required:n = true,
5466   spmside       .meta:nn = { spintenc / spmside } { #1 },
5467   spmside       .value_required:n = true,
5468   unknown       .code:n =
5469     {
5470       \str_set:Nn \l__spintenc_command_name_str { spcirc }
5471       \__spintenc_unknown_keys_cmd:n {#1}
5472     },
5473   }

```

(End of definition for read-cmd and others.)

11.46.4 Internal functions for \spcirc

```

\l__spintenc_spcirc_has_arg_bool
\l__spintenc_spcirc_intent_tl
\l__spintenc_spcirc_intent_arg_tl
\l__spintenc_spcirc_connector_tl
\l__spintenc_spcirc_connector_arg_tl
\l__spintenc_spcirc_radio_arg_tl

```

The variable `\l__spintenc_spcirc_has_arg_bool` records whether the `d()` argument was given at all (`\spcirc` with no `(...)` at all has neither a center nor a radius). `\l__spintenc_spcirc_intent_tl` holds the word (circunferencia or círculo, with an optional *sub-index* suffix) paired with the symbol. The variable `\l__spintenc_spcirc_connector_tl` holds `de-centro-en`, paired with the (center) group; the center is read through `\__spintenc_mathmlarg_point:nnn`, referenced as `$pt`, not concatenated into either word — it can be subscripted or primed (e.g. `O_{1}`), as in the Euler circle) without breaking anything. The `_arg_tl` variables hold each word's final wrapped form; a third, `\l__spintenc_spcirc_radio_arg_tl`, is spoken directly on the , delimiter after the center, replacing its own reading rather than pairing with a group.

```

5474 \bool_new:N \l__spintenc_spcirc_has_arg_bool
5475 \tl_new:N \l__spintenc_spcirc_intent_tl
5476 \tl_new:N \l__spintenc_spcirc_intent_arg_tl
5477 \tl_new:N \l__spintenc_spcirc_connector_tl
5478 \tl_new:N \l__spintenc_spcirc_connector_arg_tl
5479 \tl_new:N \l__spintenc_spcirc_radio_arg_tl

```

(End of definition for \l__spintenc_spcirc_has_arg_bool and others.)

`\__spintenc_spcirc_sym:` Typesets `\bigodot` or the plain letter C, according to `print-sym`.

```

5480 \cs_new_protected:Nn \__spintenc_spcirc_sym:
5481 {
5482   \str_if_eq:NnTF \l__spintenc_spcirc_print_sym_str { odot }
5483   {
5484     \group_begin:
5485       \textstyle\bigodot
5486     \group_end:
5487   }
5488   { C }
5489 }

```

(End of definition for __spintenc_spcirc_sym:.)

`\__spintenc_spcirc_word_and_article:`

Sets `\l__spintenc_spcirc_word_tl` to `read-arg` or `circunferencia/círculo` from `read-sty`, and `\l__spintenc_spcirc_articulo_str` to the matching article, which always follows `read-sty` even when `read-arg` overrides the word.

```

5490 \cs_new_protected:Nn \__spintenc_spcirc_word_and_article:
5491 {
5492   \tl_if_empty:NTF \l__spintenc_spcirc_read_arg_tl
5493   {
5494     \str_if_eq:NnTF \l__spintenc_spcirc_read_sty_str { circun }
5495     {
5496       \tl_set:Nn \l__spintenc_spcirc_word_tl { circunferencia }
5497       \str_set:Nn \l__spintenc_spcirc_articulo_str { la }
5498     }
5499     {

```

```

5500         \tl_set:Nn \l__spintenc_spcirc_word_tl { círculo }
5501         \str_set:Nn \l__spintenc_spcirc_articulo_str { el }
5502     }
5503 }
5504 {
5505     \tl_set:Nn \l__spintenc_spcirc_word_tl { \l__spintenc_spcirc_read_arg_tl }
5506     \str_if_eq:VnTF \l__spintenc_spcirc_read_sty_str { circun }
5507     { \str_set:Nn \l__spintenc_spcirc_articulo_str { la } }
5508     { \str_set:Nn \l__spintenc_spcirc_articulo_str { el } }
5509 }
5510 }

```

(End of definition for `\l__spintenc_spcirc_word_and_article:`)

`\l__spintenc_spcirc_radio_print:`

Dispatches on `\l__spintenc_geo_luaset_circ_radio_type_str`, reusing `\spnum/\spunit/\spside/\spmside` directly rather than reimplementing any parsing.

```

5511 \cs_new_protected:Nn \l__spintenc_spcirc_radio_print:
5512 {
5513     \str_case:Vn \l__spintenc_geo_luaset_circ_radio_type_str
5514     {
5515         { number } { \exp_args:Ne \spnum { \l__spintenc_geo_luaset_circ_radio_raw_str } }
5516         { measure } { \exp_args:Ne \spunit { \l__spintenc_geo_luaset_circ_radio_raw_str } }
5517         { label } { \l__spintenc_geo_luaset_circ_radio_raw_str }
5518         { segment }
5519         {
5520             \str_if_eq:VnTF \l__spintenc_spcirc_radio_sty_str { side }
5521             { \exp_args:Ne \spside { \l__spintenc_geo_luaset_circ_radio_raw_str } }
5522             { \exp_args:Ne \spmside { \l__spintenc_geo_luaset_circ_radio_raw_str } }
5523         }
5524     }
5525 }

```

(End of definition for `\l__spintenc_spcirc_radio_print:`)

`\l__spintenc_spcirc_build_concept:`

Builds two words: `\l__spintenc_spcirc_intent_tl` (via `\l__spintenc_spcirc_word_and_article:`, plus an optional `sub-index` suffix), paired with the symbol, and `\l__spintenc_spcirc_connector_tl` (de-centro-en), paired with the (center) group — built even when `\l__spintenc_spcirc_has_arg_bool` is `false`, though `\l__spintenc_spcirc_mathml:` never uses it in that case. Neither carries the center’s own text: it is read separately, by reference, through `\l__spintenc_mathmlarg_point:nnn` — so a subscripted or primed center (e.g. $O_{\{1\}}$, as in the Euler circle) is never an issue.

```

5526 \cs_new_protected:Nn \l__spintenc_spcirc_build_concept:
5527 {
5528     \l__spintenc_spcirc_word_and_article:
5529     \tl_set:Nn \l__spintenc_spcirc_intent_tl { \l__spintenc_spcirc_word_tl }
5530     \str_if_empty:NF \l__spintenc_spcirc_sub_index_str
5531     {
5532         \tl_put_right:Nn \l__spintenc_spcirc_intent_tl
5533         {
5534             -
5535             \bool_if:NT \l__spintenc_spcirc_read_sub_bool { sub - }
5536             \l__spintenc_spcirc_sub_index_str
5537         }
5538     }
5539     \tl_set:Nn \l__spintenc_spcirc_connector_tl { de-centro-en }

```

The key `silent`, when `true`, overrides `\l__spintenc_spcirc_intent_arg_tl`, `\l__spintenc_spcirc_connector_arg_tl` and `\l__spintenc_spcirc_radio_arg_tl` with `_-`, regardless of `read-arg:` a composite center (one with a subscript or a prime) is not reliably silenced by the per-point `:silent` alone.

```

5540     \bool_if:NTF \l__spintenc_spcirc_silent_bool
5541     {
5542         \tl_set:Nn \l__spintenc_spcirc_intent_arg_tl { _- }
5543         \tl_set:Nn \l__spintenc_spcirc_connector_arg_tl { _- }
5544         \tl_set:Nn \l__spintenc_spcirc_radio_arg_tl { _- }
5545     }

```

With `concept true`, `\l__spintenc_spcirc_intent_arg_tl` is `\l__spintenc_spcirc_intent_tl` alone (bare — the group it wraps has nothing to reference, only a silenced symbol), with `prefix` placed after the leading `_` when given; `\l__spintenc_spcirc_connector_arg_tl` is `\l__spintenc_spcirc_connector_tl:pr` and `\l__spintenc_spcirc_radio_arg_tl` — spoken directly on the `,` delimiter, replacing its own reading rather than pairing with any group — is the literal `_y-radio-`, or `_y-diámetro-` when `diameter` is `true`,

read as its own phrase right before `\__spintent_spcirc_radio_print:`'s independent reading follows as the next sibling.

```

5546     {
5547         \bool_if:NTF \l__spintent_spcirc_concept_bool
5548         {
5549             \tl_set:Nx \l__spintent_spcirc_intent_arg_tl
5550             {
5551                 -
5552                 \tl_if_empty:NF \l__spintent_spcirc_prefix_tl
5553                 { \tl_use:N \l__spintent_spcirc_prefix_tl - }
5554                 \l__spintent_spcirc_intent_tl
5555             }
5556             \tl_set:Nx \l__spintent_spcirc_connector_arg_tl
5557             { _ \l__spintent_spcirc_connector_tl :prefix( $ pt ) }
5558             \bool_if:NTF \l__spintent_spcirc_diameter_bool
5559             { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-diámetro- } }
5560             { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-radio- } }
5561         }

```

With `concept false` and `read-arg` empty, the same `_/_($pt)` silencing applies as `silent`; with `read-arg` given, none of the three is silenced instead — the connector and `y-radio/y-diámetro` words are fixed structural pieces, not customized by `read-arg`, but once a real word is being spoken there is nothing left to silence around it (same convention as `\__spintent_spside_process:nn`).

```

5562     {
5563         \tl_if_empty:NTF \l__spintent_spcirc_read_arg_tl
5564         {
5565             \tl_set:Nn \l__spintent_spcirc_intent_arg_tl { _- }
5566             \tl_set:Nx \l__spintent_spcirc_connector_arg_tl { _ ( $ pt ) }
5567             \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _- }
5568         }
5569         {
5570             \tl_set:Nx \l__spintent_spcirc_intent_arg_tl { _ \l__spintent_spcirc_intent_tl }
5571             \tl_set:Nx \l__spintent_spcirc_connector_arg_tl
5572             { _ \l__spintent_spcirc_connector_tl :prefix( $ pt ) }
5573             \bool_if:NTF \l__spintent_spcirc_diameter_bool
5574             { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-diámetro- } }
5575             { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-radio- } }
5576         }
5577     }
5578 }
5579 }

```

(End of definition for `\__spintent_spcirc_build_concept:`)

`\__spintent_spcirc_process:n`

Receives the `d()` argument in `#1`, an explicit empty group when omitted (`\spcirc` itself converts `-NoValue-` to `{ }` before calling `\__spintent_spcirc_exec:nn`) — `\l__spintent_spcirc_has_arg_bool` set to `false` in that case, and `\__spintent_luafun_geo_circ_parse_and_set:n` never called at all — there is nothing to parse. Otherwise calls it, reports `spcirc-invalid-arg` on failure, then delegates to `\__spintent_spcirc_mathml:`, which calls `\__spintent_spcirc_build_concept:` itself.

```

5580 \cs_new_protected:Nn \__spintent_spcirc_process:n
5581 {
5582     \tl_if_empty:NTF {#1}
5583     {
5584         \bool_set_false:N \l__spintent_spcirc_has_arg_bool
5585     }
5586     {
5587         \bool_set_true:N \l__spintent_spcirc_has_arg_bool
5588         \__spintent_luafun_geo_circ_parse_and_set:n { \exp_not:n {#1} }
5589         \str_if_eq:VnT \l__spintent_geo_luaseta_error_str { true }
5590         {
5591             \msg_error:nne { spintent } { spcirc-invalid-arg } { \exp_not:n {#1} }
5592         }
5593     }
5594     \__spintent_spcirc_mathml:
5595 }

```

(End of definition for `\__spintent_spcirc_process:n`)

`\__spintent_spcirc_mathml:`

Calls `\__spintent_spcirc_build_concept:`, then wraps the symbol (`\l__spintent_spcirc_intent_arg_tl`) in its own native `\MathMLintent` — this alone is the whole result when `\l__spintent_spcirc_has_arg_bool` is `false` (`\spcirc` with no `(...)` at all: no center, no radius).

Otherwise, the `(center)` group (`\l__spintenc_spcirc_connector_arg_tl`) follows as a separate sibling, never nested inside the symbol's own `\MathMLintent` — or MathCAT falls back to its own structural reading of the underlying `<mover>/<msub>` instead, the same problem already found for `\spangle`'s symbol and `\sparc`'s accent. When a radius is present, the `,` delimiter after the center carries `\l__spintenc_spcirc_radio_arg_tl` directly (replacing its own reading, not paired with any group), and `\l__spintenc_spcirc_radio_print:` follows as an ordinary sibling right after — except with `silent=true`, when it is wrapped in a literal `_` — `\MathMLintent`, which silences whatever it produces. Otherwise, wrapping it in another `\MathMLintent` would lose the reading of whatever it produces (itself a native `\MathMLintent`, from `\spnum/\spunit`, nested inside another one does not compose).

The symbol and, when `sub-index` is given, its subscript number are each silenced with `\l__spintenc_luafun_mrow_intent` *separately* — one queue entry per leaf. The two are reached differently by the shared `mathml_filter` callback's recursive lookup (the symbol via `nucleus`, the subscript via `sub`), so a single entry covering both leaves nothing registered for the second one, which the recursion then assigns whatever comes next in the queue instead — shifting every claim after it by one. The `(` delimiter is silenced the same way, but *without* the usual `{...}` around the character: `luamml` only keeps a delimiter's own math class (giving `<mo>` instead of `<mi>`, needed for MathCAT to read the group at all) when it is not itself sitting inside a brace group.

The final `)` — following `\l__spintenc_spcirc_radio_print:`, whose own output the shared queue does not reliably skip over — is typeset as `\rparen` inside its own native `\MathMLintent{silent}` instead of a bare `)`: `\rparen` keeps its own math class through the wrapping, unlike a bare delimiter character, and this sits outside the shared queue entirely, so nothing in between can claim an entry meant for it.

The center itself is read through `\l__spintenc_mathmlarg_point:nnn`, referenced as `$pt`.

```

5596 \cs_new_protected:Nn \l__spintenc_spcirc_mathml:
5597 {
5598   \l__spintenc_invisible_sep: % (luamml issues #26)
5599   \l__spintenc_spcirc_build_concept:
5600   \MathMLintent { \l__spintenc_spcirc_intent_arg_tl }
5601   {
5602     \l__spintenc_inner_sep:
5603     \l__spintenc_luafun_mrow_intent_arg:nn
5604     { } { intent = ":silent" }
5605     { \l__spintenc_spcirc_sym: }
5606     \str_if_empty:NF \l__spintenc_spcirc_sub_index_str
5607     {
5608       \c_math_subscript_token
5609       {
5610         \l__spintenc_luafun_mrow_intent_arg:nn
5611         { } { intent = ":silent" }
5612         { \l__spintenc_spcirc_sub_index_str }
5613       }
5614     }
5615   }
5616   \bool_if:NT \l__spintenc_spcirc_has_arg_bool
5617   {
5618     \str_if_empty:NTF \l__spintenc_geo_luaset_circ_radio_type_str
5619     {
5620       \MathMLintent { \l__spintenc_spcirc_connector_arg_tl }
5621       {
5622         \l__spintenc_inner_sep:
5623         \l__spintenc_luafun_mrow_intent_arg:nn { } { intent = ":silent" } (
5624         \l__spintenc_mathmlarg_point:nnV { pt } { spcirc } \l__spintenc_geo_luaset_points:
5625         \l__spintenc_luafun_mrow_intent_arg:nn { } { intent = ":silent" } )
5626       }
5627     }
5628     {
5629       \MathMLintent { \l__spintenc_spcirc_connector_arg_tl }
5630       {
5631         \l__spintenc_inner_sep:
5632         \l__spintenc_luafun_mrow_intent_arg:nn { } { intent = ":silent" } (
5633         \l__spintenc_mathmlarg_point:nnV { pt } { spcirc } \l__spintenc_geo_luaset_points:
5634       )
5635       \MathMLintent { \l__spintenc_spcirc_radio_arg_tl } { , }
5636       \bool_if:NTF \l__spintenc_spcirc_silent_bool
5637       {
5638         \MathMLintent { _- }
5639         {
5640           \l__spintenc_inner_sep:
5641           \l__spintenc_spcirc_radio_print:
5642         }
5643       }

```

```

5644         { \__spintent_spcirc_radio_print: }
5645         \MathMLintent { :silent } { \rparen }
5646     }
5647 }
5648 }

```

(End of definition for __spintent_spcirc_mathml:.)

__spintent_spcirc_exec:nn

The function `\__spintent_spcirc_exec:nn` receives the optional keys in **#1** and the argument in **#2**, delegating the rendering to `\__spintent_spcirc_print:n`.

```

5649 \cs_new_protected:Nn \__spintent_spcirc_exec:nn
5650 {
5651     \keys_set:nn { spintent / spcirc } { #1 }
5652     \__spintent_spcirc_process:n { #2 }
5653 }

```

(End of definition for __spintent_spcirc_exec:nn.)

\spcirc

The public command `\spcirc` enforces math mode, opens a local group, and delegates execution to `\__spintent_spcirc_exec:nn`. It follows the same `d()`-argument pattern as `\spmcd/\spmcn` and `\spAfig/\spPfig`; an omitted radius forwards an explicit empty group rather than the `-NoValue-` marker.

```

5654 \NewDocumentCommand \spcirc { 0{} d() }
5655 {
5656     \mode_if_math:TF
5657     {
5658         \group_begin:
5659         \attribute \__spintent_scope_attr_int = 1
5660         \tl_if_novalue:nTF {#2}
5661         { \__spintent_spcirc_exec:nn { #1 } { } }
5662         { \__spintent_spcirc_exec:nn { #1 } { #2 } }
5663         \group_end:
5664     }
5665     { \msg_error:nnn { spintent } { need-math-mode } { \spcirc } }
5666 }

```

(End of definition for `\spcirc`. This function is documented on page 30.)

11.47 The commands `\spAcirc` and `\spPcirc`

The user commands `\spAcirc` and `\spPcirc` typeset the area and the perimeter of a circle or a circumference. The argument follows `\spcirc`'s own point-and-radius grammar for a real center (with an optional radius); when no comma is present and no valid center is found, it falls back to a bare number/letter shortcut, reusing the same Lua entry point as `\spAfig/\spPfig`. A table letter matched by that fallback is rejected, since it has no meaning for a circle.

11.47.1 Definition of variables for `\spAcirc` and `\spPcirc`

```

\__spintent_spAcirc_mode_str
\__spintent_spPcirc_mode_str
\__spintent_spAcirc_print_sym_str
\__spintent_spPcirc_print_sym_str
\__spintent_spAcirc_read_sty_str
\__spintent_spPcirc_read_sty_str
\__spintent_spAcirc_radio_sty_str
\__spintent_spPcirc_radio_sty_str
\__spintent_spAcirc_sub_index_str
\__spintent_spPcirc_sub_index_str
\__spintent_spAcirc_prefix_tl
\__spintent_spPcirc_prefix_tl
\__spintent_spAcirc_read_arg_tl
\__spintent_spPcirc_read_arg_tl
\__spintent_geo_circfig_mode_str

```

These variables store the value set by each corresponding key: `read-cmd`, `print-sym`, `read-sty`, `radio-sty`, `sub-index`, `prefix` and `read-arg`, respectively.

```

5667 \clist_map_inline:nn { Acirc , Pcirc }
5668 {
5669     \str_new:c { \__spintent_sp #1 _mode_str }
5670     \str_new:c { \__spintent_sp #1 _print_sym_str }
5671     \str_new:c { \__spintent_sp #1 _read_sty_str }
5672     \str_new:c { \__spintent_sp #1 _radio_sty_str }
5673     \str_new:c { \__spintent_sp #1 _sub_index_str }
5674     \tl_new:c { \__spintent_sp #1 _prefix_tl }
5675     \tl_new:c { \__spintent_sp #1 _read_arg_tl }
5676 }

```

(End of definition for `\__spintent_spAcirc_mode_str` and others.)

The variable `\__spintent_geo_circfig_mode_str` stores whether the argument was resolved as a real circle (`circle`) or as the bare number/letter shortcut (bare).

```

5677 \str_new:N \__spintent_geo_circfig_mode_str

```

(End of definition for `\__spintent_geo_circfig_mode_str`.)

11.47.2 Definition of error messages for \spAcirc and \spPcirc

spAcirc-invalid-arg
spPcirc-invalid-arg

The error message shown when the argument does not match any of the accepted forms: empty, a center point (optionally followed by a comma and a radius), or a bare number/letter.

```

5678 \clist_map_inline:nn { Acirc , Pcirc }
5679 {
5680   \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
5681   {
5682     Invalid~argument~in~##1:~'##2'~\msg_line_context:.
5683   }
5684   {
5685     See~the~documentation~of~##1~for~the~accepted~argument.
5686   }

```

(End of definition for spAcirc-invalid-arg and spPcirc-invalid-arg.)

11.47.3 Definition of keys for \spAcirc and \spPcirc

read-cmd
prefix
print-sym
read-sty
concept
sub-index
read-sub
read-arg
silent
unknown

We define the shared (*keys*) read-cmd, prefix, print-sym, read-sty, concept, sub-index, read-sub, read-arg and silent.

```

5687 \keys_define:nn { spintent / sp #1 }
5688 {
5689   read-cmd .choices:nn = { school , mathcat }
5690   {
5691     \int_case:nn { \l_keys_choice_int }
5692     {
5693       {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
5694       {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
5695     }
5696   },
5697   read-cmd .initial:n = school,
5698   read-cmd .value_required:n = true,
5699   read-cmd / unknown .code:n =
5700     \msg_error:nneee { spintent } { unknown-choice }
5701     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
5702   prefix .code:n =
5703     { \__spintent_sanitizet_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
5704   prefix .initial:n = ,
5705   prefix .value_required:n = true,
5706   print-sym .choices:nn = { usual , odot }
5707   {
5708     \int_case:nn { \l_keys_choice_int }
5709     {
5710       {1} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { usual } }
5711       {2} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { odot } }
5712     }
5713   },
5714   print-sym .initial:n = odot,
5715   print-sym .value_required:n = true,
5716   print-sym / unknown .code:n =
5717     \msg_error:nneee { spintent } { unknown-choice }
5718     { print-sym } { usual,~odot } { \exp_not:n {##1} },
5719   read-sty .choices:nn = { circun , circle }
5720   {
5721     \int_case:nn { \l_keys_choice_int }
5722     {
5723       {1} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { circun } }
5724       {2} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { circle } }
5725     }
5726   },
5727   read-sty .initial:n = circun,
5728   read-sty .value_required:n = true,
5729   concept .bool_set:c = { l__spintent_sp #1 _concept_bool },
5730   concept .initial:n = true,
5731   concept .value_required:n = true,
5732   sub-index .code:n =
5733     { \str_set:cn { l__spintent_sp #1 _sub_index_str } {##1} },
5734   sub-index .initial:n = ,
5735   sub-index .value_required:n = true,
5736   read-sub .bool_set:c = { l__spintent_sp #1 _read_sub_bool },
5737   read-sub .initial:n = false,
5738   read-sub .value_required:n = true,

```

```

5739     read-arg             .code:n =
5740     { \__spintent_sanitiz
5741     read-arg             .initial:n = ,
5742     read-arg             .value_required:n = true,
5743     silent               .bool_set:c = { \__spintent_sp #1 _silent_bool },
5744     silent               .initial:n = false,
5745     silent               .value_required:n = true,
5746     unknown              .code:n =
5747     {
5748         \str_set:Nn \__spintent_command_name_str { sp #1 }
5749         \__spintent_unknown_keys_cmd:n {##1}
5750     },
5751 }
5752 }

```

(End of definition for read-cmd and others.)

```

\__spintent_spcircfig_short_bool
\__spintent_spcircfig_op_tl
\__spintent_spcircfig_intent_tl
\__spintent_spcircfig_intent_arg_tl

```

The variable `\__spintent_spcircfig_short_bool` records whether the argument resolved as the short number/letter form (via `\__spintent_luafun_geo_afig_parse_and_set:n/\__spintent_luafun_geo_pfig_p` the same entry point `\spAfig/\spPfig` use) rather than a real circle. `\__spintent_spcircfig_op_tl` holds `área` or `perímetro`. The variable `\__spintent_spcircfig_intent_tl` holds the full spoken phrase: `\__spintent_spcircfig_del_word_tl` (via `\__spintent_spcircfig_del_word:n`: the word, contracted with `del-/de-la-`, or `read-arg`), an optional `sub-index` suffix, and, when a center is given, `-con-centro-en` — the center itself is never concatenated into it: it is read separately, by reference, through `\__spintent_mathmlarg_point:nnn`, so a subscripted or primed center (e.g. `O_{1}`), as in the Euler circle) is never an issue.

```

5753 \bool_new:N \__spintent_spcircfig_short_bool
5754 \tl_new:N \__spintent_spcircfig_op_tl
5755 \tl_new:N \__spintent_spcircfig_del_word_tl
5756 \tl_new:N \__spintent_spcircfig_intent_tl
5757 \tl_new:N \__spintent_spcircfig_intent_arg_tl

```

(End of definition for `\__spintent_spcircfig_short_bool` and others.)

```
\__spintent_spcircfig_sym:n
```

Typesets `\bigodot` or the plain letter C, according to `print-sym`. Receives the short command name in ‘`#1`’.

```

5758 \cs_new_protected:Nn \__spintent_spcircfig_sym:n
5759 {
5760     \str_if_eq:vnTF { \__spintent_sp #1 _print_sym_str } { odot }
5761     { \bigodot }
5762     { C }
5763 }

```

(End of definition for `\__spintent_spcircfig_sym:n`.)

```
\__spintent_spcircfig_word_and_article:n
```

Sets `\__spintent_spcirc_word_tl` to `read-arg` or `circunferencia/círculo` from `read-sty`, and `\__spintent_spcirc_articulo_str` to the matching article, which always follows `read-sty` even when `read-arg` overrides the word. Receives the short command name in ‘`#1`’.

```

5764 \cs_new_protected:Nn \__spintent_spcircfig_word_and_article:n
5765 {
5766     \str_if_eq:vnTF { \__spintent_sp #1 _read_sty_str } { circun }
5767     { \str_set:Nn \__spintent_spcirc_articulo_str { la } }
5768     { \str_set:Nn \__spintent_spcirc_articulo_str { el } }
5769     \tl_if_empty:cTF { \__spintent_sp #1 _read_arg_tl }
5770     {
5771         \str_if_eq:vnTF { \__spintent_sp #1 _read_sty_str } { circun }
5772         { \tl_set:Nn \__spintent_spcirc_word_tl { circunferencia } }
5773         { \tl_set:Nn \__spintent_spcirc_word_tl { círculo } }
5774     }
5775     {
5776         \tl_set:Ne \__spintent_spcirc_word_tl
5777         {
5778             \tl_use:c { \__spintent_sp #1 _read_arg_tl }
5779         }
5780     }
5781 }

```

(End of definition for `\__spintent_spcircfig_word_and_article:n`.)

```

\__spintent_spcircfig_del_word:n Sets \l__spintent_spcircfig_del_word_tl to \__spintent_spcircfig_word_and_article:n's word,
prefixed with del- or de-la- — contracted, not de-el-, since a screen reader does not contract that on its
own. Receives the short command name in '#1'.

5782 \cs_new_protected:Nn \__spintent_spcircfig_del_word:n
5783 {
5784   \__spintent_spcircfig_word_and_article:n {#1}
5785   \str_if_eq:VnTF \__spintent_spcirc_articulo_str { el }
5786   {
5787     \tl_set:Ne \l__spintent_spcircfig_del_word_tl { del- \l__spintent_spcirc_word_tl }
5788   }
5789   {
5790     \tl_set:Ne \l__spintent_spcircfig_del_word_tl { de-la- \l__spintent_spcirc_word_tl }
5791   }
5792 }

```

(End of definition for __spintent_spcircfig_del_word:n.)

```

\__spintent_spcircfig_build_short:n Handles the short number/letter case: builds \l__spintent_spcircfig_intent_arg_tl as \l__spintent_spcircfig_intent_arg_tl
(ber word), with prefix placed after the leading _ when given, with no :prefix (no reference — there is
nothing to point to), or _- when silent is true or concept is false. Receives the short command name in
'#1'.

5793 \cs_new_protected:Nn \__spintent_spcircfig_build_short:n
5794 {
5795   \bool_if:cTF { l__spintent_sp #1 _silent_bool }
5796   {
5797     \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { _- }
5798   }
5799   {
5800     \bool_if:cTF { l__spintent_sp #1 _concept_bool }
5801     {
5802       \__spintent_spcircfig_del_word:n {#1}
5803       \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl
5804       {
5805         -
5806         \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
5807         { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
5808         \l__spintent_spcircfig_op_tl - \l__spintent_spcircfig_del_word_tl -
5809         \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
5810         \l__spintent_geo_luaset_sub_spoken_str
5811       }
5812     }
5813     {
5814       \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { _- }
5815     }
5816   }
5817 }

```

(End of definition for __spintent_spcircfig_build_short:n.)

```

\__spintent_spcircfig_finish_intent:nn Appends an optional sub-index suffix and, when #2 is non-empty, -con-centro-en to \l__spintent_spcircfig_intent_arg_tl
— shared by both branches of \__spintent_spcircfig_build_concept:nn, after either one has set the
word itself. Receives the short command name in '#1' and the argument in '#2'.

5818 \cs_new_protected:Nn \__spintent_spcircfig_finish_intent:nn
5819 {
5820   \str_if_empty:cF { l__spintent_sp #1 _sub_index_str }
5821   {
5822     \tl_put_right:Ne \l__spintent_spcircfig_intent_tl
5823     {
5824       -
5825       \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
5826       \str_use:c { l__spintent_sp #1 _sub_index_str }
5827     }
5828   }
5829   \tl_if_empty:nF {#2}
5830   {
5831     \tl_put_right:Nn \l__spintent_spcircfig_intent_tl { -con-centro-en }
5832   }
5833 }

```

(End of definition for __spintent_spcircfig_finish_intent:nn.)

_spintent_spcircfig_build_concept:nn

Builds _spintent_spcircfig_intent_arg_tl. `_` - when `silent` is `true`, whatever `#2` is, or when `false` and `concept` is `false` with `read_arg` empty (`_( $pt)` instead when `#2` is non-empty — the center is still read). Otherwise `_\\_spintent_spcircfig_intent_tl` — with `:prefix( $pt)` appended only when `#2` is non-empty — with `prefix` placed after the leading `_` when given, where `\\_spintent_spcircfig_intent_tl` is: with `concept true`, `\\_spintent_spcircfig_op_tl-\\_spintent_spcircfig_del_word_tl` (via `\\_spintent_spcircfig_del_word:n`: the word, contracted with `del-/de-la-`, or `read_arg` the same way); with `false` and `read_arg` given, `read_arg` verbatim instead, with `\\_spintent_spcircfig_op_tl` dropped entirely — `área/ perímetro` is itself the concept `concept` toggles, not the `-con-centro-en` that follows, which always stays (it identifies which figure, not the concept). No automatic article either — the word (and any article it needs) is the user's to write, same convention as `\\_spintent_spcirc_process:n`. Either way, an optional `sub-index` suffix and, when `#2` is non-empty, `-con-centro-en` follow. Receives the short command name in '`#1`' and the argument in '`#2`'.

```

5834 \\_cs_new_protected:Nn \\_spintent_spcircfig_build_concept:nn
5835 {
5836   \\bool_if:cTF { \\_spintent_sp #1 _silent_bool }
5837   {
5838     \\tl_set:Nn \\_spintent_spcircfig_intent_arg_tl { _- }
5839   }
5840   {
5841     \\bool_if:cTF { \\_spintent_sp #1 _concept_bool }
5842     {
5843       \\_spintent_spcircfig_del_word:n {#1}
5844       \\tl_set:Ne \\_spintent_spcircfig_intent_tl
5845       {
5846         \\_spintent_spcircfig_op_tl - \\_spintent_spcircfig_del_word_tl
5847       }
5848       \\_spintent_spcircfig_finish_intent:nn {#1} {#2}
5849       \\tl_set:Ne \\_spintent_spcircfig_intent_arg_tl
5850       {
5851         -
5852         \\tl_if_empty:cF { \\_spintent_sp #1 _prefix_tl }
5853         {
5854           \\tl_use:c { \\_spintent_sp #1 _prefix_tl } -
5855         }
5856         \\_spintent_spcircfig_intent_tl
5857         \\tl_if_empty:nF {#2} { :prefix ( $ pt ) }
5858       }
5859     }
5860     {
5861       \\tl_if_empty:cTF { \\_spintent_sp #1 _read_arg_tl }
5862       {
5863         \\tl_if_empty:nTF {#2}
5864         { \\tl_set:Nn \\_spintent_spcircfig_intent_arg_tl { _- } }
5865         { \\tl_set:Ne \\_spintent_spcircfig_intent_arg_tl { _ ( $ pt ) } }
5866       }
5867       {
5868         \\tl_set:Ne \\_spintent_spcircfig_intent_tl
5869         { \\tl_use:c { \\_spintent_sp #1 _read_arg_tl } }
5870         \\_spintent_spcircfig_finish_intent:nn {#1} {#2}
5871         \\tl_set:Ne \\_spintent_spcircfig_intent_arg_tl
5872         {
5873           -
5874           \\_spintent_spcircfig_intent_tl
5875           \\tl_if_empty:nF {#2} { :prefix ( $ pt ) }
5876         }
5877       }
5878     }
5879   }
5880 }

```

(End of definition for _spintent_spcircfig_build_concept:nn.)

_spintent_spcircfig_process:nn

Receives the short command name in '`#1`' and the argument in '`#2`'. Sets `\\_spintent_spcircfig_op_tl` to `área/ perímetro`. With `#2` empty, `\\_spintent_spcircfig_short_bool` is set `false` and nothing else is parsed. With a comma in `#2`, reports `sp#1-invalid-arg` immediately — `\\@@_luafun_geo_circ_parse_and_set:` (shared with `\\_spirc`) still accepts a second, comma-separated part as a generic `label`-type radius, matching almost any text, so it would never itself report an error for a comma-containing argument here, where a comma is never meaningful at all. Without a comma, calls `\\_spintent_luafun_geo_circ_parse_and_set:n`; on error, retries through `\\_spintent_luafun_geo_afig_parse_and_set:n/\\_spintent_luafun_geo_pfig_parse_and_set:n` (the same entry point `\\_spAfig/\\_spPfig` use) for the short number/letter form, rejecting a table-letter match

(not meaningful for a circle) back to an error. Reports `sp#1-invalid-arg` on failure either way, then delegates to `\__spintent_spcircfig_build_short:n/\__spintent_spcircfig_build_concept:nn` and `\__spintent_spcircfig_mathml:nn`.

- Unlike `\spcirc`, there is no radius/diameter here at all: a formula's left-hand side (`A_0`) names the figure, it does not carry the values needed to compute it — those belong on the right-hand side, elsewhere in the document.

```

5881 \cs_new_protected:Nn \__spintent_spcircfig_process:nn
5882 {
5883   \str_if_eq:eeTF {#1} { Acirc }
5884   { \tl_set:Nn \__spintent_spcircfig_op_tl { área } }
5885   { \tl_set:Nn \__spintent_spcircfig_op_tl { perímetro } }
5886   \tl_if_empty:nTF {#2}
5887   {
5888     \bool_set_false:N \__spintent_spcircfig_short_bool
5889   }
5890   {
5891     \bool_set_false:N \__spintent_spcircfig_short_bool
5892     \tl_if_in:nnTF {#2} { , }
5893     {
5894       \msg_error:nnee { spintent } { sp #1 -invalid-arg }
5895       { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5896     }
5897     {
5898       \__spintent_luafun_geo_circ_parse_and_set:n { \exp_not:n {#2} }
5899       \str_if_eq:VnT \__spintent_geo_luaaset_error_str { true }
5900       {
5901         \str_if_eq:eeTF {#1} { Acirc }
5902         { \__spintent_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
5903         { \__spintent_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
5904         \str_if_eq:VnF \__spintent_geo_luaaset_error_str { true }
5905         {
5906           \str_if_empty:NTF \__spintent_geo_luaaset_letra_word_str
5907           { \bool_set_true:N \__spintent_spcircfig_short_bool }
5908           { \str_set:Nn \__spintent_geo_luaaset_error_str { true } }
5909         }
5910       }
5911       \bool_if:NF \__spintent_spcircfig_short_bool
5912       {
5913         \str_if_eq:VnT \__spintent_geo_luaaset_error_str { true }
5914         {
5915           \msg_error:nnee { spintent } { sp #1 -invalid-arg }
5916           { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5917         }
5918       }
5919     }
5920   }
5921   \bool_if:NTF \__spintent_spcircfig_short_bool
5922   { \__spintent_spcircfig_build_short:n {#1} }
5923   { \__spintent_spcircfig_build_concept:nn {#1} {#2} }
5924   \__spintent_spcircfig_mathml:nn { #1 } { #2 }
5925 }

```

(End of definition for `\__spintent_spcircfig_process:nn`.)

`\__spintent_spcircfig_mathml:nn`

Renders `\__spintent_spcircfig_intent_arg_tl`, already built by `\__spintent_spcircfig_process:nn`. Three cases: the short case prints the symbol with the short value as its own subscript; an empty `#2` prints the symbol alone; a real center prints the symbol (plus optional `sub-index` subscript) and the center between parentheses. In every case a single native `\MathMLintent` wraps `\__spintent_mathcal:n{⟨A | P⟩}` and the whole subscript together — splitting the symbol and the center into separate `\MathMLintent` siblings inside the same subscript was tried and reads as `<msub>`'s own native structural narration instead. The center is read through `\__spintent_mathmlarg_point:nnn`, referenced as `$pt` (same convention as `\spcirc`). Receives the short command name in `'#1'` and the argument in `'#2'`.

```

5926 \cs_new_protected:Nn \__spintent_spcircfig_mathml:nn
5927 {
5928   \__spintent_invisible_sep: % (luamml issues #26)
5929   \MathMLintent { \__spintent_spcircfig_intent_arg_tl }
5930   {
5931     \__spintent_inner_sep:
5932     \__spintent_luafun_mrow_intent_arg:nn
5933     { } { intent = ":silent" }
5934   }

```

```

5935     \str_if_eq:eeTF {#1} { Acirc } { \__spintent_mathcal:n { A } } { \__spintent_mathcal:n
5936   }
5937   \bool_if:NTF \__spintent_spcircfig_short_bool
5938   {
5939     \c_math_subscript_token
5940     {
5941       \__spintent_spcircfig_sym:n {#1}
5942       \c_math_subscript_token { \__spintent_geo_luaset_print_tl }
5943     }
5944   }
5945   {
5946     \c_math_subscript_token
5947     {
5948       \MathMLint { :silent } { \__spintent_spcircfig_sym:n {#1} }
5949       \str_if_empty:cF { l__spintent_sp #1 _sub_index_str }
5950       {
5951         \c_math_subscript_token
5952         {
5953           \MathMLint { :silent }
5954           { \str_use:c { l__spintent_sp #1 _sub_index_str } }
5955         }
5956       }
5957       \tl_if_empty:nF {#2}
5958       {
5959         \MathMLint { :silent } { \lparen }
5960         \__spintent_mathmlarg_point:nnV { pt } { sp #1 } \__spintent_geo_luaset_point
5961         \MathMLint { :silent } { \rparen }
5962       }
5963     }
5964   }
5965 }
5966 }

```

(End of definition for `\__spintent_spcircfig_mathml:nn`.)

`\__spintent_spcircfig_exec:nnn`

The function `\__spintent_spcircfig_exec:nnn` receives the short name of the command in `#1`, the optional keys in `#2`, and the argument in `#3`, delegating the rendering to `\__spintent_spcircfig_process:nn`.

```

5967 \cs_new_protected:Nn \__spintent_spcircfig_exec:nnn
5968 {
5969   \keys_set:nn { spintent / sp #1 } { #2 }
5970   \__spintent_spcircfig_process:nn { #1 } { #3 }
5971 }

```

(End of definition for `\__spintent_spcircfig_exec:nnn`.)

`\spAcirc`
`\spPcirc`

Both public commands `\spAcirc` and `\spPcirc` enforce math mode, open a local group, and delegate execution to `\__spintent_spcircfig_exec:nnn`.

```

5972 \NewDocumentCommand \spAcirc { 0{} d() }
5973 {
5974   \mode_if_math:TF
5975   {
5976     \group_begin:
5977     \attribute \__spintent_scope_attr_int = 1
5978     \tl_if_novalue:nTF {#2}
5979     { \__spintent_spcircfig_exec:nnn { Acirc } { #1 } { } }
5980     { \__spintent_spcircfig_exec:nnn { Acirc } { #1 } { #2 } }
5981     \group_end:
5982   }
5983   { \msg_error:nnn { spintent } { need-math-mode } { \spAcirc } }
5984 }
5985 \NewDocumentCommand \spPcirc { 0{} d() }
5986 {
5987   \mode_if_math:TF
5988   {
5989     \group_begin:
5990     \attribute \__spintent_scope_attr_int = 1
5991     \tl_if_novalue:nTF {#2}
5992     { \__spintent_spcircfig_exec:nnn { Pcirc } { #1 } { } }
5993     { \__spintent_spcircfig_exec:nnn { Pcirc } { #1 } { #2 } }
5994     \group_end:
5995   }
5996   { \msg_error:nnn { spintent } { need-math-mode } { \spPcirc } }

```

```
5997 }
```

(End of definition for `\spAcirc` and `\spPcirc`. These functions are documented on page 31.)

11.48 The command `\spVol`

The user-level command `\spVol` provides a semantic interface for the volume of a solid, always typesetting `\mathcal{V}`.

11.48.1 Definition of variables for `\spVol`

The variable `\l__spinttent_spVol_mode_str` holds the `read-cmd` choice (`school` or `mathcat`); `\l__spinttent_spVol_sub_index_str` the raw `sub-index` text; `\l__spinttent_spVol_read_arg_tl` the sanitized `read-arg` text. `\l__spinttent_spVol_word_tl` and `\l__spinttent_spVol_suffix_tl` are scratch variables holding the word to voice (volumen) and the text to attach after it; `\l__spinttent_spVol_native_bool` records whether that text should be combined natively with the subscript (silenced) or left for MathCAT to read on its own. The variable `\l__spinttent_spVol_read_arg_full_tl` holds the text to voice when `read-arg` is given.

```
5998 \str_new:N \l__spinttent_spVol_mode_str
5999 \str_new:N \l__spinttent_spVol_sub_index_str
6000 \tl_new:N \l__spinttent_spVol_read_arg_tl
6001 \tl_new:N \l__spinttent_spVol_word_tl
6002 \tl_new:N \l__spinttent_spVol_suffix_tl
6003 \bool_new:N \l__spinttent_spVol_native_bool
6004 \tl_new:N \l__spinttent_spVol_read_arg_full_tl
```

(End of definition for `\l__spinttent_spVol_mode_str` and others.)

11.48.2 Definition of keys for `\spVol`

We define the *(keys)* `read-cmd`, `sub-index`, `read-sub`, `read-arg`, `concept` and `silent`.

```
read-cmd \keys_define:nn { spintent / spVol }
sub-index {
read-sub {
read-arg   read-cmd   .choices:nn = { school , mathcat }
concept    {
silent      \int_case:nn { \l_keys_choice_int }
unknown     {
6011         {1} { \str_set:Nn \l__spinttent_spVol_mode_str { school } }
6012         {2} { \str_set:Nn \l__spinttent_spVol_mode_str { mathcat } }
6013         }
6014     },
6015     read-cmd   .initial:n = school,
6016     read-cmd   .value_required:n = true,
6017     read-cmd / unknown .code:n =
6018     \msg_error:nneee { spintent } { unknown-choice }
6019     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
6020     sub-index   .code:n =
6021     { \str_set:Nn \l__spinttent_spVol_sub_index_str {#1} },
6022     sub-index   .initial:n = ,
6023     sub-index   .value_required:n = true,
6024     read-sub    .bool_set:N = \l__spinttent_spVol_read_sub_bool,
6025     read-sub    .initial:n = false,
6026     read-sub    .value_required:n = true,
6027     read-arg    .code:n =
6028     { \__spinttent_sanitizize_affix:Nn \l__spinttent_spVol_read_arg_tl {#1} },
6029     read-arg    .initial:n = ,
6030     read-arg    .value_required:n = true,
6031     concept     .bool_set:N = \l__spinttent_spVol_concept_bool,
6032     concept     .initial:n = true,
6033     concept     .value_required:n = true,
6034     silent      .bool_set:N = \l__spinttent_spVol_silent_bool,
6035     silent      .initial:n = false,
6036     silent      .value_required:n = true,
6037     unknown     .code:n =
6038     {
6039         \str_set:Nn \l__spinttent_command_name_str { spVol }
6040         \__spinttent_unknown_keys_cmd:n {#1}
6041     },
6042 }
```

(End of definition for `read-cmd` and others.)

11.48.3 Internal functions for \spVol

`\__spintent_spVol_print:` The function `\__spintent_spVol_print:` dispatches to `\__spintent_spVol_silent:` when `silent` is `true`, or to `\__spintent_spVol_verbal:` otherwise.

```
6043 \cs_new_protected:Nn \__spintent_spVol_print:
6044 {
6045   \__spintent_invisible_sep:
6046   \bool_if:NTF \l__spintent_spVol_silent_bool
6047     { \__spintent_spVol_silent: }
6048     { \__spintent_spVol_verbal: }
6049 }
```

(End of definition for `\__spintent_spVol_print:`.)

`\__spintent_spVol_silent:` The function `\__spintent_spVol_silent:` typesets `\mathcal{V}` with the intent `_-` — the symbol alone through the shared queue when there is no `sub-index`, or symbol and subscript together in one native `\MathMLintent` (subscript silenced) when there is.

```
6050 \cs_new_protected:Nn \__spintent_spVol_silent:
6051 {
6052   \str_if_empty:NTF \l__spintent_spVol_sub_index_str
6053     {
6054       \__spintent_luafun_mrow_intent_arg:nn
6055       { } { intent = "_-" }
6056       { \mathcal{V} }
6057     }
6058     {
6059       \MathMLintent { _- }
6060       {
6061         \__spintent_inner_sep:
6062         \mathcal{V}
6063         \c_math_subscript_token
6064         {
6065           \MathMLintent { :silent }
6066           { \l__spintent_spVol_sub_index_str }
6067         }
6068       }
6069     }
6070 }
```

(End of definition for `\__spintent_spVol_silent:`.)

`\__spintent_spVol_verbal:` The function `\__spintent_spVol_verbal:` delegates to `\__spintent_spVol_read_arg_print:` when `read-arg` is given, and otherwise, with `concept false`, to `\__spintent_spVol_symbol_print:`. With `concept true` it sets `\l__spintent_spVol_word_tl` to `volumen` and `\l__spintent_spVol_suffix_tl` to `sub-⟨value⟩/⟨value⟩` (per `read-sub`), combined natively only under `read-cmd=school` — under `read-cmd=mathcat` the suffix is left for MathCAT to read on its own. Delegates the actual typesetting to `\__spintent_spVol_attach_subscript:`

```
6071 \cs_new_protected:Nn \__spintent_spVol_verbal:
6072 {
6073   \tl_if_empty:NTF \l__spintent_spVol_read_arg_tl
6074     {
6075       \bool_if:NTF \l__spintent_spVol_concept_bool
6076       {
6077         \tl_set:Nn \l__spintent_spVol_word_tl { volumen }
6078         \str_if_eq:VnTF \l__spintent_spVol_mode_str { school }
6079           { \bool_set_true:N \l__spintent_spVol_native_bool }
6080           { \bool_set_false:N \l__spintent_spVol_native_bool }
6081         \tl_set:Ne \l__spintent_spVol_suffix_tl
6082         {
6083           \bool_if:NT \l__spintent_spVol_read_sub_bool { sub- }
6084           \l__spintent_spVol_sub_index_str
6085         }
6086         \__spintent_spVol_attach_subscript:
6087       }
6088       { \__spintent_spVol_symbol_print: }
6089     }
6090     { \__spintent_spVol_read_arg_print: }
6091 }
```

(End of definition for `\__spintent_spVol_verbal:`.)

`\__spintent_spVol_symbol_print:`

With `concept false`, the symbol itself is read instead — but only when `read-cmd` is `school`: with no `sub-index`, the shared queue tags the symbol with the intent `_V-`; with one, symbol and subscript are wrapped together in one native `\MathMLintent` (subscript silenced). With `read-cmd mathcat`, the intent is left empty: MathCAT applies its own rules to the symbol and its subscript, unmarked by us.

```

6092 \cs_new_protected:Nn \__spintent_spVol_symbol_print:
6093 {
6094   \str_if_empty:NTF \l__spintent_spVol_sub_index_str
6095   {
6096     \str_if_eq:VnTF \l__spintent_spVol_mode_str { school }
6097     {
6098       \__spintent_luafun_mrow_intent_arg:nn
6099       { } { intent = "_V-" }
6100       { \mathcal{V} } }
6101     }
6102     { \mathcal{V} } }
6103   }
6104   {
6105     \str_if_eq:VnTF \l__spintent_spVol_mode_str { school }
6106     {
6107       \MathMLintent
6108       {
6109         _V-
6110         \bool_if:NT \l__spintent_spVol_read_sub_bool { sub- }
6111         \l__spintent_spVol_sub_index_str
6112       }
6113       {
6114         \__spintent_inner_sep:
6115         \mathcal{V}
6116         \c_math_subscript_token
6117         {
6118           \MathMLintent { :silent }
6119           { \l__spintent_spVol_sub_index_str }
6120         }
6121       }
6122     }
6123     {
6124       \mathcal{V}
6125       \c_math_subscript_token
6126       { \l__spintent_spVol_sub_index_str }
6127     }
6128   }
6129 }

```

(End of definition for `\__spintent_spVol_symbol_print:`.)

`\__spintent_spVol_attach_subscript:`

The function `\__spintent_spVol_attach_subscript:` typesets `\mathcal{V}`. An empty `sub-index` tags the symbol alone through the shared queue (`\l__spintent_spVol_suffix_tl` is discarded, since there's no subscript to attach it to). A given one combines symbol, subscript, and `\l__spintent_spVol_suffix_tl` in one native `\MathMLintent` (subscript silenced) when `\l__spintent_spVol_native_bool` is `true`, or tags the symbol alone through the shared queue with the subscript left bare when `false`.

```

6130 \cs_new_protected:Nn \__spintent_spVol_attach_subscript:
6131 {
6132   \str_if_empty:NTF \l__spintent_spVol_sub_index_str
6133   {
6134     \__spintent_luafun_mrow_intent_arg:nn
6135     { } { intent = "_ \l__spintent_spVol_word_tl :nofix" }
6136     { \mathcal{V} } }
6137   }
6138   {
6139     \bool_if:NTF \l__spintent_spVol_native_bool
6140     {
6141       \MathMLintent
6142       { _ \l__spintent_spVol_word_tl - \l__spintent_spVol_suffix_tl :nofix }
6143       {
6144         \__spintent_inner_sep:
6145         \mathcal{V}
6146         \c_math_subscript_token
6147         {
6148           \MathMLintent { :silent }
6149           { \l__spintent_spVol_sub_index_str }
6150         }
6151       }
6152     }
6153     {
6154       \mathcal{V}
6155       \c_math_subscript_token
6156       { \l__spintent_spVol_sub_index_str }
6157     }
6158   }
6159 }

```

```

6151         }
6152     }
6153     {
6154         \__spintent_luafun_mrow_intent_arg:nn
6155         { } { intent = "_ \l__spintent_spVol_word_tl :nofix" }
6156         { \mathcal{V} }
6157         \c_math_subscript_token
6158         { \l__spintent_spVol_sub_index_str }
6159     }
6160 }
6161 }

```

(End of definition for `\__spintent_spVol_attach_subscript:`.)

`\__spintent_spVol_read_arg_print:`

With `read-arg` given, `sub-index/read-sub` are ignored for the *spoken* content (the subscript, if any, still displays, silenced). With `concept true`, `\l__spintent_spVol_read_arg_full_tl` becomes `volumen` followed by `read-arg`'s text; with `false`, `read-arg`'s text alone. Combined natively regardless of `read-cmd:` with no `sub-index`, the shared queue tags the symbol alone; with one, symbol and subscript are wrapped together in one native `\MathMLintent`.

```

6162 \cs_new_protected:Nn \__spintent_spVol_read_arg_print:
6163 {
6164     \bool_if:NTF \l__spintent_spVol_concept_bool
6165     {
6166         \tl_set:Nx \l__spintent_spVol_read_arg_full_tl
6167         { volumen- \tl_use:N \l__spintent_spVol_read_arg_tl }
6168     }
6169     { \tl_set_eq:NN \l__spintent_spVol_read_arg_full_tl \l__spintent_spVol_read_arg_tl }
6170     \str_if_empty:NTF \l__spintent_spVol_sub_index_str
6171     {
6172         \__spintent_luafun_mrow_intent_arg:nn
6173         { } { intent = "_ \l__spintent_spVol_read_arg_full_tl :nofix" }
6174         { \mathcal{V} }
6175     }
6176     {
6177         \MathMLintent
6178         { _ \l__spintent_spVol_read_arg_full_tl :nofix }
6179         {
6180             \__spintent_inner_sep:
6181             \mathcal{V}
6182             \c_math_subscript_token
6183             {
6184                 \MathMLintent { :silent }
6185                 { \l__spintent_spVol_sub_index_str }
6186             }
6187         }
6188     }
6189 }

```

(End of definition for `\__spintent_spVol_read_arg_print:`.)

`\spVol` The public command `\spVol` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spVol_print:`.

```

6190 \NewDocumentCommand \spVol { 0{ } }
6191 {
6192     \mode_if_math:TF
6193     {
6194         \group_begin:
6195         \attribute \l__spintent_scope_attr_int = 1
6196         \keys_set:nn { spintent / spVol } {#1}
6197         \__spintent_spVol_print:
6198         \group_end:
6199     }
6200     { \msg_error:nnn { spintent } { need-math-mode } { \spVol } }
6201 }

```

(End of definition for `\spVol`. This function is documented on page 34.)

11.49 Finish package

Finish package implementation.

```

6202 \file_input_stop:
6203 </package>

```

12 Índice de la Implementación

The italic numbers denote the pages where the corresponding entry is described, the numbers underlined and all others indicate the line on which they are implemented in the package code.

A	
\angle	126
autofit	<u>3269</u>
C	
cifras-sep	<u>289</u> , <u>574</u> , <u>861</u> , <u>1898</u> , <u>1969</u> , <u>2592</u>
Document class:	
l3doc	40
cmd-key-unknown	<u>208</u>
cmd-key-value-unknown	<u>208</u>
Commands provide by spintent:	
\spmoney	58
\spnum	48
\spsiglo	68, 70
\spunit	53
concept <u>2060</u> , <u>2272</u> , <u>3269</u> , <u>3896</u> , <u>4105</u> , <u>4392</u> , <u>4546</u> , <u>4731</u> , <u>5132</u> , <u>5388</u> , <u>5687</u> , <u>6005</u>	
concept*	<u>4731</u>
currency	<u>861</u>
currency-no-subunits	<u>830</u>
D	
diameter	<u>5388</u>
I	
inline	<u>2454</u>
inline-numeric-denominator	552
invalid-angle-format	<u>2542</u>
invalid-century-format	<u>1256</u>
invalid-currency	<u>830</u>
invalid-date-format	<u>1486</u>
invalid-time-format	<u>1407</u>
J	
join-word	<u>2670</u>
K	
Keys for \spAcirc provide by spintent:	
concept	156
prefix	155, 156
print-sym	155–157
radio-sty	155
read-arg	155, 156
read-cmd	155, 156
read-sty	155, 156
read-sub	156
silent	156
sub-index	155, 156
Keys for \spAfig provide by spintent:	
prefix	137
print-sym	136, 137
read-arg	137, 139
read-cmd	136, 137
read-sub	137
Keys for \spangle provide by spintent:	
concept	125
prefix	125
print-m	125
read-cmd	124, 125
recto	124, 125
silent	125
Keys for \sparc provide by spintent:	
concept	130
prefix	130
read-cmd	130
silent	130
Keys for \spcirc provide by spintent:	
prefix	149
print-sym	149–151
radio-sty	149, 150
read-arg	149
read-cmd	149
read-sty	149, 150
read-sub	150
sub-index	149, 150
Keys for \spcoord provide by spintent:	
silent	102
Keys for \spdiam provide by spintent:	
concept	145
read-arg	145
read-cmd	145
read-sub	145
silent	145
sub-index	145
upper-case	145
Keys for \spdiv provide by spintent:	
cifras-sep	80
notation-sym	80
read-parens	80
read-period-sep	80
read-sign	80
read-sym	81
set-sym	81
wrap-parens	80
Keys for \spdsym provide by spintent:	
prefix	78
read-sym	78
set-sym	78
suffix	78
Keys for \spmangle provide by spintent:	
concept	125
prefix	125
print-m	124, 125
read-cmd	125
recto	124, 125
silent	125
Keys for \spmarc provide by spintent:	
concept	130
prefix	130
read-cmd	130
silent	130
Keys for \spmcd provide by spintent:	
concept	84
prefix	84
read-cmd	84
result	84
silent	84
upper-case	84
Keys for \spmcm provide by spintent:	
concept	84

prefix	84	print-sym	155–157
read-cmd	84	radio-sty	155
result	84	read-arg	155, 156
silent	84	read-cmd	155, 156
upper-case	84	read-sty	155, 156
Keys for \spmone provide by spintent :		read-sub	156
cifras-sep	60	silent	156
currency	60	sub-index	155, 156
money-math	60	Keys for \spPfig provide by spintent :	
sub-units	60	prefix	137
Keys for \spmQ provide by spintent :		print-sym	136, 137
join-word	96, 98	read-arg	137, 139
read-parens	96, 98	read-cmd	136, 137
use-dfrac	96–100	read-sub	137
use-spnZ	96, 99, 100	Keys for \spPoint provide by spintent :	
wrap-parens	96, 98	concept	107
Keys for \spmside provide by spintent :		prefix	106, 107
concept	121	read-cmd	106, 107
prefix	120, 121	silent	107
print-m	120, 121	Keys for \spPoly provide by spintent :	
print-sym	145	concept	134
read-arg	120, 121	prefix	133, 134
read-cmd	120, 121, 123	print-sym	134
read-sty	120, 121	read-cmd	133, 134
silent	121, 123	silent	134
Keys for \spmsym provide by spintent :		Keys for \sprad provide by spintent :	
not-print	79	concept	145
prefix	79	read-arg	145
read-sym	79	read-cmd	145
set-sym	79	read-sub	145
suffix	79	silent	145
Keys for \spmul provide by spintent :		sub-index	145
cifras-sep	82	upper-case	145
notation-sym	82	Keys for \sprayo provide by spintent :	
read-parens	82	concept	118
read-period-sep	82	prefix	118
read-sign	82	read-cmd	118, 119
read-sym	82	read-sty	117, 118
set-sym	82	silent	118, 119
wrap-parens	82	Keys for \sprazon provide by spintent :	
Keys for \spnD provide by spintent :		inline	91
concept	88	silent	91
prefix	88	use-dfrac	91
result-num	88	Keys for \spread provide by spintent :	
result	88	read-arg	76
silent	88	silent	76
Keys for \spnM provide by spintent :		Keys for \sprecta provide by spintent :	
concept	88	prefix	113, 114
prefix	88	read-cmd	113, 114
result-num	88	silent	114
result	88	Keys for \spshort provide by spintent :	
silent	88	read-arg	65
Keys for \spnum provide by spintent :		small-caps	65
cifras-sep	47–49	Keys for \spside provide by spintent :	
notation-sym	48	concept	121
read-period-sep	48, 50	prefix	120, 121
read-sign	48	read-arg	120, 121
Keys for \spnZ provide by spintent :		read-cmd	120, 121, 123
cifras-sep	94	read-sty	120, 121
read-parens	94, 95	silent	121, 123
read-sign	94	Keys for \spsiglo provide by spintent :	
wrap-parens	94, 95	print-era	68, 70
Keys for \spPcirc provide by spintent :		Keys for \spunit provide by spintent :	
concept	156	cifras-sep	54
prefix	155, 156	notation-sym	54

print-cdot	54	right-angle-sqr-fallback	80
read-cdot	54	\rightangle	126
read-period-sep	54		
read-slash	54	S	
Keys for \spusep provide by spintent :		seq commands:	
silent	74	\seq_set_from_clist:Nn	62
size	74	set-sym	1752, 1823, 1898, 1969
Keys for \spvec provide by spintent :		\setspintent	3, 44, 196
autofit	109	silent	1534, 1673, 2060, 2272, 2454, 2885, 3150, 3269, 3519, 3756, 3896, 4105, 4392, 4546, 4731, 5132, 5388, 5687, 6005
concept	109	size	1534
prefix	109	small-caps	1134
read-cmd	109	\spAcirc	31, 155, 5972
silent	109	spAcirc-invalid-arg	5678
Keys for \spVol provide by spintent :		\spAfig	26, 136, 5103
concept	162	spAfig-invalid-arg	4693
read-arg	162	spAfig-invalid-points	4693
read-cmd	162	spAfig-read-arg-key	4693
read-sub	162	spAfig-sym-cardinality-conflict	4693
silent	162	spAfig-sym-letra-conflict	4693
sub-index	162	\spang	24, 93, 2582
		\spangle	23, 124, 4355
M		spangle-invalid-arg	4095
mathcal-fallback	80	\sparc	25, 130, 4515
\MathMLintent	128	sparc-invalid-points	4382
mc-invalid-arguments	2053	\spcirc	30, 149, 5654
mc-too-few-arguments	2298	spcirc-invalid-arg	5381
md-invalid-argument	2298	\spcoord	19, 101, 3028
Lua module provide by spintent :		spcoord-invalid-arg	2881
spintent.lua 40, 41, 43, 46-48, 53, 57-60, 65, 66, 68-72, 83, 87, 93, 96, 100, 106		\spdate	11, 72, 1510
money-math	861	\spdiam	29, 145, 5348
		\spdiv	13, 80, 1959
N		spdiv-empty-argument	1918
not-print	1823	\spdsym	12, 78, 1799
notation-sym	289, 574, 1898, 1969	spintent internal commands:	
P		__spintent_collapse_hyphens:N	128
Packages :		\l__spintent_command_name_str	208
expl3	46, 48	\l__spintent_geo_circfig_mode_str	5677
lua-unicode-math	40	\l__spintent_geo_final_print_tl	4685
luamml	43, 96, 98	\l__spintent_geo_luaset_angulo_case_-str	2853
tagpdf	64, 65, 69	\l__spintent_geo_luaset_circ_radio_raw_-str	5370
unicode-math	40, 75	\l__spintent_geo_luaset_circ_radio_type_-str	5370
prefix 1752, 1823, 2060, 2272, 3150, 3269, 3519, 3756, 3896, 4105, 4392, 4546, 4731, 5388, 5687		\l__spintent_geo_luaset_concept_str ...	2853
print-era	1264	\l__spintent_geo_luaset_error_str	2853
print-m	3896, 4105	\l__spintent_geo_luaset_intent_str ...	2853
print-sym	4546, 4731, 5132, 5388, 5687	\l__spintent_geo_luaset_letra_str	2853
		\l__spintent_geo_luaset_letra_word_str	2853
R		\l__spintent_geo_luaset_letra_word_str	2853
radio-sty	5388	\l__spintent_geo_luaset_nombre_es_num_-str	2853
read-arg .. 1134, 1673, 3896, 4731, 5132, 5388, 5687, 6005		\l__spintent_geo_luaset_points_-clist_{\l__spintent_geo_luaset_is_-name_str	2853
read-cdot	574	\l__spintent_geo_luaset_poly_sym_str ..	2853
read-cmd .. 2060, 3150, 3269, 3519, 3756, 3896, 4105, 4392, 4546, 4731, 5132, 5388, 5687, 6005		\l__spintent_geo_luaset_print_tl	2853
read-parens	1898, 1969, 2592, 2670	\l__spintent_geo_luaset_sub_spoken_str	2853
read-period-sep	289, 574, 1898, 1969	\l__spintent_geo_luaset_sub_str	2853
read-sign	289, 1898, 1969, 2592	\l__spintent_geo_luaset_tipo_str	2853
read-slash	574	\l__spintent_geo_print_sym_str	4801
read-sty	3756, 3896, 5388, 5687		
read-sub	4731, 5132, 5388, 5687, 6005		
read-sym	1752, 1823, 1898, 1969		
recto	4105		
result	2060, 2272		
result-num	2272		

<code>__spintont_inner_sep:</code>	120	<code>__spintont_right_angle_sqr:</code>	107
<code>__spintont_invisible_sep:</code>	116	<code>__spintont_sanitize_affix:Nn</code>	136
<code>\c__spintont_invisible_sep_int</code>	124	<code>\l__spintont_saved_luaaml_flag_int</code> 98, 2651	
<code>\c__spintont_invisible_sep_tl</code>	98	<code>\l__spintont_scope_attr_int</code>	74
<code>__spintont_luafun_clean_split_and_set:n</code> 76, 246		<code>\l__spintont_setspintont_input_arg_cmd_-tl</code>	159
<code>__spintont_luafun_mrow_intent_arg:nn</code> .	116	<code>__spintont_setspintont_parse_args:nn</code> .	179
<code>__spintont_luafun_set_debug_ghost:n</code> ...	71	<code>__spintont_setspintont_set_keys:nn</code> ...	168
<code>__spintont_luafun_spang_parse:n</code>	93	<code>\l__spintont_smparc_prefix_tl</code>	4377
<code>__spintont_luafun_sptime_parse:n</code>	73	<code>__spintont_sp_rad_diam_exec:nn</code>	5343
<code>__spintont_luafun_spmoney_lookup_data:n</code> 76		<code>\l__spintont_sp_rad_diam_letra_tl</code>	5129
<code>__spintont_luafun_spmQ_scale_int:nn</code> ...	76	<code>__spintont_sp_rad_diam_print:n</code>	5179
<code>__spintont_luafun_sptime_parse:n</code>	72	<code>\l__spintont_sp_rad_diam_read_arg_full_-tl</code>	5129
<code>__spintont_luafun_spunit_lookup_alias:n</code> 76		<code>__spintont_sp_rad_diam_read_arg_-print:n</code>	5305
<code>\l__spintont_luaset_arg_above_tl</code>	265	<code>\l__spintont_sp_rad_diam_word_tl</code>	5129
<code>\l__spintont_luaset_arg_below_tl</code>	265	<code>\l__spintont_spAcirc_mode_str</code>	5667
<code>\l__spintont_luaset_cal_fam_ok_str</code>	67	<code>\l__spintont_spAcirc_prefix_tl</code>	5667
<code>\l__spintont_luaset_dec_is_all_zeros_-str</code>	258	<code>\l__spintont_spAcirc_print_sym_str</code> ...	5667
<code>\l__spintont_luaset_dec_sep_tl</code>	246	<code>\l__spintont_spAcirc_radio_sty_str</code> ...	5667
<code>\l__spintont_luaset_decimal_period_str</code> 258		<code>\l__spintont_spAcirc_read_arg_tl</code>	5667
<code>\l__spintont_luaset_denom_has_number_-str</code>	265	<code>\l__spintont_spAcirc_read_sty_str</code>	5667
<code>\l__spintont_luaset_exponent_tl</code>	258	<code>\l__spintont_spAcirc_sub_index_str</code> ...	5667
<code>\l__spintont_luaset_format_part_dec_str</code> 253		<code>\l__spintont_spAfig_mode_str</code>	4686
<code>\l__spintont_luaset_format_part_int_str</code> 253		<code>\l__spintont_spAfig_prefix_tl</code>	4686
<code>\l__spintont_luaset_has_exponent_str</code> ..	258	<code>\l__spintont_spAfig_print_sym_str</code>	4686
<code>\l__spintont_luaset_has_integer_str</code> . 99, 100, 246		<code>\l__spintont_spAfig_read_arg_tl</code>	4686
<code>\l__spintont_luaset_has_natural_str</code> ...	246	<code>\l__spintont_spang_luaset_error_str</code> ...	2538
<code>\l__spintont_luaset_has_units_str</code>	265	<code>\l__spintont_spang_luaset_grado_str</code> ...	2538
<code>\l__spintont_luaset_millions_str</code>	258	<code>\l__spintont_spang_luaset_minuto_str</code> ..	2538
<code>\l__spintont_luaset_not_decimal_not_-period_str</code>	258	<code>\l__spintont_spang_luaset_segundo_str</code> .	2538
<code>\l__spintont_luaset_not_decimal_period_-str</code>	258	<code>__spintont_spang_process:n</code>	2547
<code>\l__spintont_luaset_only_part_dec_str</code> .	253	<code>__spintont_spangle_exec:nnn</code>	4350
<code>\l__spintont_luaset_only_part_int_str</code> .	253	<code>\l__spintont_spangle_intent_arg_tl</code> ...	4164
<code>\l__spintont_luaset_only_part_period_-str</code>	253	<code>\l__spintont_spangle_intent_tl</code>	4164
<code>\l__spintont_luaset_part_dec_tl</code>	246	<code>__spintont_spangle_mathml:n</code>	4206
<code>\l__spintont_luaset_part_int_tl</code> 98, 246		<code>\l__spintont_spangle_mode_str</code>	4090
<code>\l__spintont_luaset_part_period_tl</code> ...	246	<code>\l__spintont_spangle_prefix_tl</code>	4090
<code>\l__spintont_luaset_scope_attr_str</code>	67	<code>\l__spintont_spangle_print_m_now_bool</code> .	4164
<code>\l__spintont_luaset_sign_tl</code>	99, 246	<code>__spintont_spangle_process:nn</code>	4171
<code>\l__spintont_luaset_status_str</code>	265	<code>\l__spintont_spangle_pt_idx_int</code>	4164
<code>\l__spintont_luaset_sym_fam_ok_str</code>	67	<code>\l__spintont_spangle_pt_seq</code>	4164
<code>__spintont_mathcal:n</code>	94	<code>\l__spintont_spangle_refs_tl</code>	4164
<code>__spintont_mathmlarg_point:nnn</code>	3043	<code>__spintont_spangle_sym:n</code>	4145
<code>\l__spintont_mathmlarg_point_kv_tl</code> ...	3039	<code>__spintont_sparc_exec:nnn</code>	4510
<code>\l__spintont_mathmlarg_point_mode_str</code> .	3039	<code>\l__spintont_sparc_intent_arg_tl</code>	4436
<code>\l__spintont_mathmlarg_point_ref_tl</code> ...	3039	<code>\l__spintont_sparc_intent_tl</code>	4436
<code>\l__spintont_mathmlarg_point_silent_-bool</code>	3039	<code>__spintont_sparc_mathml:n</code>	4454
<code>\l__spintont_mc_args_seq</code>	2040	<code>\l__spintont_sparc_mode_str</code>	4377
<code>\l__spintont_mc_luaset_all_numeric_str</code> 2040		<code>\l__spintont_sparc_prefix_tl</code>	4377
<code>\l__spintont_mc_luaset_types_str</code>	2040	<code>__spintont_sparc_print_arc:n</code>	4430
<code>__spintont_mc_process_args:nn</code>	2101	<code>__spintont_sparc_process:nn</code>	4441
<code>\l__spintont_mc_raw_args_seq</code>	2040	<code>\l__spintont_sparc_pt_idx_int</code>	4436
<code>\l__spintont_mc_types_seq_{uuu}\l__spintont_-spmcd_print_result_bool</code>	2040	<code>\l__spintont_sparc_pt_seq</code>	4436
<code>\l__spintont_md_out_seq</code>	2263	<code>\l__spintont_sparc_refs_tl</code>	4436
<code>__spintont_print_spunit_hierarchy:</code> ...	675	<code>__spintont_sparc_wide_arc:n</code>	4426
		<code>\l__spintont_spcirc_articulo_str</code>	5372
		<code>__spintont_spcirc_build_concept:</code>	5526
		<code>\l__spintont_spcirc_connector_arg_tl</code> ..	5474
		<code>\l__spintont_spcirc_connector_tl</code>	5474
		<code>__spintont_spcirc_exec:nn</code>	5649

<code>\l__spintent_spcirc_has_arg_bool</code>	5474	<code>__spintent_spdiv_render_parens:nn</code> . . .	1932
<code>\l__spintent_spcirc_intent_arg_tl</code>	5474	<code>\l__spintent_spdsym_prefix_tl</code>	1748
<code>\l__spintent_spcirc_intent_tl</code>	5474	<code>\l__spintent_spdsym_read_sym_tl</code>	1748
<code>__spintent_spcirc_mathml:</code>	5596	<code>\l__spintent_spdsym_suffix_tl</code>	1748
<code>\l__spintent_spcirc_mode_str</code>	5372	<code>\l__spintent_spdsym_symbol_tl</code>	1748
<code>\l__spintent_spcirc_prefix_tl</code>	5372	<code>__spintent_spfig_build_concept_-</code>	
<code>\l__spintent_spcirc_print_sym_str</code>	5372	<code>vertices:n</code>	4974
<code>__spintent_spcirc_process:n</code>	5580	<code>__spintent_spfig_build_refs:</code>	4961
<code>\l__spintent_spcirc_radio_arg_tl</code>	5474	<code>__spintent_spfig_build_simple:n</code>	4834
<code>__spintent_spcirc_radio_print:</code>	5511	<code>__spintent_spfig_exec:nnn</code>	5098
<code>\l__spintent_spcirc_radio_sty_str</code>	5372	<code>\l__spintent_spfig_intent_arg_tl</code>	4801
<code>\l__spintent_spcirc_read_sty_str</code>	5372	<code>\l__spintent_spfig_intent_tl</code>	4801
<code>\l__spintent_spcirc_sub_index_str</code>	5372	<code>__spintent_spfig_mathml:n</code>	5058
<code>__spintent_spcirc_sym:</code>	5480	<code>\l__spintent_spfig_op_tl</code>	4801
<code>__spintent_spcirc_word_and_article:</code> . .	5490	<code>__spintent_spfig_points_loop:n</code>	5046
<code>\l__spintent_spcirc_word_tl</code>	5380	<code>__spintent_spfig_process:nn</code>	4810
<code>__spintent_spcircfig_build_concept:nn</code>	5834	<code>\l__spintent_spfig_pt_idx_int</code>	4801
<code>__spintent_spcircfig_build_short:n</code> . . .	5793	<code>\l__spintent_spfig_pt_seq</code>	4801
<code>__spintent_spcircfig_del_word:n</code>	5782	<code>\l__spintent_spfig_refs_tl</code>	4801
<code>__spintent_spcircfig_exec:nnn</code>	5967	<code>__spintent_spfig_sym:</code>	4793
<code>__spintent_spcircfig_finish_intent:nn</code>	5818	<code>\l__spintent_spfig_symbol_tl</code>	4801
<code>\l__spintent_spcircfig_intent_arg_tl</code> . .	5753	<code>\l__spintent_spmangle_mode_str</code>	4090
<code>\l__spintent_spcircfig_intent_tl</code>	5753	<code>\l__spintent_spmangle_prefix_tl</code>	4090
<code>__spintent_spcircfig_mathml:nn</code>	5926	<code>\l__spintent_spmarc_mode_str</code>	4377
<code>\l__spintent_spcircfig_op_tl</code>	5753	<code>__spintent_spmc_process:nnnn</code>	2157
<code>__spintent_spcircfig_process:nn</code>	5881	<code>\l__spintent_spmcd_prefix_tl</code>	2040
<code>\l__spintent_spcircfig_short_bool</code>	5753	<code>\l__spintent_spmcd_print_upper_bool</code> . . .	2040
<code>__spintent_spcircfig_sym:n</code>	5758	<code>\l__spintent_spmcd_read_terse_bool</code> . . .	2040
<code>__spintent_spcircfig_word_and_-</code>		<code>\l__spintent_spmcd_suffix_tl</code>	2040
<code>article:n</code>	5764	<code>\l__spintent_spmcm_prefix_tl</code>	2040
<code>__spintent_spcoord_exec:nn</code>	3010	<code>\l__spintent_spmcm_print_result_bool</code> . .	2040
<code>__spintent_spcoord_inner_sep:n</code>	2975	<code>\l__spintent_spmcm_print_upper_bool</code> . .	2040
<code>__spintent_spcoord_intent_infix:</code>	2981	<code>\l__spintent_spmcm_read_terse_bool</code> . . .	2040
<code>\l__spintent_spcoord_intent_str</code>	2877	<code>\l__spintent_spmcm_spmcd_luaset_error_-</code>	
<code>\l__spintent_spcoord_luaset_error_str</code> . .	2867	<code>str</code>	2040
<code>\l__spintent_spcoord_luaset_sep_str</code> . . .	2867	<code>\l__spintent_spmcm_suffix_tl</code>	2040
<code>\l__spintent_spcoord_luaset_x_has_frac_-</code>		<code>__spintent_spmoney_build_tree:</code>	1046
<code>str</code>	2867	<code>__spintent_spmoney_check_sub_units:</code> . .	997
<code>\l__spintent_spcoord_luaset_x_sign_str</code> .	2867	<code>\l__spintent_spmoney_extracted_curr_str</code> .	822
<code>\l__spintent_spcoord_luaset_x_type_str</code> .	2867	<code>\l__spintent_spmoney_extracted_num_tl</code> . .	822
<code>\l__spintent_spcoord_luaset_x_value_-</code>		<code>__spintent_spmoney_intent_int:</code>	962
<code>str</code>	2867	<code>\l__spintent_spmoney_intent_money_str</code> . .	822
<code>\l__spintent_spcoord_luaset_y_has_frac_-</code>		<code>\l__spintent_spmoney_intent_sign_str</code> . .	822
<code>str</code>	2867	<code>\l__spintent_spmoney_intent_sub_unit_-</code>	
<code>\l__spintent_spcoord_luaset_y_sign_str</code> .	2867	<code>str</code>	822
<code>\l__spintent_spcoord_luaset_y_type_str</code> .	2867	<code>\l__spintent_spmoney_luaset_conde_str</code> . .	812
<code>\l__spintent_spcoord_luaset_y_value_-</code>		<code>\l__spintent_spmoney_luaset_currency_arg_-</code>	
<code>str</code>	2867	<code>str</code>	812
<code>__spintent_spcoord_mathml:</code>	2993	<code>\l__spintent_spmoney_luaset_plur_str</code> . .	812
<code>__spintent_spcoord_print:n</code>	2952	<code>\l__spintent_spmoney_luaset_position_-</code>	
<code>\l__spintent_spcoord_raw_arg_tl</code>	2877	<code>str</code>	812
<code>__spintent_spcoord_set_wrap_parens_-</code>		<code>\l__spintent_spmoney_luaset_print_iso_-</code>	
<code>size:</code>	2896	<code>str</code>	812
<code>__spintent_spcoord_value:n</code>	2943	<code>\l__spintent_spmoney_luaset_sing_str</code> . .	812
<code>\l__spintent_spcoord_wrap_parens_l_tl</code> . .	2877	<code>\l__spintent_spmoney_luaset_status_str</code> .	812
<code>\l__spintent_spcoord_wrap_parens_r_tl</code> . .	2877	<code>\l__spintent_spmoney_luaset_subplur_str</code> .	812
<code>\l__spintent_spdate_luaset_error_str</code> . .	1484	<code>\l__spintent_spmoney_luaset_subsing_str</code> .	812
<code>\l__spintent_spdate_luaset_output_str</code> . .	1484	<code>\l__spintent_spmoney_luaset_symbol_str</code> .	812
<code>__spintent_spdate_process:n</code>	73, 1495	<code>__spintent_spmoney_normalize_key:n</code> . . .	847
<code>__spintent_spdiv_process:nnn</code>	1944	<code>__spintent_spmoney_parse_and_route:n</code> . .	1087
<code>__spintent_spdiv_render_op:nn</code>	1923	<code>__spintent_spmoney_print_dec_sub_-</code>	
		<code>unit:</code>	1046
		<code>__spintent_spmoney_print_int:</code>	933
		<code>__spintent_spmoney_print_sep_con:</code> . . .	1046

<code>\l__spintent_sprayo_refs_tl</code>	3800	<code>\l__spintent_sptime_luaset_is_fraction_-</code>	
<code>__spintent_sprazon_exec:nnn</code>	2471	<code>str</code>	1402
<code>__spintent_spread_exec:nn</code>	1690	<code>\l__spintent_sptime_luaset_seconds_str</code>	1402
<code>\l__spintent_spread_luaset_reading_str</code>	1670	<code>\l__spintent_spunit_exp_seq</code>	545
<code>\l__spintent_spread_luaset_status_str</code>	1670	<code>\l__spintent_spunit_exponent_tl</code>	545
<code>\l__spintent_spread_read_arg_tl</code>	1670	<code>__spintent_spunit_format_canonical:..</code>	608
<code>__spintent_sprecta_exec:nn</code>	3730	<code>\l__spintent_spunit_is_mult_bool</code>	545
<code>\l__spintent_sprecta_intent_arg_tl</code> ...	3505	<code>\l__spintent_spunit_luaset_canonical_-</code>	
<code>\l__spintent_sprecta_kv_tl</code>	3505	<code>str</code>	538
<code>__spintent_sprecta_mathml:n</code>	3559	<code>\l__spintent_spunit_luaset_compact_exp_-</code>	
<code>\l__spintent_sprecta_mode_str</code>	3505	<code>str</code>	538
<code>\l__spintent_sprecta_prefix_tl</code>	3505	<code>\l__spintent_spunit_luaset_is_compact_-</code>	
<code>__spintent_sprecta_print:n</code>	3550	<code>str</code>	538
<code>\l__spintent_sprecta_pt_idx_int</code>	3505	<code>\l__spintent_spunit_luaset_is_-</code>	
<code>\l__spintent_sprecta_pt_seq</code>	3505	<code>sexagesimal_str</code>	538
<code>\l__spintent_sprecta_refs_tl</code>	3505	<code>\l__spintent_spunit_luaset_lookup_status_-</code>	
<code>__spintent_spshort_execute_layout:..</code>	1195	<code>str</code>	538
<code>\l__spintent_spshort_luaset_base_str</code> ..	1112	<code>\l__spintent_spunit_luaset_read_str</code> ...	538
<code>\l__spintent_spshort_luaset_expand_str</code>	1112	<code>\l__spintent_spunit_luaset_status_str</code> .	538
<code>\l__spintent_spshort_luaset_layout_str</code>	1112	<code>\l__spintent_spunit_mult_seq</code>	545
<code>\l__spintent_spshort_luaset_output_str</code>	1112	<code>__spintent_spunit_process_base_exp:n</code> .	616
<code>\l__spintent_spshort_luaset_status_str</code>	1112	<code>__spintent_spunit_process_mult:n</code>	668
<code>\l__spintent_spshort_luaset_suffix_str</code>	1112	<code>\l__spintent_spunit_read_cdot_bool</code> ...	545
<code>__spintent_spshort_print:nn</code>	1148	<code>\l__spintent_spunit_read_slash_str</code> ...	545
<code>__spintent_spshort_process_and_-</code>		<code>__spintent_spunit_safe_exec:n</code>	687
<code>render:n</code>	1160	<code>\l__spintent_spunit_unit_name_tl</code>	545
<code>__spintent_spshort_render_bypass:n</code> ...	1171	<code>\l__spintent_spusep_arg_tl</code>	74, 75, 1531
<code>__spintent_spshort_render_error:n</code> ...	1171	<code>__spintent_spusep_convert:n</code>	1593
<code>__spintent_spshort_render_fallback:..</code>	1171	<code>\l__spintent_spusep_left_tl</code>	74, 75, 1531
<code>__spintent_spshort_render_success:..</code>	1171	<code>\l__spintent_spusep_right_tl</code>	74, 75, 1531
<code>__spintent_spshort_render_span:nn</code>	1121	<code>__spintent_spusep_set_parens:nn</code>	1582
<code>__spintent_spside_exec:nnn</code>	4063	<code>\l__spintent_spusep_silent_bool</code>	75
<code>\l__spintent_spside_intent_arg_tl</code>	3953	<code>__spintent_spvec_exec:nnn</code>	3467
<code>\l__spintent_spside_intent_tl</code>	3953	<code>\l__spintent_spvec_intent_arg_tl</code>	3258
<code>__spintent_spside_mathml:n</code>	3986	<code>\l__spintent_spvec_kv_tl</code>	3258
<code>\l__spintent_spside_mode_str</code>	3879	<code>__spintent_spvec_mathml:n</code>	3320
<code>__spintent_spside_points_loop:n</code>	4051	<code>\l__spintent_spvec_mode_str</code>	3258
<code>\l__spintent_spside_prefix_tl</code>	3879	<code>\l__spintent_spvec_prefix_tl</code>	3258
<code>__spintent_spside_process:nn</code>	3958	<code>__spintent_spvec_print:n</code>	3303
<code>\l__spintent_spside_pt_idx_int</code>	3953	<code>\l__spintent_spvec_pt_idx_int</code>	3258
<code>\l__spintent_spside_pt_seq</code>	3953	<code>\l__spintent_spvec_pt_seq</code>	3258
<code>\l__spintent_spside_read_arg_tl</code>	3879	<code>\l__spintent_spvec_refs_tl</code>	3258
<code>\l__spintent_spside_read_sty_str</code>	3879	<code>__spintent_spVol_attach_subscript:..</code>	6130
<code>\l__spintent_spside_refs_tl</code>	3953	<code>\l__spintent_spVol_mode_str</code>	5998
<code>\l__spintent_spsiglo_luaset_arabic_str</code>	1252	<code>\l__spintent_spVol_native_bool</code>	5998
<code>\l__spintent_spsiglo_luaset_error_str</code> .	1252	<code>__spintent_spVol_print:..</code>	6043
<code>\l__spintent_spsiglo_luaset_roman_min_-</code>		<code>\l__spintent_spVol_read_arg_full_tl</code> ...	5998
<code>str</code>	1252	<code>__spintent_spVol_read_arg_print:..</code>	6162
<code>\l__spintent_spsiglo_print_era_str</code> ...	1252	<code>\l__spintent_spVol_read_arg_tl</code>	5998
<code>__spintent_spsiglo_print_math:n</code>	1336	<code>__spintent_spVol_silent:..</code>	6050
<code>__spintent_spsiglo_print_text:n</code>	1336	<code>\l__spintent_spVol_sub_index_str</code>	5998
<code>__spintent_spsiglo_render_simple:nn</code> ..	1287	<code>\l__spintent_spVol_suffix_tl</code>	5998
<code>__spintent_spsiglo_render_with_-</code>		<code>__spintent_spVol_symbol_print:..</code>	6092
<code>era:nnn</code>	1287	<code>__spintent_spVol_verbal:..</code>	6071
<code>__spintent_spsiglo_tag_span:nn</code>	1323	<code>\l__spintent_spVol_word_tl</code>	5998
<code>__spintent_sptime_build_seconds:nn</code> ...	1416	<code>__spintent_unknown_keys:n</code>	235
<code>__spintent_sptime_build_tree:....</code>	72, 1434	<code>__spintent_unknown_keys:nn</code>	235
<code>\l__spintent_sptime_luaset_base_str</code> ...	1402	<code>spintent.lua</code>	70
<code>\l__spintent_sptime_luaset_error_str</code> ..	1402	<code>\spmangle</code>	24, 124, 4355
<code>\l__spintent_sptime_luaset_has_seconds_-</code>		<code>spmangle-invalid-arg</code>	4095
<code>str</code>	1402	<code>\spmarc</code>	25, 130, 4515
		<code>spmarc-invalid-points</code>	4382

